Руководство Биджа по программированию на C

Брайан «Бидж Йоргенсен» Холл

Русский перевод учебных материалов

Оригинал: Beej's Guide to C Programming, версия 0.11.2

© Brian "Beej Jorgensen" Hall. Неучебные разделы сокращены.

Содержание

- Перед началом
- Глава 2. Привет, мир!
- Глава 3. Переменные и инструкции
- Глава 4. Функции
- Глава 5. Указатели — трепещите от страха!
- Глава 6. Массивы
- Глава 7. Строки
- Глава 8. Структуры
- Глава 9. Файловый ввод и вывод
- Глава 10. typedef: создание новых типов
- Глава 11. Указатели II: арифметика
- Глава 12. Ручное выделение памяти
- Глава 13. Область видимости
- Глава 14. Типы II: ещё больше типов!
- Глава 15. Типы III: преобразования
- Глава 16. Типы IV: квалификаторы и спецификаторы
- Глава 17. Проекты из нескольких файлов
- Глава 18. Внешнее окружение
- Глава 19. Препроцессор C
- Глава 20. Структуры II: ещё больше интересного
- Глава 21. Символы и строки II
- Глава 22. Перечисляемые типы: enum
- Глава 23. Указатели III: указатели на указатели и не только
- Глава 24. Побитовые операции
- Глава 25. Функции с переменным числом аргументов
- Глава 26. Локаль и интернационализация
- Глава 27. Unicode, широкие символы и всё такое
- Глава 28. Завершение программы
- Глава 29. Обработка сигналов
- Глава 30. Массивы переменной длины (VLA)
- Глава 31. goto
- Глава 32. Типы V: составные литералы и обобщённый выбор
- Глава 33. Массивы, часть II
- Глава 34. Дальние переходы с setjmp и longjmp

- Глава 35. Неполные типы
- Глава 36. Комплексные числа
- Глава 37. Целочисленные типы фиксированной ширины
- Глава 38. Дата и время
- Глава 39. Многопоточность
- Глава 40. Атомарные операции
- Глава 41. Спецификаторы функций и выравнивания

Перед началом

Для кого это руководство

Автор предполагает, что вы уже немного программировали на другом языке — например, Python, JavaScript, Java, Rust, Go или Swift. Желательно понимать, что такое переменные, циклы и функции.

Как читать

Руководство состоит из двух томов. Этот том — учебник; второй посвящён справочнику стандартной библиотеки. Если вы только начинаете изучать C, автор рекомендует проходить учебные главы по порядку. По мере продвижения порядок становится менее важным.

Платформа и компилятор

Примеры преимущественно используют стандартный C, хотя местами затрагиваются и другие средства, например POSIX.

В Linux, BSD и других Unix-подобных системах попробуйте запустить в терминале `cc` или `gcc`: компилятор может быть уже установлен. Если его нет, воспользуйтесь инструкциями для вашего дистрибутива, чтобы установить GCC или Clang.

В Windows автор предлагает Visual Studio Community либо, для Unix-подобной среды, WSL и GCC. На Mac можно установить Xcode и инструменты командной строки.

Компиляторов существует много, и почти любой должен подойти для примеров книги. Компилятор C++ способен собрать многие, но не все программы на C, поэтому лучше использовать именно компилятор C.

Глава 2. Привет, мир!

2.1. Чего ожидать от C

> — Куда ведёт эта лестница? — Наверх.
>
> Рэй Стэнц и Питер Венкман, «Охотники за привидениями»

C — низкоуровневый язык.

Так было не всегда. Когда люди ещё высекали перфокарты из гранита, C давал невероятную свободу от утомительной работы на языках более низкого уровня, таких как ассемблер.

Но современные языки предлагают массу возможностей, которых не существовало в 1972 году, когда был создан C. Поэтому C — довольно простой язык, не особенно богатый возможностями. На нём можно сделать всё что угодно, но иногда придётся потрудиться.

Зачем же пользоваться им сегодня?

- **Для обучения.** C — не только почтенная часть истории вычислительной техники: он связан с «голым железом» так, как современные языки обычно не связаны. Изучая C, вы узнаете, как программы работают с памятью компьютера на низком уровне. Ремней безопасности здесь нет. Уверяю, ваши программы будут аварийно завершаться. И это тоже часть удовольствия!
- **Для решения практических задач.** C до сих пор используют, например, для создания операционных систем и встраиваемых систем. (Хотя язык Rust тоже присматривается к обеим областям!)

Если вы знаете другой язык, многое в C покажется простым. C вдохновил создателей многих языков, и его следы можно увидеть в Go, Rust, Swift, Python, JavaScript, Java и множестве других. Эти части будут вам знакомы.

Но есть одна вещь, на которой люди спотыкаются: указатели. Практически всё остальное знакомо, а указатели выглядят странно. Вероятно, сама идея указателей вам уже известна, но C заставляет выражать её явно, с помощью операторов, которых вы, скорее всего, раньше не встречали.

Самое коварное: стоит по-настоящему понять указатели, как они внезапно становятся простыми. А до этого они словно скользкие угри.

Всё остальное в C — это просто запоминание ещё одного способа (а иногда и того же самого!) сделать то, что вы уже делали. Указатели — вот что непривычно. И даже они, пожалуй, лишь разновидность знакомой вам идеи.

Так что готовьтесь к захватывающему приключению: мы подберёмся к сердцу компьютера настолько близко, насколько возможно без ассемблера, используя самый влиятельный язык

программирования всех времён. Да, кто-нибудь со мной поспорит, но уж в тройке лидеров он точно, верно? Держитесь!

2.2. Привет, мир!

Это канонический пример программы на C. Его используют все. Числа слева в оригинале служат лишь для ссылок и не входят в исходный код.

```
/* Hello world program */

#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n"); // Actually do the work here
}
```

Наденем прочные резиновые перчатки с длинными манжетами, возьмём скальпель и разберём программу, чтобы узнать, как она устроена. Итак, моем руки и приступаем. Режем очень осторожно…

Начнём с простого: всё между парами символов `/*` и `*/` — комментарий, который компилятор полностью игнорирует. То же относится ко всему, что находится в строке после `//`. Это позволяет оставлять сообщения себе и другим, чтобы, вернувшись к коду в далёком будущем, вы понимали, что вообще пытались сделать. Поверьте, вы забудете — такое случается.

А что такое `#include`? Ужас какой! На самом деле эта директива велит препроцессору C взять содержимое другого файла и вставить его в программу в этом месте.

Постойте, что ещё за препроцессор C? Хороший вопрос. В компиляции есть два этапа: препроцессор и компилятор. Технически этапов больше, но давайте пока представим, что их два: меньше знаешь — крепче спишь. Всё, что начинается со знака решётки (#), обрабатывает препроцессор, прежде чем компилятор приступит к работе. Распространённые *директивы препроцессора* — `#include` и `#define`. Позже мы поговорим о них подробнее.

Зачем вообще упоминать английское название решётки *octothorpe*? Всё просто: это слово настолько забавное, что я не могу удержаться и рассказываю о нём при каждой возможности. Octothorpe. Octothorpe, octothorpe, octothorpe.

После того как препроцессор C завершит работу, компилятор получает её результат и создаёт код ассемблера, машинный код или то, что ему положено создать. Машинный код — это «язык», который понимает процессор, причём понимает очень быстро. Это одна из причин, по которым программы на C обычно быстры.

Пока не беспокойтесь о технических подробностях компиляции. Достаточно знать, что исходный код проходит через препроцессор, его результат — через компилятор, а затем получается исполняемый файл, который можно запустить.

Что же означает оставшаяся часть строки, `<stdio.h>`? Это *заголовочный файл*. Его выдаёт расширение `.h`. Перед нами заголовочный файл стандартного ввода-вывода (*standard I/O*, `stdio`), который вы ещё узнаете и полюбите. Он даёт доступ к различным возможностям ввода-вывода. Технически он содержит директивы препроцессора и прототипы функций для обычных задач ввода и вывода; о прототипах мы поговорим позже.

В нашем примере выводится строка `Hello, World!`, поэтому нам нужна функция `printf()`. Файл `<stdio.h>` предоставляет к ней доступ. Если бы мы попытались использовать `printf()` без `#include <stdio.h>`, компилятор пожаловался бы.

Откуда я знаю, что для `printf()` нужно подключать `<stdio.h>`? Из документации. В системе Unix выполните `man 3 printf`: необходимые заголовочные файлы указаны в самом начале страницы руководства. Можно также обратиться к справочному разделу этой книги. :-)

Ничего себе. И это мы только первую строку разобрали! Зато, согласитесь, разобрали её полностью. Ни одной загадки не останется!

Передохните и ещё раз посмотрите на пример. Осталось всего несколько простых строк.

С возвращением! Я знаю, вы не делали перерыв, просто подыгрываю вам.

Следующая строка — `main()`. Это определение функции `main()`; всё между фигурными скобками `{` и `}` относится к её определению.

А как вызвать другую функцию? Ответ находится в строке с `printf()`, но к этому мы вернёмся чуть позже.

Функция `main` особенная по многим причинам, но одна важнее остальных: она автоматически вызывается при запуске программы. Ничто из написанного вами не вызывается до `main()`. В нашем примере это удобно: мы хотим только вывести строку и завершить работу.

И ещё: когда выполнение программы проходит конец `main()` — закрывающую фигурную скобку, — программа завершается, и вы возвращаетесь к командной строке.

Итак, программа подключает заголовочный файл `stdio.h` и объявляет функцию `main()`, которая выполняется при запуске. Что интересного находится внутри `main()`?

Рад, что вы спросили. Правда! Там всего одна интересная вещь: вызов функции `printf()`. Отличить вызов от определения можно несколькими способами; один из признаков — отсутствие фигурных скобок после него. В конце вызова ставится точка с запятой, чтобы компилятор понял, что выражение закончено. Скоро вы увидите, что точку с запятой приходится ставить почти везде.

В `printf()` передаётся один аргумент: строка, которую нужно вывести. Да, мы вызываем функцию! Вот это да! Постойте, не зазнавайтесь. Что за странное `\n` в конце строки? Большинство символов выводится так, как они записаны. Но некоторые символы неудобно изображать на экране, поэтому их записывают двухсимвольными последовательностями с обратной косой чертой. Одна из самых распространённых — `\n` («обратная косая черта, эн»

или просто «новая строка»), обозначающая символ перевода строки. После него дальнейший вывод продолжается в начале следующей строки, как после нажатия Enter.

Скопируйте код в файл `hello.c` и соберите программу. В Unix-подобной системе (например, Linux, BSD, Mac или WSL) используйте в командной строке:

```
gcc -o hello hello.c
```

Это означает: «скомпилировать `hello.c` и записать исполняемый файл с именем `hello`». После этого появится файл `hello`, который можно запустить командой:

```
./hello
```

Начальное `./` сообщает командной оболочке: «запустить из текущего каталога».

Посмотрим, что получится:

```
Hello, World!
```

Готово и проверено! Отправляем в производство!

2.3. Подробности компиляции

Поговорим ещё немного о сборке программ на C и о том, что при этом происходит за кулисами.

Как и в других языках, в C есть исходный код. Но, в зависимости от того, каким языком вы пользовались раньше, вам, возможно, не приходилось компилировать исходный код в исполняемый файл.

Компиляция — это превращение исходного кода C в программу, которую может выполнить операционная система.

Разработчики на JavaScript и Python не привыкли к отдельному этапу компиляции, хотя за кулисами он происходит! Python компилирует исходный код в так называемый байткод, который может выполнить виртуальная машина Python. Разработчики Java привыкли к компиляции, но она создаёт байткод для виртуальной машины Java.

При компиляции C создаётся машинный код — единицы и нули, которые процессор способен выполнять непосредственно и быстро.

Языки, которые обычно не компилируют, называют интерпретируемыми. Но, как мы видели на примере Java и Python, этап компиляции встречается и там. И нет правила, запрещающего интерпретировать C: интерпретаторы C существуют! Одним словом, здесь много промежуточных вариантов. В общем смысле компиляция — это преобразование исходного кода в другую форму, более удобную для выполнения.

Компилятор C — программа, которая выполняет компиляцию. Как уже говорилось, `gcc` установлен во многих Unix-подобных системах. Его часто запускают из командной строки терминала, хотя можно запускать и из IDE.

Как же собирать программы в командной строке?

2.4. Сборка с помощью gcc

Если в текущем каталоге находится исходный файл `hello.c`, его можно собрать в программу `hello`, введя в терминале:

```
gcc -o hello hello.c
```

Параметр `-o` означает «записать результат в указанный файл», а `hello.c` в конце — имя файла, который нужно скомпилировать. Если не указать имя результата, по умолчанию получится `a.out` — это имя уходит корнями в историю Unix.

Если исходный код разбит на несколько файлов, их можно скомпилировать вместе, перечислив все файлы `.c` в командной строке. Это почти как если бы они были одним файлом, хотя на самом деле правила сложнее:

```
gcc -o awesomegame ui.c characters.c npc.c items.c
```

Все они будут собраны вместе в один большой исполняемый файл.

Для начала этого достаточно. Позже мы поговорим подробнее о нескольких исходных файлах, объектных файлах и прочих занимательных вещах.

2.5. Сборка с помощью clang

На Mac стандартный компилятор — не `gcc`, а `clang`. Однако там установлена и обёртка, благодаря которой команда `gcc` тоже работает.

Настоящий компилятор `gcc` также можно установить через Homebrew или другим способом.

2.6. Сборка из IDE

Если вы пользуетесь интегрированной средой разработки (IDE), собирать программу из командной строки, вероятно, не придётся.

В Visual Studio сочетание Ctrl+F7 выполняет сборку, а Ctrl+F5 запускает программу.

В VS Code можно нажать F5 для запуска через отладчик. Потребуется установить расширение C/C++.

В Xcode Command+B выполняет сборку, а Command+R запускает программу. Чтобы получить инструменты командной строки, найдите в интернете инструкции по запросу «Xcode command line tools».

И всё же для начала советую попробовать сборку и из командной строки — это же история!

2.7. Версии C

C прошёл долгий путь, и у его версий появилось много названий, обозначающих используемый диалект языка. Обычно они связаны с годом выхода спецификации.

Самые известные версии — C89, C99, C11 и C23. В этой книге мы сосредоточимся на последней. Вот более полная таблица:

Версия	Описание
K&R C	1978 год, исходная версия. Названа в честь Брайана Кернигана и Денниса Ритчи. Ритчи разработал и написал язык, а Керниган стал соавтором книги о нём. Сегодня исходный код K&R встречается редко; он выглядит непривычно, как среднеанглийский язык для современного англоязычного читателя.
C89, ANSI C, C90	В 1989 году Американский национальный институт стандартов (ANSI) выпустил спецификацию C, задавшую направление развития языка до наших дней. Год спустя Международная организация по стандартизации (ISO) выпустила идентичный стандарт C90.
C95	Редко упоминаемое дополнение к C89, включавшее поддержку широких символов.
C99	Первая крупная переработка со множеством дополнений. Чаще всего вспоминают появление комментариев в стиле `//`. На момент написания книги это наиболее популярная используемая версия C.
C11	Крупное обновление с поддержкой Unicode и многопоточности. Используя эти возможности, вы можете потерять переносимость в среды, которые всё ещё застряли на C99. Но, честно говоря, 1999 год был уже довольно давно.
C17, C18	Обновление C11 с исправлениями ошибок. Судя по всему, официальное название — C17, но публикацию отложили до 2018 года. Насколько мне известно, названия взаимозаменяемы, хотя C17 предпочтительнее.
C23	Самая новая спецификация.

Можно заставить GCC использовать один из стандартов с помощью аргумента командной строки `-std=`. Если нужна строгая проверка соответствия стандарту, добавьте `-pedantic`.

Например:

```
gcc -std=c11 -pedantic foo.c
```

Для этой книги я компилирую программы по стандарту C23, включив все предупреждения:

```
gcc -Wall -Wextra -std=c23 -pedantic foo.c
```

Глава 3. Переменные и инструкции

В программе на C может быть много разных элементов. Давайте разберём некоторые из них, чтобы понимать, о чём говорим.

3.1. Переменные

Говорят, что переменные хранят значения. Можно также представить переменную как понятное человеку имя данных, находящихся в памяти.

Память можно представить как большой массив байтов. Если число занимает больше одного байта, оно хранится в нескольких байтах. Каждый байт можно обозначить индексом — адресом, или положением в памяти. Это упрощённая модель современной памяти, но она полезна.

Байт обычно содержит восемь битов и может представлять числа от 0 до 255. Однако C допускает и другое число битов в байте; для однозначного обозначения восьмибитной единицы используется слово «октет».

Значение переменной находится в памяти по некоторому адресу. Пользоваться числовым адресом неудобно, поэтому мы даём данным имя. Это представление пригодится при изучении указателей: переменная-указатель хранит адрес другой переменной.

3.1.1. Имена переменных

В именах можно использовать латинские буквы, цифры и подчёркивание. Имя не может начинаться с цифры; также избегайте имён, начинающихся с двух подчёркиваний либо с подчёркивания и заглавной буквы. Для символов Unicode действуют правила стандарта, перечисленные в §D.2.

3.1.2. Типы переменных

C внимательно относится к типам. Вот несколько основных примеров:

Данные	Пример	Тип C
Целое число	`3490`	`int`
Число с плавающей точкой	`3.14159`	`float`
Символ	`'c'`	`char`
Строка	`"Hello, world!"`	`char *`

Строго говоря, литерал `3.14159` имеет тип `double`, но его можно преобразовать в `float`. Запись `char *` читается как «указатель на char»; подробнее об указателях поговорим позже.

C автоматически преобразует многие числовые типы, но другие преобразования, особенно между строками и числами, нужно выполнять самостоятельно. Большинство типов C —

разновидности этих основных типов.

Перед использованием переменную нужно объявить, указав её тип. Во время работы программы изменить тип объявленной переменной нельзя; он сохраняется, пока переменная существует в своей области видимости.

Добавим две переменные в знакомую программу:

```
#include <stdio.h>

int main(void)
{
    int i;   // Целые числа со знаком: -3, -2, 0, 1, 10
    float f; // Числа с плавающей точкой, например -3.1416
    printf("Hello, World!\n");
}
```

Переменные объявлены, но ещё не использованы и не инициализированы. Неинициализированная переменная может иметь неопределённое значение: не рассчитывайте, что это будет ноль, даже если при каком-то запуске увидели ноль. Перед использованием явно задавайте значение. Некоторые переменные со статической длительностью хранения автоматически инициализируются нулём; об этом позже.

```
int main(void)
{
    int i;
    i = 2; // Присваиваем i значение 2
    printf("Hello, World!\n");
}
```

Теперь выведем значение. Функция `printf()` получает строку формата и значения. Она ищет в строке последовательности, начинающиеся со знака `%`: `%d` выводит целое число, `%f` — число с плавающей точкой, `%s` — строку.

```
#include <stdio.h>

int main(void)
{
    int i = 2;
    float f = 3.14;
    char *s = "Hello, world!";
    printf("%s i = %d and f = %f!\n", s, i, f);
}
```

Результат содержит строку, целое число и число с плавающей точкой. В следующих примерах предполагается, что используемые переменные заранее объявлены, например `int i, j;`.

3.1.3. Логический тип

Исторически в C не было отдельного логического типа. Ноль обозначает ложь, любое ненулевое значение — истину: например, 1 и −37 истинны, 0 ложно.

```
int x = 1;
if (x) {
    printf("x is true!\n");
}
```

В C23 доступны `bool`, `true` и `false`. В более ранних достаточно современных версиях подключите `<stdbool.h>`:

```
#include <stdio.h>
#include <stdbool.h> // В C23 не требуется

int main(void) {
    bool x = true;
    if (x) {
        printf("x is true!\n");
    }
}
```

В `bool` можно преобразовать и другие значения: нулевые становятся ложью, ненулевые — истиной. Но не путайте истинность числа с равенством значению `true`:

```
printf("%d\n", true == 12); // 0: ложь
```

3.2. Операторы и выражения

Быстро познакомимся с основными операторами C; подробности будут дальше.

3.2.1. Арифметика

```
i = i + 3; // Сложение и присваивание
i = i - 8; // Вычитание
i = i * 9; // Умножение
i = i / 2; // Деление
i = i % 5; // Остаток от деления
```

Для каждого действия есть сокращённая запись:

```
i += 3; // То же, что i = i + 3
i -= 8;
i *= 9;
i /= 2;
i %= 5;
```

Оператора возведения в степень нет; используйте одну из функций семейства `pow()` из `<math.h>`.

3.2.2. Тернарный оператор

Значение тернарного выражения зависит от условия:

```
y += x > 10 ? 17 : 37;
```

Если `x > 10`, к `y` прибавляется 17, иначе — 37. Эквивалентная инструкция:

```
if (x > 10)
    y += 17;
else
    y += 37;
```

Ещё один пример определяет чётность:

```
printf("The number %d is %s.\n", x, x % 2 == 0 ? "even" : "odd");
```

Если остаток от деления `x` на 2 равен нулю, выражение выбирает строку `"even"`; иначе — `"odd"`. Тернарный оператор создаёт выражение со значением, а не инструкцию управления потоком вроде `if`.

3.2.3. Префиксные и постфиксные инкремент и декремент

```
i++; // Увеличить i на единицу, постфиксная форма
i--; // Уменьшить i на единицу, постфиксная форма
++i; // Увеличить i на единицу, префиксная форма
--i; // Уменьшить i на единицу, префиксная форма
```

В префиксной форме значением выражения становится новое значение переменной. В постфиксной — её прежнее значение. Сравните:

```
i = 10;
j = 5 + i++;             // Сначала используем 10, затем увеличиваем i
printf("%d, %d\n", i, j); // 11, 15

i = 10;
j = 5 + ++i;             // Сначала увеличиваем i, затем используем 11
printf("%d, %d\n", i, j); // 11, 16
```

Эти операторы часто применяют при работе с массивами и указателями, а также в циклах `for`:

```
for (i = 0; i < 10; i++)
    printf("i is %d\n", i);
```

3.2.4. Оператор запятой

Этот нечасто используемый оператор объединяет выражения, которые выполняются слева направо:

```
x = 10, y = 20; // Сначала присвоить 10 переменной x, затем 20 переменной y
```

Казалось бы, запятую можно заменить точкой с запятой:

```
x = 10; y = 20;
```

Однако во втором случае это два отдельных выражения, а в первом — одно. Значение выражения с оператором запятой равно значению самого правого выражения:

```
x = (1, 2, 3);
printf("x is %d\n", x); // 3
```

Этот пример искусственный, но оператор запятой часто используют в циклах `for`, чтобы выполнить несколько действий в одной части заголовка:

```
for (i = 0, j = 10; i < 100; i++, j++)
    printf("%d, %d\n", i, j);
```

Мы ещё вернёмся к этому.

3.2.5. Операторы сравнения

Стандартные операторы сравнения дают логический результат:

```
a == b; // Истина, если a равно b
a != b; // Истина, если a не равно b
a < b;  // Истина, если a меньше b
a > b;  // Истина, если a больше b
a <= b; // Истина, если a меньше или равно b
a >= b; // Истина, если a больше или равно b
```

Не путайте присваивание `=` со сравнением `==`: для сравнения нужны два знака равенства. Сравнения можно использовать в условии `if`:

```
if (a <= 10)
    printf("Success!\n");
```

3.2.6. Логические операторы

Логические операторы позволяют объединять условия и отрицать их:

Оператор	Значение
`&&`	И
`||`	ИЛИ
`!`	НЕ

Пример логического И:

```
// Выполнить действие, если x меньше 10 и y больше 20
if (x < 10 && y > 20)
    printf("Doing something!\n");
```

Пример отрицания:

```
if (!(x < 12))
    printf("x is not less than 12\n");
```

У `!` более высокий приоритет, поэтому условие заключено в скобки. Разумеется, здесь можно было написать и так:

```
if (x >= 12)
    printf("x is not less than 12\n");
```

Но мне ведь нужен был пример!

3.2.7. Оператор sizeof

`sizeof` сообщает размер типа выражения или переменной в байтах. Размеры могут отличаться в разных системах, кроме размера `char` и его разновидностей: он всегда равен одному байту. Пока это может показаться не слишком полезным, но мы ещё будем обращаться к этому оператору.

Размер не может быть отрицательным. С использует для результата специальный беззнаковый целочисленный тип `size_t`, способный вместить размер любого объекта, к которому применим `sizeof`. Суффикс `_t` сокращает слово *type*. Этот тип часто встречается там, где передают или возвращают количество.

Можно измерить переменную или выражение:

```
int a = 999;
// %zu — спецификатор формата для size_t
printf("%zu\n", sizeof a);       // В системе автора: 4
printf("%zu\n", sizeof(2 + 7)); // В системе автора: 4
printf("%zu\n", sizeof 3.14);   // В системе автора: 8
```

Это размер типа результата выражения, а не длина записи самого выражения. У `2 + 7` и `a` тип `int`, поэтому их размеры одинаковы. Можно указать и имя типа; в этом случае скобки обязательны:

```
printf("%zu\n", sizeof(int));  // В системе автора: 4
printf("%zu\n", sizeof(char)); // Во всех системах: 1
```

Обычно результат `sizeof` определяется во время компиляции. Исключение связано с массивами переменной длины, о которых поговорим позже.

3.3. Управление потоком

Рассмотрим конструкции `if`, `for`, `while` и `do-while`. Сначала замечание об инструкциях и блоках.

После `if` или `while` можно поставить одну инструкцию:

```
if (x == 10) printf("x is 10\n");
```

Либо записать её на следующей строке: пробелы и переносы строк в C в основном не имеют значения, в отличие от Python.

```
if (x == 10)
    printf("x is 10\n");
```

Чтобы выполнить несколько инструкций, объедините их фигурными скобками в блок:

```
if (x == 10) {
    printf("x is 10\n");
    printf("And also this happens when x is 10\n");
}
```

Многие всегда ставят скобки, даже для одной инструкции: так легче читать код и избежать ошибок, когда отступ создаёт ложное впечатление, будто инструкция входит в условие:

```
// ПЛОХОЙ ПРИМЕР: вторая строка выполняется всегда!
if (x == 10)
    printf("This happens if x is 10\n");
    printf("This happens ALWAYS\n");
```

Циклы работают аналогично: если нужно выполнить несколько инструкций, заключайте их в блок. Конструкция управляет одной следующей инструкцией, а блок считается одной составной инструкцией.

3.3.1. Инструкция if-else

```
int i = 10;
if (i > 10) {
    printf("Yes, i is greater than 10.\n");
    printf("And this will also print if i is greater than 10.\n");
}
if (i <= 10) printf("i is less than or equal to 10.\n");
```

Если условие истинно, выполняется следующая инструкция или блок. Если ложно, управление идёт дальше. С помощью `else` можно указать действие для ложного условия:

```
int i = 99;
if (i == 10)
    printf("i is 10!\n");
else {
```

```
    printf("i is decidedly not 10.\n");
    printf("Which irritates me a little, frankly.\n");
}
```

Условия можно выстроить в цепочку:

```
int i = 99;
if (i == 10)
    printf("i is 10!\n");
else if (i == 20)
    printf("i is 20!\n");
else if (i == 99) {
    printf("i is 99! My favorite\n");
    printf("I can't tell you how happy I am.\n");
    printf("Really.\n");
} else
    printf("i is some crazy number I've never heard of.\n");
```

Для некоторых таких цепочек удобен `switch`, но он сравнивает только на равенство с целочисленными константами. Цепочка `if-else` может проверять неравенства, диапазоны, значения переменных и другие условия.

3.3.2. Инструкция while

Цикл `while` повторяет действие, пока условие истинно:

```
int i = 0;
while (i < 10) {
    printf("i is now %d!\n", i);
    i++;
}
printf("All done!\n");
```

Он выводит значения от 0 до 9. Для этого примера цикл `for` был бы компактнее. Цикл `while` также часто используют для бесконечного повторения:

```
while (1) {
    printf("1 is always true, so this repeats forever.\n");
}
```

3.3.3. Инструкция do-while

`do-while` похож на `while`, но проверяет условие после выполнения тела. Поэтому тело выполняется хотя бы один раз, даже если условие изначально ложно:

```
i = 10;
// Тело не выполнится: i не меньше 10
while (i < 10) {
    printf("while: i is %d\n", i);
    i++;
}

i = 10;
```

```
// Тело выполнится один раз до проверки условия
do {
    printf("do-while: i is %d\n", i);
    i++;
} while (i < 10);
printf("All done!\n");
```

Здесь `while` сразу пропускает тело, а `do-while` выводит сообщение, увеличивает `i`, затем проверяет условие и завершает цикл. Используйте `do-while`, когда тело должно выполниться хотя бы раз.

Пример с менее предсказуемым числом повторений: продолжим, пока не выпадет число 37.

```
#include <stdio.h>  // printf
#include <stdlib.h> // rand

int main(void)
{
    int r;
    do {
        r = rand() % 100; // Число от 0 до 99
        printf("%d\n", r);
    } while (r != 37);
}
```

При повторных запусках последовательность может оказаться одинаковой: `rand()` — генератор псевдослучайных чисел. Чтобы получить другую последовательность, его нужно инициализировать другим значением; см. функцию `srand()`.

3.3.4. Инструкция for

`for` удобен, когда число повторений известно заранее. Сравните эквивалентные циклы:

```
// Числа от 0 до 9 через while
 i = 0;
while (i < 10) {
    printf("i is %d\n", i);
    i++;
}

// То же через for
for (i = 0; i < 10; i++) {
    printf("i is %d\n", i);
}
```

Заголовок `for` состоит из трёх частей, разделённых точкой с запятой: инициализация; условие продолжения; действие после каждого выполнения тела. Все три части необязательны.

```
for (инициализация; условие; действие после каждой итерации)
```

Если условие ложно с самого начала, тело не выполнится ни разу. Оператор запятой позволяет выполнять несколько действий в одной части:

```
for (i = 0, j = 999; i < 10; i++, j--) {
    printf("%d, %d\n", i, j);
}
```

Пустой заголовок создаёт бесконечный цикл:

```
for (;;) {
    printf("I will print this again and again and again\n");
    printf("for all eternity until the heat-death of the universe.\n");
    printf("Or until you hit CTRL-C.\n");
}
```

3.3.5. Инструкция switch

`switch` выбирает действие в зависимости от значения целочисленного выражения. Он вычисляет выражение и переходит к соответствующей метке `case`; выполнение продолжается с этого места. Инструкция `break` выводит управление из `switch`.

```
#include <stdio.h>

int main(void)
{
    int goat_count = 2;

    switch (goat_count) {
    case 0:
        printf("You have no goats.\n");
        break;
    case 1:
        printf("You have a singular goat.\n");
        break;
    case 2:
        printf("You have a brace of goats.\n");
        break;
    default:
        printf("You have a bona fide plethora of goats!\n");
        break;
    }
}
```

В примере управление перейдёт к `case 2`, а затем `break` завершит `switch`. Метка `default` используется, если ни один вариант не подошёл. Все метки необязательны и могут стоять в любом порядке, но `default` обычно пишут последней.

Это похоже на цепочку:

```
if (goat_count == 0)
    printf("You have no goats.\n");
else if (goat_count == 1)
    printf("You have a singular goat.\n");
else if (goat_count == 2)
    printf("You have a brace of goats.\n");
else
    printf("You have a bona fide plethora of goats!\n");
```

Однако `switch` может быстрее переходить к нужному коду, хотя стандарт этого не гарантирует. `if-else`, в отличие от `switch`, поддерживает сравнения вроде `<` и `>=`, числа с плавающей точкой и другие типы.

Есть ещё интересная особенность: если не поставить `break`, выполнение продолжится в следующем варианте. Это называется *fall through*:

```
switch (x) {
case 1:
    printf("1\n");
    // Намеренно переходим к следующему случаю
case 2:
    printf("2\n");
    break;
case 3:
    printf("3\n");
    break;
}
```

При `x == 1` программа выведет 1, затем 2 и выйдет из `switch`. При `x == 2` она выведет только 2. До C23 намеренный переход следует отмечать комментарием, чтобы коллеги поняли замысел; GCC также распознаёт такие комментарии и подавляет предупреждение.

Можно использовать нестандартный атрибут GCC, распознаваемый также Clang:

```
switch (x) {
case 1:
    printf("1\n");
    __attribute__((fallthrough));
case 2:
    // ...
}
```

Начиная с C23 есть стандартный атрибут:

```
switch (x) {
case 1:
    printf("1\n");
    [[fallthrough]];
case 2:
    // ...
}
```

Забытый `break` — частый источник ошибок: без него выполнение продолжится в следующем случае. В языке Go разработчики выбрали противоположное поведение: для перехода к следующему случаю нужно явно написать `fallthrough`.

В `switch` используйте целочисленные типы, а не числа с плавающей точкой или строки. Символы подходят, поскольку представляются целыми числами:

```
char c = 'b';
switch (c) {
case 'a':
    printf("It's 'a'!\n");
```

```
    break;
case 'b':
    printf("It's 'b'!\n");
    break;
case 'c':
    printf("It's 'c'!\n");
    break;
}
```

Перечисления `enum` тоже можно использовать в `switch`, поскольку они относятся к целочисленным типам. Подробнее об этом — в главе про `enum`.

Глава 4. Функции

В C есть функции, как и во многих других языках, с которыми вы, возможно, знакомы.

Функции могут принимать различные аргументы и возвращать значение. Но есть важная особенность: типы аргументов и возвращаемого значения объявляются заранее — так уж любит C!

Посмотрим на функцию, которая принимает аргумент типа `int` и возвращает значение типа `int`:

```
#include <stdio.h>

int plus_one(int n)  // Определение
{
    return n + 1;
}
```

`int` перед `plus_one` указывает тип возвращаемого значения.

Запись `int n` означает, что функция принимает один аргумент типа `int`, который сохраняется в параметре `n`. Параметр — особый вид локальной переменной, в которую копируется аргумент.

Хочу особо подчеркнуть: аргументы **копируются** в параметры. Многое в C становится понятнее, если помнить, что параметр — это копия аргумента, а не сам аргумент. Скоро мы рассмотрим это подробнее.

Продолжив программу функцией `main()`, мы увидим вызов функции, возвращаемое значение которой записывается в локальную переменную `j`:

```
int main(void)
{
    int i = 10, j;

    j = plus_one(i);  // Вызов

    printf("i + 1 is %d\n", j);
}
```

Пока не забыл: обратите внимание, что я определил функцию до её использования. Если бы я этого не сделал, компилятор при обработке `main()` ещё не знал бы о ней и сообщил бы об ошибке вызова неизвестной функции. Существует более правильный способ организовать этот код с помощью прототипов функций, но о них поговорим позже.

Заметьте также, что `main()` — это функция! Она возвращает `int`.

А что такое `void`? Это ключевое слово здесь означает, что функция не принимает аргументов.

Можно также указать `void` как возвращаемый тип: это означает, что функция не возвращает значение.

```
#include <stdio.h>

// Эта функция не принимает аргументов и не возвращает значение:
void hello(void)
{
    printf("Hello, world!\n");
}

int main(void)
{
    hello();  // Выводит "Hello, world!"
}
```

4.1. Передача по значению

Ранее я говорил, что при передаче аргумента функции создаётся его копия и сохраняется в соответствующем параметре.

Если аргумент — переменная, в параметр копируется значение этой переменной. В общем случае вычисляется выражение аргумента, и полученное значение копируется в параметр.

В любом случае значение параметра существует само по себе. Оно не зависит от значений или переменных, которые вы использовали в качестве аргументов при вызове функции.

Рассмотрим пример. Прежде чем запускать его, попробуйте определить, что он выведет:

```
#include <stdio.h>

void increment(int a)
{
    a++;
}

int main(void)
{
    int i = 10;

    increment(i);

    printf("i == %d\n", i);  // Что будет выведено?
}
```

На первый взгляд, `i` равно `10`, и мы передаём его функции `increment()`. Там значение увеличивается, значит, при выводе должно получиться `11`, верно?

Но нет: выводится `10`! Как же так?

Дело в том, что значения выражений, переданных функциям, копируются в соответствующие параметры. Параметр — копия, а не оригинал.

Итак, в `main()` переменная `i` равна `10`. Мы передаём её в `increment()`, где соответствующий параметр называется `a`.

Происходит копирование, как будто выполняется присваивание `a = i`. Теперь `a` равно `10`, и в `main()` переменная `i` тоже равна `10`.

Затем мы увеличиваем `a` до `11`, но `i` вообще не трогаем! Она остаётся равной `10`.

Наконец, функция завершается. Все её локальные переменные исчезают (пока, `a`!), и мы возвращаемся в `main()`, где `i` по-прежнему равно `10`. Мы выводим это значение и получаем `10`.

Именно поэтому в предыдущем примере с `plus_one()` мы возвращали изменённое локальное значение, чтобы снова увидеть его в `main()`.

Кажется несколько ограничивающим? Возможно, вы думаете, что функция может вернуть только одно значение. Но есть и другой способ получить данные обратно; программисты на C называют его передачей по ссылке, и об этом мы поговорим в другой раз.

Однако никакое красивое название не должно отвлекать от факта: **всё без исключения**, что вы передаёте функции, копируется в соответствующий параметр, и функция работает с этой локальной копией. Помните об этом даже тогда, когда мы будем говорить о так называемой передаче по ссылке.

4.2. Прототипы функций

Помните, всего несколько разделов назад я говорил, что функцию нужно определить до использования, иначе компилятор ещё не будет о ней знать и выдаст ошибку?

Это не совсем так. Можно заранее сообщить компилятору, что вы будете использовать функцию определённого типа с определённым списком параметров. Тогда определение функции может находиться где угодно, даже в другом файле, — при условии, что её прототип объявлен до вызова.

К счастью, прототип очень прост: это копия первой строки определения функции с точкой с запятой в конце. Например, следующий код вызывает функцию, определённую позже, потому что сначала был объявлен её прототип:

```
#include <stdio.h>

int foo(void);  // Это прототип!

int main(void)
{
    int i;

    // Здесь можно вызвать foo() до её определения,
    // потому что выше уже объявлен прототип!
    i = foo();

    printf("%d\n", i);  // 3490
}
```

```
int foo(void)  // Определение, как и в прототипе!
{
    return 3490;
}
```

Если функция не объявлена до использования — ни прототипом, ни определением, — возникает так называемое *неявное объявление*. Оно допускалось первым стандартом C (C89), в котором были правила для такой ситуации, но теперь это запрещено. В новом коде нет разумной причины на него полагаться.

Вы могли заметить кое-что в наших примерах: мы постоянно используем старую добрую функцию `printf()`, не определяя её и не объявляя её прототип! Как нам сходит с рук такое беззаконие? На самом деле никак. Прототип существует: он находится в заголовочном файле `stdio.h`, который мы подключили через `#include`. Так что всё законно!

4.3. Пустые списки параметров

В C23 пустой список параметров в определении или прототипе функции означает то же самое, что и `void`:

```
void foo();
void foo(void);  // То же самое в C23
```

При этом в современном коде на C всё ещё предпочитают явно указывать `void`, главным образом потому, что до C23 поведение пустого списка параметров заметно отличалось — и не в лучшую сторону.

Сейчас мы разберёмся подробнее, но коротко правило таково: если код может компилироваться компилятором, работающим по стандарту до C23, всегда используйте `void` для обозначения пустого списка параметров во всех объявлениях функций.

Нужно рассмотреть две ситуации:

- параметры опущены в определении функции;
- параметры опущены в прототипе.

Начнём с определения функции:

```
void foo()  // Здесь на самом деле стоило бы написать void
{
    printf("Hello, world!\n");
}
```

Хотя стандарт говорит, что это означает отсутствие параметров у `foo()`, компилятор вправе не предупредить вас, если вы по ошибке передадите аргумент. А если вы так сделаете, поведение будет неопределённым. Чтобы всего этого избежать, просто укажите `void`.

В случае прототипа функции наличие или отсутствие `void` существенно меняет смысл:

```
void foo();
void foo(void);  // До C23 это не одно и то же!
```

Если опустить `void` в прототипе, компилятору сообщается, что дополнительной информации о параметрах нет. Фактически это отключает проверку типов, поэтому `void foo();` в таком случае даже не настоящий «прототип», а лишь предварительное объявление.

Поэтому при пустом списке параметров в прототипе обязательно используйте `void`. И вообще, это хорошая привычка. Возможно, когда-нибудь, когда компиляторы до C23 выйдут из употребления, мы сможем от неё отказаться.

Глава 5. Указатели — трепещите от страха!

> — Как попасть в Карнеги-холл? — Репетировать!
>
> Шутка XX века, автор неизвестен

Указатели — одна из самых пугающих вещей в языке C. Именно из-за них этот язык вообще считают сложным. Но почему?

Потому что, скажем прямо, они могут вызвать удар током через клавиатуру и навсегда приварить к ней ваши руки, обречённые работать с языком из семидесятых!

Правда? Ну, не совсем. Я просто пытаюсь подготовить вас к успеху.

Возможно, в знакомом вам языке уже есть понятие ссылки: переменная ссылается на объект определённого типа. Здесь идея во многом та же, только в C нужно более явно различать саму ссылку и то, на что она указывает.

5.1. Память и переменные

Память компьютера хранит самые разные данные: `float`, `int` и всё прочее. Чтобы с памятью было удобно работать, каждый её байт обозначают целым числом. Эти числа обычно последовательно увеличиваются по мере продвижения по памяти. Представьте множество пронумерованных ящиков, каждый из которых хранит байт данных. Или большой массив, каждый элемент которого содержит байт. Номер ящика называется его *адресом*.

Байт обычно состоит из восьми двоичных разрядов, или битов. Значит, он способен хранить беззнаковое число от 0 до 255 включительно в привычной десятичной записи.

Не все типы занимают всего один байт. Например, `int` и `float` часто занимают по четыре байта, но это зависит от системы. Чтобы узнать размер типа в байтах, используйте оператор `sizeof`:

```
// %zu is the format specifier for type size_t
printf("an int uses %zu bytes of memory\n", sizeof(int));
// That prints "4" for me, but can vary by system.
```

Интересные факты о памяти. Если тип данных, например обычный `int`, занимает больше одного байта, составляющие его байты всегда расположены в памяти рядом. Иногда они идут в ожидаемом порядке, иногда нет. C не гарантирует конкретный порядок: он зависит от платформы. Тем не менее обычно можно написать переносимый код, не задумываясь об этих надоедливых порядках байтов.

Порядок байтов называют *endianness*. Основные варианты — big-endian, когда старший байт идёт первым, и little-endian, когда старший байт идёт последним. Существует и редко

встречающийся теперь смешанный порядок.

Итак, барабанная дробь и зловещая музыка: **указатель — это переменная, которая хранит адрес**. Представьте сейчас классическую музыку из «Космической одиссеи 2001 года». Ба-бам, ба-бам, ба-бам, БА-А-АМ!

Ладно, пожалуй, слегка переусердствовал. Никакой особой тайны в указателях нет: это адреса данных. Как переменная `int` может хранить значение `12`, так переменная-указатель может хранить адрес данных.

Здесь следующие слова означают одно и то же — число, обозначающее место в памяти:

- индекс в памяти, если представлять память как большой массив;
- адрес;
- местоположение.

Я буду пользоваться ими как взаимозаменяемыми. Да, слово «местоположение» я добавил просто потому, что синонимов много не бывает. Переменная-указатель хранит этот адрес точно так же, как переменная `float` может хранить `3.14159`.

Представьте множество стикеров, последовательно пронумерованных по адресам: первый имеет индекс `0`, следующий — `1` и так далее. Помимо номера, обозначающего положение, на каждом можно написать любое число: количество ваших собак, число спутников Марса…

Или индекс другого стикера! Если вы записали число собак, это обычная переменная. А если записали индекс другого стикера — это указатель. Он *указывает* на другой стикер!

Другая аналогия — адрес дома. Дом может иметь разные свойства: двор, металлическую крышу, солнечные батареи и так далее. А ещё у него есть адрес. Адрес и дом — не одно и то же: одно — настоящий дом, другое — несколько строк текста. Но адрес — указатель на дом. Он не является самим домом, однако показывает, где его найти.

Точно так же можно работать с данными в компьютере. Переменная хранит значение и находится в памяти по некоторому адресу. Другая переменная, указатель, может хранить адрес первой. Это не сама переменная с данными, но, как и адрес дома, указатель показывает, где её найти. Мы говорим, что у нас есть «указатель на» эти данные, и можем пройти по нему, чтобы получить доступ к самим данным.

Пока это может казаться бесполезным, но при вызовах функций указатели становятся незаменимыми. Потерпите, скоро мы до этого дойдём.

Итак, если у нас есть `int` и нужен указатель на него, нам нужен способ узнать адрес этого `int`. Какой оператор для этого использовать?

К вашему невероятному удивлению, дорогой читатель, мы используем оператор *получения адреса* — амперсанд `&`. Амперсанд!

Для короткого примера введём новый спецификатор формата `printf()`, позволяющий выводить указатель. Вы уже знаете, что `%d` выводит целое десятичное число. А `%p` выводит

указатель. Он будет похож на бессмысленное число и может быть представлен в шестнадцатеричной системе, но это всего лишь индекс памяти, где хранятся данные; если данные занимают несколько байтов — адрес первого байта. Практически всегда, в том числе здесь, конкретное значение не имеет значения; я показываю его лишь для демонстрации оператора получения адреса.

```
#include <stdio.h>

int main(void)
{
    int i = 10;

    printf("The value of i is %d\n", i);
    printf("And its address is %p\n", (void *)&i);
}
```

В примере есть *приведение типа*: выражение `&i` преобразуется к типу `void *`. Это нужно, чтобы компилятор не выдавал предупреждение. Мы ещё не изучали эту тему, поэтому пока просто игнорируйте `(void *)` и представьте, что его нет.

На моём компьютере программа выводит:

```
The value of i is 10
And its address is 0x7ffddf7072a4
```

Для любопытных: это шестнадцатеричное число равно 140 727 326 896 068 в десятичной системе. Это индекс памяти, в которой хранятся данные переменной `i`: адрес `i`, её местоположение, указатель на неё. В шестнадцатеричной системе используются цифры от `0` до `9` и буквы от `A` до `F`.

Погодите, у вас 140 терабайт оперативной памяти? Конечно! А у вас нет? Ладно, вру напропалую: разумеется, нет, по крайней мере в 2024 году. Современные компьютеры используют замечательную технологию *виртуальной памяти*, благодаря которой процессам кажется, будто всё адресное пространство компьютера принадлежит им одним, независимо от объёма физической памяти. Поэтому этот огромный адрес системой виртуальной памяти процессора отображается на некоторый физический адрес. В моём компьютере 16 ГБ памяти (опять же, в 2024 году, но у меня Linux, так что этого достаточно). Терабайты памяти? Я преподаватель, а не фантастически богатый интернет-предприниматель. Ничего из этого не должно нас беспокоить, кроме того факта, что я не сказочно богат.

Это указатель, потому что он сообщает, где в памяти находится `i`. Как адрес дома на клочке бумаги показывает, где найти дом, так и это число показывает, где в памяти находится значение `i`. Оно указывает на `i`.

Повторюсь: обычно точное числовое значение адреса нас не интересует. Важно лишь, что это указатель на `i`.

5.2. Типы указателей

Всё это хорошо. Теперь вы умеете получать адрес переменной и выводить его на экран. Неплохая строчка для резюме, да? Наверняка сейчас вы готовы схватить меня за шиворот и вежливо спросить, какой, чёрт возьми, от указателей толк.

Отличный вопрос! Мы ответим на него сразу после сообщения нашего спонсора.

СЛУЖБА РОБОТИЗИРОВАННОЙ УБОРКИ ЖИЛИЩ АСМЕ. СОСТОЯНИЕ ВАШЕГО ЖИЛИЩА ЗНАЧИТЕЛЬНО УЛУЧШИТСЯ, ИНАЧЕ ВЫ БУДЕТЕ УНИЧТОЖЕНЫ. КОНЕЦ СООБЩЕНИЯ.

С возвращением к руководству Биджа. В прошлый раз мы говорили о применении указателей. Мы сохраним указатель в переменной, чтобы использовать его позже. Тип указателя можно узнать по звёздочке `*` перед именем переменной и после типа:

```
int main(void)
{
    int i;   // i's type is "int"
    int *p;  // p's type is "pointer to an int", or "int-pointer"
}
```

Здесь у нас есть переменная-указатель, которая может указывать на `int`, то есть хранить адрес `int`. Мы знаем это по типу `int *`, который читается как «указатель на int».

При присваивании переменной-указателю выражение справа должно иметь подходящий тип указателя. К счастью, при получении адреса переменной результат имеет тип указателя на тип этой переменной, поэтому такое присваивание подходит:

```
int i;
int *p;  // p is a pointer, but is uninitialized and points to garbage
p = &i;  // p is assigned the address of i--p now "points to" i
```

Слева от присваивания находится переменная типа «указатель на `int`» (`int *`), а справа — выражение того же типа, поскольку `i` имеет тип `int`: получение адреса `int` даёт указатель на `int`. Адрес объекта можно сохранить в указателе на этот объект.

Понятно? Возможно, пока не совсем: вы ещё не видели практического применения переменной-указателя. Но мы идём маленькими шагами, чтобы никто не потерялся. Теперь познакомимся с оператором, противоположным получению адреса — словно оператор получения адреса из перевёрнутого мира.

5.3. Разыменование

Можно считать, что переменная-указатель ссылается на другую переменную, указывая на неё. В мире C редко говорят о «ссылках» в этом смысле, но так название оператора будет понятнее.

Имея указатель на переменную, можно работать с исходной переменной через него, *разыменовав* указатель. В нашей аналогии это примерно как прочитать адрес дома и отправиться туда.

Что значит «получить доступ к исходной переменной»? Если у вас есть переменная `i` и указатель на неё `p`, разыменованный указатель `p` можно использовать точно так же, как исходную переменную `i`!

Почти всё готово для примера. Осталось выяснить, как выглядит оператор разыменования. Его также называют оператором косвенного обращения, поскольку доступ к значению происходит косвенно, через указатель. Это снова звёздочка: `*`. Не путайте её со звёздочкой в объявлении указателя: символ одинаковый, но в разных контекстах означает разное. Он также используется в `/*комментариях*/`, умножении и прототипах функций с массивами переменной длины.

Вот полный пример:

```
#include <stdio.h>

int main(void)
{
    int i;
    int *p;  // this is NOT a dereference--this is a type "int*"

    p = &i;  // p now points to i, p holds address of i

    i = 10;  // i is now 10
    *p = 20; // the thing p points to (namely i!) is now 20!!

    printf("i is %d\n", i);  // prints "20"
    printf("i is %d\n", *p); // "20"! dereference-p is the same as i!
}
```

Помните, что `p` хранит адрес `i`: это видно из присваивания `p = &i`. Оператор косвенного обращения говорит компьютеру использовать объект, на который указывает указатель, вместо самого указателя. Так `*p` становится своего рода другим именем для `i`.

Отлично, но зачем? Зачем всё это?

5.4. Передача указателей в качестве аргументов

Сейчас вы, вероятно, думаете: об указателях вы уже узнали немало, а применить эти знания некуда. Зачем использовать `*p`, если можно просто написать `i`?

Дело в том, что настоящая сила указателей проявляется при передаче их в функции. Почему это важно? Как вы помните, аргументы функции исправно копируются в её параметры; внутри функции можно работать с локальными копиями и вернуть одно значение.

А что, если нужно получить из функции больше одного элемента данных? Ведь вернуть можно только одно значение, верно? А если я отвечу на этот вопрос другим вопросом? Или

двумя?

Что происходит, когда указатель передают функции как аргумент? Копируется ли он в соответствующий параметр? Ещё как! Помните, я долго твердил, что *каждый аргумент* копируется в параметр и функция работает с копией? Здесь происходит то же самое: функция получает копию указателя.

Но вот в чём хитрость: заранее настроив указатель так, чтобы он указывал на переменную, мы позволяем функции разыменовать свою копию указателя и вернуться к исходной переменной! Сама функция не видит переменную напрямую, но вполне может разыменовать указатель на неё.

Это похоже на адрес дома, записанный на листке бумаги, а затем переписанный на другой листок. Теперь у вас есть два указателя на один дом, и оба одинаково хорошо позволяют его найти.

При вызове функции одна копия хранится в переменной-указателе в вызывающей области видимости, а другая — в параметре функции.

Пример! Вернёмся к нашей старой функции `increment()`, но на этот раз сделаем так, чтобы она действительно увеличивала значение у вызывающей стороны.

```
#include <stdio.h>

void increment(int *p) // note that it accepts a pointer to an int
{
    *p = *p + 1; // add one to the thing p points to
}

int main(void)
{
    int i = 10;
    int *j = &i; // note the address-of; turns it into a pointer to i

    printf("i is %d\n", i);        // prints "10"
    printf("i is also %d\n", *j); // prints "10"

    increment(j); // j is an int*--to i

    printf("i is %d\n", i); // prints "11"!
}
```

Здесь стоит обратить внимание на несколько вещей, прежде всего на то, что `increment()` принимает аргумент типа `int *`. При вызове мы передаём ей `int *`, получив указатель на переменную `int i` с помощью оператора взятия адреса. Помните: указатель хранит адрес, поэтому мы создаём указатель на переменную, применяя к ней оператор получения адреса.

Функция `increment()` получает копию указателя. И исходный указатель `j` в `main()`, и его копия `p` — параметр `increment()` — указывают на один адрес, где хранится значение `i`. Как два листка с одним адресом дома. Разыменовав любой из них, можно изменить исходную переменную `i`! Функция способна изменить переменную из другой области видимости. Здорово!

Часто этот пример записывают короче, применяя оператор получения адреса прямо в списке аргументов:

```
printf("i is %d\n", i); // prints "10"
increment(&i);
printf("i is %d\n", i); // prints "11"!
```

Общее правило: если функция должна изменить переданный объект так, чтобы результат был виден вызывающему коду, нужно передать указатель на этот объект.

5.5. Нулевой указатель NULL

Любой переменной любого типа указателя можно присвоить специальное значение `NULL`. Оно означает, что указатель ни на что не указывает.

```
int *p;
p = NULL;
```

Поскольку он не указывает на значение, его разыменование приводит к неопределённому поведению и, вероятно, к аварийному завершению:

```
int *p = NULL;
*p = 12; // CRASH or SOMETHING PROBABLY BAD. BEST AVOIDED.
```

Хотя создатель назвал нулевой указатель «ошибкой на миллиард долларов», `NULL` хорошо подходит в качестве сигнального значения и общего признака того, что указатель ещё не был настроен на объект.

Конечно, как и другие переменные, неинициализированный указатель содержит неопределённое значение, пока вы явно не присвоите ему адрес или `NULL`.

5.6. Замечание об объявлении указателей

Синтаксис объявления указателей может выглядеть немного странно. Посмотрим на пример:

```
int a;
int b;
```

Можно сократить его до одной строки, верно?

```
int a, b; // Same thing
```

И `a`, и `b` имеют тип `int`. Всё просто. А как быть здесь?

```
int a;
int *p;
```

Можно ли объединить объявления в одну строку? Да. Но где поставить `*`?

Правило такое: `*` ставится перед каждой переменной, имеющей тип указателя. В этом примере `*` относится не к `int`, а к переменной `p`. Поэтому можно написать:

```
int a, *p; // Same thing
```

Важно помнить, что следующая строка **не** объявляет два указателя:

```
int *p, q; // p is a pointer to an int; q is just an int.
```

Особенно коварно это выглядит, если программист запишет следующую допустимую строку, функционально идентичную предыдущей:

```
int* p, q; // p is a pointer to an int; q is just an int.
```

Посмотрите на это объявление и определите, какие переменные являются указателями, а какие нет:

```
int *a, b, c, *d, e, *f, g, h, *i;
```

Ответ: указатели — `a`, `d`, `f` и `i`, поскольку именно перед ними стоит `*`.

5.7. sizeof и указатели

Ещё немного синтаксиса, который иногда встречается и может сбивать с толку.

Вспомните: `sizeof` работает с типом выражения.

```
int *p;

// Prints size of an 'int'
printf("%zu\n", sizeof(int));

// p is type 'int *', so prints size of 'int*'
printf("%zu\n", sizeof p);

// *p is type 'int', so prints size of 'int'
printf("%zu\n", sizeof *p);
```

Последний вариант `sizeof` можно встретить в чужом коде. Просто помните: `sizeof` здесь определяет размер по типу выражения, а не по значениям самих переменных в этом выражении.

Глава 6. Массивы

К счастью, в C есть массивы. Я знаю, что C считается низкоуровневым языком, но по крайней мере понятие массива в него встроено. А поскольку очень многие языки вдохновлялись синтаксисом C, вы, вероятно, уже знакомы с использованием `[` и `]` для объявления массивов и обращения к ним.

Однако поддержка массивов в C довольно своеобразна: как мы узнаем позже, массивы тесно связаны с указателями. Не пугайтесь! Пока будем пользоваться ими просто как массивами.

6.1. Простой пример

Посмотрим на пример:

```
#include <stdio.h>

int main(void)
{
    int i;
    float f[4];  // Объявляем массив из четырёх float

    f[0] = 3.14159;  // Индексация начинается с нуля
    f[1] = 1.41421;
    f[2] = 1.61803;
    f[3] = 2.71828;

    // Выводим все элементы:
    for (i = 0; i < 4; i++) {
        printf("%f\n", f[i]);
    }
}
```

При объявлении массива нужно указать его размер. Здесь мы пока рассматриваем массивы с фиксированным размером; массивы переменной длины — тема отдельного разговора.

В примере мы создали массив из четырёх значений `float`: об этом говорит число в квадратных скобках объявления. Далее мы обращаемся к элементам массива, записываем и читаем их значения, также используя квадратные скобки.

Надеюсь, это похоже на то, что вы уже знаете по другим языкам!

6.2. Определение длины массива

Вообще говоря, отдельно записанную длину C не хранит: её приходится отслеживать самостоятельно. Однако при некоторых обстоятельствах узнать количество элементов можно. Есть приём, который работает там, где выражение обозначает сам массив, но обычно не работает так, как хотелось бы, если передать массив функции.

Идея проста: берём размер всего массива с помощью `sizeof` и делим его на размер одного элемента. Например, если `int` занимает 4 байта, а массив — 32 байта, в нём помещается 32 / 4 = 8 элементов `int`.

```
int x[12];                              // 12 элементов int
printf("%zu\n", sizeof x);               // Всего 48 байт
printf("%zu\n", sizeof(int));            // 4 байта на int
printf("%zu\n", sizeof x / sizeof(int)); // 48 / 4 = 12 элементов!
```

Для массива `char` значение `sizeof` всего массива равно количеству элементов, поскольку по определению `sizeof(char)` равно 1. Для других типов нужно делить на размер элемента. Численные размеры в примере зависят от реализации.

При передаче массива функции этот приём не работает, даже если указать размер в объявлении параметра:

```
void foo(int x[12])
{
    printf("%zu\n", sizeof x);              // 8?! Куда делись 48?
    printf("%zu\n", sizeof(int));            // 4 байта на int
    printf("%zu\n", sizeof x / sizeof(int)); // 8 / 4 = 2? НЕВЕРНО.
}
```

Причина в том, что при «передаче массива» функции фактически передаётся указатель на первый элемент. Именно размер указателя и измеряет здесь `sizeof`. Размеры 8 и 4 в примере относятся к конкретной системе. Подробнее об этом — в разделе о передаче одномерных массивов функциям ниже.

При помощи `sizeof` можно также узнать размер массива фиксированной длины, не объявляя сам массив, подобно тому как размер `int` узнают выражением `sizeof(int)`. Например, размер массива из 48 элементов `double`:

```
sizeof(double [48]);
```

6.3. Инициализаторы массивов

Массив можно сразу инициализировать константами:

```
#include <stdio.h>

int main(void)
{
    int i;
    int a[5] = {22, 37, 3490, 18, 95};  // Начальные значения

    for (i = 0; i < 5; i++) {
        printf("%d\n", a[i]);
    }
}
```

Значений в инициализаторе не должно быть больше, чем элементов в массиве, иначе компилятор пожалуется:

```
foo.c:6:39: warning: excess elements in array initializer
    int a[5] = {22, 37, 3490, 18, 95, 999};
                                      ^~~
```

А вот меньше значений указать можно! Оставшиеся элементы автоматически инициализируются нулями. Это общее правило: если инициализатор присутствует, элементы, которым явно не задано значение, получают ноль.

```
int a[5] = {22, 37, 3490};
// То же самое, что:
int a[5] = {22, 37, 3490, 0, 0};
```

Часто для обнуления всего массива используют сокращённую запись:

```
int a[100] = {0};
```

Она означает: «Присвой первому элементу ноль, а остальные тоже автоматически заполни нулями».

В инициализаторе можно задавать конкретные элементы по индексу. После указанного элемента C продолжает присваивать следующие значения по порядку, пока инициализатор не закончится; прочие элементы заполняются нулями. Для этого заключите индекс в квадратные скобки, поставьте `=` и укажите значение:

```
int a[10] = {0, 11, 22, [5]=55, 66, 77};
```

Поскольку значение `55` задано для индекса 5, массив получится таким:

```
0 11 22 0 0 55 66 77 0 0
```

В индексах можно использовать и простые константные выражения:

```
#define COUNT 5
int a[COUNT] = {[COUNT-3]=3, 2, 1};
```

Результат:

```
0 0 3 2 1
```

Наконец, размер массива можно не указывать: C вычислит его по инициализатору.

```
int a[3] = {22, 37, 3490};
// То же самое, что:
int a[] = {22, 37, 3490};  // Размер опущен!
```

6.4. Выход за границы!

С не мешает обратиться к элементу за пределами массива и может даже не предупредить об этом.

Возьмём предыдущий пример и продолжим вывод за концом массива. В нём всего пять элементов, но попробуем вывести десять:

```
#include <stdio.h>

int main(void)
{
    int i;
    int a[5] = {22, 37, 3490, 18, 95};

    for (i = 0; i < 10; i++) {  // ПЛОХО: выводим лишние элементы!
        printf("%d\n", a[i]);
    }
}
```

На моём компьютере программа вывела:

```
22
37
3490
18
95
32765
1847052032
1780534144
-56487472
21890
```

Ужас! Что это? Оказывается, обращение за пределы массива приводит к тому, что разработчики на С называют *неопределённым поведением*. Мы ещё поговорим об этом подробнее, а пока смысл таков: вы сделали нечто недопустимое, и во время выполнения программы может случиться что угодно.

На практике это могут быть нули, случайные числа или аварийное завершение. Но стандарт С действительно допускает, что при неопределённом поведении результат может быть каким угодно.

Коротко: никогда не делайте ничего, что вызывает неопределённое поведение. Причин для него существует много, не только выход за границы массива.

6.5. Многомерные массивы

Массивам можно добавлять измерения:

```
int a[10];
int b[2][7];
int c[4][5][6];
```

В памяти элементы располагаются по строкам. Для двумерного массива это означает, что первый индекс обозначает строку, а второй — столбец.

Многомерные массивы можно инициализировать с помощью вложенных списков:

```
#include <stdio.h>

int main(void)
{
    int row, col;
    int a[2][5] = {  // Инициализируем двумерный массив
        {0, 1, 2, 3, 4},
        {5, 6, 7, 8, 9}
    };

    for (row = 0; row < 2; row++) {
        for (col = 0; col < 5; col++) {
            printf("(%d,%d) = %d\n", row, col, a[row][col]);
        }
    }
}
```

Результат:

```
(0,0) = 0
(0,1) = 1
(0,2) = 2
(0,3) = 3
(0,4) = 4
(1,0) = 5
(1,1) = 6
(1,2) = 7
(1,3) = 8
(1,4) = 9
```

Можно также явно указать индексы при инициализации:

```
// Создаём единичную матрицу 3 x 3
int a[3][3] = {[0][0]=1, [1][1]=1, [2][2]=1};
```

Получится двумерный массив:

```
1 0 0
0 1 0
0 0 1
```

6.6. Массивы и указатели

Кажется, выше я вскользь упомянул, что массивы связаны с указателями? Давайте немного разберёмся, чтобы не запутаться. Позже мы подробнее рассмотрим настоящие отношения между ними, а пока сосредоточимся на передаче массивов функциям.

6.6.1. Получение указателя на массив

Открою секрет. Обычно, когда программист на C говорит «указатель на массив», он имеет в виду указатель на первый элемент массива. Строго говоря, это неточно: указатель на массив и указатель на его первый элемент имеют разные типы. Но с этим мы разберёмся позже.

Получим указатель на первый элемент:

```
#include <stdio.h>

int main(void)
{
    int a[5] = {11, 22, 33, 44, 55};
    int *p;

    p = &a[0];  // p указывает на первый элемент массива

    printf("%d\n", *p);  // Выводит 11
}
```

Это настолько распространённая операция, что в C для неё есть сокращение:

```
p = &a[0];  // p указывает на первый элемент массива

// То же самое:
p = a;      // Выглядит гораздо приятнее!
```

В этом выражении использование имени массива даёт указатель на его первый элемент. Мы будем часто применять это в следующих примерах.

Но подождите: разве `p` не имеет тип `int*`? И разве `*p` не даёт `11`, как и `a[0]`? Именно! Вы начинаете видеть связь массивов и указателей в C. Подробнее мы поговорим о ней в главе «Указатели II», в разделе об эквивалентности массивов и указателей.

6.6.2. Передача одномерных массивов функциям

Рассмотрим одномерный массив. Напишем несколько функций, которые принимают его и выполняют разные действия. Приготовьтесь к неожиданным объявлениям!

```
#include <stdio.h>

// Передача как указателя на первый элемент
void times2(int *a, int len)
{
    for (int i = 0; i < len; i++)
        printf("%d\n", a[i] * 2);
}

// То же самое с записью массива
void times3(int a[], int len)
{
    for (int i = 0; i < len; i++)
        printf("%d\n", a[i] * 3);
```

```
}

// То же самое с указанием размера
void times4(int a[5], int len)
{
    for (int i = 0; i < len; i++)
        printf("%d\n", a[i] * 4);
}

int main(void)
{
    int x[5] = {11, 22, 33, 44, 55};

    times2(x, 5);
    times3(x, 5);
    times4(x, 5);
}
```

Все три способа записи параметра эквивалентны:

```
void times2(int *a, int len)
void times3(int a[], int len)
void times4(int a[5], int len)
```

Опытные программисты на C гораздо чаще используют первый вариант. В последнем варианте компилятор не проверяет, что переданный массив действительно имеет указанный размер. Если в таком объявлении задано константное выражение размера, оно должно быть больше нуля.

Встречаются расширения компиляторов с массивами нулевой длины в конце структур. Такой приём использовали для структур переменной длины; в стандартном C для этой цели появились гибкие члены массива, о которых мы поговорим позже.

При передаче многомерных массивов размеры в объявлении функции действительно имеют значение — к этому мы ещё вернёмся.

6.6.3. Изменение массивов в функциях

При передаче массива функции обычно передаётся указатель на его первый элемент. Получив указатель на данные, функция может изменять эти данные! Поэтому изменения массива будут видны вызывающей стороне.

Вот пример: функция получает указатель на элементы массива и меняет их, а вызывающий код видит изменения.

```
#include <stdio.h>

void double_array(int *a, int len)
{
    // Умножаем каждый элемент на 2.
    // Это меняет значения x в main(), поскольку указатель a
    // указывает на элементы того же массива в памяти!
    for (int i = 0; i < len; i++)
        a[i] *= 2;
}
```

```
int main(void)
{
    int x[5] = {1, 2, 3, 4, 5};

    double_array(x, 5);

    for (int i = 0; i < 5; i++)
        printf("%d\n", x[i]);  // 2, 4, 6, 8, 10!
}
```

Хотя параметр `a` имеет тип `int*`, посмотрите, как мы обращаемся к данным: через запись массива `a[i]`! Да, это совершенно допустимо.

Позже, когда мы поговорим об эквивалентности массивов и указателей, всё станет понятнее. Пока достаточно знать: функция может изменять массив, и эти изменения будут видны вызывающей стороне.

6.6.4. Передача многомерных массивов функциям

С многомерными массивами ситуация немного меняется. С должен знать все размеры, кроме первого, чтобы определить, где в памяти находится нужное значение.

В этом примере мы явно указываем оба размера:

```
#include <stdio.h>

void print_2D_array(int a[2][3])
{
    for (int row = 0; row < 2; row++) {
        for (int col = 0; col < 3; col++)
            printf("%d ", a[row][col]);
        printf("\n");
    }
}

int main(void)
{
    int x[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    print_2D_array(x);
}
```

Однако здесь эти два объявления эквивалентны:

```
void print_2D_array(int a[2][3])
void print_2D_array(int a[][3])
```

Компилятору нужен второй размер, чтобы вычислить, на сколько нужно переместиться в памяти при увеличении первого индекса. Вообще говоря, ему нужны все размеры, кроме первого. Эквивалентна и запись `void print_2D_array(int (*a)[3])`, но пока мы не будем подробно её разбирать.

И помните: при компиляции проверка границ минимальна, если вам вообще повезёт, а во время выполнения C её не выполняет. Ремней безопасности нет! Не допускайте обращений за границы массива.

Глава 7. Строки

Наконец-то! Строки! Что может быть проще?

Оказывается, в C строки на самом деле не совсем строки. Именно так! Это указатели! А как же иначе!

Как и массивы, строки в C существуют лишь в очень ограниченном смысле. Но давайте разберёмся: на самом деле всё не так страшно.

7.1. Строковые литералы

Прежде чем начать, поговорим о строковых литералах в C. Это последовательности символов в двойных кавычках (`"`). Одинарные кавычки окружают символы, а это совсем другая история.

Примеры:

```
"Hello, world!\n"
"This is a test."
"When asked if this string had quotes in it, she replied, \"It does.\""
```

В конце первого литерала находится перевод строки — это встречается довольно часто.

В последнем литерале есть кавычки внутри строки. Обратите внимание: перед каждой стоит обратная косая черта (`\`). Мы говорим, что кавычки *экранированы*. Это означает, что здесь в строке должна находиться сама кавычка. Так компилятор C отличает выводимую двойную кавычку от кавычки, завершающей строку.

7.2. Строковые переменные

Теперь, когда мы знаем, как создать строковый литерал, присвоим его переменной, чтобы что-нибудь с ним сделать:

```
char *s = "Hello, world!";
```

Обратите внимание на тип: указатель на `char`. Строковая переменная `s` на самом деле указывает на первый символ строки, то есть на `H`.

Вывести строку можно с помощью спецификатора формата `%s` (от *string* — «строка»):

```
char *s = "Hello, world!";
printf("%s\n", s); // "Hello, world!"
```

7.3. Строковые переменные как массивы

Есть ещё один вариант, почти эквивалентный использованию `char *` выше:

```
char s[14] = "Hello, world!";

// or, if we were properly lazy and have the compiler
// figure the length for us:
char s[] = "Hello, world!";
```

Это означает, что к символам строки можно обращаться с помощью записи для массивов. Именно так и поступим, чтобы вывести все символы строки на одной строке:

```
#include <stdio.h>

int main(void)
{
    char s[] = "Hello, world!";

    for (int i = 0; i < 13; i++)
        printf("%c", s[i]);
    printf("\n");
}
```

Обратите внимание: для вывода одного символа мы используем спецификатор формата `%c`.

А теперь взгляните. Программа по-прежнему будет работать, если изменить определение `s` на тип `char *`:

```
#include <stdio.h>

int main(void)
{
    char *s = "Hello, world!"; // char* here

    for (int i = 0; i < 13; i++)
        printf("%c", s[i]); // But still use arrays here...?
    printf("\n");
}
```

Для вывода мы всё равно можем использовать индексирование! Это удивляет только потому, что мы ещё не поговорили об эквивалентности массивов и указателей в выражениях. Это ещё одна подсказка о тесной связи массивов и указателей.

7.4. Инициализация строк

Мы уже видели примеры инициализации строковых переменных строковыми литералами:

```
char *s = "Hello, world!";
char t[] = "Hello, again!";
```

Однако между этими двумя способами есть тонкое различие. Память строкового литерала, как и целочисленного литерала, компилятор организует автоматически. Целое число — объект фиксированного размера, и управлять им довольно просто. Строки же занимают переменное число байтов; компилятор справляется с ними, помещая их в область памяти и предоставляя указатель на неё.

В следующей форме указатель указывает на место размещения строки. Обычно из соображений производительности и безопасности это место находится далеко от остальной памяти программы — в области, доступной только для чтения:

```
char *s = "Hello, world!";
```

Поэтому, если попытаться изменить эту строку:

```
char *s = "Hello, world!";
s[0] = 'z'; // BAD NEWS: tried to mutate a string literal!
```

поведение будет неопределённым. В зависимости от системы программа, вероятно, аварийно завершится.

А вот объявление массива работает иначе. Компилятор не прячет эти байты в другой части города: они находятся совсем рядом. Это изменяемая копия строки, которую можно править по своему усмотрению:

```
char t[] = "Hello, again!"; // t is an array copy of the string
t[0] = 'z';                 // No problem
printf("%s\n", t);          // "zello, again!"
```

Запомните: если у вас есть указатель на строковый литерал, не пытайтесь изменять этот литерал! А при инициализации массива строкой в двойных кавычках получается отдельный изменяемый массив, а не указатель на литерал.

7.5. Как узнать длину строки

Никак: C не отслеживает её за вас. А когда я говорю «никак», на самом деле я имею в виду «можно». Хотя C и правда не хранит длину строки автоматически, в `<string.h>` есть функция `strlen()`, которая вычисляет длину любой строки в байтах.

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char *s = "Hello, world!";

    printf("The string is %zu bytes long.\n", strlen(s));
}
```

Функция `strlen()` возвращает значение типа `size_t`. Это целочисленный тип, поэтому его можно использовать в целочисленных вычислениях. Для вывода `size_t` применяется `%zu`.

Если вы используете базовый или восьмибитный набор символов, то привыкли, что один символ занимает один байт. Но это верно не для всех кодировок.

Программа выведет:

```
The string is 13 bytes long.
```

Отлично! Значит, длину строки узнать можно! Но если C нигде её не хранит, откуда функция знает, какова длина строки?

7.6. Завершение строки

C работает со строками несколько иначе, чем многие языки программирования, и уж точно иначе, чем почти все современные языки.

Создавая язык, можно выбрать один из двух основных способов хранения строки в памяти:

1. Хранить байты строки вместе с числом, обозначающим её длину.
2. Хранить байты строки и отмечать конец специальным байтом — *терминатором*.

Если нужны строки длиннее 255 символов, в первом варианте для хранения длины требуется как минимум два байта. Во втором варианте для завершения строки достаточно одного байта. Небольшая экономия!

Конечно, сегодня беспокоиться об экономии одного байта кажется смешным. Или трёх: многие языки без проблем позволяют создавать строки длиной в четыре гигабайта. Но раньше это было важнее.

Поэтому в C выбран второй подход. Строка в C определяется двумя основными признаками:

- указателем на первый символ строки;
- байтом со значением ноль, или символом NUL, расположенным далее в памяти и обозначающим конец строки.

Не путайте символ NUL с нулевым указателем `NULL`: для символа мы используем обозначение NUL, а для указателя — `NULL`.

В коде C символ NUL можно записать как `\0`, хотя делать это часто не приходится. Когда вы включаете в код строку в двойных кавычках, завершающий NUL добавляется автоматически и неявно:

```
char *s = "Hello!"; // Actually "Hello!\0" behind the scenes
```

Теперь напишем собственную функцию `strlen()`, которая подсчитывает символы `char` в строке, пока не встретит NUL. Будем идти по строке и одновременно считать символы. Позже мы узнаем более изящный способ с использованием арифметики указателей.

```
int my_strlen(char *s)
{
    int count = 0;

    while (s[count] != '\0') // Single quotes for single char
        count++;

    return count;
}
```

Примерно так стандартная функция `strlen()` и выполняет свою работу.

7.7. Копирование строки

Скопировать строку оператором присваивания (=) нельзя. Он лишь копирует указатель на первый символ, и в итоге получаются два указателя на одну и ту же строку:

```
#include <stdio.h>

int main(void)
{
    char s[] = "Hello, world!";
    char *t;

    // This makes a copy of the pointer, not a copy of the string!
    t = s;

    // We modify t
    t[0] = 'z';

    // But printing s shows the modification!
    // Because t and s point to the same string!

    printf("%s\n", s); // "zello, world!"
}
```

Чтобы создать копию строки, нужно скопировать её по одному байту: взять отдельные байты из одного места памяти и продублировать их в другом. Функция `strcpy()` облегчает эту задачу.

Есть и функция `strncpy()`, ограничивающая число копируемых байтов. Одни считают, что следует всегда использовать `strncpy()` для защиты от переполнения буфера. Другие считают, что её использовать не следует никогда, поскольку она не обязательно завершает строку нулевым байтом, а это ещё одна опасная ловушка. Для надёжности можно написать собственную версию `strncpy()`, которая всегда завершает строку.

Перед копированием убедитесь, что для копии достаточно места: массив назначения должен быть достаточно длинным для всех символов копируемой строки **и завершающего нулевого байта**.

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "Hello, world!";
    char t[100]; // Each char is one byte, so plenty of room

    // This makes a copy of the string!
    strcpy(t, s);

    // We modify t
    t[0] = 'z';

    // And s remains unaffected because it's a different string
    printf("%s\n", s); // "Hello, world!"

    // But t has been changed
    printf("%s\n", t); // "zello, world!"
}
```

Обратите внимание: первый аргумент `strcpy()` — указатель на место назначения, второй — указатель на источник. Я запоминаю порядок так: `t` и `s` стоят в том же порядке, в каком они стояли бы при присваивании через `=`, если бы оно работало для строк: назначение слева, источник справа.

Глава 8. Структуры

В C есть так называемая структура (`struct`) — определяемый пользователем тип, который объединяет несколько значений, возможно, разных типов.

Это удобный способ собрать несколько переменных в одну. Он полезен при передаче данных функциям — достаточно передать одну структуру вместо множества отдельных переменных, — а также помогает организовать данные и сделать код понятнее.

Если вы пришли из другого языка, вам могут быть знакомы классы и объекты. В C нет встроенных классов и объектов в смысле объектно-ориентированного программирования. Структуру можно представить как класс, содержащий только данные и не имеющий методов. При этом в терминологии C отдельные области памяти, например значения `int`, тоже называют объектами, но в другом смысле.

8.1. Объявление структуры

Структуру можно объявить следующим образом:

```
struct car {
    char *name;
    float price;
    int speed;
};
```

Часто это делают на уровне файла, вне всех функций, чтобы структура была доступна во всём файле.

Так создаётся новый тип. Его полное имя — `struct car`. Просто `car` не подойдёт.

Переменных этого типа пока нет, но мы можем их объявить:

```
struct car saturn;  // Переменная saturn типа struct car
```

Теперь у нас есть неинициализированная переменная `saturn` типа `struct car`. Её следует инициализировать! Но как присвоить значения отдельным полям?

Как и во многих других языках, позаимствовавших этот синтаксис у C, для доступа к полям используется оператор точки (`.`):

```
saturn.name = "Saturn SL/2";
saturn.price = 15999.99;
saturn.speed = 175;

printf("Name: %s\n", saturn.name);
printf("Price (USD): %f\n", saturn.price);
printf("Top Speed (km): %d\n", saturn.speed);
```

В первых строках мы задаём значения полей `struct car`, а затем выводим их.

8.2. Инициализаторы структур

Предыдущий пример получился немного громоздким. Наверняка есть способ удобнее инициализировать переменную структуры!

Можно использовать инициализатор: при определении переменной перечислить значения полей в том порядке, в каком они указаны в объявлении структуры. После определения переменной такой синтаксис уже не работает — он применяется именно при определении.

```
struct car {
    char *name;
    float price;
    int speed;
};

// С инициализатором! Порядок полей совпадает с объявлением:
struct car saturn = {"Saturn SL/2", 16000.99, 175};

printf("Name: %s\n", saturn.name);
printf("Price: %f\n", saturn.price);
printf("Top Speed: %d km\n", saturn.speed);
```

Необходимость соблюдать порядок полей немного пугает. Если кто-то изменит порядок в `struct car`, может сломаться весь остальной код!

Можно указывать поля явно:

```
struct car saturn = {.speed=175, .name="Saturn SL/2"};
```

Теперь порядок полей в объявлении структуры не имеет значения. Такой код, безусловно, безопаснее.

Как и при инициализации массивов, поля, для которых значение не указано, инициализируются нулём. Здесь это поле `.price`.

8.3. Передача структур функциям

Передать структуру функции можно двумя способами:

1. Передать саму структуру.
2. Передать указатель на неё.

Вспомните: когда вы передаёте что-либо функции, создаётся копия значения, с которой функция и работает. Неважно, что именно копируется — указатель, `int`, структура или что-то ещё.

Указатель на структуру обычно передают в двух случаях:

1. Функция должна изменить переданную структуру так, чтобы вызывающий код увидел изменения.

2. Структура достаточно велика, и скопировать её дороже, чем скопировать указатель. На 64-битной системе указатель, вероятно, занимает 8 байт.

По этим причинам указатель на структуру передают гораздо чаще, хотя передать саму структуру тоже вполне допустимо.

Попробуем передать указатель и написать функцию, которая задаёт значение поля `.price` структуры `struct car`:

```
#include <stdio.h>

struct car {
    char *name;
    float price;
    int speed;
};

int main(void)
{
    struct car saturn = {.speed=175, .name="Saturn SL/2"};

    // Передаём указатель на структуру и новую, более
    // реалистичную цену:
    set_price(&saturn, 799.99);

    printf("Price: %f\n", saturn.price);
}
```

Посмотрев на типы аргументов, можно самостоятельно вывести объявление функции `set_price()`.

`saturn` имеет тип `struct car`, поэтому `&saturn` — адрес структуры, то есть указатель на `struct car`, иначе говоря `struct car*`. Второе значение, `799.99`, передаётся параметру типа `float`.

Значит, объявление функции выглядит так:

```
void set_price(struct car *c, float new_price);
```

Остаётся написать тело. Можно было бы попробовать так:

```
void set_price(struct car *c, float new_price)
{
    c.price = new_price;  // ОШИБКА!
}
```

Это не сработает: оператор точки применяется к структурам, а не к указателям на них.

Хорошо, тогда разыменуем переменную `c`, чтобы получить саму структуру. Результат разыменования `struct car*` — структура `struct car`, на которую указывает этот указатель; к ней можно применить точку:

```
void set_price(struct car *c, float new_price)
{
    (*c).price = new_price;  // Работает, но выглядит громоздко
}
```

И это работает! Но печатать скобки и звёздочку не очень удобно. В C есть синтаксическое сокращение — оператор стрелки.

8.4. Оператор стрелки

Оператор стрелки помогает обращаться к полям структуры через указатель:

```
void set_price(struct car *c, float new_price)
{
    // (*c).price = new_price;  // Работает, но неудобно
    // Строка выше полностью эквивалентна строке ниже:
    c->price = new_price;       // Вот так!
}
```

Когда же использовать точку, а когда стрелку?

- Если перед вами структура, используйте точку (`.`).
- Если перед вами указатель на структуру, используйте стрелку (`->`).

8.5. Копирование и возврат структур

Здесь всё просто: присвойте одну структуру другой!

```
struct car a, b;
b = a;  // Копируем структуру
```

Возврат структуры из функции — именно структуры, а не указателя на неё — аналогично создаёт копию для переменной, принимающей результат.

Это не *глубокая копия*: все поля копируются как есть, в том числе указатели. Глубокое копирование следовало бы по указателям в структуре и копировало также данные, на которые они указывают. При поверхностном копировании копируются только указатели, а не эти данные. Встроенного средства глубокого копирования в C нет.

8.6. Сравнение структур

Единственный надёжный общий способ — сравнивать поля по одному.

Может показаться, что подойдёт `memcmp()`, но она не учитывает возможные байты заполнения между полями структуры.

Если предварительно обнулить структуру с помощью `memset()`, побайтовое сравнение может сработать, но отдельные элементы всё равно могут сравниваться не так, как вы

ожидаете. Поэтому сравнение соответствующих полей остаётся надёжным подходом.

Глава 9. Файловый ввод и вывод

Мы уже видели примеры ввода и вывода в консоли с помощью `printf()`. Теперь пойдём немного дальше.

9.1. Тип данных FILE *

Для ввода и вывода в C используется объект, доступный через указатель типа `FILE *`. Он содержит сведения, необходимые подсистеме ввода-вывода: какой файл открыт, какова текущая позиция в нём и так далее.

Стандарт называет такие объекты потоками: данные могут поступать из файла или другого источника. Здесь мы будем иногда взаимозаменяемо употреблять слова «файл» и «поток», но файл — лишь частный случай потока.

Чуть позже мы откроем поток по имени файла, но сначала познакомимся с тремя потоками, уже открытыми для программы:

Имя	Назначение
`stdin`	Стандартный ввод, обычно клавиатура
`stdout`	Стандартный вывод, обычно экран
`stderr`	Стандартный поток ошибок, обычно тоже экран

Мы уже неявно использовали эти потоки. Например, следующие вызовы эквивалентны:

```
printf("Hello, world!\n");
fprintf(stdout, "Hello, world!\n"); // Вывод в указанный поток
```

Хотя `stdout` и `stderr` обычно выводятся на экран, разделение не лишнее. Операционная система позволяет перенаправить их в разные файлы: обычный вывод окажется в одном, сообщения об ошибках — в другом. Например, в оболочке POSIX на Unix-подобной системе:

```
./foo > output.txt 2> errors.txt # Команда для Unix-подобных систем
```

Поэтому серьёзные сообщения об ошибках стоит отправлять в `stderr`. Как это делать, увидим позже.

9.2. Чтение текстовых файлов

Потоки обычно подразделяют на текстовые и двоичные. Для текстовых потоков система может преобразовывать данные, прежде всего обозначения конца строки. В разных системах использовались CR, LF и CRLF; сегодня Unix-подобные системы обычно используют LF, а Windows — CRLF. Текстовый файл логически представляет собой

последовательность строк. Для переносимости входные данные должны заканчиваться переводом строки.

Общее правило: если файл можно редактировать обычным текстовым редактором, это текстовый файл; в противном случае он двоичный. К двоичным файлам мы вернёмся позже.

Создадим файл `hello.txt`:

```
Hello, world!
```

Откроем его, прочитаем один символ и закроем:

```
#include <stdio.h>

int main(void)
{
    FILE *fp; // Открытый файл
    fp = fopen("hello.txt", "r"); // Открыть для чтения
    int c = fgetc(fp); // Прочитать один символ
    printf("%c\n", c); // Вывести символ
    fclose(fp); // Закрыть файл
}
```

`fopen()` возвращает `FILE *`, с которым затем можно работать. Ради краткости проверка ошибки здесь опущена, но в настоящей программе она необходима: при ошибке, например если файл не найден, `fopen()` возвращает `NULL`.

Режим `"r"` означает открытие текстового потока для чтения. Другие строки режима позволяют, например, записывать данные или добавлять их в конец файла. Для чтения символа мы вызвали `fgetc()`. Почему `c` имеет тип `int`, а не `char`? Скоро узнаем!

После работы поток следует закрыть. При завершении программы потоки закрываются автоматически, но явное закрытие — хороший порядок. Объект `FILE *` отслеживает позицию: каждый следующий вызов `fgetc()` читает следующий символ. Однако читать файл по одному символу не всегда удобно.

9.3. Конец файла: EOF

Макрос `EOF` обозначает специальное значение, которое `fgetc()` возвращает при попытке чтения за концом файла или при ошибке чтения.

Именно поэтому `fgetc()` и похожие функции возвращают `int`, а не `char`: `EOF` — не обычный символ, и функция должна отличать его от любого прочитанного байта. Тип `int` позволяет представить и значение байта, и признак конца файла.

Прочитаем весь файл в цикле:

```
#include <stdio.h>

int main(void)
```

```
{
    FILE *fp;
    int c;
    fp = fopen("hello.txt", "r");
    while ((c = fgetc(fp)) != EOF)
        printf("%c", c);
    fclose(fp);
}
```

Если условие кажется странным, начните с внутренних скобок: сначала результат `fgetc()` присваивается `c`, затем сравнивается с `EOF`. Мы просто объединили действия в одной строке. Это характерный для C приём, поэтому стоит к нему привыкнуть. Программа выведет:

```
Hello, world!
```

Однако текстовые файлы часто удобнее обрабатывать целыми строками.

9.3.1. Построчное чтение

Для этого есть `fgets()`. Ей передают указатель на буфер `char`, максимальный размер буфера и поток `FILE *`. При конце файла или ошибке она возвращает `NULL`. При успешном чтении она добавляет завершающий символ NUL. Если строка не помещается в буфер, чтение остановится посреди неё, а следующий вызов продолжит чтение оставшейся части.

Пусть файл `quote.txt` содержит:

```
A wise man can learn more from
a foolish question than a fool
can learn from a wise answer.
--Bruce Lee
```

Программа читает строки и выводит перед каждой её номер:

```
#include <stdio.h>

int main(void)
{
    FILE *fp;
    char s[1024]; // Достаточно для любой строки в нашем примере
    int linecount = 0;
    fp = fopen("quote.txt", "r");
    while (fgets(s, sizeof s, fp) != NULL)
        printf("%d: %s", ++linecount, s);
    fclose(fp);
}
```

Результат:

```
1: A wise man can learn more from
2: a foolish question than a fool
```

```
3: can learn from a wise answer.
4: --Bruce Lee
```

9.4. Форматированный ввод

Подобно тому как `printf()` и `fprintf()` выполняют форматированный вывод, `fscanf()` выполняет форматированный ввод.

Функции семейства `scanf()` опасны при обработке недоверенных данных. Если не ограничить ширину поля `%s`, можно переполнить буфер. Некорректное числовое преобразование также может привести к неопределённому поведению. Для недоверенного ввода безопаснее читать строку с ограничением ширины и затем преобразовывать числа функциями вроде `strtol()` или `strtod()`.

Пусть файл `whales.txt` содержит записи о китах: название, длину в метрах и массу в тоннах.

```
blue 29.9 173
right 20.7 135
gray 14.9 41
humpback 16.0 30
```

Можно прочитать строки функцией `fgets()`, а затем разобрать через `sscanf()` — такой подход устойчивее к повреждённому файлу. Но для примера воспользуемся непосредственно `fscanf()`:

```
#include <stdio.h>

int main(void)
{
    FILE *fp;
    char name[1024];
    float length;
    int mass;
    fp = fopen("whales.txt", "r");
    while (fscanf(fp, "%s %f %d", name, &length, &mass) != EOF)
        printf("%s whale, %d tonnes, %.1f meters\n", name, mass, length);
    fclose(fp);
}
```

`fscanf()` при чтении пропускает начальные пробельные символы и возвращает `EOF`, если конец файла или ошибка возникли до первого успешного преобразования. Результат примера:

```
blue whale, 173 tonnes, 29.9 meters
right whale, 135 tonnes, 20.7 meters
gray whale, 41 tonnes, 14.9 meters
humpback whale, 30 tonnes, 16.0 meters
```

Обратите внимание: пример выше предполагает корректный входной файл. В реальном коде ограничивайте ширину `%s`, проверяйте результат открытия и число успешно

прочитанных полей: проверка только на `EOF` не обнаруживает все ошибки формата.

9.5. Запись текстовых файлов

Аналогично функциям чтения `fgetc()`, `fgets()` и `fscanf()` существуют функции записи `fputc()`, `fputs()` и `fprintf()`.

Откройте файл через `fopen()` в режиме `"w"`. Внимание: существующий файл при этом немедленно обрезается до нулевой длины и перезаписывается.

```
#include <stdio.h>

int main(void)
{
    FILE *fp;
    int x = 32;
    fp = fopen("output.txt", "w");
    fputc('B', fp);
    fputc('\n', fp); // Перевод строки
    fprintf(fp, "x = %d\n", x);
    fputs("Hello, world!\n", fp);
    fclose(fp);
}
```

Получим `output.txt`:

```
B
x = 32
Hello, world!
```

Поскольку `stdout` тоже является потоком, присваивание `fp = stdout;` вместо открытия файла направило бы вывод в консоль. Попробуйте!

9.6. Двоичный ввод и вывод

До сих пор речь шла о тексте. В двоичном потоке подсистема ввода-вывода не выполняет преобразования, которые возможны для текстового потока: программа получает последовательность байтов.

Для открытия добавьте `b` к режиму: `"rb"` для чтения и `"wb"` для записи. Поскольку двоичные данные могут содержать байты NUL, которые в C обозначают конец строки, функции семейства `fprintf()` редко используют для двоичных файлов. Вместо них обычно применяются `fread()` и `fwrite()`.

Напишем две программы. Первая запишет последовательность байтов за один вызов; вторая прочитает их по одному и выведет. Обычно эффективнее было бы прочитать все байты сразу, но здесь важно показать принцип.

```
#include <stdio.h>

int main(void)
```

```
{
    FILE *fp;
    unsigned char bytes[6] = {5, 37, 0, 88, 255, 12};
    fp = fopen("output.bin", "wb"); // Двоичная запись

    // Аргументы: адрес данных, размер элемента,
    // число элементов, поток.
    fwrite(bytes, sizeof(char), 6, fp);
    fclose(fp);
}
```

Два средних аргумента `fwrite()` задают размер одного элемента и их количество. Это удобно для массива записей фиксированной длины. В примере размер каждого элемента равен размеру `char`, а элементов шесть.

Получившийся файл не выглядит осмысленно в текстовом редакторе: это произвольные двоичные данные. Программа шестнадцатеричного дампа покажет байты:

```
05 25 00 58 ff 0c
```

На многих Unix-системах для этого есть `hexdump`:

```
$ hexdump -C output.bin
00000000  05 25 00 58 ff 0c  |.%.X..|
```

Слева показано смещение строки от начала файла, в середине — значения байтов (до 16 в строке), справа между символами `|` — попытка показать соответствующие печатные символы. Непечатные символы представлены точкой. Для случайных двоичных данных эта часть мало что значит, но текст ASCII был бы виден. Шестнадцатеричные значения соответствуют записанным десятичным.

Теперь прочитаем байты обратно. `fread()` возвращает число полностью прочитанных элементов, а при конце файла или ошибке может вернуть ноль. Здесь размер элемента равен одному байту, поэтому результат совпадает с числом прочитанных байтов:

```
#include <stdio.h>

int main(void)
{
    FILE *fp;
    unsigned char c;
    fp = fopen("output.bin", "rb"); // Двоичное чтение
    while (fread(&c, sizeof(char), 1, fp) > 0)
        printf("%d\n", c);
    fclose(fp);
}
```

Результат — исходные числа:

```
5
37
0
88
```

```
255
12
```

Ура!

9.6.1. Особенности структур и чисел

Как мы видели в разделе о структурах, компилятор вправе добавлять в `struct` байты выравнивания. Разные компиляторы, архитектуры и настройки могут располагать их по-разному. Поэтому непосредственная запись всей структуры через `fwrite()` не является переносимым форматом.

Это не единственная проблема: разные архитектуры могут по-разному представлять числа в памяти. Запишем двухбайтовое число, используя шестнадцатеричную запись, чтобы были видны отдельные байты: старший `0x12` и младший `0x34`.

```
unsigned short v = 0x1234; // Для этого примера — два байта
fwrite(&v, sizeof v, 1, fp);
```

Казалось бы, в поток попадёт сначала `0x12`, затем `0x34`. Но на машине автора дамп показывает:

```
34 12
```

Всё наоборот! Дело в порядке байтов архитектуры (*endianness*): одни системы записывают сначала старший байт, другие — младший. Поэтому прямая запись многобайтового числа из памяти не переносима между системами. Именно поэтому в предыдущем примере использовались отдельные байты.

Похожая проблема возникает с числами с плавающей точкой: большинство систем использует одинаковый формат, но не все; гарантий нет.

Решение — сериализация: данные записываются в явно определённом, известном формате, который можно одинаково обрабатывать на разных платформах. Для этого существуют готовые библиотеки, например Protocol Buffers. Они берут на себя детали и позволяют обмениваться данными между программами на C и других языках, поддерживающих тот же формат.

Сериализуйте двоичные данные перед записью в поток: так файлы останутся переносимыми при передаче между архитектурами.

Глава 10. typedef: создание новых типов

Ну, не столько создание новых типов, сколько получение новых имён для существующих. На первый взгляд это кажется бессмысленным, но на самом деле помогает сделать код чище.

10.1. typedef в теории

Суть проста: вы берёте существующий тип и создаёте для него другое имя с помощью `typedef`.

Вот так:

```
typedef int antelope; // Make "antelope" an alias for "int"

antelope x = 10;      // Type "antelope" is the same as type "int"
```

Так можно поступить с любым существующим типом. Можно даже создать несколько имён, перечислив их через запятую:

```
typedef int antelope, bagel, mushroom; // These are all "int"
```

Очень полезно, правда? Можно написать `mushroom` вместо `int`! Наверняка вы в восторге от этой возможности!

Ладно, профессор Сарказм, сейчас мы рассмотрим более распространённые способы её применения.

10.1.1. Область видимости

На `typedef` распространяются обычные правила области видимости.

Поэтому объявления `typedef` часто встречаются на уровне файла, то есть в «глобальной» области видимости: тогда все функции могут свободно пользоваться новыми именами типов.

10.2. typedef на практике

Переименование `int` во что-то другое не слишком увлекательно. Посмотрим, где `typedef` действительно часто встречается.

10.2.1. typedef и структуры

Иногда для структуры с помощью `typedef` создают новое имя, чтобы не писать слово `struct` снова и снова.

```
struct animal {
    char *name;
    int leg_count, speed;
};

// original name        new name
//       |                 |
//       v                 v
// |-----------|         |----|
typedef struct animal animal;

struct animal y; // This works
animal z;        // This also works because "animal" is an alias
```

Лично мне этот приём не нравится. Я предпочитаю ясность, которую придаёт коду слово `struct` в типе: программисты сразу понимают, с чем имеют дело. Но приём очень распространён, поэтому я его здесь привожу.

Теперь запишем тот же пример способом, который вы часто встретите: поместим определение `struct animal` внутрь `typedef`. Всё можно объединить так:

```
// original name
//       |
//       v
// |-----------|
typedef struct animal {
    char *name;
    int leg_count, speed;
} animal; // <-- new name

struct animal y; // This works
animal z;        // This also works because "animal" is an alias
```

Это в точности то же самое, что предыдущий пример, только короче.

Но это ещё не всё! Есть и другой распространённый сокращённый вариант с использованием так называемых *анонимных структур*. Мы поговорим о них подробнее позже. Во многих случаях имя структуре давать не обязательно, и применение `typedef` — один из них.

Запишем тот же пример с анонимной структурой:

```
// Anonymous struct! It has no name!
//      |
//      v
// |----|
typedef struct {
    char *name;
    int leg_count, speed;
} animal; // <-- new name
```

```
//struct animal y; // ERROR: this no longer works--no such struct!
animal z;          // This works because "animal" is an alias
```

Другой пример, который можно встретить:

```
typedef struct {
    int x, y;
} point;

point p = {.x=20, .y=40};

printf("%d, %d\n", p.x, p.y); // 20, 40
```

10.2.2. typedef и другие типы

Использовать `typedef` с простым типом вроде `int` тоже бывает полезно: это позволяет отвлечься от конкретного типа, чтобы позже его было проще изменить.

Например, если у вас `float` встречается в ста зиллионах мест, менять все эти вхождения на `double` будет мучительно, если однажды возникнет такая необходимость.

Но если заранее немного подготовиться:

```
typedef float app_float;

// and

app_float f1, f2, f3;
```

то при переходе на другой тип, например `long double`, достаточно будет изменить `typedef`:

```
// voila!
//       |---------|
typedef long double app_float;

// and no need to change this line:
app_float f1, f2, f3; // Now these are all long doubles
```

10.2.3. typedef и указатели

Можно создать имя для типа указателя:

```
typedef int *intptr;

int a = 10;
intptr x = &a; // "intptr" is type "int*"
```

Мне этот приём очень не нравится. Он скрывает тот факт, что `x` имеет тип указателя: ведь в объявлении не видно `*`.

По моему скромному мнению, лучше явно показывать, что объявляется указатель, чтобы другие разработчики сразу это видели и не принимали `x` за переменную другого типа.

Но, по последним подсчётам, примерно 832 007 человек придерживаются другого мнения.

10.2.4. typedef и регистр букв

Я видел самые разные способы написания имён, созданных с помощью `typedef`:

```
typedef struct {
    int x, y;
} my_point; // lower snake case

typedef struct {
    int x, y;
} MyPoint;  // CamelCase

typedef struct {
    int x, y;
} Mypoint;  // Leading uppercase

typedef struct {
    int x, y;
} MY_POINT; // UPPER SNAKE CASE
```

Спецификация C11 не предписывает какой-то один вариант и приводит примеры как полностью в верхнем, так и полностью в нижнем регистре.

В книге Кернигана и Ритчи преимущественно используются имена с заглавной первой буквой, но встречаются и примеры в верхнем регистре, и `snake_case` с суффиксом `_t`.

Если у вас принят стандарт оформления кода, придерживайтесь его. Если нет — выберите какой-нибудь и придерживайтесь его.

10.3. Массивы и typedef

Синтаксис немного странный, и, по моему опыту, это редко встречается, но с помощью `typedef` можно создать имя для типа массива с определённым количеством элементов:

```
// Make type five_ints an array of 5 ints
typedef int five_ints[5];

five_ints x = {11, 22, 33, 44, 55};
```

Мне это не нравится, потому что скрывает тот факт, что переменная является массивом, но такая возможность существует.

Глава 11. Указатели II: арифметика

Пора глубже разобраться в указателях! Если предыдущие сведения об указателях ещё не усвоены, вернитесь к первой главе руководства на эту тему.

11.1. Арифметика указателей

С указателями можно выполнять арифметические операции, прежде всего сложение и вычитание. Что они означают?

Если к указателю на объект некоторого типа прибавить единицу, он переместится к следующему объекту этого типа, непосредственно расположенному за первым в памяти.

Перемещая указатель и обращаясь к разным участкам памяти, всегда проверяйте, что перед разыменованием он указывает на допустимое место. Если попытаться прочитать содержимое произвольного места, поведение будет неопределённым; часто программа аварийно завершится.

Этот материал тесно связан с эквивалентностью массивов и указателей, о которой речь пойдёт ниже.

11.1.1. Прибавление к указателям

Возьмём массив чисел и указатель на его первый элемент:

```
int a[5] = {11, 22, 33, 44, 55};
int *p = &a[0]; // Можно также написать int *p = a;
printf("%d\n", *p); // Выводит 11
printf("%d\n", *(p + 1)); // Выводит 22!
```

Что произошло? C знает, что `p` указывает на `int`, а значит, знает размер `int` и то, сколько байтов нужно пропустить до следующего элемента. Оператор `sizeof` сообщает размер объекта в байтах.

Прибавление нуля не меняет указатель, поэтому следующие записи эквивалентны:

```
printf("%d\n", *p);       // 11
printf("%d\n", *(p + 0)); // 11
```

Так можно пройти по массиву без записи с индексом:

```
int a[5] = {11, 22, 33, 44, 55};
int *p = &a[0];
for (int i = 0; i < 5; i++) {
    printf("%d\n", *(p + i)); // То же, что p[i]!
}
```

Результат тот же, что при индексировании массива. Мы приближаемся к эквивалентности массивов и указателей!

Как это работает? Память можно представить как большой массив байтов. Положение в памяти называют индексом, ячейкой или адресом; указатель содержит адрес.

Например, пусть число 3490 хранится по адресу 23 237 489 202. Указатель на него содержит именно этот адрес. Следующее число, 4096, может находиться по адресу 23 237 489 210: в этом примере каждый `int` занимает восемь байтов.

Прибавление единицы к указателю на `int` переместит его на `sizeof(int)` байтов, к следующему `int`. Указатель на `float` при этом переместился бы на `sizeof(float)` байтов. Прибавив два, можно обратиться ещё к следующему элементу, и так далее.

11.1.2. Изменение указателей

В предыдущем разделе мы прибавляли целое число к указателю в выражении. Теперь изменим сам указатель. К указателю можно прибавлять целые числа и вычитать их из него.

Добавим в конец массива число 999 как признак окончания данных:

```
int a[] = {11, 22, 33, 44, 55, 999}; // 999 — признак конца
int *p = &a[0];                      // p указывает на 11

while (*p != 999) { // Пока значение не равно 999
    printf("%d\n", *p);
    p++;            // Перейти к следующему int
}
```

Сначала `p` указывает на 11, после увеличения — на 22, затем на 33 и так далее. Когда он укажет на 999, цикл завершится.

11.1.3. Вычитание указателей

Из указателя можно вычесть число и перейти к предыдущему адресу. Кроме того, можно вычесть один указатель из другого и узнать расстояние между ними, например число элементов `int` между двумя указателями `int *`. Это допустимо только в пределах одного массива; иначе поведение неопределённо. Указатель также может указывать непосредственно за последним элементом массива и участвовать в такой арифметике, но разыменовывать его там нельзя.

Строка в C — массив `char`. Напишем ещё один вариант `strlen()`: найдём указатель на завершающий символ NUL и вычтем из него указатель на начало строки.

```
#include <stdio.h>

int my_strlen(char *s)
{
    char *p = s;              // Начинаем с начала строки
    while (*p != '\0')        // Ищем NUL
        p++;
```

```
    return p - s;             // Разность указателей
}

int main(void)
{
    printf("%d\n", my_strlen("Hello, world!")); // 13
}
```

Помните: вычитать друг из друга можно только указатели на элементы одного массива!

11.2. Эквивалентность массивов и указателей

Вот основная формула:

```
a[b] == *(a + b)
```

Эти выражения эквивалентны. Здесь есть небольшое упрощение: `a` и `b` могут быть сложными выражениями, поэтому нужны дополнительные скобки, чтобы обеспечить правильный порядок вычислений. Стандарт C11, §6.5.2.1¶2, формулирует точнее: `E1[E2]` идентично `(*((E1)+(E2)))`.

Для доступа к элементам массива или указателя на его элемент можно использовать любую из двух форм:

```
#include <stdio.h>

int main(void)
{
    int a[] = {11, 22, 33, 44, 55};
    int *p = a; // Указывает на первый элемент

    for (int i = 0; i < 5; i++)
        printf("%d\n", a[i]);
    for (int i = 0; i < 5; i++)
        printf("%d\n", p[i]);
    for (int i = 0; i < 5; i++)
        printf("%d\n", *(a + i));
    for (int i = 0; i < 5; i++)
        printf("%d\n", *(p + i));
    for (int i = 0; i < 5; i++)
        printf("%d\n", *(p++)); // Перемещаем указатель
    // printf("%d\n", *(a++)); // Ошибка: массив перемещать нельзя
}
```

Существенное различие: указатель можно изменить так, чтобы он указывал на другой адрес, а переменную массива — нет. Присваивать значения можно отдельным элементам массива. Чтобы скопировать один массив в другой, используйте, например, `memcpy()` или цикл.

11.2.1. Массивы и указатели при вызове функций

Именно здесь эта эквивалентность особенно часто встречается. Если функция принимает указатель:

```
int my_strlen(char *s)
```

ей можно передать как массив, так и указатель:

```
char s[] = "Antelopes";
char *t = "Wombats";
printf("%d\n", my_strlen(s)); // Работает
printf("%d\n", my_strlen(t)); // Тоже работает
```

По той же причине следующие объявления параметра функции эквивалентны:

```
int my_strlen(char *s)
int my_strlen(char s[])
```

11.3. Указатели void

Ключевое слово `void` в объявлениях функций обозначает отсутствие параметров или возвращаемого значения. Но `void *` — другое дело: это указатель на объект, тип которого не указан. Он позволяет писать более универсальный код в языке со строгой типизацией.

Есть два основных применения:

1. Функция обрабатывает объект побайтово. Например, `memcpy()` копирует байты между адресами, которые могут указывать на объекты разных типов. Указатель `char *` позволяет пройти по байтам объекта любого типа.
2. Функция вызывает переданную ей функцию обратного вызова и передаёт данные, не зная их типа. Вызываемая функция знает нужный тип и преобразует `void *` в соответствующий указатель. Так работают стандартные `qsort()` и `bsearch()`.

Рассмотрим `memcpy()`:

```
void *memcpy(void *s1, void *s2, size_t n);
```

Она копирует `n` байтов из области, начинающейся по адресу `s2`, в область по адресу `s1`. Оба аргумента — `void *`. Например, скопируем строку (хотя для строк подходит и `strcpy()`):

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "Goats!";
    char t[100];
    memcpy(t, s, 7); // Семь байтов, включая завершающий NUL
    printf("%s\n", t); // Goats!
}
```

Или скопируем числа:

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    int a[] = {11, 22, 33};
    int b[3];
    memcpy(b, a, 3 * sizeof(int)); // Три объекта int
    printf("%d\n", b[1]); // 22
}
```

Нужно указывать число байтов, а `int` занимает более одного байта и его размер зависит от системы. Оператор `sizeof(int)` сообщает нужный размер; три элемента занимают `3 * sizeof(int)` байтов. В примере со строкой точнее было бы написать `7 * sizeof(char)`, но `char` по определению занимает один байт.

Скопировать можно даже `float` или структуру, хотя здесь проще использовать присваивание:

```
struct antelope my_antelope;
struct antelope my_clone_antelope;
// ...
memcpy(&my_clone_antelope, &my_antelope, sizeof my_antelope);
```

`memcpy()` универсальна: имея адреса источника и назначения и число байтов, она копирует данные любого типа. Без `void *` пришлось бы писать отдельные функции для `int`, `float`, `double`, `char`, `unsigned char` и других типов. С `void *` достаточно одной функции.

Однако есть ограничения:

1. Нельзя выполнять арифметику над `void *`.
2. Нельзя разыменовывать `void *`.
3. Нельзя применять к нему оператор `->`, поскольку тот тоже разыменовывает указатель.
4. Нельзя использовать индексирование: оно также включает арифметику и разыменование.

Этим операциям нужно знать размер типа объекта, а у `void *` тип не указан. Как же им пользоваться? Перед работой преобразуйте его в указатель нужного типа. В C достаточно присваивания; можно также использовать явное приведение типа.

```
char a = 'X';
void *p = &a;
char *q = p;
printf("%c\n", *p); // Ошибка: void * разыменовать нельзя
printf("%c\n", *q); // X
```

Попробуем написать собственную `memcpy()`. Мы знаем количество байтов и можем копировать их как `char`:

```
void *my_memcpy(void *dest, void *src, int byte_count)
{
    char *s = src, *d = dest; // Преобразуем указатели
    while (byte_count--) {
        *d++ = *s++;
    }
    return dest;
}
```

Сначала мы преобразуем указатели в `char *`. Затем цикл уменьшает `byte_count`, пока значение не станет нулём. При постфиксном декременте сначала вычисляется значение выражения для `while`, затем переменная уменьшается. Присваивание копирует байт, а постфиксные инкременты перемещают оба указателя к следующим байтам. Многие функции работы с памятью и строками возвращают адрес назначения: он может пригодиться вызывающему коду.

Так можно пройти по байтам любого объекта C: `float`, структуры и т. д. Рассмотрим практический пример со стандартной `qsort()`, способной сортировать любые данные благодаря `void *`. Пока можно не обращать внимания на `const` — мы ещё его не изучали.

```
#include <stdio.h>
#include <stdlib.h>

struct animal {
    char *name;
    int leg_count;
};

// qsort вызывает эту функцию для сравнения двух элементов.
int compar(const void *elem1, const void *elem2)
{
    const struct animal *animal1 = elem1;
    const struct animal *animal2 = elem2;

    // Отрицательное, нулевое или положительное значение задаёт порядок.
    if (animal1->leg_count > animal2->leg_count)
        return 1;
    if (animal1->leg_count < animal2->leg_count)
        return -1;
    return 0;
}

int main(void)
{
    struct animal a[4] = {
        {.name="Dog", .leg_count=4},
        {.name="Monkey", .leg_count=2},
        {.name="Antelope", .leg_count=4},
        {.name="Snake", .leg_count=0}
    };

    // Массив, количество элементов, размер элемента и функция сравнения.
    qsort(a, 4, sizeof(struct animal), compar);

    for (int i = 0; i < 4; i++)
        printf("%d: %s\n", a[i].leg_count, a[i].name);
}
```

Если предоставить `qsort()` функцию сравнения двух элементов массива, она сможет отсортировать его, не зная конкретного типа элементов. Она переставляет блоки байтов в соответствии с результатами переданной функции сравнения.

Глава 12. Ручное выделение памяти

В этой важной области C, вероятно, отличается от языков, которые вы уже знаете: речь о ручном управлении памятью.

Другие языки используют подсчёт ссылок, сборку мусора и другие способы определять, когда выделить память для данных и когда освободить её, если на данные больше нет ссылок. Это удобно: можно просто перестать ссылаться на объект и довериться тому, что связанную с ним память когда-нибудь освободят.

Но в C дело обстоит не совсем так.

Конечно, некоторые переменные C создаются и уничтожаются автоматически. Это автоматические переменные — обычные локальные переменные с областью видимости блока. Здесь всё в порядке.

А если нужно, чтобы данные существовали дольше определённого блока? Тут и пригодится ручное управление памятью.

Вы можете явно попросить C выделить определённое количество байтов и использовать их как хотите. Они останутся выделенными, пока вы явно не освободите память или не завершится программа. Некоторые системы позволяют выделять память, переживающую программу, но это зависит от системы и выходит за рамки руководства.

Важно освобождать память, которая больше не нужна! Иначе происходит *утечка памяти*: процесс продолжает занимать эту память до завершения.

Если вы выделили память вручную, то, закончив работу с ней, должны вручную её освободить.

Как это сделать? Мы изучим несколько новых функций и используем оператор `sizeof`, чтобы узнать, сколько байтов выделять.

Разработчики C обычно говорят, что автоматические локальные переменные размещаются «в стеке», а выделенная вручную память — «в куче». Стандарт C не описывает ни то ни другое, но разработчики поймут, о чём речь.

Все функции этой главы объявлены в `<stdlib.h>`.

12.1. Выделение и освобождение: malloc() и free()

Функция `malloc()` принимает количество байтов и возвращает нетипизированный указатель `void*` на выделенный блок памяти. Его можно присвоить указателю нужного типа; обычно этот тип связан с количеством выделяемых байтов.

Сколько же байтов выделять? Нам поможет `sizeof`. Чтобы выделить место для одного `int`, передадим в `malloc()` значение `sizeof(int)`.

Закончив использовать память, вызовите `free()`: это сообщит, что память больше не нужна и может использоваться для других целей. В качестве аргумента передайте тот же указатель, который вернула `malloc()`, или его копию. Использование области памяти после `free()` — неопределённое поведение.

Попробуем выделить место для `int`, сохранить там значение и вывести его:

```
// Allocate space for a single int (sizeof(int) bytes-worth):
int *p = malloc(sizeof(int));

*p = 12;             // Store something there
printf("%d\n", *p);   // Print it: 12

free(p);             // All done with that memory

//*p = 3490;          // ERROR: undefined behavior! Use after free()!
```

Конечно, в этом искусственном примере особой пользы от выделения памяти нет: обычный автоматический `int` тоже подошёл бы. Но позднее мы увидим преимущества такого подхода, особенно при работе с более сложными структурами данных.

Есть и другой распространённый приём: `sizeof` позволяет узнать размер типа результата выражения, поэтому можно использовать имя переменной. Следующая строка эквивалентна выделению выше:

```
int *p = malloc(sizeof *p);  // *p is an int, so same as sizeof(int)
```

12.2. Проверка ошибок

Функции выделения памяти возвращают указатель на новый блок либо `NULL`, если по какой-либо причине память выделить не удалось.

Некоторые ОС, например Linux, могут быть настроены так, что `malloc()` не возвращает `NULL` даже при нехватке памяти. Тем не менее всегда стоит писать код с проверкой ошибки.

```
int *x;

x = malloc(sizeof(int) * 10);

if (x == NULL) {
    printf("Error allocating 10 ints\n");
    // do something here to handle it
}
```

Распространённый приём — совместить присваивание и проверку в одной строке:

```
int *x;

if ((x = malloc(sizeof(int) * 10)) == NULL) {
    printf("Error allocating 10 ints\n");
```

```
    // do something here to handle it
}
```

12.3. Выделение памяти для массива

Мы видели, как выделить место для одного значения. А как насчёт нескольких значений в массиве?

В C массив — это несколько однотипных элементов, следующих друг за другом в непрерывной области памяти. Мы уже знаем, как выделить такую область. Если нам нужно 3490 байтов, можно просто запросить их:

```
char *p = malloc(3490);  // Voila
```

Действительно, получится место для массива из 3490 элементов `char`, поскольку каждый `char` занимает один байт: `sizeof(char)` равно `1`. Чтобы использовать такой массив как строку C, в нём также нужен завершающий нулевой символ.

Обратите внимание: новая память не инициализирована и содержит неопределённые данные. При желании очистите её с помощью `memset()` или используйте `calloc()`, о которой речь ниже.

Чтобы выделить место для элементов другого типа, умножим размер одного элемента на их количество. Затем к ним можно обращаться через указатель или запись массива:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    // Allocate space for 10 ints
    int *p = malloc(sizeof(int) * 10);

    // Assign them values 0-45:
    for (int i = 0; i < 10; i++)
        p[i] = i * 5;

    // Print all values 0, 5, 10, 15, ..., 40, 45
    for (int i = 0; i < 10; i++)
        printf("%d\n", p[i]);

    // Free the space
    free(p);
}
```

Главное здесь — вызов `malloc()`. Один `int` занимает `sizeof(int)` байтов, а нам нужно десять, поэтому выделяем:

```
sizeof(int) * 10
```

Приём работает для любого типа: получите его размер через `sizeof` и умножьте на количество элементов массива.

12.4. Альтернатива: calloc()

Эта функция похожа на `malloc()`, но есть два важных отличия:

- Ей передаются количество элементов и размер одного элемента, а не один общий размер. Она словно создана для массивов.
- Она заполняет выделенные байты нулями.

Память, полученную через `calloc()`, тоже освобождают с помощью `free()`.

Сравним `calloc()` и `malloc()`:

```
// Allocate space for 10 ints with calloc(), initialized to 0:
int *p = calloc(10, sizeof(int));

// Allocate space for 10 ints with malloc(), initialized to 0:
int *q = malloc(10 * sizeof(int));
memset(q, 0, 10 * sizeof(int));  // set to 0
```

Результат одинаков, но `malloc()` по умолчанию память не обнуляет. Для `memset()` подключают `<string.h>`. Обнуление байтов не обязательно представляет нулевое значение любого возможного типа C.

12.5. Изменение размера с realloc()

Допустим, вы выделили место для десяти элементов `int`, а потом решили, что нужно двадцать. Что делать?

Можно выделить новую область и скопировать в неё данные с помощью `memcpy()`. Но иногда перемещать ничего не требуется. Функция `realloc()` умеет выбрать подходящее действие.

Она принимает указатель на ранее выделенную память и новый размер области в байтах, затем увеличивает или уменьшает её и возвращает указатель. Иногда это будет прежний указатель, если данные не пришлось перемещать, а иногда — новый.

При вызове `realloc()` обязательно указывайте количество *байтов*, а не просто элементов массива:

```
num_floats *= 2;

np = realloc(p, num_floats);                  // WRONG: need bytes, not number of elements!
np = realloc(p, num_floats * sizeof(float));  // Better!
```

Выделим массив из 20 элементов `float`, а затем увеличим его до 40. Результат `realloc()` сначала сохраним в другой указатель, чтобы проверить его на `NULL`. Если всё успешно,

присвоим результат исходному указателю. Если сразу перезаписать исходный указатель, то при ошибке можно потерять доступ к прежней памяти.

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    // Allocate space for 20 floats
    float *p = malloc(sizeof *p * 20);  // sizeof *p same as sizeof(float)

    // Assign them fractional values 0.0-1.0:
    for (int i = 0; i < 20; i++)
        p[i] = i / 20.0;

    // But wait! Let's actually make this an array of 40 elements
    float *new_p = realloc(p, sizeof *p * 40);

    // Check to see if we successfully reallocated
    if (new_p == NULL) {
        printf("Error reallocing\n");
        return 1;
    }

    // If we did, we can just reassign p
    p = new_p;

    // And assign the new elements values in the range 1.0-2.0
    for (int i = 20; i < 40; i++)
        p[i] = 1.0 + (i - 20) / 20.0;

    // Print all values 0.0-2.0 in the 40 elements:
    for (int i = 0; i < 40; i++)
        printf("%f\n", p[i]);

    // Free the space
    free(p);
}
```

Заметьте, что результат `realloc()` мы в итоге записали в ту же переменную `p`, которая передавалась функции. Это распространённый приём.

Если запись `sizeof *p` кажется странной, вспомните: `sizeof` определяет размер типа выражения. Тип `*p` — `float`, поэтому здесь выражение эквивалентно `sizeof(float)`.

Почему нет вызова `free(new_p)`, хотя именно этот указатель вернула `realloc()`? Потому что мы скопировали `new_p` в `p`: оба указывают на один и тот же блок. Его нужно освободить только один раз, передав `free()` любой из этих указателей.

12.5.1. Чтение строк произвольной длины

В следующем примере я хочу показать две вещи:

1. Как с помощью `realloc()` увеличивать буфер по мере чтения данных.
2. Как после чтения уменьшить его до необходимого размера.

Цикл многократно вызывает `fgetc()` и добавляет символы в буфер, пока не встретится перевод строки или конец файла. Затем буфер уменьшается и возвращается вызывающему коду.

```
#include <stdio.h>
#include <stdlib.h>

// Read a line of arbitrary size from a file
//
// Returns a pointer to the line.
// Returns NULL on EOF or error.
//
// It's up to the caller to free() this pointer when done with it.
//
// Note that this strips the newline from the result. If you need
// it in there, probably best to switch this to a do-while.

char *readline(FILE *fp)
{
    int offset = 0;  // Index next char goes in the buffer
    int bufsize = 4; // Preferably power of 2 initial size
    char *buf;       // The buffer
    int c;           // The character we've read in

    buf = malloc(bufsize);  // Allocate initial buffer

    if (buf == NULL)  // Error check
        return NULL;

    // Main loop--read until newline or EOF
    while (c = fgetc(fp), c != '\n' && c != EOF) {

        // Check if we're out of room in the buffer accounting
        // for the extra byte for the NUL terminator
        if (offset == bufsize - 1) {  // -1 for the NUL terminator
            bufsize *= 2;            // 2x the space

            char *new_buf = realloc(buf, bufsize);

            if (new_buf == NULL) {
                free(buf);  // On error, free and bail
                return NULL;
            }

            buf = new_buf;  // Successful realloc
        }

        buf[offset++] = c;  // Add the byte onto the buffer
    }

    // We hit newline or EOF...

    // If at EOF and we read no bytes, free the buffer and
    // return NULL to indicate we're at EOF:
    if (c == EOF && offset == 0) {
        free(buf);
        return NULL;
    }

    // Shrink to fit
    if (offset < bufsize - 1) {  // If we're short of the end
        char *new_buf = realloc(buf, offset + 1);  // +1 for NUL terminator
```

```
        // If successful, point buf to new_buf;
        // otherwise we'll just leave buf where it is
        if (new_buf != NULL)
            buf = new_buf;
    }

    // Add the NUL terminator
    buf[offset] = '\0';

    return buf;
}

int main(void)
{
    FILE *fp = fopen("foo.txt", "r");

    char *line;

    while ((line = readline(fp)) != NULL) {
        printf("%s\n", line);
        free(line);
    }

    fclose(fp);
}
```

При таком увеличении памяти часто — хотя это и не правило — на каждом шаге удваивают размер, чтобы сократить число вызовов `realloc()`.

Обратите внимание: `readline()` возвращает указатель на буфер, выделенный через `malloc()`. Поэтому вызывающий код должен явно освободить его с помощью `free()`, когда закончит работу.

12.5.2. realloc() с NULL

Небольшой любопытный факт: следующие строки эквивалентны:

```
char *p = malloc(3490);
char *p = realloc(NULL, 3490);
```

Это бывает удобно в цикле выделения памяти, когда не хочется отдельно обрабатывать первый вызов `malloc()`:

```
int *p = NULL;
int length = 0;

while (!done) {
    // Allocate 10 more ints:
    length += 10;
    p = realloc(p, sizeof *p * length);

    // Do amazing things
    // ...
}
```

Здесь начальный `malloc()` не нужен, потому что сначала `p` равен `NULL`.

12.6. Выделение выровненной памяти

Вероятно, вам это не понадобится. Не будем глубоко углубляться в тему, но существует понятие *выравнивания памяти*: адрес памяти, то есть значение указателя, должен быть кратен некоторому числу.

Например, система может требовать, чтобы 16-битные значения начинались по адресам, кратным двум, а 64-битные — по адресам, кратным двум, четырём или восьми. Это зависит от процессора. Одним системам выравнивание нужно для быстрого доступа, другим — чтобы вообще иметь возможность обратиться к памяти.

При использовании `malloc()`, `calloc()` или `realloc()` C предоставляет область, выровненную для обычных типов, включая структуры. Но иногда данные можно выровнять по меньшей границе или по какой-либо причине необходимо большее выравнивание. Возможно, это чаще встречается во встроенных системах.

В таких случаях выравнивание можно указать с помощью `aligned_alloc()`. Оно задаётся положительной целой степенью двойки: 2, 4, 8, 16 и так далее. Этот аргумент передаётся перед количеством байтов.

Автор отмечает также ограничение: число выделяемых байтов должно быть кратно выравниванию, хотя соответствующее правило обсуждалось в отчёте об ошибке стандарта C № 460.

Выделим память с выравниванием по границе 64 байта:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    // Allocate 256 bytes aligned on a 64-byte boundary
    char *p = aligned_alloc(64, 256);  // 256 == 64 * 4

    // Copy a string in there and print it
    strcpy(p, "Hello, world!");
    printf("%s\n", p);

    // Free the space
    free(p);
}
```

Замечание о `realloc()` и `aligned_alloc()`: `realloc()` не гарантирует сохранения специального выравнивания. Если оно необходимо после изменения размера, придётся выделить новый блок и скопировать данные через `memcpy()`.

Вот нестандартная функция `aligned_realloc()`, если она вам понадобится:

```
void *aligned_realloc(void *ptr, size_t old_size, size_t alignment, size_t size)
{
    char *new_ptr = aligned_alloc(alignment, size);

    if (new_ptr == NULL)
        return NULL;

    size_t copy_size = old_size < size? old_size: size;  // get min

    if (ptr != NULL)
        memcpy(new_ptr, ptr, copy_size);

    free(ptr);

    return new_ptr;
}
```

Обратите внимание: она всегда копирует данные и тратит на это время, тогда как настоящая `realloc()` по возможности избегает копирования. Поэтому такой способ не слишком эффективен. Старайтесь избегать необходимости менять размер данных со специальным выравниванием.

Глава 13. Область видимости

Область видимости определяет, какие переменные доступны в том или ином контексте.

13.1. Область видимости блока

Именно такую область видимости имеет большинство переменных, объявляемых разработчиками. Сюда входит и то, что в других языках могут называть «областью видимости функции», то есть переменные, объявленные внутри функций.

Основное правило: если переменная объявлена внутри блока, ограниченного фигурными скобками, область её видимости — этот блок.

Если один блок вложен в другой, переменные, объявленные во внутреннем блоке, локальны для него и не видны во внешней области.

Когда область видимости переменной заканчивается, обращаться к ней больше нельзя, а её значение можно считать отправившимся в великое небесное хранилище отброшенных битов.

Пример с вложенными областями видимости:

```
#include <stdio.h>

int main(void)
{
    int a = 12; // Local to outer block, but visible in inner block

    if (a == 12) {
        int b = 99; // Local to inner block, not visible in outer block

        printf("%d %d\n", a, b); // OK: "12 99"
    }

    printf("%d\n", a); // OK, we're still in a's scope

    printf("%d\n", b); // ILLEGAL, out of b's scope
}
```

13.1.1. Где определять переменные

Ещё один любопытный факт: переменные можно определять практически в любом месте блока. Они принадлежат его области видимости, но использовать их до определения нельзя.

```
#include <stdio.h>

int main(void)
{
    int i = 0;

    printf("%d\n", i); // OK: "0"
```

```
    //printf("%d\n", j); // ILLEGAL--can't use j before it's defined

    int j = 5;

    printf("%d %d\n", i, j); // OK: "0 5"
}
```

Исторически C требовал определять все переменные до любого исполняемого кода в блоке, но начиная со стандарта C99 это уже не требуется.

13.1.2. Сокрытие переменных

Если во внутренней области видимости объявлена переменная с тем же именем, что и во внешней, внутри вложенной области приоритет имеет внутренняя переменная. Иными словами, она скрывает внешнюю на время действия внутреннего объявления.

```
#include <stdio.h>

int main(void)
{
    int i = 10;

    {
        int i = 20;

        printf("%d\n", i); // Inner scope i, 20 (outer i is hidden)
    }

    printf("%d\n", i); // Outer scope i, 10
}
```

Возможно, вы заметили, что в этом примере я просто вставил блок в строке 7, даже без оператора `for` или `if` перед ним! Это совершенно допустимо. Иногда разработчик таким образом объединяет группу локальных переменных для быстрого вычисления, но встречается это редко.

13.2. Область видимости файла

Если определить переменную вне блока, она имеет *область видимости файла*. Она видна всем функциям, которые следуют за её объявлением в файле, и используется ими совместно. Исключение: если внутри блока объявлена переменная с тем же именем, она скроет переменную уровня файла.

Это ближе всего к тому, что в другом языке называют «глобальной» областью видимости.

Например:

```
#include <stdio.h>

int shared = 10; // File scope! Visible to the whole file after this!

void func1(void)
```

```
{
    shared += 100; // Now shared holds 110
}

void func2(void)
{
    printf("%d\n", shared); // Prints "110"
}

int main(void)
{
    func1();
    func2();
}
```

Если бы `shared` была объявлена в конце файла, программа не скомпилировалась бы. Объявление должно находиться до функций, которые используют переменную.

Есть способы дополнительно управлять объектами на уровне файла с помощью `static` и `extern`, но о них мы поговорим позже.

13.3. Область видимости цикла for

Честно говоря, я не знаю, как правильно это назвать: в разделе 6.8.5.3, пункте 1 стандарта C11 специального названия нет. В этом руководстве мы уже несколько раз пользовались этой возможностью: переменную можно объявить в первой части заголовка цикла `for`.

```
for (int i = 0; i < 10; i++)
    printf("%d\n", i);

printf("%d\n", i); // ILLEGAL--i is only in scope for the for-loop
```

В этом примере время жизни `i` начинается при её определении и продолжается в течение цикла. Если тело цикла заключено в блок, переменные, определённые в заголовке `for`, видны и в этой вложенной области.

Разумеется, если только внутренняя область не скрывает их. Этот необычный пример выводит `999` пять раз:

```
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 5; i++) {
        int i = 999; // Hides the i in the for-loop scope
        printf("%d\n", i);
    }
}
```

13.4. Замечание об области видимости функции

В стандарте C действительно упоминается *область видимости функции*, но она используется исключительно для меток, о которых мы ещё не говорили. Подробнее об этом — в другой раз.

Глава 14. Типы II: ещё больше типов!

Мы уже знакомы с `char`, `int` и `float`. Пора выяснить, какие ещё типы предлагает C.

14.1. Целые числа со знаком и без знака

До сих пор `int` обозначал целое число со знаком: положительное, отрицательное или ноль. У C есть и беззнаковые целочисленные типы, диапазон которых начинается с нуля. Их обозначают словом `unsigned`:

```
int a;          // Со знаком
signed int a;   // Со знаком
signed a;       // Со знаком, сокращение для signed int
unsigned int b; // Без знака
unsigned c;     // Сокращение для unsigned int
```

Зачем отказываться от отрицательных чисел? Чтобы представить больше положительных значений при том же числе битов. Бит — двоичная цифра, равная нулю или единице. Каждая комбинация битов обозначает число. В типе со знаком примерно половина комбинаций отводится отрицательным значениям, а в беззнаковом все используются для нуля и положительных значений.

Для примера: если целый тип занимает 64 бита и числа со знаком представлены в дополнительном коде, диапазоны будут такими:

Тип	Минимум	Максимум
`int`	−9 223 372 036 854 775 808	9 223 372 036 854 775 807
`unsigned int`	0	18 446 744 073 709 551 615

Максимальное значение беззнакового типа примерно вдвое больше. Фактический размер `int` зависит от реализации; 64 бита здесь лишь пример. На многих системах `int` занимает 32 бита.

14.2. Символьные типы

`char` может хранить символ:

```
char c = 'B';
printf("%c\n", c); // "B"
```

Но на самом деле это целочисленный тип:

```
char c = 'B';
printf("%d\n", c); // 66 (!!)
```

`char` занимает ровно один байт: `sizeof(char) == 1`. Однако стандарт C не требует, чтобы байт состоял именно из восьми битов. На практически всех современных машинах это так; для последовательности ровно из восьми битов существует однозначное название «октет».

Тип `char` может быть знаковым или беззнаковым в зависимости от реализации. Если знак важен, укажите его явно:

```
char a;          // Может быть как знаковым, так и беззнаковым
signed char b;   // Со знаком
unsigned char c; // Без знака
```

При восьмибитном байте и обычном дополнительном коде диапазоны таковы:

Тип	Минимум	Максимум
`signed char`	−128	127
`unsigned char`	0	255

Диапазон простого `char` определяется реализацией. Если `char` — число, с ним можно выполнять арифметику:

```
#include <stdio.h>

int main(void)
{
    char a = 10, b = 20;
    printf("%d\n", a + b); // 30!
}
```

Символьная константа вроде `'B'` тоже имеет числовое значение. Оно зависит от используемой системой кодировки. Если первые 128 символов соответствуют ASCII, значение `'B'` равно 66:

```
#include <stdio.h>

int main(void)
{
    char a = 10;
    char b = 'B'; // Значение ASCII: 66
    printf("%d\n", a + b); // 76!
}
```

Популярная кодировка Unicode совместима с ASCII для основных латинских букв, цифр и знаков пунктуации, поэтому на распространённых системах эти значения обычно совпадают с ASCII.

14.3. Дополнительные целые типы: short, long, long long

Кроме `char` и `int`, есть `short int`, `long int` и `long long int`, а также их беззнаковые варианты. Часть `int` обычно опускают:

```
// Эти объявления эквивалентны:
long long int x;
long long x;
// Как и эти:
short int x;
short x;
```

Заголовок `<limits.h>` содержит макросы с фактическими границами типов реализации. Используйте их, когда важны диапазоны, и не полагайтесь на жёстко заданные значения. Стандарт гарантирует минимальные диапазоны; на конкретной системе они могут быть шире.

В следующей таблице приведены характерные минимальные размеры и диапазоны при восьмибитном байте. Для старых стандартов нижняя граница знакового диапазона могла быть на единицу ближе к нулю; современный C23 требует представления в дополнительном коде. Значения `char` зависят от знаковости:

Тип	Минимум байтов	Минимум	Максимум
`char`	1	−128 или 0	127 или 255
`signed char`	1	−128	127
`short`	2	−32768	32767
`int`	2	−32768	32767
`long`	4	−2147483648	2147483647
`long long`	8	−9223372036854775808	9223372036854775807
`unsigned char`	1	0	255
`unsigned short`	2	0	65535
`unsigned int`	2	0	65535
`unsigned long`	4	0	4294967295
`unsigned long long`	8	0	18446744073709551615

Типа `long long long` нет — бесконечно добавлять `long` нельзя!

Например, на одной 64-битной системе автора `short` занимает 2 байта, `int` — 4, `long` и `long long` — по 8. Соответственно, `int` имеет диапазон от −2147483648 до 2147483647, `unsigned int` — от 0 до 4294967295; знаковые 64-битные типы — от −9223372036854775808 до 9223372036854775807, беззнаковые — от 0 до 18446744073709551615. На другой системе результаты могут отличаться.

Макросы из `<limits.h>`:

Тип	Минимум	Максимум
`char`	`CHAR_MIN`	`CHAR_MAX`

Тип	Минимум	Максимум
`signed char`	`SCHAR_MIN`	`SCHAR_MAX`
`short`	`SHRT_MIN`	`SHRT_MAX`
`int`	`INT_MIN`	`INT_MAX`
`long`	`LONG_MIN`	`LONG_MAX`
`long long`	`LLONG_MIN`	`LLONG_MAX`
`unsigned char`	0	`UCHAR_MAX`
`unsigned short`	0	`USHRT_MAX`
`unsigned int`	0	`UINT_MAX`
`unsigned long`	0	`ULONG_MAX`
`unsigned long long`	0	`ULLONG_MAX`

Если `CHAR_MAX == UCHAR_MAX`, простой `char` беззнаковый. Для беззнаковых типов отдельные макросы минимума не нужны: минимум равен нулю.

14.4. Числа с плавающей точкой: double и long double

Стандарт описывает модель числа с плавающей точкой с помощью знака `s`, основания `b`, показателя степени `e`, точности `p` и цифр значащей части f_k:

```
x = s × b^e × Σ(k=1…p) f_k × b^(-k),  emin ≤ e ≤ emax
```

Всё сразу стало понятно? Ладно, посмотрим на практическую сторону. Упоминаемые ниже макросы находятся в `<float.h>`.

Число кодируется битами; очень распространён формат IEEE 754. Упрощённо оно состоит из значащей части (мантиссы) и показателя степени. Например, при основании 10 одна и та же значащая часть 12345 с разными показателями даёт:

```
12345 × 10⁻³ = 12.345
12345 × 10⁴  = 123450000
12345 × 10⁰  = 12345
```

На компьютере основание обычно равно 2; проверить его можно по макросу `FLT_RADIX`. Поскольку комбинаций битов конечное число, точно представимых значений тоже конечное число. Для большей точности можно использовать более ёмкие типы: после `float` идут `double` и `long double`. Последний не связан с `long int`, кроме названия.

Стандарт не устанавливает конкретное число байтов для этих типов. На системе автора `float` занимает 4 байта, `double` — 8, `long double` — 16. Дополнительные биты позволяют представлять числа с большей точностью, но конкретные свойства зависят от реализации.

14.4.1. Сколько десятичных цифр?

Сколько значащих десятичных цифр можно преобразовать в тип с плавающей точкой, а затем вывести обратно, получив те же цифры? Гарантированные количества задают макросы:

Тип	Макрос	Минимум
`float`	`FLT_DIG`	6
`double`	`DBL_DIG`	10
`long double`	`LDBL_DIG`	10

Если `FLT_DIG` равен 6, шестизначное десятичное значение можно восстановить с этой точностью. Некоторые значения корректно возвращаются и с большим числом цифр, но такой гарантии для всех значений нет. Например, последовательность на системе автора начинает расходиться с ожидаемой после нескольких знаков:

```
0.12345
0.123456
0.1234567
0.12345678
0.123456791
0.1234567910
```

Следующий пример демонстрирует потерю точности при сложении двух `float`. Указанный результат относится к системе с `FLT_DIG == 6` и двоичной арифметикой IEEE 754:

```
#include <stdio.h>
#include <float.h>

int main(void)
{
    // В каждом числе шесть значащих десятичных цифр.
    float f = 3.14159f;
    float g = 0.00000265358f;

    printf("%.5f\n", f);  // 3.14159
    printf("%.11f\n", g); // 0.00000265358

    f += g; // Математическая сумма: 3.14159265358
    printf("%.11f\n", f); // 3.14159274101
}
```

Суффикс `f` задаёт тип константы `float`; без него она имела бы тип `double`. Заметьте, что гарантия восстановления десятичных цифр не означает точного двоичного представления каждого десятичного числа и не устраняет ошибки последующих вычислений.

14.4.2. Преобразование в десятичный вид и обратно

Значения с плавающей точкой расположены не непрерывно: между соседними представимыми числами есть промежутки. Десятичное число из такого промежутка при

преобразовании округляется к представимому значению.

Обратный вопрос отличается от предыдущего: сколько десятичных значащих цифр нужно вывести, чтобы затем преобразовать строку обратно и восстановить исходное значение с плавающей точкой? Для гарантированного восстановления используются:

Макрос	Назначение
`FLT_DECIMAL_DIG`	Количество цифр для восстановления `float`
`DBL_DECIMAL_DIG`	Для `double`
`LDBL_DECIMAL_DIG`	Для `long double`
`DECIMAL_DIG`	Достаточная точность для наиболее широкого стандартного типа

Допустим, `DBL_DIG` равно 15, а `DBL_DECIMAL_DIG` — 17. Сложение значений `0.123456789012345` и `0.000000000000006` математически даёт `0.1234567890123456`. В примере автора вывод с дополнительными знаками показывает `0.12345678901234559`. Эти цифры отражают представимое двоичное значение после округления. Если сократить запись до 15 значащих цифр, обратное преобразование может дать другое значение. Поэтому для сохранения значения `double` при преобразовании в текст и обратно нужен `DBL_DECIMAL_DIG`, а не только `DBL_DIG`.

Для оценки точности относительно исходных десятичных чисел ориентируйтесь на `FLT_DIG`, `DBL_DIG` и `LDBL_DIG`; для надёжного обратного преобразования используйте соответствующие макросы `*_DECIMAL_DIG`.

14.5. Типы числовых констант

У числовой константы вроде `1234` тоже есть тип. Разберёмся, как он выбирается и как указать его явно.

14.5.1. Шестнадцатеричная и восьмеричная запись

Префикс `0x` задаёт шестнадцатеричное число:

```
int a = 0x1A2B; // Шестнадцатеричное число
int b = 0x1a2b; // Регистр цифр не важен
printf("%x", a); // 1a2b
```

Начальный ноль без `x` означает восьмеричную запись:

```
int a = 012;
printf("%o\n", a); // 12 в восьмеричной системе
```

Будьте внимательны: добавляя нули слева для выравнивания, можно случайно изменить систему счисления:

```
int x = 11111; // Десятичное 11111
int y = 00111; // Десятичное 73 (восьмеричное 111)
int z = 01111; // Десятичное 585 (восьмеричное 1111)
```

Двоичная запись

Префикс `0b` позволяет записать двоичную константу:

```
int x = 0b101010;
printf("%d\n", x); // Десятичное 42
```

В старых стандартах это было распространённым расширением компиляторов; в C23 двоичные целочисленные константы стандартизованы. Возможность вывода через `%b` также появилась в C23; в более ранних версиях можно выводить биты по одному с помощью побитовых операций.

14.5.2. Целочисленные константы

Суффиксы позволяют задать семейство типа константы:

```
int x = 1234;
long int x = 1234L;
long long int x = 1234LL;
unsigned int x = 1234U;
unsigned long int x = 1234UL;
unsigned long long int x = 1234ULL;
```

Это отдельные примеры объявлений: в одном блоке повторно объявлять `x` так нельзя. Суффиксы могут быть прописными или строчными; `U` и `L` либо `LL` могут стоять в любом порядке.

Тип	Суффикс
`int`	отсутствует
`long int`	`L`
`long long int`	`LL`
`unsigned int`	`U`
`unsigned long int`	`UL`
`unsigned long long int`	`ULL`

На деле отсутствие суффикса не всегда означает `int`: компилятор выбирает первый тип из предусмотренного стандартом списка, способный представить значение. Список зависит и от основания. Для правил C11 порядок такой:

Суффикс	Десятичная константа	Восьмеричная или шестнадцатеричная
Нет	`int`, `long`, `long long`	`int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`
`U`	`unsigned int`, `unsigned long`, `unsigned long long`	То же
`L`	`long`, `long long`	`long`, `unsigned long`, `long long`, `unsigned long long`
`UL`	`unsigned long`, `unsigned long long`	То же
`LL`	`long long`	`long long`, `unsigned long long`
`ULL`	`unsigned long long`	То же

Например, для `123456789U` сначала проверяется `unsigned int`, затем при необходимости `unsigned long`, затем `unsigned long long`. Выбирается первый подходящий тип.

14.5.3. Константы с плавающей точкой

Константа `1.23` без суффикса имеет тип `double`, а не `float`. Суффикс `f` или `F` задаёт `float`, суффикс `l` или `L` — `long double`:

```
float x = 3.14f;
double x = 3.14;
long double x = 3.14L;
```

И здесь это отдельные примеры. В записи `float x = 3.14;` справа действительно стоит `double`, но C автоматически преобразует его в `float` при присваивании.

Научная запись

Число вида `1.2345 × 10³`, равное `1234.5`, можно записать в C с помощью `e`:

```
1.2345 × 10⁻³ = 0.0012345  →  1.2345e-3
1.2345 × 10⁸  = 123450000  →  1.2345e+8
```

Спецификатор `%e` выводит число в таком виде:

```
printf("%e\n", 123456.0); // 1.234560e+05
```

Перед десятичной точкой при записи может быть больше одной цифры, хотя при выводе нормализованной записи остаётся одна:

```
double x = 123.456e+3; // 123456
```

Знак `+` у положительной степени можно опустить: `1.2345e10` равно `1.2345e+10`. Суффиксы также применимы: `1.2345e10F`, `1.2345e10L`.

Шестнадцатеричные константы с плавающей точкой

Такие константы начинаются с `0x`, но показатель степени обязателен, записывается после `p` и задаёт степень двойки. Например, `0xa.1p3` означает 10,0625 × 2³ = 80,5. Спецификатор `%a` выводит шестнадцатеричную запись:

```
double x = 0xa.1p3;
printf("%a\n", x); // 0x1.42p+6
printf("%f\n", x); // 80.500000
```

Глава 15. Типы III: преобразования

В этой главе мы поговорим о преобразовании одного типа в другой. В C есть несколько способов это делать, и некоторые могут отличаться от привычных вам по другим языкам.

Прежде чем обсуждать, как вызвать преобразование, разберёмся, что происходит при его выполнении.

15.1. Преобразование строк

В отличие от многих языков, C преобразует строки в числа и обратно не так просто, как одни числовые типы в другие. Для этого приходится вызывать функции, которые выполняют всю грязную работу.

15.1.1. Число в строку

Чтобы преобразовать число в строку, можно воспользоваться `sprintf()` или `snprintf()`. Они работают примерно как `printf()`, но записывают результат в строку, которую потом можно вывести или использовать иначе. Разница в том, что `snprintf()` позволяет указать максимальное количество записываемых байтов и тем самым не допустить выхода за конец буфера, поэтому она безопаснее.

Например, преобразуем часть значения π в строку:

```
#include <stdio.h>

int main(void)
{
    char s[10];
    float f = 3.14159;

    // Convert "f" to string, storing in "s", writing at most 10 characters
    // including the NUL terminator

    snprintf(s, 10, "%f", f);

    printf("String value: %s\n", s); // String value: 3.141590
}
```

Для целых чисел, как обычно, можно использовать `%d` или `%u`.

15.1.2. Строка в число

В C для этого есть несколько семейств функций. Назовём их семействами `atoi` и `strtol`.

Для простого преобразования строки в число попробуйте функции `atoi` из `<stdlib.h>`. Они плохо подходят для обработки ошибок, поэтому применять их следует осторожно. Если преобразованное значение невозможно представить в целевом целом типе, поведение не определено.

Функция	Преобразование
`atoi`	Строка в `int`
`atof`	Строка в число с плавающей точкой (`double`)
`atol`	Строка в `long int`
`atoll`	Строка в `long long int`

Хотя стандарт этого и не утверждает, буква `a` в начале названия означает ASCII: таким образом, `atoi()` — это «ASCII в целое число». Сегодня такое название несколько чрезмерно сосредоточено на ASCII.

Вот пример преобразования строки в `float`:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *pi = "3.14159";
    float f;

    f = atof(pi);

    printf("%f\n", f);
}
```

Однако некорректный ввод обрабатывается неудобно:

```
int x = atoi("what"); // "What" ain't no number I ever heard of
```

Здесь результат `0` нельзя отличить от результата корректного преобразования строки `"0"` средствами самой `atoi()`; эта функция не предоставляет надёжного способа сообщить об ошибке. В оригинале автор относит этот пример к неопределённому поведению, однако при отсутствии преобразуемого числа `atoi` возвращает ноль; неопределённость возникает, в частности, когда результат не представим в `int`.

Чтобы лучше обрабатывать ошибки, рассмотрим функции семейства `strtol`, также объявленные в `<stdlib.h>`. Они к тому же поддерживают больше типов и систем счисления!

Функция	Преобразование
`strtol`	Строка в `long int`
`strtoll`	Строка в `long long int`
`strtoul`	Строка в `unsigned long int`
`strtoull`	Строка в `unsigned long long int`
`strtof`	Строка в `float`
`strtod`	Строка в `double`

Функция	Преобразование
`strtold`	Строка в `long double`

Все эти функции используются похожим образом; для многих они становятся первой встречей с указателями на указатели! Не волнуйтесь: разобраться проще, чем кажется.

Преобразуем строку в `unsigned long`, пока отбросив информацию об ошибках, то есть о неподходящих символах во входной строке:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *s = "3490";

    // Convert string s, a number in base 10, to an unsigned long int.
    // NULL means we don't care to learn about any error information.

    unsigned long int x = strtoul(s, NULL, 10);

    printf("%lu\n", x); // 3490
}
```

Обратите внимание: даже если мы не сохраняем информацию о недопустимых символах, `strtoul()` не вызывает из-за них неопределённое поведение. Если ни одна цифра не преобразована, она возвращает `0`; при частичном преобразовании возвращается значение прочитанной части.

Кроме того, мы указали, что число десятичное, то есть его основание равно 10. Значит ли это, что можно преобразовывать числа и в других системах счисления? Конечно! Попробуем двоичную:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *s = "101010"; // What's the meaning of this number?

    // Convert string s, a number in base 2, to an unsigned long int.

    unsigned long int x = strtoul(s, NULL, 2);

    printf("%lu\n", x); // 42
}
```

Хорошо, это забавно, но зачем там `NULL`?

Этот аргумент помогает выяснить, возникла ли ошибка при обработке строки. Он имеет тип указателя на указатель на `char`. Звучит страшно, но стоит разобраться — и всё станет просто.

Передадим заведомо неправильное число и посмотрим, как `strtoul()` показывает, где находится первая недопустимая цифра:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *s = "34x90"; // "x" is not a valid digit in base 10!
    char *badchar;

    // Convert string s, a number in base 10, to an unsigned long int.

    unsigned long int x = strtoul(s, &badchar, 10);

    // It tries to convert as much as possible, so gets this far:

    printf("%lu\n", x); // 34

    // But we can see the offending bad character because badchar
    // points to it!

    printf("Invalid character: %c\n", *badchar); // "x"
}
```

Здесь `strtoul()` изменяет указатель `badchar`, чтобы показать место, где возникла проблема. Мы передаём адрес `badchar`, иначе функция не могла бы изменить его так, чтобы это изменение было видно вызывающему коду. Это аналогично передаче указателя на `int` функции, которая должна изменить значение этого `int`.

А если всё прошло успешно? Тогда `badchar` будет указывать на завершающий символ NUL в конце строки. Это можно проверить:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *s = "3490"; // "x" is not a valid digit in base 10!
    char *badchar;

    // Convert string s, a number in base 10, to an unsigned long int.

    unsigned long int x = strtoul(s, &badchar, 10);

    // Check if things went well

    if (*badchar == '\0') {
        printf("Success! %lu\n", x);
    } else {
        printf("Partial conversion: %lu\n", x);
        printf("Invalid character: %c\n", *badchar);
    }
}
```

Вот и всё. Функции вроде `atoi()` могут пригодиться для быстрого преобразования контролируемых данных, но семейство `strtol()` даёт гораздо больше возможностей для

обработки ошибок и выбора основания системы счисления.

Для полной проверки также нужно убедиться, что преобразована хотя бы одна цифра (`badchar != s`), и проверить переполнение через `errno`.

15.2. Преобразования char

Допустим, у нас есть символ цифры, например `'5'`. Это то же самое, что значение `5`? Давайте проверим:

```
printf("%d %d\n", 5, '5');
```

На моей системе с UTF-8 программа выводит:

```
5 53
```

Значит, нет. А что такое 53? Это кодовая позиция символа `'5'` в UTF-8 и ASCII. В каждой кодировке символу соответствует некоторое значение.

Как же преобразовать символ `'5'`, который, по-видимому, имеет значение 53, в число 5? С помощью одной хитрости!

Стандарт C гарантирует, что коды символов цифр идут подряд именно в таком порядке:

```
0 1 2 3 4 5 6 7 8 9
```

Подумайте минуту, как этим воспользоваться. Дальше будет ответ…

Посмотрим на символы и их коды в UTF-8:

```
 0  1  2  3  4  5  6  7  8  9
48 49 50 51 52 53 54 55 56 57
```

Как видите, `'5'` имеет код 53, а `'0'` — 48. Поэтому, чтобы получить числовое значение любой цифры, можно вычесть из её символа `'0'`:

```
char c = '6';

int x = c;       // x has value 54, the code point for '6'
int y = c - '0'; // y has value 6, just like we want
```

Преобразование в обратную сторону также возможно: достаточно прибавить значение к `'0'`.

```
int x = 6;
char c = x + '0'; // c has value 54

printf("%d\n", c); // prints 54
printf("%c\n", c); // prints 6 with %c
```

Такой способ может показаться странным, и по современным меркам он действительно странный. Но в старые времена, когда компьютеры буквально делали из дерева, преобразование выполняли именно так. Оно работало, поэтому C ничего менять не стал.

15.3. Числовые преобразования

15.3.1. Логический тип

При преобразовании нуля в `bool` результат равен `0`. Во всех остальных случаях — `1`.

15.3.2. Преобразования целых типов

Если целое число преобразуется в беззнаковый тип и не помещается в него, результат циклически приводится к диапазону этого типа, как показание одометра.

На практике в вашей реализации, вероятно, просто отбрасываются старшие биты. Например, при преобразовании 16-битного числа `0x1234` в восьмибитное получится `0x34`.

Если число преобразуется в знаковый тип и не помещается в него, результат определяется реализацией: произойдёт нечто документированное, но подробности придётся искать в документации.

На практике система может усечь исходную битовую последовательность и интерпретировать её как знаковое число в дополнительном коде. Например, моя система преобразует `unsigned char` со значением 192 в `signed char` со значением −64. В дополнительном коде обоим соответствует двоичная последовательность `11000000`.

15.3.3. Преобразования целых чисел и чисел с плавающей точкой

При преобразовании числа с плавающей точкой в целый тип дробная часть безжалостно отбрасывается. Ладно, просто отбрасывается.

Но есть подвох: если число слишком велико и не представимо в целевом целом типе, поведение не определено. Так что не делайте этого.

При преобразовании целого числа или числа с плавающей точкой в тип с плавающей точкой C старается представить значение как можно точнее. Если точное представление невозможно, выбирается приближённое значение в соответствии с правилами реализации. Значение, выходящее за представимый диапазон, требует отдельной осторожности; правила зависят от конкретного преобразования и реализации.

15.4. Неявные преобразования

Это преобразования, которые компилятор автоматически выполняет при смешивании типов.

15.4.1. Целочисленные повышения

В ряде случаев, если `int` способен представить все значения типа `char` или `short` — знакового или беззнакового, — значение повышается до `int`. Если нет, оно повышается до `unsigned int`.

Благодаря этому можно написать:

```
char x = 10, y = 20;
int i = x + y;
```

Здесь C повышает `x` и `y` до `int` до выполнения сложения.

Целочисленные повышения выполняются при обычных арифметических преобразованиях, в вариативных функциях, с унарными операторами `+` и `-`, а также при передаче значений функциям без прототипов. Вариативные функции принимают переменное число аргументов. Вызовы без прототипов сейчас редки: компилятор предупреждает о них, а использовать прототипы правильно; эта возможность сохранилась по историческим причинам.

15.4.2. Обычные арифметические преобразования

Это автоматические преобразования, которые C выполняет при запрошенных вами числовых операциях. Кстати, стандарт C11, раздел 6.3.1.8, действительно называет их именно так. Здесь речь только о числовых типах; строки рассмотрим позже.

Эти преобразования отвечают на вопросы о смешивании типов, например:

```
int x = 3 + 1.2; // Mixing int and double
                 // 4.2 is converted to int
                 // 4 is stored in x

float y = 12 * 2; // Mixing float and int
                  // 24 is converted to float
                  // 24.0 is stored in y
```

Станут ли значения `int` или `float`? Как это работает?

Вот упрощённое изложение:

1. Если один из операндов имеет тип с плавающей точкой, остальные приводятся к подходящему общему типу с плавающей точкой.
2. В противном случае для целочисленных операндов выполняются целочисленные повышения, после чего выбирается общий тип по правилам ранга и знаковости. Иногда знаковый тип преобразуется в беззнаковый.

Если нужны все подробности, обратитесь к C11, раздел 6.3.1.8. Хотя, скорее всего, вам этого не захочется.

В общем случае помните: при смешивании целого числа с числом с плавающей точкой целое преобразуется в соответствующий тип с плавающей точкой. Для смешанных целочисленных типов действует набор правил выбора общего типа; он не гарантирует отсутствия переполнения или неожиданного беззнакового результата.

Наконец, при преобразовании между типами с плавающей точкой компилятор старается получить точное значение, а если это невозможно — подходящее приближение. При выходе за представимый диапазон нужно учитывать правила реализации и применимого стандарта.

15.4.3. void*

Тип `void *` интересен тем, что его можно преобразовать к любому типу указателя на объект и обратно:

```
int x = 10;
void *p = &x; // &x is type int*, but we store it in a void*
int *q = p;   // p is void*, but we store it in an int*
```

15.5. Явные преобразования

Это преобразования типов, которые вы запрашиваете сами.

Преобразование может произойти при присваивании значения одного типа переменной другого типа с помощью `=`. Кроме того, можно явно выполнить преобразование с помощью приведения типа.

15.5.1. Приведение типа

Чтобы явно изменить тип выражения, поставьте перед ним новый тип в скобках. Некоторые разработчики на C не одобряют этот приём без крайней необходимости, но вы наверняка встретите его в чужом коде.

Рассмотрим преобразование `int` в `long`, чтобы сохранить результат в `long`.

Этот пример искусственный: приведение здесь совершенно не нужно, потому что при присваивании выражение `x + 12` автоматически преобразуется в `long int` соответственно типу `y`.

```
int x = 10;
long int y = (long int)x + 12;
```

Здесь, хотя `x` имеет тип `int`, выражение `(long int)x` имеет тип `long int`. Мы говорим: «Мы привели `x` к `long int`».

Чаще приведение можно встретить при преобразовании `void *` в конкретный тип указателя, чтобы его можно было разыменовать.

Например, функция обратного вызова для стандартной `qsort()` получает аргументы типа `void *`:

```
int compar(const void *elem1, const void *elem2)
{
    if (*((const int*)elem2) > *((const int*)elem1)) return 1;
    if (*((const int*)elem2) < *((const int*)elem1)) return -1;
    return 0;
}
```

Но можно записать и более понятно с помощью присваивания:

```
int compar(const void *elem1, const void *elem2)
{
    const int *e1 = elem1;
    const int *e2 = elem2;

    return *e2 - *e1;
}
```

Учтите: вычитание в этом последнем варианте может переполнить `int`. Первый вариант с отдельными сравнениями этого избегает.

Ещё приведение часто используют, чтобы избежать предупреждения при выводе указателя с помощью `%p`, который ожидает именно `void *`:

```
int x = 3490;
int *p = &x;

printf("%p\n", p);
```

Это вызывает предупреждение:

```
warning: format ‘%p’ expects argument of type ‘void *’, but argument
2 has type ‘int *’
```

Исправить его можно приведением:

```
printf("%p\n", (void *)p);
```

Ещё один случай — явное преобразование указателей без промежуточного `void *`, хотя это тоже встречается довольно редко:

```
long x = 3490;
long *p = &x;
unsigned char *c = (unsigned char *)p;
```

Третий распространённый случай — функции работы с символами из `<ctype.h>`: значения, знаковость которых вызывает сомнения, следует приводить к `unsigned char`, чтобы избежать неопределённого поведения. Отдельно допустимо передавать специальное значение `EOF`.

Повторюсь: на практике приведение типов нужно редко. Если вы его используете, возможно, ту же задачу можно решить иначе или приведение вообще не требуется.

А может быть, требуется. Лично я стараюсь его избегать, но не боюсь применять, когда это необходимо.

Глава 16. Типы IV: квалификаторы и спецификаторы

Теперь, когда мы познакомились с дополнительными типами, выясняется, что им можно придавать свойства, управляющие их поведением. Для этого служат квалификаторы типа и спецификаторы класса хранения.

16.1. Квалификаторы типа

Они позволяют объявлять постоянные значения и давать компилятору подсказки для оптимизации.

16.1.1. const

Это самый распространённый квалификатор типа. Он означает, что переменная постоянна, и попытка изменить её вызовет возмущение компилятора.

```
const int x = 2;
x = 4;  // COMPILER PUKING SOUNDS, can't assign to a constant
```

Изменить значение `const` нельзя. Этот квалификатор часто встречается в списках параметров функций:

```
void foo(const int x)
{
    printf("%d\n", x + 30);  // OK, doesn't modify "x"
}
```

16.1.1.1. const и указатели

Здесь становится немного хитрее: для указателей есть два варианта с разным смыслом.

Во-первых, можно запретить изменение значения, на которое указывает указатель. Для этого поставьте `const` рядом с именем типа, перед звёздочкой:

```
int x[] = {10, 20};
const int *p = x;

p++;      // We can modify p, no problem
*p = 30;  // Compiler error! Can't change what it points to
```

Несколько сбивает с толку то, что эти две записи эквивалентны:

```
const int *p;  // Can't modify what p points to
int const *p;  // Can't modify what p points to, just like the previous line
```

Итак, менять значение по указателю нельзя, а сам указатель — можно. А если мы хотим наоборот: менять значение, но не сам указатель?

Переместите `const` за звёздочку:

```
int *const p;  // We can't modify "p" with pointer arithmetic
p++;           // Compiler error!
```

Но значение, на которое он указывает, менять можно:

```
int x = 10;
int *const p = &x;

*p = 20;  // Set "x" to 20, no problem
```

Можно сделать постоянными оба уровня:

```
const int *const p;  // Can't modify p or *p!
```

Наконец, если уровней косвенности несколько, применяйте `const` к нужным уровням. Постоянство одного указателя не означает, что указатель, на который он ссылается, тоже постоянен. Их можно задать явно:

```
char **p;
p++;       // OK!
(*p)++;    // OK!

char **const p;
p++;       // Error!
(*p)++;    // OK!

char *const *p;
p++;       // OK!
(*p)++;    // Error!

char *const *const p;
p++;       // Error!
(*p)++;    // Error!
```

16.1.1.2. Корректное использование const

Компилятор предупредит о таком коде:

```
const int x = 20;
int *p = &x;
```

Предупреждение будет примерно таким:

```
initialization discards 'const' qualifier from pointer type target
```

Что произошло? Посмотрим на типы по обе стороны присваивания:

```
const int x = 20;
int *p = &x;
//  ^     ^
//  |     |
// int*  const int*
```

Компилятор предупреждает, что справа находится указатель на `const`, а слева — нет. При этом квалификатор `const` правого выражения отбрасывается.

Можно попытаться сделать следующее, но так делать неправильно: компилятор предупредит, а результатом будет неопределённое поведение.

```
const int x = 20;
int *p = &x;

*p = 40;              // Undefined behavior--maybe it modifies "x", maybe not!
printf("%d\n", x);    // 40, if you're lucky
```

16.1.2. restrict

Коротко: использовать его необязательно, и вы можете игнорировать его, когда встретите. При правильном использовании он может повысить производительность; при неправильном приводит к неопределённому поведению.

`restrict` подсказывает компилятору, что определённая область памяти будет доступна через данный указатель, а не через другой. Если разработчик объявляет указатель с `restrict`, а затем обращается к изменяемому объекту другим способом, например через другой указатель, поведение может быть неопределённым.

По сути вы говорите C: «Я гарантирую, что обращаюсь к этой памяти только через данный указатель. Если я соврал, допускается неопределённое поведение».

Компилятор использует эту информацию для оптимизации. Например, если указатель с `restrict` многократно разыменовывается в цикле, компилятор может сохранить значение в регистре и записать конечный результат в память один раз после цикла. Если другой указатель обращается к той же памяти внутри цикла, результат может оказаться неверным.

Если объект, на который указывает `restrict`-указатель, не изменяется, это ограничение не играет той же роли: речь об оптимизациях, связанных с записью в память.

Напишем функцию обмена двух переменных. С помощью `restrict` обещаем, что не передадим ей два указателя на один объект. А затем нарушим обещание:

```
void swap(int *restrict a, int *restrict b)
{
    int t;

    t = *a;
    *a = *b;
    *b = t;
}
```

```
int main(void)
{
    int x = 10, y = 20;

    swap(&x, &y);  // OK! "a" and "b", above, point to different things

    swap(&x, &x);  // Undefined behavior! "a" and "b" point to the same thing
}
```

Если убрать оба `restrict`, оба вызова будут допустимы, хотя у компилятора может оказаться меньше возможностей для оптимизации.

Ограничение `restrict` действует в соответствующем блоке. Для параметра функции это блок её тела. Если указатель относится к массиву, правило касается отдельных элементов: другие указатели могут читать и записывать другие элементы массива, не обращаясь к тем же объектам, что и ограниченный указатель. Для объявления на уровне файла ограничение охватывает всю программу.

Вы можете встретить `restrict` в библиотечных функциях, например `printf()`:

```
int printf(const char * restrict format, ...);
```

Здесь компилятору сообщают об ограничениях доступа к строке формата внутри функции.

Если в параметре функции вместо указателя используется запись массива, `restrict` можно указать так:

```
void foo(int p[restrict])     // With no size
void foo(int p[restrict 10])  // Or with a size
```

Но запись через указатель встречается чаще.

16.1.3. volatile

Вряд ли вы встретите или будете использовать это ключевое слово, если не работаете непосредственно с оборудованием.

`volatile` сообщает компилятору, что значение может измениться извне и его следует считывать при каждом обращении.

Например, по некоторому адресу в памяти может находиться значение аппаратного таймера, постоянно обновляемое независимо от программы. Если компилятор надолго сохранит его в регистре ради оптимизации, изменения в памяти не отразятся в регистре.

Объявляя значение `volatile`, вы сообщаете: «То, на что указывает этот указатель, может измениться в любой момент по причинам вне кода программы».

```
volatile int *p;
```

16.1.4. _Atomic

Это необязательная возможность C, которую мы рассмотрим в главе об атомарных операциях.

16.2. Спецификаторы класса хранения

Спецификаторы класса хранения похожи на квалификаторы: они дают компилятору дополнительную информацию о переменной.

16.2.1. auto

Это ключевое слово почти не встречается, поскольку для переменных области видимости блока `auto` подразумевается по умолчанию. Следующие объявления эквивалентны:

```
{
    int a;       // auto is the default...
    auto int a;  // So this is redundant
}
```

`auto` указывает, что объект имеет автоматическую продолжительность хранения: он существует в соответствующем блоке, а его память автоматически освобождается при выходе из него.

Есть одна особенность: значение автоматической переменной неопределённо, пока вы явно её не инициализировали. Иногда говорят, что она содержит «случайные» или «мусорные» данные, хотя оба описания не совсем удачны. В любом случае без инициализации вы не знаете её значения.

Всегда инициализируйте автоматические переменные перед использованием!

16.2.2. static

У этого слова два значения в зависимости от того, объявлена переменная на уровне файла или внутри блока. Начнём с блока.

16.2.2.1. static в области видимости блока

Здесь мы фактически говорим: «Мне нужен один экземпляр этой переменной, общий для всех вызовов функции». Иными словами, её значение сохраняется между вызовами.

Переменная `static` внутри блока с инициализатором инициализируется один раз при запуске программы, а не при каждом вызове функции.

```
#include <stdio.h>

void counter(void)
{
    static int count = 1;  // This is initialized one time
```

```
    printf("This has been called %d time(s)\n", count);

    count++;
}

int main(void)
{
    counter();  // "This has been called 1 time(s)"
    counter();  // "This has been called 2 time(s)"
    counter();  // "This has been called 3 time(s)"
    counter();  // "This has been called 4 time(s)"
}
```

Видите, как значение `count` сохраняется между вызовами?

Заметим также, что статические переменные в блоке по умолчанию инициализируются нулём:

```
static int foo;      // Default starting value is `0`...
static int foo = 0;  // So the `0` assignment is redundant
```

В многопоточной программе необходимо следить, чтобы несколько потоков не изменяли одну и ту же переменную без должной синхронизации.

16.2.2.2. static на уровне файла

Вне блоков, на уровне файла, смысл меняется. Такие переменные и без `static` сохраняются между вызовами функций.

Здесь `static` означает, что переменная недоступна за пределами данного исходного файла. Вроде глобальной, но только внутри этого файла. Подробнее об этом — в разделе о проектах из нескольких исходных файлов.

16.2.3. extern

Спецификатор `extern` позволяет ссылаться на объекты в других исходных файлах.

Предположим, файл `bar.c` целиком состоит из следующего:

```
// bar.c

int a = 37;
```

Здесь объявлена переменная `int a` на уровне файла. А если в другом файле, `foo.c`, мы хотим обратиться к этой `a`? Это легко сделать через `extern`:

```
// foo.c

extern int a;

int main(void)
{
```

```
    printf("%d\n", a);  // 37, from bar.c!

    a = 99;

    printf("%d\n", a);  // Same "a" from bar.c, but it's now 99
}
```

Можно объявить `extern int a` и внутри блока; она всё равно будет ссылаться на `a` из `bar.c`:

```
// foo.c

int main(void)
{
    extern int a;

    printf("%d\n", a);  // 37, from bar.c!

    a = 99;

    printf("%d\n", a);  // Same "a" from bar.c, but it's now 99
}
```

Если бы `a` в `bar.c` была объявлена с `static`, это не сработало бы: статические переменные уровня файла не видны за его пределами.

И последнее замечание о `extern` для функций: для них он подразумевается по умолчанию, поэтому указывать его необязательно. Если функция должна быть видна только в одном исходном файле, её можно объявить с `static`.

16.2.4. register

Это ключевое слово подсказывает компилятору, что переменная используется часто и доступ к ней желательно сделать как можно быстрее. Компилятор не обязан соглашаться.

Современные оптимизаторы компиляторов C довольно хорошо разбираются в этом сами, поэтому сейчас `register` встречается редко. Существуют доводы в пользу его использования, некоторые перечислены ниже. Но, честно говоря, я ни разу не применял его в собственном коде C.

Тем не менее, если очень хочется:

```
#include <stdio.h>

int main(void)
{
    register int a;  // Make "a" as fast to use as possible.

    for (a = 0; a < 10; a++)
        printf("%d\n", a);
}
```

Однако за это приходится платить: у переменной `register` нельзя взять адрес.

```
register int a;
int *p = &a;  // COMPILER ERROR! Can't take address of a register
```

То же относится к элементам массива `register`: преобразование такого массива в указатель приводит к неопределённому поведению. Можно получить его размер через `sizeof`, но особой пользы от этого нет. Получается, массивы `register` практически бесполезны.

GCC и Clang сообщают об ошибках для следующих операций. Исключение — GCC для `a[2]` выдаёт предупреждение лишь с параметром `-pedantic`:

```
register int a[] = {11, 22, 33, 44, 55};
int *p = a;        // Undefined behavior!
int b = *(a + 2);  // Undefined behavior!
int c = a[2];      // Undefined behavior!
```

Поскольку адрес переменной `register` взять нельзя, компилятор может использовать это при оптимизации. Кроме того, `register` у переменной `const` не позволяет случайно передать её указатель другой функции, которая намеренно проигнорирует `const`.

Немного истории: внутри процессора есть небольшие специальные «переменные» — регистры. Доступ к ним намного быстрее, чем к оперативной памяти, поэтому их использование может ускорить программу. Регистры находятся не в оперативной памяти, и у них нет соответствующего адреса памяти; поэтому нельзя получить их адрес или указатель на них.

Однако, как уже сказано, современные компиляторы умеют генерировать оптимальный код и использовать регистры, когда это возможно, независимо от ключевого слова `register`. Более того, стандарт разрешает компилятору рассматривать его как `auto`. Никаких гарантий нет.

16.2.5. _Thread_local

В многопоточной программе это средство позволяет сделать так, чтобы у каждого потока была собственная копия глобальной или статической переменной блока. Оно помогает избегать гонок и взаимного вмешательства потоков.

В области видимости блока его нужно использовать вместе с `extern` или `static`.

Кроме того, подключив `<threads.h>`, можно использовать более приятное имя `thread_local` как синоним `_Thread_local`. Подробнее об этом — в разделе о потоках.

Глава 17. Проекты из нескольких файлов

До сих пор мы рассматривали учебные программы, которые по большей части помещались в одном файле. Но сложные программы на C состоят из множества файлов, которые компилируются и связываются в один исполняемый файл.

В этой главе мы познакомимся с распространёнными приёмами организации крупных проектов.

17.1. Подключения и прототипы функций

Очень часто часть функций определена в одном файле, а вызвать их нужно из другого.

Попробуем сначала сделать это напрямую, получим предупреждение или ошибку, а затем исправим проблему правильным способом.

Для сборки нужно указать в командной строке все исходные файлы:

```
#       output file    source files
#            v             v
#          |----|       |---------|
gcc -o foo foo.c bar.c
```

Здесь `foo.c` и `bar.c` собираются в исполняемый файл `foo`.

В следующих примерах имя файла указано первым комментарием в исходном коде. Посмотрим на `bar.c`:

```
// File bar.c

int add(int x, int y)
{
    return x + y;
}
```

А вот `foo.c`, содержащий `main`:

```
// File foo.c

#include <stdio.h>

int main(void)
{
    printf("%d\n", add(2, 3)); // 5!
}
```

Видите, как мы вызываем `add()` из `main()`, хотя `add()` находится совсем в другом исходном файле? Она определена в `bar.c`, а вызов находится в `foo.c`!

Если собрать программу командой:

```
gcc -o foo foo.c bar.c
```

получим ошибку:

```
error: implicit declaration of function 'add' is invalid in C99
```

Либо вы получите предупреждение. Не игнорируйте его! В C никогда не игнорируйте предупреждения — устраняйте их все.

Как вы помните из раздела о прототипах, современные стандарты C запрещают неявные объявления, и вводить их в новый код нет причин. Нужно это исправить.

Неявное объявление означает, что мы используем функцию, в данном случае `add()`, заранее ничего о ней не сообщив языку C. Компилятору нужно знать, что функция возвращает, какие типы аргументов принимает и так далее.

Ранее мы уже видели решение: прототип функции. Если добавить его в `foo.c` перед вызовом, всё заработает:

```
// File foo.c

#include <stdio.h>

int add(int, int); // Add the prototype

int main(void)
{
    printf("%d\n", add(2, 3)); // 5!
}
```

Больше никакой ошибки!

Но писать прототип каждый раз, когда нужна функция, неудобно. Ведь мы только что использовали `printf()` и не вводили её прототип. Почему?

На самом деле мы его подключили! Он находится в `stdio.h`, который мы включили директивой `#include`.

Можно ли сделать то же самое для `add()`: написать прототип и поместить его в заголовочный файл? Конечно!

По соглашению заголовочные файлы C имеют расширение `.h`. Часто, хотя и не всегда, их имя совпадает с именем соответствующего файла `.c`. Создадим для `bar.c` файл `bar.h` и поместим туда прототип:

```
// File bar.h

int add(int, int);
```

Теперь изменим `foo.c`, чтобы он подключал этот файл. Предположив, что файл находится в том же каталоге, укажем его в двойных кавычках, а не в угловых скобках:

```
// File foo.c

#include <stdio.h>

#include "bar.h" // Include from current directory

int main(void)
{
    printf("%d\n", add(2, 3)); // 5!
}
```

Обратите внимание: прототипа в самом `foo.c` больше нет — мы подключили его из `bar.h`. Теперь любой файл, которому нужна функция `add()`, может получить её объявление через `#include "bar.h"`, не переписывая прототип.

Как вы, наверное, догадались, `#include` буквально вставляет содержимое указанного файла в это место исходного кода, как если бы вы ввели его вручную.

Соберём и запустим программу:

```
./foo
5
```

Мы действительно получили результат `2 + 3`! Ура!

Но пока не открывайте любимый напиток. Мы почти закончили! Осталось добавить ещё один стандартный элемент.

17.2. Повторные подключения

Нередко заголовочный файл сам подключает другие заголовки, необходимые соответствующим файлам C. А почему бы и нет?

В итоге один заголовочный файл может подключаться несколько раз из разных мест. Иногда это не вызывает проблем, а иногда приводит к ошибкам компиляции. При этом мы не можем контролировать все места, где его подключают!

Ещё хуже: может возникнуть безумная ситуация, когда `a.h` подключает `b.h`, а `b.h` — `a.h`. Получается бесконечный цикл `#include`!

При попытке собрать такой код возникнет ошибка:

```
error: #include nested depth 200 exceeds maximum of 200
```

Может быть, на 201-м шаге цикл бы наконец разрешился…

Нам нужно добиться того, чтобы после первого подключения файла последующие директивы `#include` для него игнорировались.

Этот приём настолько распространён, что его стоит автоматически применять при создании каждого заголовочного файла!

Обычно для этого используют макрос препроцессора, определяемый при первом подключении файла. При последующих подключениях сначала проверяется, определён ли он уже.

Имя макроса часто получают из имени заголовка: например, `bar.h` переводят в верхний регистр, а точку заменяют подчёркиванием — получается `BAR_H`.

В самом начале файла поставим проверку: если файл уже был подключён, всё его содержимое фактически пропускается.

Не начинайте такое имя с подчёркивания перед заглавной буквой или с двух подчёркиваний: эти варианты зарезервированы.

```
#ifndef BAR_H // If BAR_H isn't defined...
#define BAR_H // Define it (with no particular value)

// File bar.h

int add(int, int);

#endif // End of the #ifndef BAR_H
```

Благодаря этому содержимое заголовка фактически включается только один раз, сколько бы мест ни пытались его подключить.

17.3. static и extern

С помощью `extern` можно ссылаться на объекты из других файлов.

Ключевое слово `static` позволяет сделать переменные и функции уровня файла невидимыми из других исходных файлов, даже если там используется `extern`.

Подробнее см. разделы книги о спецификаторах класса хранения `static` и `extern`.

17.4. Компиляция в объектные файлы

Это не входит в стандарт языка, но встречается практически повсеместно в мире C.

Файлы C можно скомпилировать в промежуточные представления — *объектные файлы*. Они содержат машинный код, то есть единицы и нули самих инструкций, который ещё не собран в исполняемый файл.

В Windows объектные файлы имеют расширение `.OBJ`, в Unix-подобных системах — `.o`.

В GCC их можно получить с флагом `-c`, означающим «только компиляция»:

```
gcc -c foo.c  # produces foo.o
gcc -c bar.c  # produces bar.o
```

Затем свяжем их в один исполняемый файл:

```
gcc -o foo foo.o bar.o
```

Вуаля: из двух объектных файлов получен исполняемый файл `foo`.

Но зачем утруждаться? Разве нельзя просто сделать так:

```
gcc -o foo foo.c bar.c
```

и убить двух зайцев одним выстрелом?

Для маленьких программ это нормально. Я сам постоянно так делаю.

Но в крупных проектах можно использовать тот факт, что компиляция исходных файлов в объектные сравнительно медленна, а связывание большого числа объектных файлов происходит сравнительно быстро.

Особенно хорошо это видно при использовании утилиты `make`, которая пересобирает только исходные файлы, изменённые после создания соответствующих результатов.

Допустим, у вас тысяча файлов C. Сначала можно скомпилировать их все в объектные файлы — медленно, — а затем объединить объектные файлы в исполняемую программу — быстро.

Теперь допустим, что вы изменили только один файл C. Вот где начинается волшебство: нужно пересобрать только соответствующий ему объектный файл! Затем заново собрать исполняемый файл, что происходит быстро. Все остальные файлы C трогать не нужно.

Иными словами, пересобирая только необходимые объектные файлы, мы существенно сокращаем время компиляции. Разумеется, при «чистой» сборке приходится создавать все объектные файлы заново.

Глава 18. Внешнее окружение

Когда вы запускаете программу, оболочка просит операционную систему создать процесс и выполнить её. Но вне программы существует целый мир, с которым можно взаимодействовать из C. Рассмотрим несколько способов.

18.1. Аргументы командной строки

Многие утилиты принимают аргументы. Например, в Unix-подобной системе команда

```
ls *.txt
```

покажет файлы с окончанием `.txt` (в Windows аналогичная команда — `dir`). Исторически Unix-оболочка раскрывает шаблон до передачи программе, а программы Windows часто получают сам шаблон и обрабатывают его самостоятельно.

Допустим, программа `add` складывает переданные числа:

```
./add 10 30 5
45
```

Сначала узнаем, как получить аргументы. Для этого понадобится новая форма `main()`. Программа `foo` выведет все аргументы:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    for (int i = 0; i < argc; i++) {
        printf("arg %d: %s\n", i, argv[i]);
    }
}
```

Вызов `./foo i like turtles` даст:

```
arg 0: ./foo
arg 1: i
arg 2: like
arg 3: turtles
```

Нулевой аргумент обычно содержит имя программы; остальные следуют за ним. `argc` — число аргументов, а `argv` — массив строк. Цикл проходит по массиву и выводит строки. Эти имена параметров не обязательны, но настолько привычны, что другие имена удивят программистов C.

Теперь напишем сумматор: пропустим `argv[0]`, преобразуем остальные строки в числа, сложим и выведем результат.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    int total = 0;
    for (int i = 1; i < argc; i++) { // Начинаем с первого аргумента
        int value = atoi(argv[i]); // strtol() лучше обрабатывает ошибки
        total += value;
    }
    printf("%d\n", total);
}
```

Примеры:

```
$ ./add
0
$ ./add 1
1
$ ./add 1 2
3
$ ./add 1 2 3
6
$ ./add 1 2 3 4
10
```

Некорректный ввод и переполнение требуют дополнительной обработки; попробуйте добавить её самостоятельно.

18.1.1. Последний элемент argv равен NULL

После последней строки массива находится нулевой указатель:

```
argv[argc] == NULL
```

Это всегда верно и иногда оказывается полезным.

18.1.2. Альтернативная запись: char **argv

В объявлении параметра функции запись массива эквивалентна записи указателя:

```
void foo(char a[])
void foo(char *a)
```

Поэтому эти формы `main` тоже эквивалентны:

```
int main(int argc, char *argv[])
int main(int argc, char **argv)
```

Да, это указатель на указатель: можно представить его как указатель на строку, хотя точнее он указывает на значение, которое само указывает на `char`. Поскольку `argv[i]` эквивалентно `*(argv + i)`, можно проходить аргументы арифметикой указателей до завершающего `NULL`:

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    int total = 0;

    // Подавить предупреждение о неиспользуемом argc:
    (void)argc;

    for (char **p = argv + 1; *p != NULL; p++) {
        int value = atoi(*p); // strtol() лучше обрабатывает ошибки
        total += value;
    }
    printf("%d\n", total);
}
```

Автор предпочитает индексирование `argv`, но такой стиль тоже встречается.

18.1.3. Ещё несколько фактов

- В некоторых окружениях имя программы недоступно; тогда при наличии `argv[0]` эта строка может быть пустой.
- Стандарт оставляет реализации свободу в том, откуда берутся аргументы и как они формируются, хотя на распространённых системах они работают описанным образом.
- Можно изменять `argc`, `argv` и строки, на которые указывает `argv`; не делайте строки длиннее выделенного места.
- На некоторых Unix-подобных системах изменение `argv[0]` влияет на вывод команды `ps`, показывающей процессы. Например, при исходном выводе `4078 tty1 S 0:00 ./foo` вызов `strcpy(argv[0], "Hi! ");` может изменить отображаемое имя на `Hi!`. Новая строка должна помещаться в исходное место. Это поведение не определено стандартом C и сильно зависит от системы.

18.2. Код завершения

Почему `main()` возвращает `int`? Это код завершения, передаваемый программе, которая запустила вашу. По нему запускающая программа может узнать результат работы.

Программа может завершиться, например, через `return` из `main()` или вызов одной из функций семейства `exit()`. Если выполнение дошло до конца `main()` без явного `return`, C автоматически возвращает 0. Для обычных функций такой специальной оговорки нет.

Обычно оболочка запускает программу, ждёт её завершения, затем принимает следующую команду. Между завершением программы и продолжением работы оболочки передаётся статус, который обычно обозначает успех или вид ошибки.

В `<stdlib.h>` определены переносимые значения:

Статус	Значение
`EXIT_SUCCESS` или `0`	Успешное завершение
`EXIT_FAILURE`	Завершение с ошибкой

Программа умножения требует ровно два аргумента:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    if (argc != 3) {
        printf("usage: mult x y\n");
        return EXIT_FAILURE;
    }
    printf("%d\n", atoi(argv[1]) * atoi(argv[2]));
    return 0; // Успех, как EXIT_SUCCESS
}
```

Примеры:

```
$ ./mult
usage: mult x y
$ ./mult 3 4 5
usage: mult x y
$ ./mult 3 4
12
```

Сама оболочка не обязана выводить статус. В Bash или другой POSIX-оболочке посмотреть его можно через `echo $?`; в Windows CMD — `echo %errorlevel%`, в PowerShell — `$LastExitCode`:

```
$ ./mult
usage: mult x y
$ echo $?
1
$ ./mult 3 4
12
$ echo $?
0
```

В системе автора `EXIT_FAILURE` равно 1. Стандарт не требует именно этого численного значения.

18.2.1. Другие коды завершения

В Unix обычно ноль означает успех, а ненулевой код — ошибку. Разные ненулевые значения позволяют обозначить разные ошибки. Bash работает с кодами от 0 до 255, причём некоторые значения зарезервированы. На Linux при передаче значения за пределами этого диапазона доступны младшие восемь битов (побитовое И с `0xff`). Это уже

особенности среды, а не общая гарантия стандарта C. Сценарии оболочки могут использовать статус, чтобы решить, что делать дальше.

18.3. Переменные окружения

C не определяет устройство переменных окружения, поэтому ниже описана модель, распространённая на основных платформах. Оболочка и другие запускающие программы могут передавать дочерней программе значения в виде пар имени и строки. В Bash команда `set` показывает переменные оболочки; дочерним процессам передаются экспортированные переменные окружения. Например:

```
HISTFILE=/home/beej/.bash_history
HISTFILESIZE=500
HISTSIZE=500
HOME=/home/beej
HOSTNAME=FBILAPTOP
HOSTTYPE=x86_64
IFS=$' \t\n'
```

Например, имя `HOSTTYPE` связано со строкой `x86_64`. С точки зрения C значения являются строками, даже если выглядят как числа; для получения числа используйте, например, `atoi()` или `strtol()`.

Стандартная функция `getenv()` ищет переменную и возвращает указатель на строку значения либо `NULL`, если переменной нет:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *val = getenv("FROTZ");
    if (val == NULL) {
        printf("Cannot find the FROTZ environment variable\n");
        return EXIT_FAILURE;
    }
    printf("Value: %s\n", val);
}
```

Если переменная ещё не задана, программа сообщит об этом. В Bash её можно экспортировать:

```
export FROTZ="C is awesome!"
./foo
```

Вывод:

```
Value: C is awesome!
```

В CMD используется `set FROTZ=value`, в PowerShell — `$Env:FROTZ="value"`. Таким образом программа может менять поведение в зависимости от окружения.

18.3.1. Изменение переменных окружения

Стандарт C не задаёт общего способа изменения окружения, но многие системы предоставляют свои функции. В Unix-подобных системах изучите `putenv()`, `setenv()` и `unsetenv()`; в Windows — `_putenv()`.

18.3.2. Другие способы в Unix-подобных системах

В POSIX-системах часто доступна переменная `environ`. Это расширение, не входящее в стандарт C:

```
extern char **environ;
```

Она указывает на массив строк, заканчивающийся `NULL`. Её можно объявить самостоятельно; в некоторых системах объявление доступно в нестандартном для C заголовке `<unistd.h>`. Каждая строка имеет вид `"key=value"`, поэтому для получения ключа и значения её нужно разобрать.

```
#include <stdio.h>

extern char **environ; // Обязательно extern и имя environ

int main(void)
{
    for (char **p = environ; *p != NULL; p++) {
        printf("%s\n", *p);
    }

    // Или так:
    for (int i = 0; environ[i] != NULL; i++) {
        printf("%s\n", environ[i]);
    }
}
```

Пример вывода:

```
SHELL=/bin/bash
COLORTERM=truecolor
TERM_PROGRAM_VERSION=1.53.2
LOGNAME=beej
HOME=/home/beej
...
```

По возможности используйте более переносимую `getenv()`. Если нужно перебрать всё окружение, может пригодиться `environ`.

Ещё один распространённый, но менее переносимый способ — третий параметр `main()`. Он позволяет обойтись без отдельного `extern`, но не является стандартной формой C:

```
#include <stdio.h>

int main(int argc, char **argv, char **env)
```

```
{
    (void)argc; (void)argv; // Подавить предупреждения

    for (char **p = env; *p != NULL; p++) {
        printf("%s\n", *p);
    }

    // Или так:
    for (int i = 0; env[i] != NULL; i++) {
        printf("%s\n", env[i]);
    }
}
```

Это похоже на использование `environ`, но ещё менее переносимо.

Глава 19. Препроцессор C

До компиляции программа проходит этап, называемый препроцессированием. Почти как если бы поверх языка C существовал другой язык, который выполняется первым. Он выдаёт код C, который затем компилируется.

Мы уже немного видели это в действии с `#include`! Это и есть препроцессор C: встретив директиву, он включает указанный файл прямо на её месте, словно вы напечатали его там сами. А затем компилятор собирает результат.

Но возможности препроцессора гораздо шире простого включения файлов. Можно определять макросы, которые будут заменены другим текстом, и даже макросы с аргументами!

19.1. #include

Начнём с уже знакомой директивы. Она включает другие файлы в исходный код и очень часто используется для заголовочных файлов.

Стандарт допускает разные варианты поведения `#include`, но рассмотрим практический вариант, встречающийся автору на всех знакомых системах.

Условно разделим заголовочные файлы на системные и локальные. Встроенные заголовки, такие как `stdio.h`, `stdlib.h`, `math.h`, обычно подключают в угловых скобках:

```
#include <stdio.h>
#include <stdlib.h>
```

Скобки сообщают компилятору, что заголовок следует искать в системных каталогах. Для локальных файлов используют двойные кавычки:

```
#include "myheader.h"
```

Как правило, можно указывать и относительные пути, используя прямые косые черты и точки:

```
#include "mydir/myheader.h"
#include "../someheader.py"
```

Не используйте обратную косую черту (`\`) как разделитель пути в `#include`: это неопределённое поведение. Используйте прямую (`/`), даже в Windows.

Итак, для системных заголовков используйте угловые скобки, а для собственных — двойные кавычки.

19.2. Простые макросы

Макрос — это идентификатор, который заменяется другим фрагментом кода ещё до того, как компилятор увидит программу. Считайте его заполнителем: встретив идентификатор, препроцессор подставляет определённый вами текст.

Макросы задаются директивой `#define`. Например:

```
#include <stdio.h>

#define HELLO "Hello, world"
#define PI 3.14159

int main(void)
{
    printf("%s, %f\n", HELLO, PI);
}
```

Мы определили два макроса. В остальных местах кода, где они встретятся, вместо них будут подставлены заданные значения. С точки зрения компилятора это то же самое, что написать:

```
#include <stdio.h>

int main(void)
{
    printf("%s, %f\n", "Hello, world", 3.14159);
}
```

Видите, как `HELLO` заменился строкой `"Hello, world"`, а `PI` — числом `3.14159`? Для компилятора эти значения словно изначально стояли в коде.

Сами по себе макросы не имеют определённого типа. Они целиком заменяются текстом из `#define`. Если получившийся код C неверен, компилятор выдаст ошибку.

Макрос можно определить и без значения:

```
#define EXTRA_HAPPY
```

В этом случае макрос существует и определён, но его замена пуста. Встретив его в тексте, препроцессор заменит его ничем. Позже мы увидим, зачем это нужно.

Имена макросов принято писать ПРОПИСНЫМИ_БУКВАМИ, хотя технически это необязательно.

Таким образом можно задавать постоянные значения, фактически доступные глобально, в том числе там, где переменная `const` не подойдёт: например, в метках `case` и размерах массивов фиксированной длины. Впрочем, в интернете не прекращаются споры о том, что в общем случае лучше: типизированная переменная `const` или макрос `#define`.

Макросы могут также заменять или изменять ключевые слова — переменные `const` на это не способны. Однако такую практику следует применять осторожно.

19.3. Условная компиляция

Препроцессор может решать, передавать ли определённые блоки кода компилятору или полностью удалять их до компиляции. Для этого код заключают в условные блоки, похожие на конструкции `if`–`else`.

19.3.1. Если определено: #ifdef и #endif

Попробуем компилировать код в зависимости от того, определён ли макрос:

```
#include <stdio.h>

#define EXTRA_HAPPY

int main(void)
{
#ifdef EXTRA_HAPPY
    printf("I'm extra happy!\n");
#endif

    printf("OK!\n");
}
```

Мы определили `EXTRA_HAPPY` как пустой макрос, затем проверили наличие определения через `#ifdef`. Если оно есть, код до `#endif` включается в компиляцию. В данном случае программа выведет:

```
I'm extra happy!
OK!
```

Если закомментировать определение:

```
//#define EXTRA_HAPPY
```

макрос не будет определён, и соответствующий код не попадёт в компиляцию. Вывод будет состоять только из `OK!`.

Важно помнить, что решение принимается во время компиляции: код включается или удаляется. Обычный оператор `if`, напротив, вычисляется во время выполнения программы.

19.3.2. Если не определено: #ifndef

Директива `#ifndef` проверяет обратное условие: макрос *не* определён. Можно изменить предыдущий пример и выводить разные сообщения:

```
#ifdef EXTRA_HAPPY
    printf("I'm extra happy!\n");
```

```
#endif

#ifndef EXTRA_HAPPY
    printf("I'm just regular\n");
#endif
```

В следующем разделе мы увидим более чистый способ. А пока вспомним, как защитить заголовочный файл от повторного включения:

```
#ifndef MYHEADER_H  // First line of myheader.h
#define MYHEADER_H

int x = 12;

#endif  // Last line of myheader.h
```

Макрос сохраняет своё определение при переходе между файлами и несколькими `#include`. При первом включении он ещё не определён: мы определяем его и передаём содержимое компилятору. При следующем включении `MYHEADER_H` уже существует, поэтому содержимое заголовка не передаётся компилятору.

19.3.3. #else

В условный блок можно добавить `#else`:

```
#ifdef EXTRA_HAPPY
    printf("I'm extra happy!\n");
#else
    printf("I'm just regular\n");
#endif
```

Если `EXTRA_HAPPY` не определён, будет выбран блок `#else` и выведено `I'm just regular`.

19.3.4. Иначе если: #elifdef и #elifndef

Эти директивы появились в C23! Они пригодятся для более сложной цепочки условий. `#elifdef` означает «иначе, если определено»:

```
#ifdef MODE_1
    printf("This is mode 1\n");
#elifdef MODE_2
    printf("This is mode 2\n");
#elifdef MODE_3
    printf("This is mode 3\n");
#else
    printf("This is some other mode\n");
#endif
```

Аналогично `#elifndef` означает «иначе, если не определено».

19.3.5. Общее условие: #if и #elif

Эти директивы похожи на `#ifdef` и `#ifndef`: можно использовать `#else`, а завершает конструкцию `#endif`. Отличие в том, что после `#if` записывается константное выражение. Код включается, если оно истинно, то есть не равно нулю.

```
#include <stdio.h>

#define HAPPY_FACTOR 1

int main(void)
{
#if HAPPY_FACTOR == 0
    printf("I'm not happy!\n");
#elif HAPPY_FACTOR == 1
    printf("I'm just regular\n");
#else
    printf("I'm extra happy!\n");
#endif

    printf("OK!\n");
}
```

Компилятор даже не увидит ветви с невыполненными условиями. После препроцессирования останется:

```
#include <stdio.h>

int main(void)
{
    printf("I'm just regular\n");
    printf("OK!\n");
}
```

Один не слишком изящный, но удобный приём — быстро отключить большой участок кода, обернув его в `#if 0` и `#endif`:

```
#if 0
    printf("All this code");      /* is effectively */
    printf("commented out");     // by the #if 0
#endif
```

Обычные комментарии `/* ... */` для этого подходят не всегда, потому что они не могут вкладываться друг в друга.

Что делать, если компилятор до C23 не поддерживает `#elifdef` и `#elifndef`? Препроцессор предоставляет оператор `defined`, который можно применять в `#if`. Следующие записи эквивалентны:

```
#ifdef FOO
#if defined FOO
#if defined(FOO)  // Parentheses optional
```

А эти проверяют отсутствие определения:

```
#ifndef FOO
#if !defined FOO
#if !defined(FOO)  // Parentheses optional
```

Здесь для отрицания используется обычный логический оператор `!`. Поэтому код с неподдерживаемым `#elifdef`:

```
#ifdef FOO
    x = 2;
#elifdef BAR  // POTENTIAL ERROR: Not supported before C23
    x = 3;
#endif
```

можно заменить следующим:

```
#if defined FOO
    x = 2;
#elif defined BAR
    x = 3;
#endif
```

19.3.6. Удаление макроса: #undef

Когда определённый макрос больше не нужен, его можно отменить директивой `#undef`:

```
#include <stdio.h>

int main(void)
{
#define GOATS

#ifdef GOATS
    printf("Goats detected!\n");  // prints
#endif

#undef GOATS  // Make GOATS no longer defined

#ifdef GOATS
    printf("Goats detected, again!\n");  // doesn't print
#endif
}
```

19.4. Встроенные макросы

Стандарт определяет множество встроенных макросов, которые можно проверять и использовать при условной компиляции.

19.4.1. Обязательные макросы

Следующие макросы определены:

Макрос	Значение
`__DATE__`	Дата компиляции файла в формате `Mmm dd yyyy`
`__TIME__`	Время компиляции в формате `hh:mm:ss`
`__FILE__`	Строка с именем файла
`__LINE__`	Номер строки файла, в которой расположен макрос
`__func__`	Имя текущей функции в виде строки
`__STDC__`	Равно 1 для соответствующего стандарту компилятора C
`__STDC_HOSTED__`	1 для размещённой реализации, иначе 0
`__STDC_VERSION__`	Версия C, константа `long int` вида `yyyymmL`, например `201710L`

Строго говоря, `__func__` — не макрос, а предварительно определённый идентификатор, но он настолько похож на макрос, что включён сюда. Размещённая реализация обычно означает полное окружение C под операционной системой; встроенная система без ОС может использовать автономную реализацию.

Соберём их вместе:

```
#include <stdio.h>

int main(void)
{
    printf("This function: %s\n", __func__);
    printf("This file: %s\n", __FILE__);
    printf("This line: %d\n", __LINE__);
    printf("Compiled on: %s %s\n", __DATE__, __TIME__);
    printf("C Version: %ld\n", __STDC_VERSION__);
}
```

На системе автора программа вывела:

```
This function: main
This file: foo.c
This line: 7
Compiled on: Nov 23 2020 17:16:27
C Version: 201710
```

`__FILE__`, `__func__` и `__LINE__` особенно полезны в сообщениях разработчикам об ошибках. Макрос `assert()` из `<assert.h>` использует такую информацию для указания места неудачной проверки.

Версии STDC_VERSION

Для основных выпусков стандарта автор приводит следующие значения:

Выпуск	Версия ISO/IEC	`__STDC_VERSION__`
C89	ISO/IEC 9899:1990	не определён
C89 с поправкой	ISO/IEC 9899:1990/Amd.1:1995	`199409L`
C99	ISO/IEC 9899:1999	`199901L`
C11	ISO/IEC 9899:2011/Amd.1:2012	`201112L`

Изначально в C89 этого макроса не было. Значения версий должны строго возрастать, поэтому можно проверять минимальную версию. В оригинале пример проверки выглядит так:

```
#if __STDC_VERSION__ >= 1999901L
```

Обратите внимание: в приведённой константе, вероятно, опечатка; для проверки C99 нужно сравнивать с `199901L`.

19.4.2. Необязательные макросы

Реализация может определять следующие макросы, но не обязана:

Макрос	Значение
`__STDC_ISO_10646__`	Если определён, `wchar_t` содержит значения Unicode.
`__STDC_MB_MIGHT_NEQ_WC__`	Значение 1 означает, что значения многобайтовых и широких символов могут не совпадать.
`__STDC_UTF_16__`	1 означает использование UTF-16 в `char16_t`.
`__STDC_UTF_32__`	1 означает использование UTF-32 в `char32_t`.
`__STDC_ANALYZABLE__`	1 означает поддержку расширения, ограничивающего некоторые виды неопределённого поведения для статического анализа.
`__STDC_IEC_559__`	1 означает поддержку чисел с плавающей точкой IEEE-754 / IEC 60559.
`__STDC_IEC_559_COMPLEX__`	1 означает поддержку комплексных чисел IEC 60559.
`__STDC_LIB_EXT1__`	1 означает поддержку дополнительных «безопасных» функций библиотеки с суффиксом `_s`.
`__STDC_NO_ATOMICS__`	1 означает отсутствие поддержки `_Atomic` или `<stdatomic.h>`.
`__STDC_NO_COMPLEX__`	1 означает отсутствие комплексных типов или `<complex.h>`.
`__STDC_NO_THREADS__`	1 означает отсутствие поддержки `<threads.h>`.
`__STDC_NO_VLA__`	1 означает отсутствие поддержки массивов переменной длины.

19.5. Макросы с аргументами

Макросы могут не только заменять простой текст: им можно передавать аргументы, которые также будут подставлены.

Часто возникает вопрос, когда использовать параметризованные макросы вместо функций. Короткий ответ: используйте функции. Но макросов много в существующем коде и стандартной библиотеке. Их часто применяют для коротких математических выражений и возможностей, различающихся между платформами.

19.5.1. Макрос с одним аргументом

Начнём с простого макроса возведения в квадрат:

```
#include <stdio.h>

#define SQR(x) x * x  // Not quite right, but bear with me

int main(void)
{
    printf("%d\n", SQR(12));  // 144
}
```

Его смысл: «Везде, где встретится `SQR` со значением, подставь это значение, умноженное на себя». Вызов превращается в:

```
printf("%d\n", 12 * 12);  // 144
```

Но в макросе есть распространённая ошибка. Что будет при `SQR(3 + 4)`? Мы рассчитываем получить 7² = 49, а получаем 19!

```
printf("%d\n", SQR(3 + 4));  // 19!!??
```

Посмотрим на подстановку:

```
printf("%d\n", 3 + 4 * 3 + 4);  // 19!
```

Умножение имеет более высокий приоритет, поэтому сначала вычисляется 4 × 3, и получается 3 + 12 + 4 = 19. Не то, чего мы хотели.

Это настолько распространённая проблема, что при создании математических макросов с параметрами стоит автоматически заключать каждый параметр в скобки:

```
#define SQR(x) (x) * (x)  // Better... but still not quite good enough!
```

Теперь выражение раскрывается в `(3 + 4) * (3 + 4)` и даёт 49. Но проблема может снова возникнуть, если рядом окажется оператор с приоритетом выше умножения. Поэтому безопаснее заключить в скобки и весь результат:

```
#define SQR(x) ((x) * (x))  // Good!
```

Возьмите это за привычку при написании математических макросов. Также помните, что аргумент здесь подставляется дважды: выражения с побочными эффектами требуют особой осторожности.

19.5.2. Макросы с несколькими аргументами

Аргументов может быть несколько:

```
#define TRIANGLE_AREA(w, h) (0.5 * (w) * (h))
```

Напишем макросы, которые решают квадратное уравнение. Напомним: для уравнения `a*x² + b*x + c = 0` корни находятся по формуле `x = (-b ± sqrt(b² - 4ac)) / (2a)`. Знак ± означает два решения. Создадим макрос для каждого:

```
#define QUADP(a, b, c) ((-(b) + sqrt((b) * (b) - 4 * (a) * (c))) / (2 * (a)))
#define QUADM(a, b, c) ((-(b) - sqrt((b) * (b) - 4 * (a) * (c))) / (2 * (a)))
```

Теперь создадим ещё один макрос для вывода обоих ответов через `printf()`:

```
// macro replacement
//      |-----------|  |---------------------------|
#define QUAD(a, b, c) QUADP(a, b, c), QUADM(a, b, c)
```

Он раскрывается в два значения через запятую. Используем его как своего рода объединённый аргумент:

```
printf("x = %f or x = %f\n", QUAD(2, 10, 5));
```

Полная программа:

```
#include <stdio.h>
#include <math.h>  // For sqrt()

#define QUADP(a, b, c) ((-(b) + sqrt((b) * (b) - 4 * (a) * (c))) / (2 * (a)))
#define QUADM(a, b, c) ((-(b) - sqrt((b) * (b) - 4 * (a) * (c))) / (2 * (a)))
#define QUAD(a, b, c) QUADP(a, b, c), QUADM(a, b, c)

int main(void)
{
    printf("2*x^2 + 10*x + 5 = 0\n");
    printf("x = %f or x = %f\n", QUAD(2, 10, 5));
}
```

Вывод:

```
2*x^2 + 10*x + 5 = 0
x = -0.563508 or x = -4.436492
```

Подстановка любого корня даёт приблизительно ноль; небольшая погрешность возникает из-за округления: `2 × (-0.563508)² + 10 × (-0.563508) + 5 ≈ 0.000003`.

19.5.3. Макросы с переменным числом аргументов

После известных именованных аргументов можно поставить многоточие (`...`), позволяющее передать переменное число дополнительных аргументов. При раскрытии они попадают в `__VA_ARGS__` как список через запятую:

```
#include <stdio.h>

// Combine the first two arguments to a single number,
// then have a commalist of the rest of them:
#define X(a, b, ...) (10*(a) + 20*(b)), __VA_ARGS__

int main(void)
{
    printf("%d %f %s %d\n", X(5, 4, 3.14, "Hi!", 12));
}
```

Подстановка в вызове даёт:

```
printf("%d %f %s %d\n", (10*(5) + 20*(4)), 3.14, "Hi!", 12);
```

Результат:

```
130 3.140000 Hi! 12
```

Можно также превратить `__VA_ARGS__` в строку, поставив перед ним `#`:

```
#define X(...) #__VA_ARGS__
printf("%s\n", X(1,2,3));  // Prints "1, 2, 3"
```

19.5.4. Преобразование в строку

Как уже упоминалось, любой аргумент макроса можно превратить в строку, поставив перед ним `#` в тексте замены:

```
#define STR(x) #x
printf("%s\n", STR(3.14159));
```

После подстановки получаем:

```
printf("%s\n", "3.14159");
```

Применим это для вывода имени и значения любой переменной `int`:

```
#include <stdio.h>
```

```
#define PRINT_INT_VAL(x) printf("%s = %d\n", #x, x)

int main(void)
{
    int a = 5;

    PRINT_INT_VAL(a);  // prints "a = 5"
}
```

Последняя строка раскрывается так:

```
printf("%s = %d\n", "a", a);
```

19.5.5. Объединение

С помощью `##` можно склеить два аргумента:

```
#define CAT(a, b) a ## b
printf("%f\n", CAT(3.14, 1592));  // 3.141592
```

19.6. Многострочные макросы

Макрос можно продолжить на следующих строках, экранируя перевод строки обратной косой чертой (`\`). Напишем макрос, который выводит числа от нуля до произведения двух аргументов, не включая само произведение:

```
#include <stdio.h>

#define PRINT_NUMS_TO_PRODUCT(a, b) do { \
    int product = (a) * (b);             \
    for (int i = 0; i < product; i++) {  \
        printf("%d\n", i);               \
    }                                    \
} while(0)

int main(void)
{
    PRINT_NUMS_TO_PRODUCT(2, 4);  // Outputs numbers from 0 to 7
}
```

Обратите внимание: обратная косая черта стоит в конце каждой строки, кроме последней, показывая продолжение макроса. Весь код также заключён в цикл `do ... while(0)` с фигурными скобками.

Вторая особенность может показаться странной, но она нужна, чтобы корректно обработать точку с запятой, которую программист ставит после вызова макроса.

Одних фигурных скобок недостаточно. Посмотрите на этот пример:

```
#include <stdio.h>

#define FOO(x) { (x)++; }
```

```
int main(void)
{
    int i = 0;

    if (i == 0)
        FOO(i);
    else
        printf(":-(\n");

    printf("%d\n", i);
}
```

На первый взгляд всё просто, но при компиляции возникает синтаксическая ошибка: `else` без предшествующего `if`. Вот во что раскрывается код:

```
if (i == 0) {
    (i)++;
};  // <-- Trouble with a capital-T!
else
    printf(":-(\n");
```

Точка с запятой завершает оператор `if`, и `else` оказывается отдельно. Поэтому многострочный макрос заключают в `do ... while(0)`.

19.7. Пример: макрос проверки утверждения

Проверки утверждений помогают обнаруживать условия, которые, по вашему мнению, не должны возникать. В C для этого есть `assert()`: если условие ложно, программа завершается с указанием файла и номера строки.

Но этого недостаточно: нельзя указать дополнительное сообщение и нет простого переключателя для всех проверок. Первую проблему можно решить макросами.

Если написано:

```
ASSERT(x < 20, "x must be under 20");
```

то хотелось бы получить примерно следующее, предполагая, что вызов находится в строке 220 файла `foo.c`:

```
if (!(x < 20)) {
    fprintf(stderr, "foo.c:220: assertion x < 20 failed: ");
    fprintf(stderr, "x must be under 20\n");
    exit(1);
}
```

Имя файла получим из `__FILE__`, номер строки — из `__LINE__`. Сообщение уже является строкой, а выражение `x < 20` нужно превратить в строку с помощью `#`. Обратные косые черты позволят сделать макрос многострочным:

```
#define ASSERT(c, m) \
    do { \
        if (!(c)) { \
            fprintf(stderr, __FILE__ ":%d: assertion %s failed: %s\n", \
                    __LINE__, #c, m); \
            exit(1); \
        } \
    } while(0)
```

`__FILE__` перед строковым литералом выглядит необычно, но это тоже строковый литерал, а соседние литералы автоматически объединяются. `__LINE__`, напротив, имеет тип `int`.

Если вызвать `ASSERT(x < 20, "x must be under 20")` при `x = 30`, получится сообщение вида:

```
foo.c:23: assertion x < 20 failed: x must be under 20
```

Осталось сделать переключатель. Для этого подойдёт условная компиляция:

```
#include <stdio.h>
#include <stdlib.h>

#define ASSERT_ENABLED 1

#if ASSERT_ENABLED
#define ASSERT(c, m) \
    do { \
        if (!(c)) { \
            fprintf(stderr, __FILE__ ":%d: assertion %s failed: %s\n", \
                    __LINE__, #c, m); \
            exit(1); \
        } \
    } while(0)
#else
#define ASSERT(c, m)  // Empty macro if not enabled
#endif

int main(void)
{
    int x = 30;

    ASSERT(x < 20, "x must be under 20");
}
```

При включённых проверках вывод снова будет таким:

```
foo.c:23: assertion x < 20 failed: x must be under 20
```

19.8. Директива #error

Эта директива вызывает ошибку компиляции. Обычно её помещают внутрь условия, чтобы предотвратить сборку при невыполнении необходимых требований:

```
#ifndef __STDC_IEC_559__
#error I really need IEEE-754 floating point to compile. Sorry!
```

```
#endif
```

Некоторые компиляторы также поддерживают нестандартную директиву `#warning`, которая выводит предупреждение, но не останавливает компиляцию; в стандарт C11 она не входит.

19.9. Директива #embed

Это нововведение C23. Автор отмечает, что на момент написания раздела его компиляторы ещё не поддерживали эту директиву, поэтому советует относиться к изложению с осторожностью.

Суть в том, что байты файла можно включить в код как целочисленные константы, словно вы написали их вручную. Допустим, двоичный файл `foo.bin` содержит четыре байта со значениями 11, 22, 33 и 44:

```
int a[] = {
#embed "foo.bin"
};
```

Это будет эквивалентно записи:

```
int a[] = {11,22,33,44};
```

Так можно инициализировать массив двоичными данными без предварительного ручного преобразования в код: препроцессор сделает это сам. Например, можно включить небольшой файл изображения, чтобы не загружать его во время выполнения.

Допустимы также угловые скобки:

```
int a[] = {
#embed <foo.bin>
};
```

В этом случае препроцессор ищет файл в местах, определяемых реализацией. При использовании кавычек, если ресурс не найден, выполняется поиск как для угловых скобок.

Подобно `#include`, директива фактически вставляет значения до обработки компилятором, поэтому её можно использовать в разных местах:

```
return
#embed "somevalue.dat"
;

int x =
#embed "xvalue.dat"
;
```

Всегда ли это байты со значениями от 0 до 255? Обычно да, но точнее ширина элемента равна `CHAR_BIT` битам. На большинстве систем это 8, поэтому диапазон будет от 0 до 255.

Значения всегда неотрицательны. Реализация может предоставлять способы изменить размер элемента, например через параметры или командную строку.

Размер файла в битах должен быть кратен размеру элемента. Для восьмибитного элемента это просто означает целое число байтов.

19.9.1. Параметры #embed

У `#embed` есть параметры. Например, пока ещё не рассмотренный `limit()`:

```
int a[] = {
#embed "/dev/random" limit(5)
};
```

Если `limit` уже определён где-то ещё, можно окружить имя двойными подчёркиваниями — смысл останется тем же:

```
int a[] = {
#embed "/dev/random" __limit__(5)
};
```

19.9.2. Параметр limit()

Он задаёт максимальное количество включаемых элементов. Если файл короче указанного предела, будут включены только имеющиеся байты.

В Unix `/dev/random` — символьное устройство, которое может выдавать практически бесконечный поток случайных чисел. Включать бесконечное число байтов в программу тяжело для оперативной памяти, поэтому `limit` позволяет остановиться. В выражении предела разрешено использовать макросы `#define`.

19.9.3. Параметр if_empty

Этот параметр задаёт результат на случай, когда файл существует, но пуст. Пусть `foo.dat` содержит один байт со значением 123:

```
int x =
#embed "foo.dat" if_empty(999)
;
```

Получится:

```
int x = 123;  // When foo.dat contains a 123 byte
```

А если файл пуст, подставится:

```
int x = 999;  // When foo.dat is empty
```

Если предел `limit` равен нулю, будет всегда выбран `if_empty`, независимо от содержимого файла:

```
int x =
#embed "foo.dat" limit(0) if_empty(999)
;
```

19.9.4. Параметры prefix() и suffix()

Эти параметры добавляют текст до и после включаемых данных. Они применяются только к непустым данным: если файл пуст, ни один из них не действует.

Например, включим три случайных числа, добавив перед ними 11, а после них — 99:

```
int x[] = {
#embed "/dev/urandom" limit(3) prefix(11,) suffix(,99)
};
```

Один из возможных результатов:

```
int x[] = {11,135,116,220,99};
```

Использовать оба параметра необязательно: можно выбрать один, оба или ни одного.

То, что они применяются только к непустым файлам, можно использовать интересным образом. Предположим, мы хотим инициализировать массив данными `foo.dat` и добавить нулевой элемент:

```
int x[] = {
#embed "foo.dat" suffix(,0)
};
```

Если файл содержит 11, 22 и 33, получится `int x[] = {11,22,33,0};`. Но если файл пуст, результатом будет `int x[] = {};`, что нам не подходит.

Исправим пример:

```
int x[] = {
#embed "foo.dat" suffix(,)
0
};
```

Для пустого файла `suffix` не выводится, поэтому результат — `int x[] = {0};`.

19.9.5. Идентификатор __has_embed()

Он позволяет проверить, доступен ли файл для включения и пуст ли он. Применяется вместе с `#if`.

В следующем примере программа включает пять чисел из устройства случайных чисел. Если оно недоступно, берутся данные из `myrandoms.dat`, а если и тот отсутствует — заданные константы:

```
int random_nums[] = {
#if __has_embed("/dev/urandom")
#embed "/dev/urandom" limit(5)
#elif __has_embed("myrandoms.dat")
#embed "myrandoms.dat" limit(5)
#else
    140,178,92,167,120
#endif
};
```

Технически результатом `__has_embed()` является одно из трёх значений:

Результат	Значение
`__STDC_EMBED_NOT_FOUND__`	Файл не найден.
`__STDC_EMBED_FOUND__`	Файл найден и не пуст.
`__STDC_EMBED_EMPTY__`	Файл найден и пуст.

Автор отмечает, что на момент написания имел основания считать `__STDC_EMBED_NOT_FOUND__` равным нулю, а остальные значения — ненулевыми, но не нашёл подтверждения в использованной версии проекта стандарта.

19.9.6. Другие параметры

Реализация компилятора может определять дополнительные параметры. Ищите их в документации компилятора. Например:

```
#embed "foo.bin" limit(12) frotz(lamp)
#embed "foo.bin" limit(12) fmc::frotz(lamp)
```

Префикс может использоваться для разделения пространств имён. Перед использованием такого параметра разумно проверить его поддержку через `__has_embed()`: кроме отсутствия файла, этот оператор возвращает ложное значение и при неподдерживаемом дополнительном параметре.

Нужен файл, о существовании которого мы знаем заранее. Можно использовать `__FILE__`: он раскрывается в имя текущего исходного файла, который должен существовать. Проверим параметр:

```
#if __has_embed(__FILE__ fmc::frotz(lamp))
    puts("fmc::frotz(lamp) is supported!");
#else
    puts("fmc::frotz(lamp) is NOT supported!");
#endif
```

19.9.7. Включение многобайтовых значений

Можно ли включать не отдельные байты, а, например, многобайтовые значения `int`? Стандарт C23 этого не поддерживает, хотя в будущем реализации могут добавить расширения.

19.10. Директива #pragma

Название этой необычной директивы происходит от слова «прагматический». С её помощью можно делать всё, что компилятор позволяет делать с `#pragma`. Обычно её добавляют, когда этого требует документация.

19.10.1. Нестандартные директивы pragma

Пример нестандартной директивы: параллельное выполнение цикла `for` несколькими потоками, если компилятор поддерживает расширение OpenMP:

```
#pragma omp parallel for
for (int i = 0; i < 10; i++) { ... }
```

Существует множество вариантов `#pragma`, описанных в документации различных компиляторов. Нераспознанные директивы `#pragma` игнорируются.

19.10.2. Стандартные директивы pragma

Существуют и стандартные варианты, начинающиеся с `STDC`:

```
#pragma STDC pragma_name on-off
```

Вместо `on-off` указывается `ON`, `OFF` или `DEFAULT`. Возможные имена:

Имя	Описание
`FP_CONTRACT`	Разрешает объединять выражения с плавающей точкой в одну операцию, чтобы избежать промежуточных ошибок округления.
`FENV_ACCESS`	Установите `ON`, если программа обращается к флагам состояния вычислений с плавающей точкой. При `OFF` оптимизации могут сделать их значения несогласованными или неверными.
`CX_LIMITED_RANGE`	`ON` разрешает пропускать проверки переполнения при комплексной арифметике. По умолчанию — `OFF`.

Например:

```
#pragma STDC FP_CONTRACT OFF
#pragma STDC CX_LIMITED_RANGE ON
```

Для `CX_LIMITED_RANGE` стандарт указывает, что директива позволяет реализации применять следующие формулы, когда программист определил, что они безопасны:

```
(x + iy) * (u + iv) = (xu - yv) + i(yu + xv)
(x + iy) / (u + iv) = [(xu + yv) + i(yu - xv)] / (u² + v²)
|x + iy| = sqrt(x² + y²)
```

19.10.3. Оператор _Pragma

Это другой способ задать pragma, который можно использовать внутри макроса. Следующие формы эквивалентны:

```
#pragma "Unnecessary" quotes
_Pragma("\"Unnecessary\" quotes")
```

При необходимости оператор можно применять в макросе:

```
#define PRAGMA(x) _Pragma(#x)
```

19.11. Директива #line

Она позволяет переопределить значения `__LINE__` и `__FILE__`. Автор отмечает, что лично ему это не требовалось; один из возможных случаев — работа других препроцессоров, генерирующих программы на C.

Чтобы установить номер строки, например 300:

```
#line 300
```

Далее `__LINE__` продолжит отсчёт с этого значения. Чтобы переопределить также имя файла:

```
#line 300 "newfilename"
```

19.12. Пустая директива

Одинокий `#` на отдельной строке игнорируется препроцессором. Честно говоря, автор не знает, зачем это может понадобиться. Встречаются такие примеры:

```
#ifdef FOO
#
#else
    printf("Something");
#endif
```

Это просто оформление: строку с `#` можно удалить без последствий. Возможно, кто-то использует её и для визуальной согласованности:

```
#
#ifdef FOO
    x = 2;
#endif
#
#if BAR == 17
    x = 12;
#endif
#
```

Но, по мнению автора, выглядит это некрасиво. Ещё упоминалось удаление комментариев: в GCC комментарий после `#` не виден компилятору, однако автор не нашёл подтверждения, что стандарт требует такого поведения. Считайте это одной из маленьких загадок C.

Глава 20. Структуры II: ещё больше интересного

Оказывается, со структурами `struct` можно делать гораздо больше, чем мы уже обсудили. Это целый набор разных возможностей, и мы собрали их в этой главе. Если вы освоили основы структур, здесь можно дополнить свои знания.

20.1. Инициализаторы вложенных структур и массивов

Помните, что поля структуры можно инициализировать так?

```
struct foo x = {.a=12, .b=3.14};
```

На самом деле эти инициализаторы предоставляют больше возможностей. Например, поле вложенной структуры можно инициализировать, последовательно указав его имя:

```
struct foo x = {.a.b.c=12};
```

Рассмотрим пример:

```
#include <stdio.h>

struct cabin_information {
    int window_count;
    int o2level;
};

struct spaceship {
    char *manufacturer;
    struct cabin_information ci;
};

int main(void)
{
    struct spaceship s = {
        .manufacturer="General Products",
        .ci.window_count = 8, // <-- NESTED INITIALIZER!
        .ci.o2level = 21
    };

    printf("%s: %d seats, %d%% oxygen\n",
           s.manufacturer, s.ci.window_count, s.ci.o2level);
}
```

Обратите внимание на вложенные инициализаторы: при определении `s` типа `struct spaceship` мы инициализируем поля `struct cabin_information`.

Тот же инициализатор можно записать более привычным образом; оба варианта работают:

```
struct spaceship s = {
    .manufacturer="General Products",
```

```
    .ci={
        .window_count = 8,
        .o2level = 21
    }
};
```

И это ещё не всё: сюда можно добавить и инициализаторы массивов. Добавим массив сведений о пассажирах и посмотрим, как он инициализируется:

```
#include <stdio.h>

struct passenger {
    char *name;
    int covid_vaccinated; // Boolean
};

#define MAX_PASSENGERS 8

struct spaceship {
    char *manufacturer;
    struct passenger passenger[MAX_PASSENGERS];
};

int main(void)
{
    struct spaceship s = {
        .manufacturer="General Products",
        .passenger = {
            // Initialize a field at a time
            [0].name = "Gridley, Lewis",
            [0].covid_vaccinated = 0,

            // Or all at once
            [7] = {.name="Brown, Teela", .covid_vaccinated=1},
        }
    };

    printf("Passengers for %s ship:\n", s.manufacturer);

    for (int i = 0; i < MAX_PASSENGERS; i++)
        if (s.passenger[i].name != NULL)
            printf("    %s (%svaccinated)\n",
                   s.passenger[i].name,
                   s.passenger[i].covid_vaccinated? "": "not ");
}
```

20.2. Анонимные структуры

Это «структуры без имени». Мы уже упоминали их в разделе о `typedef`, но повторим ещё раз.

Вот обычная структура:

```
struct animal {
    char *name;
    int leg_count, speed;
};
```

А вот аналог без имени:

```
struct { // <-- No name!
    char *name;
    int leg_count, speed;
};
```

Хорошо-о-о. Получается, структура есть, но имени нет, и использовать её в дальнейшем невозможно? Кажется бессмысленным. Признаю: в этом примере так и есть. Но применить её всё-таки можно несколькими способами.

Один из них встречается редко: поскольку структура без имени представляет тип, можно просто указать после неё имена переменных и пользоваться ими.

```
struct { // <-- No name!
    char *name;
    int leg_count, speed;
} a, b, c; // 3 variables of this struct type

a.name = "antelope";
c.leg_count = 4; // for example
```

Однако и это не слишком полезно. Гораздо чаще безымянные структуры используют с `typedef`, чтобы обращаться к типу позже, например при передаче переменных функциям:

```
typedef struct { // <-- No name!
    char *name;
    int leg_count, speed;
} animal; // New type: animal

animal a, b, c;

a.name = "antelope";
c.leg_count = 4; // for example
```

Лично я редко использую такие структуры. Мне приятнее видеть полное `struct animal` перед именем переменной в объявлении. Но это всего лишь моё мнение.

20.3. Самоссылочные структуры

Для графоподобных структур данных полезно хранить указатели на связанные узлы. Но тогда внутри определения узла нужен указатель на узел. Проблема курицы и яйца!

Оказывается, в C это делается без затруднений. Вот, например, узел связного списка:

```
struct node {
    int data;
    struct node *next;
};
```

Важно, что `next` — указатель. Благодаря этому объявление возможно, хотя на этом месте `struct node` ещё не является полностью определённым типом: для указателя не требуется

знать полный размер структуры.

Вот незамысловатая программа со связным списком:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

int main(void)
{
    struct node *head;

    // Hackishly set up a linked list (11)->(22)->(33)
    head = malloc(sizeof(struct node));
    head->data = 11;
    head->next = malloc(sizeof(struct node));
    head->next->data = 22;
    head->next->next = malloc(sizeof(struct node));
    head->next->next->data = 33;
    head->next->next->next = NULL;

    // Traverse it
    for (struct node *cur = head; cur != NULL; cur = cur->next) {
        printf("%d\n", cur->data);
    }
}
```

Результат:

```
11
22
33
```

Для краткости пример не проверяет ошибки `malloc` и не освобождает выделенную память; в практической программе следует делать и то и другое.

20.4. Гибкие массивы внутри структур

В старые добрые времена, когда код C вырезали из дерева, некоторым хотелось выделять структуры с массивом переменной длины в конце.

Сначала покажем старый способ, затем перейдём к современному.

Допустим, нужна структура для строки и её длины:

```
struct len_string {
    int length;
    char data[8];
};
```

Но здесь максимальная длина жёстко задана числом 8, а это немного. Что если выделить с помощью `malloc()` дополнительное место после структуры и позволить данным выйти за пределы массива? Выделим дополнительно 40 байтов:

```
struct len_string *s = malloc(sizeof *s + 40);
```

Поле `data` находится последним, и при выходе за его границу данные попадают в уже выделенную область. Поэтому старый трюк применялся только к последнему полю:

```
// Copy more than 8 bytes!
strcpy(s->data, "Hello, world!"); // Won't crash. Probably.
```

Некоторые компиляторы также поддерживали обходной приём с массивом нулевой длины:

```
struct len_string {
    int length;
    char data[0];
};
```

Тогда каждый дополнительно выделенный байт предполагалось использовать для строки:

```
// Copy more than 8 bytes!
strcpy(s->data, "Hello, world!"); // Won't crash. Probably.
```

Однако доступ за пределами массива приводит к неопределённому поведению, а массив нулевой длины не является стандартным объявлением массива C. В наши дни такое варварство больше не требуется.

Начиная с C99 аналогичный результат можно получить законным способом: объявить массив без размера. Технически его тип называется неполным:

```
struct len_string {
    int length;
    char data[];
};
```

Гибкий массив должен быть последним полем структуры. Теперь можно выделить больше памяти, чем занимает сама `struct len_string`, как в этой функции, создающей её из строки C:

```
struct len_string *len_string_from_c_string(char *s)
{
    int len = strlen(s);

    // Allocate "len" more bytes than we'd normally need
    struct len_string *ls = malloc(sizeof *ls + len);

    ls->length = len;

    // Copy the string into those extra bytes
    memcpy(ls->data, s, len);
```

```
    return ls;
}
```

Здесь копируются ровно `len` байтов, без завершающего NUL: `data` представляет данные с явно хранимой длиной, а не готовую строку C.

20.5. Байты заполнения

Помните: C может добавлять байты заполнения между полями и после структуры. Нельзя рассчитывать, что поля будут непосредственно соседствовать в памяти. Некоторые компиляторы предоставляют способы принудительной плотной упаковки; например, в GCC можно изучить `__attribute__((packed))`.

Посмотрим на программу, выводящую два числа: сумму размеров отдельных типов полей и размер всей структуры. Можно было бы ожидать, что они равны: размер целого ведь равен сумме размеров частей?

```
#include <stdio.h>

struct foo {
    int a;
    char b;
    int c;
    char d;
};

int main(void)
{
    printf("%zu\n", sizeof(int) + sizeof(char) + sizeof(int) + sizeof(char));
    printf("%zu\n", sizeof(struct foo));
}
```

Но в моей системе получается:

```
10
16
```

Числа разные! Компилятор добавил шесть байтов заполнения для более эффективной работы. У вас результат может отличаться, но без специальных указаний нельзя быть уверенным в отсутствии заполнения.

20.6. offsetof

В предыдущем разделе мы увидели, что компилятор может вставлять заполняющие байты внутрь структуры. Как узнать расположение полей? Для этого есть `offsetof`, определённый в `<stddef.h>`.

Изменим предыдущую программу так, чтобы она выводила смещения отдельных полей:

```
#include <stdio.h>
#include <stddef.h>

struct foo {
    int a;
    char b;
    int c;
    char d;
};

int main(void)
{
    printf("%zu\n", offsetof(struct foo, a));
    printf("%zu\n", offsetof(struct foo, b));
    printf("%zu\n", offsetof(struct foo, c));
    printf("%zu\n", offsetof(struct foo, d));
}
```

У меня выводится:

```
0
4
8
12
```

Поля начинаются с интервалом в четыре байта. Это немного странно: ведь `char` занимает всего один байт? Компилятор добавляет по три байта заполнения после каждого `char`. Предположительно на моём процессоре так быстрее.

20.7. Имитация ООП

Со структурами можно проделать несколько необычный трюк, похожий на объектно-ориентированное программирование.

Указатель на структуру, подходящим образом преобразованный, указывает на её первое поле, и наоборот. Это позволяет организовать следующее:

```
struct parent {
    int a, b;
};

struct child {
    struct parent super; // MUST be first
    int c, d;
};
```

Теперь указатель на `struct child` можно передать функции, которой нужен указатель на начальный `struct parent`: поле `super` находится первым в `struct child`. Кстати, `super` — не ключевое слово; я просто позаимствовал терминологию ООП.

Пример:

```
#include <stdio.h>

struct parent {
    int a, b;
};

struct child {
    struct parent super; // MUST be first
    int c, d;
};

// Making the argument `void*` so we can pass any type into it
// (namely a struct parent or struct child)
void print_parent(void *p)
{
    // Expects a struct parent--but a struct child will also work
    // because the pointer points to the struct parent in the first
    // field:
    struct parent *self = p;

    printf("Parent: %d, %d\n", self->a, self->b);
}

void print_child(struct child *self)
{
    printf("Child: %d, %d\n", self->c, self->d);
}

int main(void)
{
    struct child c = {.super.a=1, .super.b=2, .c=3, .d=4};

    print_child(&c);
    print_parent(&c); // Also works even though it's a struct child!
}
```

В последней строке `main()` мы передали `print_parent()` указатель на `struct child`. Это работает, потому что первым полем `struct child` является `struct parent`.

Основа приёма — правило стандарта C: подходящим образом преобразованный указатель на структуру указывает на её начальный член и наоборот. Стандарт также ограничивает доступ к сохранённому значению объекта выражениями совместимых типов; автор исходит из того, что подходящее преобразование даёт указатель на действительный тип начального поля.

20.8. Битовые поля

По моему опыту, битовые поля используются редко, но их можно встретить, особенно в низкоуровневых приложениях, которые упаковывают биты.

Посмотрим на пример:

```
#include <stdio.h>

struct foo {
    unsigned int a;
```

```
    unsigned int b;
    unsigned int c;
    unsigned int d;
};

int main(void)
{
    printf("%zu\n", sizeof(struct foo));
}
```

У меня программа выводит `16`, что логично: `unsigned int` в моей системе занимает четыре байта.

Но что, если значения `a` и `b` помещаются в пять битов, а `c` и `d` — в три? Всего 16 битов. Зачем резервировать 128, если используются только 16?

Можно попросить C упаковать значения, указав ширину каждого поля в битах после двоеточия:

```
struct foo {
    unsigned int a:5;
    unsigned int b:5;
    unsigned int c:3;
    unsigned int d:3;
};
```

Теперь размер `struct foo` в моей системе равен четырём байтам вместо шестнадцати: четыре значения упакованы, что экономит память в четыре раза.

Компромисс: пятибитные поля вмещают только значения от 0 до 31, а трёхбитные — от 0 до 7. Но жизнь ведь состоит из компромиссов.

20.8.1. Несмежные битовые поля

Подвох: вместе упаковываются соседние битовые поля. Если между ними расположены обычные поля, экономии может не быть:

```
struct foo {          // sizeof(struct foo) == 16 (for me)
    unsigned int a:1; // since a is not adjacent to c.
    unsigned int b;
    unsigned int c:1;
    unsigned int d;
};
```

В этом примере `a` и `c` не соседствуют и в моей системе занимают отдельные единицы хранения. Вместе с `b` и `d` получается четыре `int` по четыре байта, всего 16.

Перестановка сокращает размер до 12 байтов в моей системе:

```
struct foo { // sizeof(struct foo) == 12 (for me)
    unsigned int a:1;
    unsigned int c:1;
    unsigned int b;
```

```
    unsigned int d;
};
```

Теперь `a` и `c` соседствуют, и компилятор помещает их вместе в одну единицу хранения, а `b` и `d` занимают по одной. Итого три `int`, или 12 байтов. Размещайте битовые поля рядом, чтобы компилятор мог объединить их. Конкретная упаковка зависит от реализации.

20.8.2. Знаковые и беззнаковые int

Если объявить битовое поле просто как `int`, разные компиляторы могут считать его знаковым или беззнаковым — как и в случае с `char`. Поэтому явно указывайте знаковость.

20.8.3. Безымянные битовые поля

Иногда по аппаратным причинам необходимо зарезервировать несколько битов, не используя их в коде. Например, в байте значимы верхние два и нижний один бит, а средние пять вам не нужны; предполагается восьмибитный `char`, то есть `CHAR_BIT == 8`.

Можно сделать так:

```
struct foo {
    unsigned char a:2;
    unsigned char dummy:5;
    unsigned char b:1;
};
```

В коде используются `a` и `b`, но не `dummy`: оно лишь занимает пять битов. C позволяет сделать это аккуратнее, оставив поле без имени:

```
struct foo {
    unsigned char a:2;
    unsigned char :5; // <-- unnamed bit-field!
    unsigned char b:1;
};
```

Для аппаратных форматов учитывайте, что порядок размещения битовых полей зависит от реализации; сам по себе этот пример не гарантирует нужный порядок битов на любой платформе.

20.8.4. Безымянные битовые поля нулевой ширины

Ещё немного тонкостей. Допустим, биты упаковываются в `unsigned int`, но некоторые соседние поля должны начаться в следующей единице хранения.

В следующем примере компилятор может упаковать все поля в одну:

```
struct foo {
    unsigned int a:1;
    unsigned int b:2;
    unsigned int c:3;
```

```
    unsigned int d:4;
};
```

Как поместить `a` и `b` в одну единицу, а `c` и `d` — в другую? Вставить безымянное поле ширины ноль:

```
struct foo {
    unsigned int a:1;
    unsigned int b:2;
    unsigned int :0; // <--Zero-width unnamed bit-field
    unsigned int c:3;
    unsigned int d:4;
};
```

Это похоже на принудительный разрыв страницы в текстовом редакторе: компилятору сообщают прекратить упаковку в текущую единицу и начать следующую. В моей системе результат занимает два `unsigned int`, то есть восемь байтов.

20.9. Объединения

Объединения `union` похожи на структуры, но их поля перекрываются в памяти. Объединение достаточно велико для самого крупного поля с учётом требований выравнивания; обычно в конкретный момент используется одно поле. Так один участок памяти можно повторно использовать для разных типов данных.

Объявляются объединения подобно структурам, только используется `union`:

```
union foo {
    int a, b, c, d, e, f;
    float g, h;
    char i, j, k, l;
};
```

Полей много. Будь это структура, в моей системе она занимала бы 36 байтов. Но в объединении поля перекрываются. Самые большие — `int` или `float`, по четыре байта, и `sizeof(union foo)` в моей системе действительно равен 4.

Компромисс: переносимым образом обычно можно использовать одно из этих полей за раз. Однако…

20.9.1. Объединения и переинтерпретация типа

Можно записать значение в одно поле объединения, а прочитать другое, но результат зависит от представления данных на платформе. Это называется *type punning*, переинтерпретацией типа, и применяется при понимании низкоуровневых деталей.

Поскольку члены объединения делят память, запись в один влияет на остальные. Чтение другого поля может дать неожиданный результат:

```
#include <stdio.h>

union foo {
    float b;
    short a;
};

int main(void)
{
    union foo x;

    x.b = 3.14159;

    printf("%f\n", x.b); // 3.14159, fair enough

    printf("%d\n", x.a); // But what about this?
}
```

В моей системе выводится:

```
3.141590
4048
```

Байтовое представление `float` для `3.14159` при чтении соответствующей части как `short` дало `4048` в моей системе. У вас результат может отличаться.

20.9.2. Указатели на объединения

Указатель на объединение можно преобразовать в указатель на тип одного из его полей и получить значение этим способом.

В примере объединение содержит `int` и `float`. Мы преобразуем указатель на него к `int *` и `float *`; приведение подавляет предупреждения компилятора. При разыменовании получаем значения, записанные в соответствующие поля:

```
#include <stdio.h>

union foo {
    int a, b, c, d, e, f;
    float g, h;
    char i, j, k, l;
};

int main(void)
{
    union foo x;

    int *foo_int_p = (int *)&x;
    float *foo_float_p = (float *)&x;

    x.a = 12;
    printf("%d\n", x.a);        // 12
    printf("%d\n", *foo_int_p); // 12, again

    x.g = 3.141592;
    printf("%f\n", x.g);          // 3.141592
```

```
    printf("%f\n", *foo_float_p); // 3.141592, again
}
```

Верно и обратное: указатель на поле можно преобразовать в указатель на объединение и обратиться к его членам:

```
union foo x;
int *foo_int_p = (int *)&x;             // Pointer to int field
union foo *p = (union foo *)foo_int_p; // Back to pointer to union

p->a = 12; // This line the same as...
x.a = 12;  // this one.
```

Все эти примеры показывают, что члены объединения начинаются в одном месте памяти — там же, где само объединение.

20.9.3. Общие начальные последовательности в объединениях

Если объединение содержит структуры с общей начальной последовательностью, к соответствующим полям этой последовательности можно обращаться через любой из таких членов объединения.

Что? Вот две структуры с общей начальной последовательностью:

```
struct a {
    int x;   //
    float y; // Common initial sequence

    char *p;
};

struct b {
    int x;   //
    float y; // Common initial sequence

    double *p;
    short z;
};
```

Видите? Обе начинаются с `int`, за которым следует `float`. Члены последовательности должны иметь совместимые типы; здесь это `x` и `y`.

Создадим объединение:

```
union foo {
    struct a sa;
    struct b sb;
};
```

Правило гарантирует возможность читать соответствующие поля общей начальной последовательности через любую из структур: `f.sa.x` соответствует `f.sb.x`, а `f.sa.y` — `f.sb.y`. Имена полей не важны, важна совместимость типов.

Это позволяет безопасно хранить общие сведения в структурах внутри объединения. Например, общее поле может указывать, какая структура сейчас используется. Иначе как функция, получившая объединение, узнала бы, на какой его член смотреть?

Рассмотрим структуры с общей начальной последовательностью:

```
#include <stdio.h>

struct common {
    int type; // common initial sequence
};

struct antelope {
    int type; // common initial sequence
    int loudness;
};

struct octopus {
    int type; // common initial sequence
    int sea_creature;
    float intelligence;
};
```

Поместим их в объединение:

```
union animal {
    struct common common;
    struct antelope antelope;
    struct octopus octopus;
};

#define ANTELOPE 1
#define OCTOPUS 2
```

Пока ничего особенного не произошло: поле `type` кажется бесполезным. Но общей функции вывода `union animal` нужно отличить антилопу от осьминога. Благодаря общей начальной последовательности тип конкретного `union animal x` можно узнать любым из этих способов:

```
int type = x.common.type; // or...
int type = x.antelope.type; // or...
int type = x.octopus.type;
```

Все они относятся к одному значению. Структура `common` позволяет проверять тип, не называя конкретное животное.

Вот функция вывода:

```
void print_animal(union animal *x)
{
    switch (x->common.type) {
        case ANTELOPE:
            printf("Antelope: loudness=%d\n", x->antelope.loudness);
            break;

        case OCTOPUS:
```

```
            printf("Octopus : sea_creature=%d\n", x->octopus.sea_creature);
            printf("          intelligence=%f\n", x->octopus.intelligence);
            break;

        default:
            printf("Unknown animal type\n");
    }
}

int main(void)
{
    union animal a = {.antelope.type=ANTELOPE, .antelope.loudness=12};
    union animal b = {.octopus.type=OCTOPUS, .octopus.sea_creature=1,
                      .octopus.intelligence=12.8};

    print_animal(&a);
    print_animal(&b);
}
```

Функции передаётся объединение, и она заранее не знает, какой вид животного в нём используется. Поэтому она проверяет `type`, а затем обращается к нужной структуре.

Того же результата можно добиться и одними структурами, но этот подход позволяет воспользоваться экономией памяти объединения.

20.10. Объединения и безымянные структуры

Как вы знаете, можно объявить структуру без имени:

```
struct {
    int x, y;
} s;
```

Это определяет переменную `s` безымянного структурного типа с полями `x` и `y`. Поэтому допустимо следующее:

```
s.x = 34;
s.y = 90;
printf("%d %d\n", s.x, s.y);
```

Такие структуры можно помещать и в объединения:

```
union foo {
    struct { // unnamed!
        int x, y;
    } a;

    struct { // unnamed!
        int z, w;
    } b;
};
```

А затем обращаться к ним обычным способом:

```
union foo f;

f.a.x = 1;
f.a.y = 2;
f.b.z = 3;
f.b.w = 4;
```

Никаких проблем!

20.11. Передача и возврат структур и объединений

Структуру или объединение можно передать функции по значению, а не через указатель: как обычно, в параметр копируется объект, словно при присваивании.

Структуру или объединение можно также вернуть из функции; возвращаемый объект тоже передаётся по значению:

```
#include <stdio.h>

struct foo {
    int x, y;
};

struct foo f(void)
{
    return (struct foo){.x=34, .y=90};
}

int main(void)
{
    struct foo a = f(); // Copy is made

    printf("%d %d\n", a.x, a.y);
}
```

Интересный факт: оператор `.` можно применять непосредственно к результату вызова функции:

```
printf("%d %d\n", f().x, f().y);
```

Конечно, в этом примере функция неэффективно вызывается дважды. То же относится к возврату указателей на структуры и объединения, только в таком случае нужно использовать оператор-стрелку `->`.

Глава 21. Символы и строки II

Мы говорили, что `char` — небольшой целочисленный тип; символьная константа в одинарных кавычках тоже имеет числовое значение. Строковый литерал в двойных кавычках представляет массив символов, который в большинстве выражений преобразуется в указатель на первый символ. Изменять строковый литерал нельзя.

Существуют и другие виды символов и строк. Они ведут к одной из самых глубоких тем языка: многобайтовым и широким символам, Unicode и локализации. Пока только заглянем в эту тему.

21.1. Управляющие последовательности

Обычные символы и строки записываются так:

```
char *s = "Hello!";
char t = 'c';
```

Но как записать специальный символ или саму одинарную кавычку? Так нельзя:

```
char t = '''; // Ошибка
```

Для этого служат управляющие, или escape-последовательности: обратная косая черта `\` и следующий за ней символ вместе приобретают специальный смысл. Например:

```
char t = '\'';
```

Теперь `\'` обозначает саму кавычку, а не конец символьной константы. Слово «экранирование» здесь не следует путать с клавишей Esc или кодом ASCII ESC.

21.1.1. Частые последовательности

По скромному мнению автора, они составляют 99,2% всех случаев — число, конечно, выдуманное!

Последовательность	Значение
`\n`	Новая строка
`\'`	Одинарная кавычка
`\"`	Двойная кавычка внутри строки
`\\`	Обратная косая черта

```
printf("Use \\n for newline\n"); // Use \n for newline
printf("Say \"hello\"!\n");     // Say "hello"!
```

```
printf("%c\n", '\'');             // '
```

21.1.2. Более редкие последовательности

Последовательность	Значение
`\a`	Сигнал: терминал может издать звук или мигнуть
`\b`	Возврат на одну позицию; символ не удаляется
`\f`	Перевод страницы; современное поведение зависит от среды
`\r`	Возврат к началу текущей строки
`\t`	Горизонтальная табуляция до следующей позиции табуляции
`\v`	Вертикальная табуляция
`\?`	Буквальный вопросительный знак; исторически помогал избежать триграфов

Обновление состояния в одной строке

С помощью `\r` или `\b` можно обновлять вывод на той же строке без прокрутки. Вот обратный отсчёт от 10. Если компилятор не поддерживает потоки C, на POSIX-системе можно использовать нестандартную для C функцию `sleep()` из `<unistd.h>` либо аналог для вашей платформы.

```
#include <stdio.h>
#include <threads.h>

int main(void)
{
    for (int i = 10; i >= 0; i--) {
        printf("\rT minus %d second%s... \b", i, i != 1? "s": "");
        fflush(stdout); // Принудительно обновить вывод
        thrd_sleep(&(struct timespec){.tv_sec=1}, NULL); // Пауза в 1 секунду
    }
    printf("\rLiftoff! \n");
}
```

`\r` возвращает курсор в начало строки, и новый отсчёт перезаписывает старый. Тернарный оператор выбирает `second` для единицы и `seconds` для остальных значений. Пробел после многоточия помогает стереть лишний символ, когда число становится на разряд короче; `\b` возвращает курсор на позицию перед этим пробелом.

Терминальный вывод часто буферизуется построчно. Здесь новой строки до конца отсчёта нет, поэтому `fflush(stdout)` принудительно отправляет накопленный вывод сразу. Если финальное сообщение короче предыдущего, для полного стирания старого текста могут понадобиться дополнительные пробелы.

Зачем экранировать вопросительный знак?

В старых версиях C существовали триграфы: некоторые тройки символов, начинающиеся с `??`, заменялись другими символами. Например, `??!` мог преобразоваться в `|`. Поэтому строка с `Doesn't it??!` могла вывести неожиданный результат. Экранирование второго вопросительного знака устраняло проблему:

```
printf("Doesn't it?\?!\n");
```

Триграфы предназначались для старых терминалов без некоторых символов на клавиатуре. В современном C они удалены, но в старом коде или режиме старого стандарта об этой особенности полезно помнить.

21.1.3. Числовые последовательности

Байт можно указать восьмеричным или шестнадцатеричным значением, а символ Unicode — универсальным именем символа:

Пример	Значение
`\123`	Байт с восьмеричным значением 123
`\x4D`	Байт с шестнадцатеричным значением 4D
`\u2620`	Кодовая точка Unicode 2620: четыре шестнадцатеричные цифры
`\U0001243F`	Кодовая точка Unicode 1243F: восемь шестнадцатеричных цифр

Восьмеричная escape-последовательность может содержать от одной до трёх цифр; при необходимости добавьте нули слева, чтобы отделить её от следующих цифр. Пример буквы `B` между `A` и `C` в ASCII-совместимой кодировке:

```
printf("A\102C\n"); // 102 в восьмеричной системе — B в ASCII
```

Шестнадцатеричная последовательность `\x` поглощает следующие шестнадцатеричные цифры до первого символа, который к ним не относится; следите за границей последовательности. Следующий пример вручную задаёт байты UTF-8 символа маркера `•`:

```
printf("\xE2\x80\xA2 Bullet 1\n");
printf("\xE2\x80\xA2 Bullet 2\n");
printf("\xE2\x80\xA2 Bullet 3\n");
```

На терминале UTF-8 получится:

```
• Bullet 1
• Bullet 2
• Bullet 3
```

На терминале с другой кодировкой результат может выглядеть неправильно. Удобнее указать кодовую точку Unicode напрямую: у маркера это шестнадцатеричное значение `2022`.

```
printf("\u2022 Bullet 1\n");
printf("\u2022 Bullet 2\n");
printf("\u2022 Bullet 3\n");
```

После `\u` нужны ровно четыре шестнадцатеричные цифры, после `\U` — восемь. При необходимости добавьте начальные нули:

```
printf("\U00002022 Bullet 1\n");
```

Так тоже работает, хотя запись получается длиннее.

Глава 22. Перечисляемые типы: enum

В C есть ещё один способ задавать именованные целочисленные константы — `enum`.

Например:

```
enum {
    ONE=1,
    TWO=2
};

printf("%d %d", ONE, TWO);  // 1 2
```

В некоторых отношениях это лучше или просто отличается от использования `#define`. Основные различия:

- Значения `enum` могут быть только целочисленного типа.
- С помощью `#define` можно определить что угодно.
- В отладчике значения `enum` часто отображаются под своими символьными именами.
- Числа, заданные через `#define`, отображаются просто как числа, и при отладке труднее понять, что они означают.

Поскольку это целочисленные значения, их можно использовать везде, где используются целые числа, в том числе при задании размеров массивов и в метках `case`.

Разберёмся подробнее.

22.1. Поведение enum

22.1.1. Нумерация

Значения `enum` нумеруются автоматически, если вы не укажете их явно. По умолчанию нумерация начинается с `0` и увеличивается на единицу:

```
enum {
    SHEEP,  // Значение 0
    WHEAT,  // Значение 1
    WOOD,   // Значение 2
    BRICK,  // Значение 3
    ORE     // Значение 4
};

printf("%d %d\n", SHEEP, BRICK);  // 0 3
```

Как мы уже видели, можно задать конкретные целые значения:

```
enum {
    X=2,
    Y=18,
```

```
    Z=-2
};
```

Повторяющиеся значения не являются проблемой:

```
enum {
    X=2,
    Y=2,
    Z=2
};
```

Если значения опущены, нумерация продолжается по возрастанию от последнего явно указанного значения. Например:

```
enum {
    A,      // 0, начальное значение по умолчанию
    B,      // 1
    C=4,    // 4, задано вручную
    D,      // 5
    E,      // 6
    F=3,    // 3, задано вручную
    G,      // 4
    H       // 5
};
```

22.1.2. Завершающая запятая

Если вам так больше нравится, следующий вариант совершенно допустим:

```
enum {
    X=2,
    Y=18,
    Z=-2,  // <-- завершающая запятая
};
```

В языках последних десятилетий такая запись стала популярнее, так что вам может быть приятно увидеть её здесь.

22.1.3. Область видимости

Для `enum` область видимости работает ожидаемым образом. Если определение находится на уровне файла, оно видно во всём файле. Если внутри блока — оно локально для этого блока.

Очень часто `enum` определяют в заголовочных файлах, чтобы подключать их директивой `#include` на уровне файла.

22.1.4. Стиль

Как вы заметили, имена констант `enum` принято записывать прописными буквами, используя подчёркивания для разделения слов.

Это не требование, но очень, очень распространённое соглашение.

22.2. enum — это тип

Важно знать: `enum` представляет собой тип — подобно тому, как типом является `struct`.

Перечислению можно дать имя, чтобы впоследствии ссылаться на его тип и объявлять переменные этого типа.

Но если `enum` — целочисленный тип, почему бы просто не использовать `int`?

В C главная причина — ясность кода: это удобный способ выразить свои намерения с помощью типа. C, в отличие от C++, не проверяет, находятся ли значения в диапазоне конкретного `enum`.

Рассмотрим пример: объявим переменную `r` типа `enum resource`, которая может хранить эти значения:

```
// Именованное перечисление, тип — "enum resource"
enum resource {
    SHEEP,
    WHEAT,
    WOOD,
    BRICK,
    ORE
};

// Объявляем переменную "r" типа "enum resource"
enum resource r = BRICK;

if (r == BRICK) {
    printf("I'll trade you a brick for two sheep.\n");
}
```

Конечно, для перечисления можно также использовать `typedef`, хотя лично мне это не нравится:

```
typedef enum {
    SHEEP,
    WHEAT,
    WOOD,
    BRICK,
    ORE
} RESOURCE;

RESOURCE r = BRICK;
```

Ещё один допустимый, но редкий сокращённый вариант — объявить переменные одновременно с перечислением:

```
// Объявляем перечисление и инициализированные переменные этого типа:
enum {
    SHEEP,
    WHEAT,
```

```
    WOOD,
    BRICK,
    ORE
} r = BRICK, s = WOOD;
```

Перечислению также можно дать имя для последующего использования. В большинстве случаев вы, вероятно, именно этого и захотите:

```
// Объявляем перечисление и инициализированные переменные этого типа:
enum resource {  // <-- теперь тип называется "enum resource"
    SHEEP,
    WHEAT,
    WOOD,
    BRICK,
    ORE
} r = BRICK, s = WOOD;
```

Одним словом, перечисления — отличный способ писать аккуратный код с понятными типами и областями видимости.

Глава 23. Указатели III: указатели на указатели и не только

Здесь мы рассмотрим приёмы работы с указателями среднего и продвинутого уровня. Если вы ещё не вполне освоили указатели, перед чтением повторите предыдущие главы об указателях и арифметике указателей.

23.1. Указатели на указатели

Если можно иметь указатель на переменную, а переменная сама может быть указателем, можно ли иметь указатель на переменную-указатель?

Да! Это указатель на указатель, который хранится в переменной соответствующего типа.

Прежде чем подробно разбираться, попробуем интуитивно понять, как они работают. Представляйте адрес как число, обозначающее позицию в памяти компьютера, обычно содержащую интересующее нас значение.

Сам указатель тоже должен где-то храниться — в памяти, как и всё остальное. Со значениями, хранящимися только в регистрах, есть тонкости, но для наших целей их можно не рассматривать.

Раз указатель хранится в памяти, у него тоже должен быть адрес. Это число — адрес указателя, то есть указатель на указатель.

Начнём с обычного указателя на `int`, знакомого по предыдущим главам:

```
#include <stdio.h>

int main(void)
{
    int x = 3490; // Type: int
    int *p = &x;  // Type: pointer to an int

    printf("%d\n", *p); // 3490
}
```

Всё просто: у нас есть `int` и `int *`, причём `p` указывает на `x`. Разыменовав `p`, мы выводим `3490`.

Но указатель можно получить на любую переменную. Значит, можно получить указатель и на `p`? Другими словами, какой тип у следующего выражения?

```
int x = 3490; // Type: int
int *p = &x;  // Type: pointer to an int

&p // <-- What type is the address of p? AKA a pointer to p?
```

Если `x` — это `int`, то `&x` — указатель на `int`, сохранённый в `p` типа `int *`. Понятно? Если нет, перечитайте этот абзац!

Следовательно, `&p` — указатель на `int *`, то есть «указатель на указатель на `int`». Понятно? Если нет, перечитайте предыдущий абзац!

Этот тип записывается с двумя звёздочками: `int **`. Посмотрим на него в действии:

```
#include <stdio.h>

int main(void)
{
    int x = 3490;  // Type: int
    int *p = &x;   // Type: pointer to an int
    int **q = &p;  // Type: pointer to pointer to int

    printf("%d %d\n", *p, **q); // 3490 3490
}
```

Придумаем примерные адреса и представим, как эти переменные могут выглядеть в памяти:

Переменная	Адрес хранения	Хранящееся значение
`x`	28350	3490 — значение из программы
`p`	29122	28350 — адрес `x`
`q`	30840	29122 — адрес `p`

А теперь я выведу значения указателей через `%p` на своём компьютере и составлю таблицу с настоящими адресами в шестнадцатеричной записи. У вас почти наверняка получатся другие числа.

Переменная	Адрес хранения	Хранящееся значение
`x`	`0x7ffd96a07b94`	3490 — значение из программы
`p`	`0x7ffd96a07b98`	`0x7ffd96a07b94` — адрес `x`
`q`	`0x7ffd96a07ba0`	`0x7ffd96a07b98` — адрес `p`

Адреса различаются только последним байтом, поэтому сосредоточьтесь на нём. В моей системе `int` занимает четыре байта: от адреса `x` до `p` разница равна четырём, а от `p` до `q` — восьми. Все указатели в моей системе занимают восемь байтов. Стандарт совершенно не гарантирует именно такого расположения переменных или одинакового размера всех типов указателей.

Имеет ли в моей системе значение, `int *` это или `int **`? Нет: оба адреса представлены восемью байтами независимо от того, что находится по адресу.

Обратите внимание, что в предыдущем примере мы дважды разыменовали `q`, чтобы снова получить `3490`. Вот что важно:

- С помощью `&` можно получить указатель на объект, в том числе на указатель.
- С помощью `*` можно получить объект, на который указывает указатель, в том числе другой указатель.

Поэтому `&` можно считать способом получения указателя, а `*` — обратным действием, ведущим к объекту, на который указывают.

С точки зрения типа каждое применение `&` добавляет один уровень указателя:

Есть	Выражение	Тип результата
`int x`	`&x`	`int *`
`int *x`	`&x`	`int **`
`int **x`	`&x`	`int ***`
`int ***x`	`&x`	`int ****`

Каждое разыменование `*` делает обратное:

Есть	Выражение	Тип результата
`int ****x`	`*x`	`int ***`
`int ***x`	`*x`	`int **`
`int **x`	`*x`	`int *`
`int *x`	`*x`	`int`

Можно поставить несколько звёздочек подряд, как в примере с `**q`: каждая снимает один уровень косвенности.

Есть	Выражение	Тип результата
`int ****x`	`***x`	`int *`
`int ***x`	`**x`	`int *`
`int **x`	`**x`	`int`

В общем случае `&*E == E`: получение адреса отменяет разыменование. Необычно, но это верно даже когда `E` — нулевой указатель.

С `&` иначе: нельзя просто поставить несколько таких операторов подряд; результат приходится сохранять в промежуточной переменной:

```
int x = 3490;   // Type: int
int *p = &x;    // Type: int *, pointer to an int
int **q = &p;   // Type: int **, pointer to pointer to int
int ***r = &q;  // Type: int ***, pointer to pointer to pointer to int
int ****s = &r; // Type: int ****, you get the idea
int *****t = &s; // Type: int *****
```

23.1.1. Указатели на указатели и const

Помните объявление:

```
int *const p;
```

Оно означает, что изменить `p` нельзя. Попытка выполнить `p++` вызовет ошибку компиляции.

Но как это работает с `int **` или `int ***`? Где ставить `const` и что он означает?

Начнём с простого: `const` непосредственно рядом с именем относится к этой переменной. Поэтому неизменяемый указатель типа `int ***` можно объявить так:

```
int ***const p;

p++; // Not allowed
```

А теперь более странная ситуация:

```
int main(void)
{
    int x = 3490;
    int *const p = &x;
    int **q = &p;
}
```

При сборке я получаю предупреждение:

```
warning: initialization discards ‘const’ qualifier from pointer target type
    7 | int **q = &p;
      |           ^
```

Что происходит? Компилятор сообщает, что квалификатор `const` отбрасывается при присваивании. Скорее всего, мы этого не хотели.

Тип `p` — `int *const`, поэтому тип `&p` — `int *const *`. Но `q` имеет тип `int **`: квалификация `const` для указателя, на который он будет указывать, отличается и теряется.

Исправим это, сохранив квалификатор в типе `q`:

```
int x = 3490;
int *const p = &x;
int *const *q = &p;
```

Теперь всё хорошо. Можно добавить ещё больше `const`: сейчас `q` сам не является `const`, но объект, на который он указывает, является. Сделаем неизменяемыми оба указателя:

```
int x = 3490;
int *const p = &x;
int *const *const q = &p; // More const!
```

Это тоже работает. Теперь нельзя изменить ни `q`, ни указатель, на который он указывает.

23.2. Многобайтовые значения

Ранее мы уже намекали: не всякое значение помещается в одном байте. Объекты могут занимать несколько байтов; узнать их количество можно с помощью `sizeof`. Оператор `&` даёт адрес объекта, соответствующий началу его размещения.

Ещё один интересный факт: если пройти по байтам объекта, получится его *объектное представление*. Для просмотра байтов любого объекта следует использовать указатель на `unsigned char`.

Напишем собственную версию `memcpy()`, которая делает именно это:

```
void *my_memcpy(void *dest, const void *src, size_t n)
{
    // Make local variables for src and dest, but of type unsigned char
    const unsigned char *s = src;
    unsigned char *d = dest;

    while (n-- > 0) // For the given number of bytes
        *d++ = *s++; // Copy source byte to dest byte

    // Most copy functions return a pointer to the dest as a convenience
    // to the caller
    return dest;
}
```

Здесь есть хорошие примеры постфиксного увеличения и уменьшения, которые стоит изучить. Эта версия, вероятно, менее эффективна, чем системная. Но ей можно передать указатели на любые объекты — `int *`, `struct animal *` и так далее — и она скопирует их байты. Как и `memcpy`, она не предназначена для перекрывающихся областей.

Теперь выведем байты объектного представления структуры, чтобы увидеть возможное заполнение и его значения. Компилятор не обязан добавлять байты заполнения, а их значения не определены заранее.

```
#include <stdio.h>

struct foo {
    char a;
    int b;
};

int main(void)
{
    struct foo x = {0x12, 0x12345678};
    unsigned char *p = (unsigned char *)&x;

    for (size_t i = 0; i < sizeof x; i++) {
        printf("%02X\n", p[i]);
    }
}
```

Структура `foo` устроена так, что компилятор, вероятно, добавит заполнение, хотя и не обязан. Затем мы получаем `unsigned char *` на первый байт `x`. Зная `sizeof x`, можно пройти по байтам и вывести их в шестнадцатеричном виде.

Ниже мой результат с пояснениями:

```
12 | x.a == 0x12
AB |
BF | padding bytes with "random" value
26 |
78 | x.b == 0x12345678
56 |
34 |
12 |
```

Во всех системах `sizeof(char)` равен 1; первый байт содержит записанное `0x12`. Затем идут байты заполнения, которые у меня менялись от запуска к запуску. Наконец, в моей системе `sizeof(int)` равен 4: последние четыре байта соответствуют `0x12345678`, но идут в обратном порядке.

Это зависит от архитектуры. У меня используется порядок little-endian: младший байт хранится первым. В big-endian первым был бы `12`, а последним — `78`. Стандарт не предписывает конкретное представление.

Так мы заглянули внутрь памяти, где хранится более сложный объект.

23.3. Нулевой указатель и ноль

В контексте нулевого указателя могут встречаться следующие записи:

- `NULL`;
- `0`;
- `'\0'`;
- `(void *)0`.

Лично я всегда использую `NULL`, когда имею в виду нулевой указатель, хотя иногда встречаются другие варианты. Сравнивать указатель с `'\0'` странно — для указателя лучше использовать `NULL`. Конечно, при обработке строк часто сравнивают *значение, на которое указывает указатель*, с `'\0'`, и это правильно.

`0` — константа нулевого указателя. При сравнении с указателем или присваивании ему она преобразуется в нулевой указатель соответствующего типа.

23.4. Указатели как целые числа

Указатели можно приводить к целым типам и обратно, но обычно это требуется только в низкоуровневом коде для оборудования. Результат такого преобразования зависит от реализации, поэтому код непереносим и может вести себя неожиданно.

Если реализация предоставляет `uintptr_t`, определённый в `<stdint.h>`, указатель на `void` можно преобразовать к этому типу и обратно без потери исходного значения указателя. Аналогичную гарантию даёт необязательный знаковый тип `intptr_t`. Эти типы могут отсутствовать.

23.5. Приведение указателей к другим типам

Есть только одно безопасное преобразование указателей:

1. Преобразование к `intptr_t` или `uintptr_t` с возможностью обратного преобразования там, где эти типы доступны.
2. Преобразование указателя на объект в `void *` и обратно.

ДВА! Два безопасных преобразования.

3. Преобразование указателя на объект к `char *`, `signed char *` или `unsigned char *` для просмотра его представления.

ТРИ! Три безопасных преобразования!

4. Преобразование указателя на структуру к указателю на её первый член и обратно.

ЧЕТЫРЕ! Четыре безопасных преобразования!

Если привести указатель к другому типу и обращаться через него к несовместимому объекту, поведение может быть неопределённым из-за правил строгого алиасинга. Нужно учитывать также выравнивание и другие требования стандарта.

Обычный алиасинг — это возможность обращаться к одному объекту несколькими способами. Эти способы служат альтернативными именами одного объекта.

Правила строгого алиасинга ограничивают типы выражений, через которые допустимо обращаться к значению объекта; помимо совместимых типов, стандарт предусматривает отдельные исключения, включая символьные типы.

Например, так делать, безусловно, можно:

```
int a = 1;
int *p = &a;
```

`p` указывает на совместимый тип `int`, всё хорошо. Но следующее небезопасно для доступа к `a`, поскольку `int` и `float` несовместимы:

```
int a = 1;
float *p = (float *)&a;
```

Следующая программа показывает алиасинг: она рассматривает переменную `int32_t` через указатель на `struct words` с двумя значениями `int16_t`. Типы несовместимы, поэтому

нарушаются правила строгого алиасинга. Компилятор может предположить, что эти указатели не ссылаются на один объект, хотя мы заставляем их это делать. Нехорошо!

Посмотрим, удастся ли что-нибудь сломать:

```
#include <stdio.h>
#include <stdint.h>

struct words {
    int16_t v[2];
};

void fun(int32_t *pv, struct words *pw)
{
    for (int i = 0; i < 5; i++) {
        (*pv)++;

        // Print the 32-bit value and the 16-bit values:

        printf("%x, %x-%x\n", *pv, pw->v[1], pw->v[0]);
    }
}

int main(void)
{
    int32_t v = 0x12345678;

    struct words *pw = (struct words *)&v; // Violates strict aliasing

    fun(&v, pw);
}
```

В `fun()` передаются два несовместимых указателя: `int32_t *` и `struct words *`. Но оба указывают на один объект с исходным значением `0x12345678`.

На первый взгляд поля `struct words` должны показывать две 16-битные половины этого числа. В цикле мы увеличиваем значение `int32_t`; поскольку структура указывает на ту же память, можно ожидать, что изменятся и её поля.

При одном запуске результат выглядит ожидаемым: слева 32-битное значение, справа его половины. Я вывожу половины в обратном порядке, поскольку использую little-endian.

```
12345679, 1234-5679
1234567a, 1234-567a
1234567b, 1234-567b
1234567c, 1234-567c
1234567d, 1234-567d
```

Всё совпадает… ДО ЗАВТРАШНЕГО ДНЯ! Попробуем собрать GCC с `-O3` и `-fstrict-aliasing`:

```
12345679, 1234-5678
1234567a, 1234-5679
1234567b, 1234-567a
```

```
1234567c, 1234-567b
1234567d, 1234-567c
```

Разница на единицу! Но ведь память одна и та же! Как такое возможно? Ответ: такой алиасинг вызывает неопределённое поведение. Возможен любой результат, и ничего хорошего в этом нет.

Если код нарушает правила строгого алиасинга, его работа зависит от способа компиляции, который может быть вне вашего контроля. Разве что вы всемогущее божество. Вряд ли, извините.

GCC можно попросить не использовать эти предположения флагом `-fno-strict-aliasing`. При сборке примера с `-O3` и этим флагом у автора вывод получается ожидаемым.

Переинтерпретация типа через указатели раньше встречалась довольно часто:

```
int a = 0x12345678;
short b = *((short *)&a); // Violates strict aliasing
```

Для относительно безопасной переинтерпретации обратитесь к разделу об объединениях и type punning; для копирования представления также применяют `memcpy()`.

23.6. Разность указателей

Как вы знаете из главы об арифметике указателей, из одного указателя можно вычесть другой, чтобы получить расстояние в элементах массива. При этом оба указателя должны относиться к одному массиву, включая допустимый указатель за его конец.

Тип разности зависит от реализации. Для переносимости храните результат в `ptrdiff_t`, определённом в `<stddef.h>`:

```
int cats[100];
int *f = cats + 20;
int *g = cats + 60;

ptrdiff_t d = g - f; // difference is 40
```

Для вывода десятичного значения используется префикс `t`:

```
printf("%td\n", d); // Print decimal: 40
printf("%tX\n", d); // Print hex: 28
```

Во второй строке оригинала `%tX` ожидает соответствующий беззнаковый тип, тогда как `d` имеет знаковый тип `ptrdiff_t`; для переносимого вывода лучше использовать подходящее преобразование и согласованный спецификатор.

23.7. Указатели на функции

Функции представлены выполняемыми инструкциями, и C позволяет получать указатели на функции и вызывать их через такие указатели.

Это полезно, например, когда указатель на функцию передаётся как аргумент другой функции, а та затем вызывает переданную функцию.

Сложность в том, что C необходимо знать точный тип указателя: например, «указатель на функцию, которая принимает два аргумента `int` и возвращает `void`». Как записать всё это в объявлении?

Похоже на прототип функции, но с дополнительными скобками:

```
// Declare p to be a pointer to a function.
// This function returns a float, and takes two ints as arguments.
float (*p)(int, int);
```

Имена параметров указывать не обязательно. Если указать, они игнорируются:

```
// Declare p to be a pointer to a function.
// This function returns a float, and takes two ints as arguments.
float (*p)(int a, int b);
```

Как получить адрес функции для присваивания? Есть сокращение, похожее на получение указателя на массив: достаточно указать имя функции без скобок. Можно поставить перед ним `&`, но это необязательно и обычно не делается.

Получив указатель, можно вызвать функцию, добавив скобки и список аргументов. В простом примере мы создадим своего рода второе имя функции через указатель, а затем вызовем её. Программа выведет `3490`:

```
#include <stdio.h>

void print_int(int n)
{
    printf("%d\n", n);
}

int main(void)
{
    // Assign p to point to print_int:

    void (*p)(int) = print_int;

    p(3490); // Call print_int via the pointer
}
```

Обратите внимание: тип `p` отражает возвращаемое значение и типы параметров `print_int`. Иначе C пожалуется на несовместимые типы указателей.

Ещё один пример показывает передачу указателя на функцию другой функции. Мы напишем функцию, принимающую два целых числа и указатель на операцию над ними, а затем выводящую результат:

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int mult(int a, int b)
{
    return a * b;
}

void print_math(int (*op)(int, int), int x, int y)
{
    int result = op(x, y);

    printf("%d\n", result);
}

int main(void)
{
    print_math(add, 5, 7);  // 12
    print_math(mult, 5, 7); // 35
}
```

Обдумайте пример. Мы передаём указатель на функцию в `print_math()`, а она вызывает эту функцию для выполнения вычисления. Так поведение `print_math()` меняется в зависимости от переданной операции: в одном вызове мы передаём `add`, в другом `mult`.

Сигнатура `print_math()` впечатляет. И, хотите верьте, хотите нет, по сравнению с некоторыми возможными объявлениями она довольно проста. Например, синтаксис объявления типов языка Go был во многом вдохновлён стремлением сделать его противоположным синтаксису C.

Разберём сигнатуру: параметров всего три, хотя их не сразу видно:

```
//                  op              x      y
//        |-----------------|      |---|  |---|
void print_math(int (*op)(int, int), int x, int y)
```

Первый параметр `op` — указатель на функцию, которая принимает два `int` и возвращает `int`. Это соответствует сигнатурам и `add()`, и `mult()`. Второй и третий параметры, `x` и `y`, — обычные `int`.

Медленно проследите взглядом сигнатуру, выделяя составные части. Особенно заметна последовательность `(*op)(`: скобки и звёздочка выдают указатель на функцию.

Наконец, вернитесь к главе «Указатели II», где приведён пример указателя на функцию со стандартной `qsort()`.

Глава 24. Побитовые операции

Эти числовые операции позволяют управлять отдельными битами переменных — вполне подходящая возможность для такого низкоуровневого языка, как C. Впрочем, подобные операции есть и в других языках; интересно, что многие современные языки используют для них те же операторы, что и C.

Если вы ещё не знакомы с побитовыми операциями, можете обратиться к статье в Википедии.

24.1. Побитовые И, ИЛИ, исключающее ИЛИ и НЕ

Для каждой из этих операций к операндам сначала применяются обычные арифметические преобразования. Оба операнда в данном случае должны иметь целочисленный тип. Затем выполняется соответствующая побитовая операция.

Операция	Оператор	Пример
И	`&`	`a = b & c`
ИЛИ	`|`	`a = b | c`
Исключающее ИЛИ	`^`	`a = b ^ c`
НЕ	`~`	`a = ~c`

Обратите внимание на сходство с логическими операторами `&&` и `||`.

Как и у `+=` и `-=`, у этих операций есть сокращённые формы присваивания:

Оператор	Пример	Полная запись
`&=`	`a &= c`	`a = a & c`
`|=`	`a |= c`	`a = a | c`
`^=`	`a ^= c`	`a = a ^ c`

24.2. Побитовый сдвиг

К каждому операнду применяется целочисленное продвижение; оба операнда должны иметь целочисленный тип. Затем выполняется побитовый сдвиг. Тип результата совпадает с типом левого операнда после продвижения.

Новые биты заполняются нулями, за возможным исключением, связанным с поведением, определяемым реализацией, о котором сказано ниже.

Операция	Оператор	Пример
Сдвиг влево	`<<`	`a = b << c`

Операция	Оператор	Пример
Сдвиг вправо	`>>`	`a = b >> c`

Для сдвигов тоже есть сокращённые формы:

Оператор	Пример	Полная запись
`>>=`	`a >>= c`	`a = a >> c`
`<<=`	`a <<= c`	`a = a << c`

Остерегайтесь неопределённого поведения: количество позиций сдвига не должно быть отрицательным, равным ширине левого операнда после продвижения или превышать её.

Обратите внимание и на поведение, определяемое реализацией: результат сдвига отрицательного числа вправо зависит от реализации. Сдвигать вправо знаковый `int` можно, если его значение положительно.

Глава 25. Функции с переменным числом аргументов

Так называют функции, которые принимают произвольное количество аргументов.

Обычная функция принимает определённое число аргументов, например:

```
int add(int x, int y)
{
    return x + y;
}
```

Её можно вызвать только с двумя аргументами, соответствующими параметрам `x` и `y`:

```
add(2, 3);
add(5, 12);
```

Если передать больше или меньше аргументов, компилятор выдаст ошибку:

```
add(2, 3, 4); // ERROR
add(5);       // ERROR
```

Функции с переменным числом аргументов до некоторой степени обходят это ограничение. Мы уже встречали знаменитый пример — `printf()`:

```
printf("Hello, world!\n");
printf("The number is %d\n", 2);
printf("The number is %d and pi is %f\n", 2, 3.14159);
```

Кажется, ей всё равно, сколько аргументов передано. Но не совсем: вызов без аргументов даст ошибку.

```
printf(); // ERROR
```

В рассматриваемом здесь привычном способе объявления используется хотя бы один именованный параметр. Остальные аргументы могут иметь разные типы, как у `printf()`. Посмотрим, как это работает.

25.1. Многоточие в объявлении функции

Сначала перечисляются обязательные параметры, а за ними ставится `...` — буквально три точки:

```
void func(int a, ...) // Literally 3 dots here
```

Пример:

```
#include <stdio.h>

void func(int a, ...)
{
    printf("a is %d\n", a); // Prints "a is 2"
}

int main(void)
{
    func(2, 3, 4, 5, 6);
}
```

Первый аргумент мы получили через переменную `a`. А как добраться до остальных?

25.2. Получение дополнительных аргументов

Для работы с ними подключите `<stdarg.h>`. Специальная переменная типа `va_list` хранит сведения о текущей позиции в списке аргументов. Вызов `va_start()` начинает обработку, `va_arg()` последовательно получает аргументы, а `va_end()` завершает работу.

В показанной форме `va_start()` получает последний именованный параметр перед `...`, чтобы определить, где начинаются дополнительные аргументы. При каждом вызове `va_arg()` необходимо указать тип следующего аргумента.

Ниже функция складывает произвольное число целых. Первый аргумент сообщает, сколько чисел нужно сложить, а значит, сколько раз вызывать `va_arg()`:

```
#include <stdio.h>
#include <stdarg.h>

int add(int count, ...)
{
    int total = 0;
    va_list va;

    va_start(va, count); // Start with arguments after "count"

    for (int i = 0; i < count; i++) {
        int n = va_arg(va, int); // Get the next int
        total += n;
    }

    va_end(va); // All done
    return total;
}

int main(void)
{
    printf("%d\n", add(4, 6, 2, -4, 17)); // 6 + 2 - 4 + 17 = 21
    printf("%d\n", add(2, 22, 44));         // 22 + 44 = 66
}
```

При вызове `printf()` форматная строка и её спецификаторы, например `%d`, сообщают функции, какие дополнительные аргументы ожидать.

Если синтаксис `va_arg()` кажется необычным из-за имени типа среди аргументов, вы не одиноки. Эти средства реализованы с помощью макросов препроцессора.

25.3. Как работает `va_list`

`va_list` — непрозрачный тип: его внутреннее устройство не предназначено для использования программистом. Переменная этого типа хранит состояние обхода и позволяет очередному вызову `va_arg()` определить следующий аргумент.

Перед использованием переменную нужно инициализировать. Вызов `va_start(va, count)` подготавливает `va` к получению аргумента, следующего за `count`. Далее вызывайте `va_arg()`, передавая переменную `va_list` и тип извлекаемого значения. Определить правильный тип должен сам программист: в примере выше это `int`, а `printf()` использует спецификаторы формата.

Закончив обработку, вызовите `va_end()`. Стандарт требует сделать это до повторной инициализации той же переменной через `va_start()` или `va_copy()`.

Обычная последовательность:

1. `va_start()` — инициализировать переменную `va_list`.
2. Вызывать `va_arg()` для получения значений.
3. `va_end()` — завершить работу с переменной.

Функция `va_copy()` создаёт копию состояния `va_list`. Если исходный список ещё не начали обходить через `va_arg()`, копия будет в той же позиции. Если уже извлечено пять аргументов, копия отразит и это. Она полезна, когда нужно заглянуть вперёд, сохранив текущую позицию.

25.4. Библиотечные функции, принимающие `va_list`

С помощью `va_list` можно написать собственную разновидность `printf()` и не разбирать все спецификаторы формата вручную. Для этого существуют функции, принимающие готовый `va_list`: `vprintf()`, `vfprintf()`, `vsprintf()`, `vsnprintf()` и другие аналоги с префиксом `v`.

Создадим `my_printf()`, которая работает подобно `printf()`, но принимает дополнительный первый аргумент:

```
#include <stdio.h>
#include <stdarg.h>

int my_printf(int serial, const char *format, ...)
{
    va_list va;
    int rv;

    // Do my custom work
    printf("The serial number is: %d\n", serial);
```

```
    // Then pass the rest off to vprintf()
    va_start(va, format);
    rv = vprintf(format, va);
    va_end(va);

    return rv;
}

int main(void)
{
    int x = 10;
    float y = 3.2;

    my_printf(3490, "x is %d, y is %f\n", x, y);
}
```

Мы инициализировали `va_list` и передали его в `vprintf()`, которая умеет обрабатывать форматную строку. Не забудьте после этого вызвать `va_end()`.

25.5. Подводные камни макросов для переменных аргументов

`va_start()` и `va_end()` могут быть макросами. Одно из возможных следствий — появление новой локальной области видимости, например если первый макрос содержит `{`, а второй — `}`.

Поэтому остерегайтесь такого кода:

```
va_start(va, format); // may contain {
int rv = vprintf(format, va);
va_end(va);           // may contain }
return rv;
```

Если `va_start()` открывает новую область видимости, переменная `rv` окажется локальной для неё, и оператор `return` не сработает. Проблема может проявиться лишь на компиляторах, где макросы устроены именно так.

Глава 26. Локаль и интернационализация

Локализация помогает программе работать в разных странах и языковых средах. Например, символы десятичного разделителя, разделителя тысяч и валюты различаются. В США число может быть записано как `100,000.00`, а в Бразилии — `100.000,00`.

У локалей есть имена, и программа может выбрать нужную. Однако стандарт C гарантирует лишь ограниченную встроенную локаль, а многие подробности зависят от системы. Полная переносимость здесь непроста.

26.1. Быстрая настройка локали

Подключите `<locale.h>`. Чтобы использовать настройки текущего окружения для всех категорий, вызовите:

```
setlocale(LC_ALL, ""); // Использовать локаль окружения
```

Есть ещё одна переносимая настройка:

```
setlocale(LC_ALL, "C"); // Стандартная локаль C
```

Она действует при запуске программы по умолчанию, поэтому обычно явно вызывать её не нужно. Можно указать и другое имя, поддерживаемое системой:

```
setlocale(LC_ALL, "en_US.UTF-8"); // Непереносимо!
```

Но это сработает только там, где существует локаль с таким именем. Пустая строка просит реализацию выбрать настройки окружения без указания конкретного имени. Проверяйте возвращаемое значение `setlocale()`: при невозможности установить локаль функция возвращает `NULL`.

26.2. Денежные настройки

После установки локали можно узнать её правила для валюты с помощью `localeconv()`:

```
struct lconv *x = localeconv();
```

Функция возвращает указатель на статически размещённую структуру `lconv`. В именах полей `p` относится к неотрицательным суммам, `n` — к отрицательным, `int` — к международному представлению. Хотя некоторые поля имеют тип `char`, их значения интерпретируются как целые числа. Значение `CHAR_MAX` из `<limits.h>` во многих таких полях означает, что соответствующая настройка недоступна.

Поле	Значение
`char *mon_decimal_point`	Десятичный разделитель денежных сумм, например `"."`
`char *mon_thousands_sep`	Разделитель групп, например `","`
`char *mon_grouping`	Правило группировки разрядов
`char *positive_sign`	Знак положительной суммы, например `"+"` или пустая строка
`char *negative_sign`	Знак отрицательной суммы, например `"-"`
`char *currency_symbol`	Символ валюты, например `"$"`
`char frac_digits`	Число знаков после десятичного разделителя
`char p_cs_precedes`	1, если символ валюты стоит перед неотрицательной суммой; 0 — после
`char n_cs_precedes`	То же для отрицательной суммы
`char p_sep_by_space`	Правило пробела для неотрицательной суммы
`char n_sep_by_space`	Правило пробела для отрицательной суммы
`char p_sign_posn`	Положение знака неотрицательной суммы
`char n_sign_posn`	Положение знака отрицательной суммы
`char *int_curr_symbol`	Международное обозначение валюты, например `"USD "`
`char int_frac_digits`	Международное число дробных разрядов
`char int_p_cs_precedes`, `int_n_cs_precedes`	Международное положение символа валюты
`char int_p_sep_by_space`, `int_n_sep_by_space`	Международные правила пробела
`char int_p_sign_posn`, `int_n_sign_posn`	Международные положения знака

26.2.1. Группировка денежных разрядов

`mon_grouping` указывает на последовательность байтов, описывающих размеры групп слева от десятичного разделителя, начиная справа. Например, в `$100,000,000.00` каждая группа содержит три цифры. Последовательность `3 3 3` описала бы три такие группы.

Чтобы не перечислять группы бесконечно, значение 0 означает повтор предыдущего размера. Следовательно, `3 0` задаёт группы по три цифры: `$1,000`, `$10,000,000` и т. д.

Правила могут быть сложнее. Последовательность `4 3 2 1 0` описывает, например:

```
$1,0,0,0,0,00,000,0000.00
```

Значение `CHAR_MAX` означает прекращение группировки. Правило `3 2 CHAR_MAX` может дать `100000000,00,000.00`. Если `CHAR_MAX` стоит в начале, группировки нет вовсе.

26.2.2. Пробелы и положение знака

Поля `*_sep_by_space` определяют пробелы:

Значение	Правило
0	Между символом валюты и суммой нет пробела
1	Символ валюты и связанный с ним знак отделяются пробелом от суммы
2	Если знак и символ валюты соседствуют, они разделяются пробелом; иначе знак отделяется пробелом от суммы

Поля `*_sign_posn` задают положение знака:

Значение	Правило
0	Заключить сумму и символ валюты в скобки
1	Поставить знак перед символом валюты и суммой
2	Поставить знак после символа валюты и суммы
3	Поставить знак непосредственно перед символом валюты
4	Поставить знак непосредственно после символа валюты

26.2.3. Пример значений

На системе автора получились следующие значения; элементы `mon_grouping` показаны как числа:

```
mon_decimal_point = "."
mon_thousands_sep = ","
mon_grouping = 3 3 0
positive_sign = ""
negative_sign = "-"
currency_symbol = "$"
frac_digits = 2
p_cs_precedes = 1
n_cs_precedes = 1
p_sep_by_space = 0
n_sep_by_space = 0
p_sign_posn = 1
n_sign_posn = 1
int_curr_symbol = "USD "
int_frac_digits = 2
int_p_cs_precedes = 1
int_n_cs_precedes = 1
int_p_sep_by_space = 1
int_n_sep_by_space = 1
int_p_sign_posn = 1
int_n_sign_posn = 1
```

26.3. Категории локали

Вместо `LC_ALL` можно настроить отдельную категорию:

Макрос	Что регулирует
`LC_ALL`	Все перечисленные категории
`LC_COLLATE`	Поведение `strcoll()` и `strxfrm()`
`LC_CTYPE`	Функции обработки символов, кроме `isdigit()` и `isxdigit()`
`LC_MONETARY`	Денежные значения, возвращаемые `localeconv()`
`LC_NUMERIC`	Числовое форматирование, включая десятичный разделитель в семействе `printf()`
`LC_TIME`	Форматирование даты и времени функциями `strftime()` и `wcsftime()`

Часто устанавливают `LC_ALL`, но можно выбрать и отдельные настройки. Особенно важна `LC_CTYPE`, поскольку она связана с обработкой широких символов — к этой большой теме мы ещё вернёмся.

Глава 27. Unicode, широкие символы и всё такое

Прежде чем начать, отметим: эта область языка C активно развивается. После выхода C23 здесь вероятны изменения.

Многих интересует обманчиво простой вопрос: «Как использовать определённый набор символов в C?» Мы к нему вернёмся. Возможно, всё уже работает в вашей системе, а возможно, придётся воспользоваться сторонней библиотекой.

В этой главе мы рассмотрим основы Unicode и кодировок, наборы символов исходного кода и исполнения, использование Unicode и UTF-8, а также типы `wchar_t`, `char16_t` и `char32_t`.

27.1. Что такое Unicode?

Раньше в США и значительной части мира было принято использовать для хранения символов семи- или восьмибитную кодировку. Всего получалось 128 или 256 символов, включая непечатаемые.

Для мира, ориентированного на США, этого хватало, но существуют и другие алфавиты. Например, в китайском языке десятки тысяч знаков, и в один байт они не помещаются.

Появилось множество способов представлять собственные наборы символов, что превратилось в кошмар совместимости. Чтобы решить эту проблему, придумали Unicode — единый набор символов. В нём есть китайские знаки, латиница, греческий алфавит, клинопись, шахматные символы, эмодзи и многое другое; новые символы постоянно добавляются.

27.2. Кодовые точки

Нам понадобятся два понятия. Путаница в том, что оба связаны с числами, но обозначают разные вещи.

Условно определим *кодовую точку* как числовое значение, представляющее символ, например букву `B` или знак `π`. Кодовые точки также могут обозначать непечатаемые управляющие символы.

Например, в Unicode числу 66 соответствует `B`, а числу 960 — `π`. В других наборах символов соответствия могут быть иными, но сосредоточимся на Unicode. Его кодовые точки охватывают значения от нуля до более чем миллиона.

27.3. Кодирование

Восьмибитный байт хранит значения от 0 до 255. Для `B`, чей код равен 66, этого достаточно. А для `π` с кодом 960 — уже нет. Для ещё больших значений понадобится несколько байтов.

Как представить эти числа в памяти? Это и определяет *кодировка*. Кодовая точка сообщает условный номер символа, а кодировка — способ записать этот номер в памяти.

Кодировок много; рассмотрим распространённые варианты Unicode:

Кодировка	Описание
UTF-8	Кодировка, использующая переменное число байтов на символ. Автор рекомендует её.
UTF-16	Кодировка на основе 16-битных единиц; для некоторых кодовых точек используются две единицы.
UTF-32	Кодировка на основе 32-битных единиц.

Для UTF-16 и UTF-32 важен порядок байтов: `BE` означает порядок от старшего байта, `LE` — от младшего. Иногда порядок задаётся начальной меткой BOM с кодовой точкой `0xFEFF`, но она необязательна.

Посмотрим на шестнадцатеричные примеры: две цифры соответствуют одному восьмибитному байту.

Символ	Код	UTF-16BE	UTF-32BE	UTF-16LE	UTF-32LE	UTF-8
A	41	0041	00000041	4100	41000000	41
B	42	0042	00000042	4200	42000000	42
~	7E	007E	0000007E	7E00	7E000000	7E
π	3C0	03C0	000003C0	C003	C0030000	CF80
€	20AC	20AC	000020AC	AC20	AC200000	E282AC

Обратите внимание на закономерности. В показанных примерах UTF-16BE и UTF-32BE непосредственно представляют кодовую точку 16- или 32-битным значением. Для UTF-16 это справедливо только для значений, представимых одной единицей. В варианте little-endian порядок байтов меняется.

В UTF-8 первые кодовые точки занимают по одному байту, а другие — разное количество байтов. Это кодировка переменной длины. После первых 128 кодовых точек значения байтов уже не совпадают непосредственно с числом кодовой точки.

Подробности кодирования UTF-8 выходят за рамки книги. Но важно, что UTF-8 обратно совместима с семибитной ASCII: любой документ ASCII также является корректным UTF-8, хотя обратное неверно. Вероятно, именно это сильно помогло распространению UTF-8.

27.4. Наборы символов исходного кода и исполнения

При программировании на C задействованы как минимум три набора символов:

- тот, в котором исходный файл записан на диске;

- набор исходных символов, в который компилятор преобразует файл в начале компиляции;
- набор символов исполнения, в который компилятор преобразует исходные символы для выполняемой программы.

Эти наборы могут совпадать или различаться. Вероятно, компилятор позволяет выбрать их при сборке.

Базовый набор исходных символов и символов исполнения включает латинские буквы, цифры, перечисленные ниже знаки, пробел и управляющие символы форматирования:

```
A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z
a b c d e f g h i j k l m
n o p q r s t u v w x y z
0 1 2 3 4 5 6 7 8 9
! " # % & ' ( ) * + , - . / :
; < = > ? [ \ ] ^ _ { | } ~
space tab vertical-tab
form-feed end-of-line
```

С этими символами исходный код остаётся максимально переносимым. Набор исполнения дополнительно содержит символы сигнала, возврата на одну позицию, возврата каретки и новой строки.

Однако большинство разработчиков свободно используют расширенные наборы, особенно теперь, когда Unicode и UTF-8 распространены. В описанном базовом наборе даже нет `@`, `$` и обратного апострофа! Вводить Unicode, пользуясь только базовым набором, неудобно, хотя и возможно благодаря управляющим последовательностям.

27.5. Unicode в C

В C можно указать кодовую точку Unicode, а компилятор преобразует её в набор символов исполнения. Возьмём символ евро с кодом `0x20AC`: в C оба способа его указания используют шестнадцатеричную запись.

В строке применим `\u`, после которого обязательно идут ровно четыре шестнадцатеричные цифры, при необходимости с ведущими нулями:

```
char *s = "\u20AC1.23";
printf("%s\n", s);  // €1.23
```

Для более крупных кодовых точек используется заглавная `\U` и восемь шестнадцатеричных цифр:

```
char *s = "\U0001D4D1";
printf("%s\n", s);  // Prints a mathematical letter "B"
```

Следующие записи эквивалентны:

```
\u03C0
\U000003C0
```

Они обозначают кодовые точки Unicode, а не конкретную кодировку. При компиляции они преобразуются в набор символов исполнения. Если кодовую точку невозможно в нём представить, результат зависит от компилятора.

Почему бы не написать символ непосредственно?

```
char *s = "€1.23";
printf("%s\n", s);  // €1.23
```

На современном компиляторе это, вероятно, сработает: он преобразует набор символов исходного файла в набор исполнения. Но компилятор может не принять символы вне поддерживаемого расширенного набора, а знак евро не входит в базовый.

Автор отмечает ограничение прежних редакций стандарта: `\u` и `\U` нельзя использовать для кодов ниже `0xA0`, кроме `0x24` (`$`), `0x40` (`@`) и `0x60` (обратный апостроф). В новых редакциях правила могут отличаться. При некоторых ограничениях универсальные имена символов можно использовать и в идентификаторах, но здесь мы сосредоточены на строках.

27.6. Короткое замечание об UTF-8

Возможно, ваш исходный файл, расширенные исходные символы и расширенные символы исполнения повсюду используют UTF-8, а библиотеки также ожидают UTF-8. Это замечательное будущее, в котором UTF-8 есть везде.

Если это так и вас не беспокоит переносимость на другие системы, используйте Unicode в исходном коде и данных, обычные строки C и радуйтесь. Многое будет работать, поскольку строки UTF-8, как и другие строки C, можно завершать нулевым байтом. Возможно, упрощение работы с символами стоит потери переносимости.

Однако есть нюансы:

- `strlen()` сообщает количество байтов, а не обязательно количество символов. `mbstowcs()` при преобразовании в широкие символы возвращает их количество; POSIX позволяет передать `NULL` в качестве первого аргумента, если нужно только количество.
- Функции `strtok()`, `strchr()` (вместо неё можно использовать `strstr()`), семейства `strspn()`, `toupper()`, `tolower()`, `isalpha()` и другие не обязательно правильно обрабатывают многобайтовые символы. Будьте осторожны с любой побайтовой операцией.
- Форматы `printf()`, например `%.12s`, позволяют ограничить вывод числом байтов. Убедитесь, что граница вывода совпадает с границей символа.
- Выделяя память под строку через `malloc()` или объявляя массив `char`, учитывайте, что символ может занимать больше одного байта — до `MB_LEN_MAX` из `<limits.h>`.

Символы базового набора гарантированно занимают один байт.

Возможно, существуют и другие подводные камни.

27.7. Разные типы символов

Мы уже знакомы с `char`, но существуют и другие типы.

27.7.1. Многобайтовые символы

Строка C — массив `char` — рассматривается как многобайтовая строка, состоящая из многобайтовых символов. Это название сохраняется, даже если каждый конкретный символ занимает один байт:

```
char c[128] = "Hello, world!";  // Multibyte string
```

Символ вне базового набора может состоять из нескольких байтов, вплоть до `MB_LEN_MAX`. На экране он может выглядеть одним символом, но в памяти занимать больше места. Как мы видели, можно указать Unicode:

```
char *s = "\u20AC1.23";
printf("%s\n", s);  // €1.23
```

Однако:

```
char *s = "\u20AC1.23";     // €1.23
printf("%zu\n", strlen(s));  // 7!
```

На системе автора длина равна 7: `strlen()` считает байты, а не символы. Позже мы увидим, как работать с широкими символами.

C допускает символьные константы с многобайтовыми символами, но их поведение зависит от реализации, и компилятор может предупредить. Например, GCC предупреждает для следующих строк; на системе автора они выводят представление UTF-8:

```
printf("%x\n", '€');
printf("%x\n", '\u20ac');
```

27.7.2. Широкие символы

Широкий символ — отдельное значение, способное представлять символ текущей локали. Это напоминает кодовые точки Unicode, хотя конкретное представление может не совпадать с ними.

Многобайтовая строка — массив байтов, а строка широких символов — массив широких значений. Это позволяет рассуждать по символам, а не по отдельным байтам переменной длины.

Главный тип для этого — `wchar_t`, словно расширенный `char`. Для строк `wchar_t` есть множество функций, например получения длины, позволяющих работать с ними, не вникая каждый раз в кодировку.

27.8. Использование широких символов и wchar_t

Для `wchar_t` подключите `<wchar.h>`. Его размер зависит от реализации: он может быть 16 или 32 бита. Шестнадцати бит недостаточно для всех кодовых точек Unicode, но стандарт этого и не требует: тип должен представлять символы текущей локали. На платформах с 16-битным `wchar_t`, в том числе Windows, это может усложнять работу с Unicode.

Широкую строку или символ записывают с префиксом `L`. Для вывода используются `%ls` и `%lc`:

```
wchar_t *s = L"Hello, world!";
wchar_t c = L'B';

printf("%ls %lc\n", s, c);
```

Являются ли значения кодовыми точками Unicode, зависит от реализации. Это можно проверить по макросу `__STDC_ISO_10646__`. Если он определён, его значение имеет форму `yyyymm` и указывает версию стандарта Unicode, на которую можно рассчитывать.

27.8.1. Преобразование между многобайтовыми и широкими строками

В именах функций используются сокращения `mb` — многобайтовый символ, `wc` — широкий символ, `mbs` — многобайтовая строка, `wcs` — широкая строка.

Функция	Преобразование
`mbtowc()`	Многобайтовый символ в широкий.
`wctomb()`	Широкий символ в многобайтовый.
`mbstowcs()`	Многобайтовая строка в широкую.
`wcstombs()`	Широкая строка в многобайтовую.

Преобразуем многобайтовую строку и сравним длины:

```
#include <stdio.h>
#include <stdlib.h>
#include <wchar.h>
#include <string.h>
#include <locale.h>

int main(void)
{
    // Get out of the C locale to one that likely has the euro symbol
    setlocale(LC_ALL, "");

    // Original multibyte string with a euro symbol (Unicode point 20ac)
    char *mb_string = "The cost is \u20ac1.23";  // €1.23
```

```
    size_t mb_len = strlen(mb_string);

    // Wide character array that will hold the converted string
    wchar_t wc_string[128];  // Holds up to 128 wide characters

    // Convert the MB string to WC; this returns the number of wide chars
    size_t wc_len = mbstowcs(wc_string, mb_string, 128);

    // Print result--note the %ls for wide char strings
    printf("multibyte: \"%s\" (%zu bytes)\n", mb_string, mb_len);
    printf("wide char: \"%ls\" (%zu characters)\n", wc_string, wc_len);
}
```

На системе автора выводится:

```
multibyte: "The cost is €1.23" (19 bytes)
wide char: "The cost is €1.23" (17 characters)
```

Число байтов может отличаться в зависимости от локали. Помимо преобразования, `mbstowcs()` возвращает длину широкой строки в широких символах. В системах POSIX можно передать `NULL` как назначение и 0 как предел, чтобы только узнать длину:

```
setlocale(LC_ALL, "");

// The following string has 7 characters
size_t len_in_chars = mbstowcs(NULL, "§¶°±π€•", 0);
printf("%zu", len_in_chars);  // 7
```

Для обратного преобразования применяется `wcstombs()`. Поддержка конкретных символов и успешность преобразования зависят от локали; в реальной программе проверяйте ошибки.

27.9. Возможности для работы с широкими символами

Для широких строк существует множество аналогов привычных функций многобайтовых строк. Например, наряду с `strlen()` есть `wcslen()`.

27.9.1. wint_t

Многие функции принимают или возвращают отдельные символы через тип `wint_t`. Он связан с `wchar_t`: это целый тип, способный представить значения расширенного набора символов и специальное значение конца файла `WEOF`.

27.9.2. Ориентация потока ввода-вывода

Коротко: не смешивайте побайтовые функции, например `fprintf()`, с функциями широкого ввода-вывода, например `fwprintf()`. Выберите для потока одну ориентацию и используйте соответствующие функции.

Новый поток не имеет ориентации. Первая операция ввода или вывода задаёт её: широкая операция вроде `fwprintf()` делает поток широким, а побайтовая вроде `fprintf()` —

байтовым. Функция `fwide()` позволяет установить ориентацию неориентированного потока или узнать текущую. Для изменения ориентации уже используемого потока применяется `freopen()`.

27.9.3. Функции ввода-вывода

Обычно подключают `<stdio.h>` и `<wchar.h>`.

Функция	Назначение
`wprintf()`, `wscanf()`	Форматированный вывод в консоль и ввод из неё.
`getwchar()`, `putwchar()`	Посимвольный ввод из консоли и вывод в неё.
`fwprintf()`, `fwscanf()`	Форматированный вывод в поток и ввод из него.
`fgetwc()`, `fputwc()`	Посимвольный ввод из потока и вывод в него.
`fgetws()`, `fputws()`	Построчный ввод из потока и вывод в него.
`swprintf()`, `swscanf()`	Форматированный вывод в строку и ввод из неё.
`vfwprintf()`, `vfwscanf()`	Форматированный ввод-вывод в поток с переменным списком аргументов.
`vswprintf()`, `vswscanf()`	Форматированный ввод-вывод для строки с переменным списком аргументов.
`vwprintf()`, `vwscanf()`	Форматированный ввод-вывод для консоли с переменным списком аргументов.
`ungetwc()`	Возвращает широкий символ обратно во входной поток.
`fwide()`	Узнаёт или задаёт ориентацию потока.

27.9.4. Преобразование типов

Обычно нужен `<wchar.h>`:

Функция	Преобразует строку в
`wcstod()`	`double`
`wcstof()`	`float`
`wcstold()`	`long double`
`wcstol()`	`long`
`wcstoll()`	`long long`
`wcstoul()`	`unsigned long`
`wcstoull()`	`unsigned long long`

27.9.5. Копирование строк и памяти

Для этих функций обычно подключают `<wchar.h>`:

Функция	Назначение
`wcscpy()`	Копирует строку.
`wcsncpy()`	Копирует строку с ограничением длины.
`wmemcpy()`	Копирует память.
`wmemmove()`	Копирует память, области которой могут перекрываться.
`wcscat()`	Объединяет строки.
`wcsncat()`	Объединяет строки с ограничением длины.

27.9.6. Сравнение строк и памяти

Обычно нужен `<wchar.h>`:

Функция	Назначение
`wcscmp()`	Лексикографически сравнивает строки.
`wcsncmp()`	Сравнивает строки с ограничением длины.
`wcscoll()`	Сравнивает строки по порядку текущей локали.
`wmemcmp()`	Лексикографически сравнивает области памяти.
`wcsxfrm()`	Преобразует строки так, чтобы последующее сравнение через `wcscmp()` соответствовало `wcscoll()`.

27.9.7. Поиск в строках

Обычно нужен `<wchar.h>`:

Функция	Назначение
`wcschr()`	Ищет символ в строке.
`wcsrchr()`	Ищет символ с конца строки.
`wmemchr()`	Ищет символ в памяти.
`wcsstr()`	Ищет подстроку.
`wcspbrk()`	Ищет любой символ из заданного набора.
`wcsspn()`	Находит длину начального участка из символов заданного набора.
`wcscspn()`	Находит длину начального участка до первого символа заданного набора.

Функция	Назначение
`wcstok()`	Разбивает строку на лексемы.

27.9.8. Длина и другие операции

Обычно нужен `<wchar.h>`:

Функция	Назначение
`wcslen()`	Возвращает длину строки.
`wmemset()`	Заполняет память широкими символами.
`wcsftime()`	Форматирует дату и время.

27.9.9. Классификация символов

Подключите `<wctype.h>`:

Функция	Проверяет, является ли символ
`iswalnum()`	буквой или цифрой;
`iswalpha()`	буквой;
`iswblank()`	горизонтальным пробельным символом;
`iswcntrl()`	управляющим;
`iswdigit()`	цифрой;
`iswgraph()`	печатным, кроме пробела;
`iswlower()`	буквой нижнего регистра;
`iswprint()`	печатным, включая пробел;
`iswpunct()`	знаком пунктуации;
`iswspace()`	пробельным символом;
`iswupper()`	буквой верхнего регистра;
`iswxdigit()`	шестнадцатеричной цифрой.

`towlower()` переводит символ в нижний регистр, а `towupper()` — в верхний.

27.10. Состояние разбора и возобновляемые функции

Немного заглянем внутрь процесса многобайтового преобразования. Представьте, что программа превращает последовательность многобайтовых символов в широкие или наоборот. В какой-то момент она может находиться посреди разбора символа и ждать дополнительные байты, прежде чем определить значение.

Состояние разбора хранится в непрозрачной переменной типа `mbstate_t` и используется при преобразовании. Если посреди работы переключиться на другую последовательность или перейти к другому месту входных данных, состояние может оказаться неподходящим.

Но выше мы выполняли преобразования и не упоминали `mbstate_t`! Дело в том, что функции вроде `mbstowcs()` и `wctomb()` используют собственное внутреннее состояние. Оно общее для соответствующей функции, поэтому такие вызовы могут быть небезопасны в многопоточном коде.

К счастью, C определяет возобновляемые версии, которым можно передавать собственное состояние отдельно для каждого потока. Для многопоточной работы используйте их.

Чтобы установить начальное состояние `mbstate_t`, заполните его нулями:

```
mbstate_t mbs;

// Set the state to the initial state
memset(&mbs, 0, sizeof mbs);
```

В именах возобновляемых функций после обозначения исходного типа добавляется `r`:

- `mbrtowc()` — многобайтовый символ в широкий;
- `wcrtomb()` — широкий символ в многобайтовый;
- `mbsrtowcs()` — многобайтовая строка в широкую;
- `wcsrtombs()` — широкая строка в многобайтовую.

Они похожи на обычные варианты, но принимают указатель на собственный `mbstate_t`. Функции работы со строками также изменяют указатель на исходную строку, что помогает определить место ошибки; поэтому может понадобиться сохранить копию исходного указателя.

Перепишем предыдущий пример с собственным состоянием:

```
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>
#include <wchar.h>
#include <string.h>
#include <locale.h>

int main(void)
{
    // Get out of the C locale to one that likely has the euro symbol
    setlocale(LC_ALL, "");

    // Original multibyte string with a euro symbol (Unicode point 20ac)
    char *mb_string = "The cost is \u20ac1.23";  // €1.23
    size_t mb_len = strlen(mb_string);

    // Wide character array that will hold the converted string
    wchar_t wc_string[128];  // Holds up to 128 wide characters

    // Set up the conversion state
```

```
    mbstate_t mbs;
    memset(&mbs, 0, sizeof mbs);  // Initial state

    // mbsrtowcs() modifies the input pointer to point at the first
    // invalid character, or NULL if successful. Let's make a copy of
    // the pointer for mbsrtowcs() to mess with so our original is
    // unchanged.
    //
    // This example will probably be successful, but we check farther
    // down to see.
    const char *invalid = mb_string;

    // Convert the MB string to WC; this returns the number of wide chars
    size_t wc_len = mbsrtowcs(wc_string, &invalid, 128, &mbs);

    if (invalid == NULL) {
        printf("No invalid characters found\n");

        // Print result--note the %ls for wide char strings
        printf("multibyte: \"%s\" (%zu bytes)\n", mb_string, mb_len);
        printf("wide char: \"%ls\" (%zu characters)\n", wc_string, wc_len);
    } else {
        ptrdiff_t offset = invalid - mb_string;
        printf("Invalid character at offset %td\n", offset);
    }
}
```

Автор также приводит пример сброса внутреннего состояния функций преобразования:

```
mbstowcs(NULL, NULL, 0);   // Reset the parse state for mbstowcs()
mbstowcs(dest, src, 100);  // Parse some stuff
```

Каждый ориентированный на широкие символы поток ввода-вывода управляет собственным `mbstate_t`. Некоторые побайтовые функции вроде `printf()` и `scanf()` также используют внутреннее состояние. Возобновляемые функции тоже используют внутреннее состояние, если передать `NULL` вместо указателя на `mbstate_t`.

27.11. Кодировки Unicode и C

Рассмотрим возможности и ограничения C для UTF-8, UTF-16 и UTF-32.

27.11.1. UTF-8

Возможности стандартной библиотеки для UTF-8 довольно ограниченны. Но можно явно указать, что строковый литерал должен быть закодирован в UTF-8, поставив перед ним `u8`:

```
char *s = u8"Hello, world!";
printf("%s\n", s);  // Hello, world!--if you can output UTF-8
```

Можно ли добавить символы Unicode?

```
char *s = u8"€123";
```

Да, если расширенный набор исходных символов их поддерживает. В противном случае можно указать кодовую точку через `\u` или `\U`, как показано выше.

В остальном стандартная библиотека не предлагает общего переносимого способа преобразовывать произвольный ввод в UTF-8 или разбирать UTF-8, если текущая локаль не использует UTF-8. Поэтому можно выбрать локаль пользователя:

```
setlocale(LC_ALL, "");
```

Либо узнать имя подходящей локали в системе и указать его явно:

```
setlocale(LC_ALL, "en_US.UTF-8");  // Non-portable name
```

Ещё один вариант — сторонняя библиотека.

27.11.2. UTF-16, UTF-32, char16_t и char32_t

`char16_t` и `char32_t` — типы для представления соответственно 16- и 32-битных единиц. Их нельзя безусловно считать широкими символами, если они не представляют любой символ текущей локали, хотя стандарт нередко называет их широкими символьными типами.

Они предназначены для более удобной работы с Unicode. Чтобы использовать их, подключите `<uchar.h>`. Автор отмечает, что на использованной им системе OS X этот заголовок отсутствовал. Если нужны только типы, он предлагает:

```
#include <stdint.h>

typedef int_least16_t char16_t;
typedef int_least32_t char32_t;
```

Но функции преобразования таким способом не получить. Для объявления строк и символов применяются префиксы `u` и `U`:

```
char16_t *s = u"Hello, world!";
char16_t c = u'B';

char32_t *t = U"Hello, world!";
char32_t d = U'B';
```

Хранятся ли эти значения в UTF-16 и UTF-32, зависит от реализации. Если `__STDC_UTF_16__` или `__STDC_UTF_32__` определены как 1, соответствующий тип использует эту кодировку. При этом используется собственный порядок байтов системы.

Автор приводит пример сравнения с кодовой точкой:

```
char16_t pi = u"\u03C0";  // pi symbol

#if __STDC_UTF_16__
    pi == 0x3C0;  // Always true
#else
```

```
    pi == 0x3C0;  // Probably not true
#endif
```

Замечание к примеру: для одиночного символа здесь нужен символьный литерал `u'\u03C0'`, а не строковый литерал в двойных кавычках.

27.11.3. Многобайтовые преобразования

Для преобразования между многобайтовой кодировкой и `char16_t` или `char32_t` есть вспомогательные функции. Как сказано выше, результат не обязательно является UTF-16 или UTF-32, если соответствующий макрос не установлен в 1.

Все эти функции возобновляемые: им передают собственное состояние `mbstate_t`. Они работают с отдельными символами или единицами кодирования; при использовании нескольких единиц `char16_t` ситуация становится сложнее.

Функция	Преобразование
`mbrtoc16()`	Многобайтовый символ в `char16_t`.
`mbrtoc32()`	Многобайтовый символ в `char32_t`.
`c16rtomb()`	`char16_t` в многобайтовый символ.
`c32rtomb()`	`char32_t` в многобайтовый символ.

27.11.4. Сторонние библиотеки

Для серьёзного преобразования между конкретными кодировками существуют зрелые библиотеки. Автор отмечает, что сам ни одной из этих двух не пользовался:

- **iconv** — распространённый стандартизованный POSIX API преобразования кодировок, доступный на основных платформах;
- **ICU**, International Components for Unicode — библиотека для работы с Unicode и интернационализацией.

Глава 28. Завершение программы

Оказывается, завершить программу можно многими способами. Можно даже установить обработчики, которые выполнятся при её завершении.

В этой главе мы рассмотрим эти способы. Значение кода завершения уже обсуждалось в соответствующем разделе — при необходимости вернитесь к нему.

Функции завершения из этой главы объявлены в `<stdlib.h>`; для `assert()` нужен `<assert.h>`.

28.1. Нормальное завершение

Начнём с обычных способов, а затем перейдём к более редким и необычным.

При нормальном завершении открытые потоки ввода-вывода сбрасываются, а временные файлы удаляются. Иными словами, программа завершается аккуратно, с необходимой очисткой. Почти всегда следует выбирать этот способ, если нет причин поступить иначе.

28.1.1. Возврат из main()

Возможно, вы заметили, что `main()` возвращает `int`, хотя я редко, если вообще когда-либо, использовал в ней `return`.

Дело в том, что только для `main()` действует особое правило: если выполнение доходит до конца функции, подразумевается `return 0`. Подчеркну: это особое правило относится только к `main()`, а не к другим функциям.

Можно явно возвращать значение из `main()` в любой момент. Некоторые программисты считают более правильным всегда ставить `return` в конце, но если его опустить, C сделает это за вас.

Итак, правила:

- Оператор `return` в `main()` задаёт код завершения программы. В других функциях `return` лишь возвращает управление вызывающему коду.
- Если выполнение дошло до конца `main()` без явного возврата, результат такой же, как при возврате `0` или `EXIT_SUCCESS`.

28.1.2. exit()

Эта функция уже встречалась нам несколько раз. Если вызвать `exit()` в любом месте программы, она завершится в этой точке. Аргумент `exit()` задаёт код завершения.

28.1.3. Регистрация обработчиков завершения через atexit()

Можно зарегистрировать функции, которые вызовутся при завершении программы — как при возврате из `main()`, так и при вызове `exit()`.

Для регистрации передайте имя функции в `atexit()`. Можно зарегистрировать несколько обработчиков; они выполняются в обратном порядке регистрации.

```
#include <stdio.h>
#include <stdlib.h>

void on_exit_1(void)
{
    printf("Exit handler 1 called!\n");
}

void on_exit_2(void)
{
    printf("Exit handler 2 called!\n");
}

int main(void)
{
    atexit(on_exit_1);
    atexit(on_exit_2);

    printf("About to exit...\n");
}
```

Вывод:

```
About to exit...
Exit handler 2 called!
Exit handler 1 called!
```

28.2. Быстрое завершение с quick_exit()

Оно похоже на обычное завершение, но:

- буферы открытых файлов могут не сброситься;
- временные файлы могут не удалиться;
- обработчики `atexit()` не вызываются.

Однако можно зарегистрировать обработчики через `at_quick_exit()` аналогично `atexit()`:

```
#include <stdio.h>
#include <stdlib.h>

void on_quick_exit_1(void)
{
    printf("Quick exit handler 1 called!\n");
}

void on_quick_exit_2(void)
{
    printf("Quick exit handler 2 called!\n");
}

void on_exit(void)
{
```

```
    printf("Normal exit--I won't be called!\n");
}

int main(void)
{
    at_quick_exit(on_quick_exit_1);
    at_quick_exit(on_quick_exit_2);

    atexit(on_exit);  // This won't be called

    printf("About to quick exit...\n");

    quick_exit(0);
}
```

Пример вывода:

```
About to quick exit...
Quick exit handler 2 called!
Quick exit handler 1 called!
```

Всё работает подобно `exit()` и `atexit()`, но сброс файловых буферов и очистка могут не выполняться.

28.3. Немедленное завершение: _Exit()

`_Exit()` завершает программу немедленно. Обработчики завершения не выполняются; сброс файлов и удаление временных файлов не гарантируются. Используйте эту функцию, когда необходимо выйти прямо сейчас.

28.4. Условное завершение: assert()

`assert()` проверяет требуемое условие и завершает программу, если оно ложно. Разработчики часто используют проверки для ошибок из разряда «этого никогда не должно случиться».

```
#define PI 3.14159
assert(PI > 3);  // Sure enough, it is, so carry on
```

Или:

```
goats -= 100;
assert(goats >= 0);  // Can't have negative goats
```

Если `goats` станет меньше нуля, возможен такой результат:

```
goat_counter: goat_counter.c:8: main: Assertion `goats >= 0' failed.
Aborted
```

После этого управление вернётся в командную строку. Такой вывод не очень удобен для конечного пользователя, поэтому проверки обычно предназначены для ситуаций, которые пользователь не должен видеть. Также нередко пишут собственные макросы проверок, которыми проще управлять.

28.5. Аварийное завершение: abort()

Эта функция пригодится, если случилось нечто серьёзное и нужно сообщить об аварийном завершении внешнему окружению. Она также не гарантирует очистку открытых файлов и других ресурсов. Автор отмечает, что редко встречал её применение.

Заглядывая вперёд, к сигналам: `abort()` вызывает сигнал `SIGABRT`, который завершает процесс. Дальнейшее поведение зависит от системы; в Unix-подобных системах при завершении нередко создаётся дамп памяти процесса.

Глава 29. Обработка сигналов

Автор советует в большинстве случаев использовать более развитые средства своей операционной системы: в Unix-подобных системах, например, семейство `sigaction()`. В Windows механизм отличается; сигналы в стиле Unix для консольных программ могут имитироваться.

29.1. Что такое сигнал?

Сигнал возникает при определённых событиях. Программа может быть прервана для его обработки и иногда продолжить работу после неё. Удобно представить это как автоматически вызываемую функцию.

Стандарт C определяет несколько сигналов:

Сигнал	Значение
`SIGABRT`	Аварийное завершение, например при вызове `abort()`
`SIGFPE`	Арифметическое исключение
`SIGILL`	Недопустимая инструкция
`SIGINT`	Прерывание, обычно при нажатии Ctrl+C
`SIGSEGV`	Недопустимое обращение к памяти
`SIGTERM`	Запрос на завершение

Функция `signal()` позволяет установить обработчик, игнорирование или действие по умолчанию.

29.2. Функция signal()

`signal()` принимает номер сигнала и действие: указатель на функцию-обработчик, `SIG_IGN` для игнорирования или `SIG_DFL` для восстановления действия по умолчанию.

Программа ниже игнорирует Ctrl+C; завершить её можно клавишей Enter:

```
#include <stdio.h>
#include <signal.h>

int main(void)
{
    char s[1024];
    signal(SIGINT, SIG_IGN); // Игнорировать SIGINT от Ctrl+C
    printf("Try hitting ^C... (hit RETURN to exit)\n");
    fgets(s, sizeof s, stdin);
}
```

Если убрать вызов `signal()`, Ctrl+C обычно завершит программу.

29.3. Обработчики сигналов

Можно передать собственную функцию. Возможности такой функции сильно ограничены. Посмотрим на объявление `signal()`:

```
void (*signal(int sig, void (*func)(int)))(int);
```

Легко читается? Нет? Разберём: `sig` — номер сигнала, `func` — указатель на функцию, принимающую `int` и возвращающую `void`:

```
void f(int x);
```

Сама `signal()` возвращает указатель на прежний обработчик того же типа либо `SIG_ERR` при ошибке.

Следующий пример демонстрирует обработчик, при котором выход происходит после двух нажатий Ctrl+C. **В примере есть неопределённое поведение**: доступ к обычной глобальной переменной и вызовы `printf()`/`exit()` из асинхронного обработчика не являются переносимо безопасными. Это учебная иллюстрация, не образец для практического применения.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

int count = 0;

void sigint_handler(int signum)
{
    // Реализация может восстановить действие по умолчанию перед вызовом.
    signal(SIGINT, sigint_handler);
    (void)signum; // Подавить предупреждение

    count++;                          // Неопределённое поведение
    printf("Count: %d\n", count); // Неопределённое поведение
    if (count == 2) {
        printf("Exiting!\n");     // Неопределённое поведение
        exit(0);
    }
}

int main(void)
{
    signal(SIGINT, sigint_handler);
    printf("Try hitting ^C...\n");
    for (;;); // Бесконечное ожидание
}
```

Вызов `signal()` внутри обработчика повторно устанавливает обработчик, поскольку реализация может вернуть действие `SIG_DFL` перед его вызовом. Предыдущий обработчик можно сохранить:

```
// Указатель на функцию, принимающую int и возвращающую void:
void (*old_handler)(int);
old_handler = signal(SIGINT, sigint_handler);
```

Выражение `(void)signum` лишь сообщает компилятору, что параметр намеренно не используется.

29.4. Что действительно разрешено?

Переносимые обработчики сигналов сильно ограничены. Нельзя свободно вызывать библиотечные функции вроде `printf()` или обращаться к обычным объектам со статической либо потоковой длительностью хранения. Для обмена с основной программой стандарт допускает, в частности, присваивание переменной `volatile sig_atomic_t`; также есть отдельные возможности для атомарных объектов без блокировок. Правила платформы могут предоставлять дополнительные гарантии, но это уже не переносимый C.

`sig_atomic_t` — целочисленный тип; его диапазон определяется макросами `SIG_ATOMIC_MIN` и `SIG_ATOMIC_MAX`. Автор рассматривает следующий вариант с инкрементом, но предупреждает: `count++` не просто присваивает значение, а сначала читает его; в строго переносимом обработчике это не следует считать безопасным.

```
#include <stdio.h>
#include <signal.h>

volatile sig_atomic_t count = 0;

void sigint_handler(int signum)
{
    (void)signum;
    signal(SIGINT, sigint_handler);
    count++; // Чтение и изменение: не переносимо безопасно
}

int main(void)
{
    signal(SIGINT, sigint_handler);
    printf("Hit ^C twice to exit.\n");
    while (count < 2);
}
```

Для простой передачи события предпочтительнее присвоить флагу фиксированное значение, а реагировать на него в обычном потоке программы. Если же достаточно, чтобы второе нажатие выполнило действие по умолчанию, обработчик может восстановить его после первого:

```
#include <stdio.h>
#include <signal.h>

void sigint_handler(int signum)
{
    (void)signum;
    signal(SIGINT, SIG_DFL);
}
```

```
int main(void)
{
    signal(SIGINT, sigint_handler);
    printf("Hit ^C twice to exit.\n");
    while (1);
}
```

Для более сложного счётчика могут подойти атомарные объекты без блокировок, если они доступны в реализации. Во многих практических задачах удобнее использовать средства обработки сигналов своей ОС.

29.5. Почему стоит выбрать средства ОС

Платформенные механизмы не являются частью стандарта C и потому менее переносимы между ОС, но обычно значительно богаче. ОС также может определять сигналы, которых нет в стандарте C. При этом и переносимый код на одном `signal()` написать непросто.

Глава 30. Массивы переменной длины (VLA)

C позволяет объявить массив, размер которого определяется во время выполнения. Это даёт динамический размер, как при `malloc()`, но без необходимости явно вызывать `free()`.

Многие разработчики не любят VLA: например, их запретили в ядре Linux. Причины рассмотрим ниже. VLA были обязательны в C99, а с C11 поддержка автоматических VLA стала необязательной. Макрос `__STDC_NO_VLA__`, если определён как 1, указывает на соответствующие ограничения реализации:

```
#if __STDC_NO_VLA__ == 1
#error Sorry, need VLAs for this program!
#endif
```

При проверке конкретного компилятора учитывайте версию стандарта и документацию реализации.

30.1. Основы

Обычный массив объявляется с постоянным размером:

```
int v[10];
```

Для VLA размер может вычисляться во время работы программы:

```
int n = 10;
int v[n];
```

Попросим пользователя ввести длину и заполним элементы значениями индекса, умноженного на 10:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    char buf[32];
    printf("Enter a number: "); fflush(stdout);
    fgets(buf, sizeof buf, stdin);
    n = strtoul(buf, NULL, 10);

    int v[n];
    for (int i = 0; i < n; i++)
        v[i] = i * 10;
    for (int i = 0; i < n; i++)
        printf("v[%d] = %d\n", i, v[i]);
}
```

`fflush()` обеспечивает вывод приглашения без завершающего перевода строки. Размер массива фиксируется при выполнении его объявления и впоследствии не меняется. В скобках допустимо выражение:

```
int v[x * 100];
```

Ограничения: VLA нельзя объявить в области файла или как `static` внутри блока; обычный список инициализаторов для него использовать нельзя. Вычисленный размер должен быть положительным: ноль или отрицательное значение приводят к неопределённому поведению. Пример ввода выше иллюстративен; в реальной программе проверяйте ошибки чтения, преобразования и допустимый размер до объявления массива.

30.2. sizeof и VLA

`sizeof` работает с VLA как с обычным массивом, но размер определяется во время выполнения. Число элементов можно вычислить обычным способом:

```
size_t num_elems = sizeof v / sizeof v[0];
```

Арифметика указателей также работает привычно:

```
#include <stdio.h>

int main(void)
{
    int n = 5;
    int v[n];
    int *p = v;
    *(p+2) = 12;
    printf("%d\n", v[2]); // 12
    p[3] = 34;
    printf("%d\n", v[3]); // 34
}
```

Можно узнать размер типа VLA, не объявляя сам массив:

```
int x = 12;
printf("%zu\n", sizeof(int [x])); // На системе автора: 48
```

30.3. Многомерные VLA

Переменными могут быть один или несколько размеров:

```
int w = 10;
int h = 20;
int x[h][w];
int y[5][w];
int z[10][w][20];
```

Доступ к элементам осуществляется как у обычных массивов.

30.4. Передача одномерного VLA в функцию

Одномерный VLA передаётся так же, как обычный массив:

```
#include <stdio.h>

int sum(int count, int *v)
{
    int total = 0;
    for (int i = 0; i < count; i++)
        total += v[i];
    return total;
}

int main(void)
{
    int x[5];
    int a = 5;
    int y[a];
    for (int i = 0; i < a; i++)
        x[i] = y[i] = i + 1;
    printf("%d\n", sum(5, x));
    printf("%d\n", sum(a, y));
}
```

Размер можно указать в списке параметров после параметра, который его задаёт:

```
int sum(int count, int v[count])
{
    // ...
}
```

В прототипе вместо размера можно использовать `*`:

```
void do_something(int count, int v[count]);
void do_something(int, int v[*]);
```

Эта форма `*` предназначена для области прототипа; в определении функции используется выражение размера. Параметр массива при этом корректируется до указателя.

30.5. Передача многомерных VLA

Передадим размеры и матрицу в функцию вывода таблицы умножения:

```
#include <stdio.h>

void print_matrix(int h, int w, int m[h][w])
{
    for (int row = 0; row < h; row++) {
        for (int col = 0; col < w; col++)
            printf("%2d ", m[row][col]);
        printf("\n");
```

```
    }
}

int main(void)
{
    int rows = 4;
    int cols = 7;
    int matrix[rows][cols];
    for (int row = 0; row < rows; row++)
        for (int col = 0; col < cols; col++)
            matrix[row][col] = row * col;
    print_matrix(rows, cols, matrix);
}
```

30.5.1. Частично переменные размеры

Некоторые размеры могут быть постоянными. Например, число записей переменно, а каждая содержит пять элементов:

```
#include <stdio.h>

void print_records(int count, int record[count][5])
{
    for (int i = 0; i < count; i++) {
        for (int j = 0; j < 5; j++)
            printf("%2d ", record[i][j]);
        printf("\n");
    }
}

int main(void)
{
    int rec_count = 3;
    int records[rec_count][5];
    for (int i = 0; i < rec_count; i++)
        for (int j = 0; j < 5; j++)
            records[i][j] = (i+1)*(j+2);
    print_records(rec_count, records);
}
```

30.6. Совместимость с обычными массивами

Если размеры согласованы, можно передавать VLA функции, объявленной для обычного массива:

```
int foo(int m[5][3]) { /* ... */ }
// ...
int w = 3, h = 5;
int matrix[h][w];
foo(matrix);
```

И наоборот:

```
int foo(int h, int w, int m[h][w]) { /* ... */ }
// ...
```

```
int matrix[3][5];
foo(3, 5, matrix);
```

Несовпадение размеров может привести к неопределённому поведению.

30.7. typedef и VLA

`typedef` запоминает размеры, вычисленные в момент выполнения объявления, а не пересчитывает их при каждом использовании имени типа:

```
#include <stdio.h>

int main(void)
{
    int w = 10;
    typedef int goat[w];
    goat x; // Массив из 10 int

    for (int i = 0; i < w; i++)
        x[i] = i*i;
    for (int i = 0; i < w; i++)
        printf("%d\n", x[i]);

    w = 20;
    // goat по-прежнему обозначает массив из 10 int:
    // именно таким был w при объявлении typedef.
}
```

В этом отношении размер остаётся фиксированным, но использовать обычный список инициализаторов по-прежнему нельзя.

30.8. Особенности переходов

С `goto` рядом с VLA нужно быть осторожным: некоторые переходы запрещены. При определённых переходах через `longjmp()` также может возникнуть утечка памяти VLA. Подробности — в главах об этих конструкциях.

30.9. Общие проблемы

В ядре Linux VLA запретили по нескольким причинам:

- во многих местах хватало массива фиксированного размера;
- код работы с VLA может быть медленнее, что важно для ОС;
- компиляторы C поддерживают их неодинаково;
- VLA часто размещаются на ограниченном стеке, и чрезмерный размер, переданный случайно или намеренно, может вызвать серьёзные проблемы.

Кроме того, у программы нет обычного способа проверить неудачное выделение VLA: она может просто аварийно завершиться. С фиксированными массивами большие размеры тоже опасны, но случайно запросить огромный размер переменного массива гораздо легче.

Глава 31. goto

Оператор `goto` всеобщее почитаем, и его можно представить без каких-либо споров.

Шутка! Годами шли дискуссии о том, считается ли `goto` вредным. По мнению этого программиста, следует выбирать конструкции, позволяющие получить лучший код с учётом удобства сопровождения и скорости. Иногда это может быть и `goto`!

В этой главе мы посмотрим, как `goto` работает в C, а затем рассмотрим типичные случаи его применения. Во всех приведённых случаях без `goto` можно обойтись, используя переменные и циклы; просто некоторые считают, что здесь `goto` даёт лучший код.

31.1. Простой пример

Используем `goto`, чтобы пропустить строку кода и перейти к *метке*. Метка — идентификатор, который может служить целью перехода; после него ставится двоеточие (`:`).

```
#include <stdio.h>

int main(void)
{
    printf("One\n");
    printf("Two\n");

    goto skip_3;

    printf("Three\n");

skip_3:

    printf("Five!\n");
}
```

Результат:

```
One
Two
Five!
```

`goto` передаёт управление указанной метке, пропуская всё между оператором и меткой. Переходить можно как вперёд, так и назад:

```
infinite_loop:
    print("Hello, world!\n");
    goto infinite_loop;
```

При обычном выполнении метки пропускаются. Следующий код выведет все числа по порядку, как если бы меток не было:

```
printf("Zero\n");
label_1:
label_2:
printf("One\n");
label_3:
printf("Two\n");
label_4:
printf("Three\n");
```

Как вы заметили, метки принято выравнивать по левому краю. Это улучшает читаемость: место перехода легче быстро найти взглядом.

Метки имеют область видимости функции. Независимо от глубины вложенности блоков, к метке можно перейти из любого места той же функции. Но к метке другой функции перейти нельзя: она вне области видимости. В разных функциях можно использовать одинаковые имена меток, но в одной функции повторять имя нельзя.

31.2. continue с меткой

В некоторых языках оператору `continue` можно указать метку. В C такой возможности нет, но вместо неё легко использовать `goto`.

Посмотрите на `continue` во вложенных циклах:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d, %d\n", i, j);
        continue; // Always goes to next j
    }
}
```

Как и всегда, `continue` переходит к следующей итерации ближайшего окружающего цикла. А если мы хотим перейти к следующей итерации внешнего цикла с `i`?

Можно использовать `break`, чтобы вернуться к внешнему циклу:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d, %d\n", i, j);
        break; // Gets us to the next iteration of i
    }
}
```

Это работает при двух уровнях вложенности. Но если добавить ещё один цикл, какой оператор перенесёт нас к следующей итерации `i`?

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        for (int k = 0; k < 3; k++) {
            printf("%d, %d, %d\n", i, j, k);
            continue; // Gets us to the next iteration of k
            break;    // Gets us to the next iteration of j
            ????;     // Gets us to the next iteration of i???
```

```
        }
    }
}
```

Оператор `goto` даёт решение:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        for (int k = 0; k < 3; k++) {
            printf("%d, %d, %d\n", i, j, k);
            goto continue_i; // Now continuing the i loop!!
        }
    }

continue_i: ;
}
```

В конце стоит пустой оператор `;`: в старых версиях C метка должна предшествовать оператору, а не просто концу составного оператора или объявлению переменной. В C23 это ограничение ослаблено.

31.3. Экстренный выход

Если вы глубоко внутри вложенного кода, `goto` позволяет выйти наружу зачастую чище, чем дополнительные вложенные `if` и переменные-флаги:

```
// Pseudocode
for (...) {
    for (...) {
        while (...) {
            do {
                if (some_error_condition)
                    goto bail;
            } while (...);
        }
    }
}

bail:
// Cleanup here
```

Без `goto` пришлось бы проверять флаг ошибки во всех циклах, чтобы выйти из каждого.

31.4. break с меткой

Как `continue` переходит только к следующей итерации ближайшего цикла, так и `break` выходит только из ближайшего цикла:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d, %d\n", i, j);
        break; // Only breaks out of the j loop
    }
}
```

```
printf("Done!\n");
```

Но `goto` позволяет выйти дальше:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d, %d\n", i, j);
        goto break_i; // Now breaking out of the i loop!
    }
}

break_i:
printf("Done!\n");
```

31.5. Многоуровневая очистка

Если несколько функций инициализируют разные подсистемы и одна из них завершилась ошибкой, следует освободить только те подсистемы, которые уже были успешно инициализированы.

Рассмотрим условный пример. Мы запускаем подсистемы и проверяем результат каждой; `-1` означает ошибку. При ошибке останавливаем только успешно запущенные подсистемы:

```
if (init_system_1() == -1)
    goto shutdown;

if (init_system_2() == -1)
    goto shutdown_1;

if (init_system_3() == -1)
    goto shutdown_2;

if (init_system_4() == -1)
    goto shutdown_3;

do_main_thing(); // Run our program

shutdown_system4();
shutdown_3:
shutdown_system3();
shutdown_2:
shutdown_system2();
shutdown_1:
shutdown_system1();
shutdown:
print("All subsystems shut down.\n");
```

Обратите внимание: подсистемы останавливаются в обратном порядке. Если не удалось запустить четвёртую, выключатся третья, вторая и первая.

31.6. Оптимизация хвостового вызова

В некотором роде. Только для рекурсивных функций.

Если вы не знакомы с оптимизацией хвостовых вызовов (tail call optimization, TCO), это способ при определённых условиях избежать расходования места в стеке при вызове функции. Подробности выходят за рамки руководства.

Но если рекурсивную функцию можно оптимизировать таким образом, допустимо использовать следующий приём. Перейти таким способом в другую функцию нельзя: метки имеют область видимости функции.

Возьмём простой пример — факториал. Вот рекурсивная версия, которая допускает такую оптимизацию:

```
#include <stdio.h>
#include <complex.h>

int factorial(int n, int a)
{
    if (n == 0)
        return a;

    return factorial(n - 1, a * n);
}

int main(void)
{
    for (int i = 0; i < 8; i++)
        printf("%d! == %ld\n", i, factorial(i, 1));
}
```

В этом примере оригинала `factorial` возвращает `int`, поэтому для результата нужен `%d`, а не `%ld`.

Заменим рекурсивный вызов двумя действиями:

1. Присвоим параметрам значения, которые они получили бы при следующем вызове.
2. Перейдём к метке в начале функции.

```
#include <stdio.h>

int factorial(int n, int a)
{
tco: // add this

    if (n == 0)
        return a;

    // replace return by setting new parameter values and
    // goto-ing the beginning of the function

    //return factorial(n - 1, a * n);

    int next_n = n - 1; // See how these match up with
    int next_a = a * n; // the recursive arguments, above?

    n = next_n; // Set the parameters to the new values
    a = next_a;
```

```
    goto tco; // And repeat!
}

int main(void)
{
    for (int i = 0; i < 8; i++)
        printf("%d! == %d\n", i, factorial(i, 1));
}
```

Перед переходом я использовал временные переменные для следующих значений параметров. Видите, как они соответствуют аргументам рекурсивного вызова?

Зачем временные переменные? Можно было написать:

```
a *= n;
n -= 1;
goto tco;
```

Это работает. Но если неосторожно поменять две строки местами:

```
n -= 1; // BAD NEWS
a *= n;
```

возникнет проблема: `n` изменено до того, как используется для изменения `a`. При рекурсивном вызове всё происходит не так. Временные переменные помогают избежать этой ошибки, а компилятор, скорее всего, всё равно их оптимизирует.

31.7. Повторный запуск прерванных системных вызовов

Эта тема не входит в стандарт C, но часто встречается в Unix-подобных системах.

Некоторые длительные системные вызовы могут завершиться ошибкой при получении сигнала. При этом `errno` устанавливается в `EINTR`: вызов просто был прерван. В таких случаях программист часто хочет повторить его:

```
retry:
byte_count = read(0, buf, sizeof(buf) - 1); // Unix read() syscall

if (byte_count == -1) { // An error occurred...
    if (errno == EINTR) { // But it was just interrupted
        printf("Restarting...\n");
        goto retry;
    }
```

Это фрагмент оригинала; внешний блок `if` в нём не завершён. Во многих Unix-подобных системах можно передать `sigaction()` флаг `SA_RESTART`, попросив ОС автоматически повторять некоторые медленные системные вызовы вместо завершения с `EINTR`.

Ещё раз: это особенность Unix, не часть стандарта C. Но похожий приём можно использовать везде, где вызов функции требуется повторить.

31.8. goto и вытеснение потоков

Этот пример взят из книги *Operating Systems: Three Easy Pieces*, авторы которой тоже считают, что качественные книги должны быть бесплатны для скачивания. Не то чтобы у меня было собственное мнение на этот счёт.

```
retry:
pthread_mutex_lock(L1);

if (pthread_mutex_trylock(L2) != 0) {
    pthread_mutex_unlock(L1);
    goto retry;
}

save_the_day();

pthread_mutex_unlock(L2);
pthread_mutex_unlock(L1);
```

Поток успешно захватывает мьютекс `L1`, но может не получить второй ресурс, защищённый `L2`, например если его удерживает другой поток. Тогда он освобождает `L1` и с помощью `goto` аккуратно повторяет попытку.

Будем надеяться, что наш героический поток в итоге захватит оба мьютекса и спасёт положение, избежав зловещей взаимной блокировки. При этом такой цикл сам по себе не гарантирует своевременного успеха и может активно расходовать процессорное время.

31.9. goto и область видимости переменных

Мы уже видели, что метки имеют область видимости функции. Но при переходе через инициализацию переменной возможны странности.

Здесь мы переходим из места, где `x` не видна, в середину её области видимости:

```
goto label;

{
    int x = 12345;

label:
    printf("%d\n", x);
}
```

Программа компилируется, но я получаю предупреждение:

```
warning: ‘x’ is used uninitialized in this function
```

При запуске у меня выводится `0`, хотя у вас результат может отличаться. Мы вошли в область видимости `x`, поэтому к имени можно обратиться, но пропустили инициализацию значением `12345`. Значение не определено; читать его нельзя безопасно.

Исправление — выполнить инициализацию после метки:

```
goto label;

{
    int x;

label:
    x = 12345;
    printf("%d\n", x);
}
```

Ещё пример:

```
{
    int x = 10;

label:
    printf("%d\n", x);
}

goto label;
```

При первом проходе всё хорошо: `x` равно 10, и это значение выводится. Но после `goto` мы снова входим в область видимости `x`, минуя инициализацию. Значение становится неопределённым.

У меня снова выводится 10, и так бесконечно, но это лишь случайность. После перехода чтение неинициализированной переменной нельзя считать корректным.

31.10. goto и массивы переменной длины

Для VLA и `goto` есть правило: нельзя перейти извне области видимости VLA внутрь неё.

Попробуем:

```
int x = 10;

goto label;

{
    int v[x];

label:
    printf("Hi!\n");
}
```

Получится ошибка:

```
error: jump into scope of identifier with variably modified type
```

Можно перейти к месту перед объявлением VLA:

```
int x = 10;

goto label;

{
label: ;
    int v[x];
    printf("Hi!\n");
}
```

Тогда VLA будет правильно выделен до того, как неизбежно освободится при выходе из области видимости.

Глава 32. Типы V: составные литералы и обобщённый выбор

Это последняя глава о типах! Мы обсудим две вещи:

- как создавать безымянные объекты и зачем это нужно;
- как получать код, зависящий от типа.

Эти темы не особенно связаны, но ни одна не заслуживает отдельной главы. Поэтому я собрал их здесь, как настоящий бунтарь!

32.1. Составные литералы

Эта удобная возможность языка позволяет создавать объект нужного типа на лету, не присваивая его переменной. Можно создавать простые типы, массивы, структуры — что угодно.

Одно из основных применений — передача сложных аргументов функции без создания временной переменной для хранения значения.

Чтобы создать составной литерал, поместите имя типа в скобки, а после него — список инициализаторов. Например, безымянный массив `int` выглядит так:

```
(int []){1,2,3,4}
```

Сама по себе эта строка ничего полезного не делает: она создаёт безымянный массив из четырёх `int`, а затем оставляет его неиспользованным.

Можно сохранить указатель на массив:

```
int *p = (int []){1,2,3,4};
printf("%d\n", p[1]); // 2
```

Но это кажется длинным способом создать массив. Ведь можно было просто написать:

```
int p[] = {1, 2, 3, 4};
printf("%d\n", p[1]); // 2
```

Хотя это не совсем то же самое: здесь `p` — массив, а не указатель на `int`. Рассмотрим более полезный пример.

32.1.1. Передача безымянных объектов функциям

Допустим, есть функция суммирования массива `int`:

```
int sum(int p[], int count)
{
    int total = 0;

    for (int i = 0; i < count; i++)
        total += p[i];

    return total;
}
```

Обычно для вызова пришлось бы объявить массив, сохранить в нём значения и передать его функции:

```
int a[] = {1, 2, 3, 4};
int s = sum(a, 4);
```

Безымянный объект позволяет обойтись без переменной и передать массив непосредственно. Здесь вместо `a` первым аргументом идёт безымянный массив:

```
//          p[]          count
//    |-----------------|  |
int s = sum((int []){1, 2, 3, 4}, 4);
```

Довольно изящно!

32.1.2. Безымянные структуры

Аналогичный приём работает и со структурами. Сначала обойдёмся без безымянного объекта: определим структуру координат `x` и `y`, создадим её экземпляр и передадим функции вывода.

```
#include <stdio.h>

struct coord {
    int x, y;
};

void print_coord(struct coord c)
{
    printf("%d, %d\n", c.x, c.y);
}

int main(void)
{
    struct coord t = {.x=10, .y=20};

    print_coord(t); // prints "10, 20"
}
```

Всё понятно? Теперь вместо переменной `t` передадим `print_coord()` безымянную структуру:

```
//struct coord t = {.x=10, .y=20};

print_coord((struct coord){.x=10, .y=20}); // prints "10, 20"
```

И это тоже работает!

32.1.3. Указатели на безымянные объекты

В предыдущем примере, несмотря на использование структуры, мы передавали в `print_coord()` её копию, а не указатель.

Адрес безымянного объекта можно получить обычным способом с помощью `&`. Вообще говоря, если оператор применим к переменной некоторого типа, его можно применить и к безымянному объекту этого типа.

Изменим код так, чтобы передавать указатель:

```
#include <stdio.h>

struct coord {
    int x, y;
};

void print_coord(struct coord *c)
{
    printf("%d, %d\n", c->x, c->y);
}

int main(void)
{
    // Note the &
    //           |
    print_coord(&(struct coord){.x=10, .y=20}); // prints "10, 20"
}
```

Этот способ удобен и для передачи указателей на простые объекты:

```
// Pass a pointer to an int with value 3490
foo(&(int){3490});
```

Проще простого.

32.1.4. Время жизни безымянных объектов

Время жизни безымянного объекта, созданного внутри блока обычным составным литералом, заканчивается при выходе из этого блока. Наиболее опасная ситуация: создать объект, получить указатель на него и выйти из блока. Указатель будет ссылаться на уже не существующий объект.

Поэтому следующий код имеет неопределённое поведение:

```
int *p;

{
    p = &(int){10};
}

printf("%d\n", *p); // INVALID: The (int){10} fell out of scope
```

Аналогично нельзя вернуть из функции указатель на такой локальный безымянный объект: его время жизни заканчивается при выходе из блока функции.

```
#include <stdio.h>

int *get3490(void)
{
    // Don't do this
    return &(int){3490};
}

int main(void)
{
    printf("%d\n", *get3490()); // INVALID: (int){3490} fell out of scope
}
```

Думайте о них как об обычных локальных переменных: возвращать указатель на локальную переменную тоже нельзя.

32.1.5. Забавный пример

Можно указать любой подходящий тип и создать безымянный объект. Например, следующие варианты фактически эквивалентны:

```
int x = 3490;

printf("%d\n", x);           // 3490 (variable)
printf("%d\n", 3490);        // 3490 (constant)
printf("%d\n", (int){3490}); // 3490 (unnamed object)
```

Последний вариант работает, но выглядит нелепо: проще использовать предыдущую строку. Надеюсь, он всё же помогает лучше понять синтаксис.

32.2. Обобщённый выбор

Обобщённый выбор позволяет выбирать разные части кода в зависимости от типа управляющего выражения.

Скоро мы рассмотрим пример, но важно помнить: выбор выполняется при компиляции, а не во время работы программы. Никакого анализа типов во время выполнения здесь нет.

Выражение начинается с `_Generic`, немного напоминает `switch` и содержит как минимум два элемента. Первый — выражение с некоторым типом; переменная здесь тоже является

выражением. Остальные элементы описывают, какой результат выбрать для соответствующего типа.

Что? Давайте посмотрим:

```
#include <stdio.h>

int main(void)
{
    int i;
    float f;
    char c;

    char *s = _Generic(i,
        int: "that variable is an int",
        float: "that variable is a float",
        default: "that variable is some type"
    );

    printf("%s\n", s);
}
```

Компилятор рассматривает тип первого выражения, здесь переменной `i`, затем находит соответствующий вариант. `i` имеет тип `int`, поэтому выбирается строка, соответствующая `int`. По результату это похоже на строку:

```
char *s = "that variable is an int";
```

Если подходящий тип не найден, выбирается необязательный вариант `default`. Если нет ни подходящего типа, ни `default`, возникает ошибка компиляции.

Поскольку писать `_Generic` снова и снова неудобно, его часто помещают в макрос, который легко использовать многократно. Создадим `TYPESTR(x)`, возвращающий строку с типом аргумента: например, `TYPESTR(1)` даст `"int"`.

```
#include <stdio.h>

#define TYPESTR(x) _Generic((x), \
    int: "int", \
    long: "long", \
    float: "float", \
    double: "double", \
    default: "something else")

int main(void)
{
    int i;
    long l;
    float f;
    double d;
    char c;

    printf("i is type %s\n", TYPESTR(i));
    printf("l is type %s\n", TYPESTR(l));
    printf("f is type %s\n", TYPESTR(f));
    printf("d is type %s\n", TYPESTR(d));
```

```
    printf("c is type %s\n", TYPESTR(c));
}
```

Результат:

```
i is type int
l is type long
f is type float
d is type double
c is type something else
```

Это ожидаемо: при компиляции результат соответствует следующим вызовам:

```
printf("i is type %s\n", "int");
printf("l is type %s\n", "long");
printf("f is type %s\n", "float");
printf("d is type %s\n", "double");
printf("c is type %s\n", "something else");
```

Рассмотрим ещё один пример. Макросы позволят написать:

```
int i = 10;
char *s = "Foo!";

PRINT_VAL(i);
PRINT_VAL(s);
```

и получить:

```
i = 10
s = Foo!
```

Для этого понадобится немного магии макросов:

```
#include <stdio.h>
#include <string.h>

// Macro that gives back a format specifier for a type
#define FMTSPEC(x) _Generic((x), \
    int: "%d", \
    long: "%ld", \
    float: "%f", \
    double: "%f", \
    char *: "%s")
// TODO: add more types

// Macro that prints a variable in the form "name = value"
#define PRINT_VAL(x) do { \
    char fmt[512]; \
    snprintf(fmt, sizeof fmt, #x " = %s\n", FMTSPEC(x)); \
    printf(fmt, (x)); \
} while(0)

int main(void)
{
    int i = 10;
```

```
    float f = 3.14159;
    char *s = "Hello, world!";

    PRINT_VAL(i);
    PRINT_VAL(f);
    PRINT_VAL(s);
}
```

Вывод:

```
i = 10
f = 3.141590
s = Hello, world!
```

Можно было упаковать всё в один большой макрос, но я разделил его на два, чтобы глаза не кровоточили.

Глава 33. Массивы, часть II

В этой главе рассмотрим ещё несколько различных возможностей массивов:

- квалификаторы типов в параметрах-массивах;
- ключевое слово `static` в параметрах-массивах;
- частичную инициализацию многомерных массивов.

Встречаются они не слишком часто, но мы их изучим, поскольку они входят в современные стандарты.

33.1. Квалификаторы типов массивов в списках параметров

Как вы помните, в списках параметров функций эти два объявления эквивалентны:

```
int func(int *p) {...}
int func(int p[]) {...}
```

Вы также можете помнить, что к переменной-указателю можно добавлять квалификаторы типов:

```
int *const p;
int *volatile p;
int *const volatile p;
// etc.
```

Но как это сделать, если параметр записан как массив? Оказывается, квалификаторы помещаются в квадратные скобки. После них можно указать необязательный размер. Следующие объявления эквивалентны по типу параметра:

```
int func(int *const volatile p) {...}
int func(int p[const volatile]) {...}
int func(int p[const volatile 10]) {...}
```

В многомерном массиве квалификаторы ставятся в первой паре квадратных скобок.

33.2. static для массивов в списках параметров

Подобным образом в объявлении параметра-массива можно использовать ключевое слово `static`.

Я никогда не встречал его в реальном коде. После него всегда указывается размер:

```
int func(int p[static 4]) {...}
```

В этом примере компилятор может считать, что переданный функции указатель обеспечивает доступ по меньшей мере к четырём элементам. Иной вызов приводит к неопределённому поведению.

```
int func(int p[static 4]) {...}

int main(void)
{
    int a[] = {11, 22, 33, 44};
    int b[] = {11, 22, 33, 44, 55};
    int c[] = {11, 22};

    func(a); // OK! a is 4 elements, the minimum
    func(b); // OK! b is at least 4 elements
    func(c); // Undefined behavior! c is under 4 elements!
}
```

По сути, так задаётся минимальное количество доступных элементов.

Важно: компилятор не обязан запрещать передачу меньшего массива. Он может не выдать предупреждения и не обнаружить проблему во время выполнения.

Указывая здесь `static`, вы говорите: «Я сверхсекретно ОБЕЩАЮ никогда не передавать массив меньше указанного». Компилятор отвечает: «Ладно» — и верит вам. Опираясь на это обещание, он может выполнять некоторые оптимизации.

33.3. Эквивалентные инициализаторы

C довольно гибок в вопросах инициализации массивов. Мы уже видели, что отсутствующие значения заменяются нулями.

Например, массив из пяти элементов можно инициализировать значениями 1, 2, 0, 0, 0 так:

```
int a[5] = {1, 2};
```

А так можно заполнить весь массив нулями:

```
int a[5] = {0};
```

Но при инициализации многомерных массивов становится интереснее. Создадим массив из трёх строк и двух столбцов:

```
int a[3][2];
```

Напишем программу, которая инициализирует его и выводит результат:

```
#include <stdio.h>

int main(void)
{
    int a[3][2] = {
```

```
        {1, 2},
        {3, 4},
        {5, 6}
    };

    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 2; col++)
            printf("%d ", a[row][col]);
        printf("\n");
    }
}
```

При запуске получаем ожидаемое:

```
1 2
3 4
5 6
```

Пропустим некоторые элементы инициализатора и увидим, что они становятся нулями:

```
int a[3][2] = {
    {1, 2},
    {3}, // Left off the 4!
    {5, 6}
};
```

Результат:

```
1 2
3 0
5 6
```

Теперь уберём всю среднюю строку инициализатора:

```
int a[3][2] = {
    {1, 2},
    // {3, 4}, // Just cut this whole thing out
    {5, 6}
};
```

Получается, возможно, неожиданный результат:

```
1 2
5 6
0 0
```

Но если подумать, мы задали инициализаторы только для двух строк, и они заполнили первые две. Остальные элементы инициализированы нулями.

Пока всё хорошо. В общем случае, если пропустить часть инициализатора, соответствующие элементы станут нулями.

А теперь что-нибудь безумное:

```
int a[3][2] = { 1, 2, 3, 4, 5, 6 };
```

Что? Массив двумерный, а инициализатор одномерный! Оказывается, так можно, хотя GCC с соответствующими предупреждениями сообщит об этом.

Элементы заполняются слева направо: сначала строка 0, затем строка 1 и строка 2. Поэтому при выводе получаем:

```
1 2
3 4
5 6
```

Если часть значений пропустить:

```
int a[3][2] = { 1, 2, 3 };
```

оставшиеся элементы заполняются нулями:

```
1 2
3 0
0 0
```

Поэтому, чтобы заполнить весь массив нулями, можно сделать так:

```
int a[3][2] = {0};
```

Но для двумерного массива я рекомендую использовать двумерный инициализатор: код так читабельнее. Исключение — заполнение всего массива нулями: `{0}` принято использовать независимо от размерности.

Глава 34. Дальние переходы с setjmp и longjmp

Мы уже видели `goto`, который выполняет переход внутри функции. А `longjmp()` позволяет вернуться к более ранней точке выполнения — в функцию, вызвавшую текущую.

Здесь есть много ограничений и оговорок, но это может пригодиться, чтобы выйти из глубокой цепочки вызовов и вернуться к предыдущему состоянию. По опыту автора, такую возможность используют очень редко.

34.1. Использование setjmp и longjmp

Мы поставим в процессе выполнения своего рода закладку с помощью `setjmp()`. Позже вызовем `longjmp()`, чтобы вернуться к этой точке, даже если тем временем были вызваны другие функции.

Следующий пример заходит на два уровня вложенности, а затем выходит обратно. Переменная `env` на уровне файла хранит состояние при вызове `setjmp()`, чтобы `longjmp()` смогла его восстановить. Это и есть наша «закладка».

Тип `env` — непрозрачный тип `jmp_buf`, объявленный в `<setjmp.h>`.

```
#include <stdio.h>
#include <setjmp.h>

jmp_buf env;

void depth2(void)
{
    printf("Entering depth 2\n");
    longjmp(env, 3490);  // Bail out
    printf("Leaving depth 2\n");  // This won't happen
}

void depth1(void)
{
    printf("Entering depth 1\n");
    depth2();
    printf("Leaving depth 1\n");  // This won't happen
}

int main(void)
{
    switch (setjmp(env)) {
        case 0:
            printf("Calling into functions, setjmp() returned 0\n");
            depth1();
            printf("Returned from functions\n");  // This won't happen
            break;

        case 3490:
            printf("Bailed back to main, setjmp() returned 3490\n");
            break;
```

```
    }
}
```

Вывод:

```
Calling into functions, setjmp() returned 0
Entering depth 1
Entering depth 2
Bailed back to main, setjmp() returned 3490
```

Сопоставив вывод с кодом, вы заметите нечто необычное: `setjmp()` возвращает управление дважды!

Вот как это работает. Возврат `0` означает, что закладка успешно установлена. Ненулевой результат означает, что выполнение вернулось к ранее установленной закладке; это значение передаётся в `longjmp()`.

Таким образом можно отличить установку закладки от последующего возврата к ней. При первом вызове `setjmp()` сохраняет состояние в `env` и возвращает `0`. Позже `longjmp()` с тем же `env` восстанавливает состояние, а `setjmp()` возвращает значение, переданное в `longjmp()`.

34.2. Подводные камни

На низком уровне идея довольно проста. Обычно указатель стека отслеживает положение локальных переменных в памяти, а счётчик команд — адрес выполняемой инструкции. Эти понятия относятся к архитектуре и реализации C и не являются частью самого стандарта.

Чтобы вернуться к прежней функции, можно восстановить указатель стека и счётчик команд из `jmp_buf`, а также установить нужное возвращаемое значение. Но различные факторы усложняют ситуацию и создают немало ловушек с неопределённым поведением.

34.2.1. Значения локальных переменных

Если после `longjmp()` нужно сохранить значения автоматических локальных переменных функции, вызвавшей `setjmp()`, объявите их с `volatile`. Строго говоря, это необходимо для тех переменных, которые изменились между вызовами `setjmp()` и `longjmp()`.

Например:

```
int x = 20;

if (setjmp(env) == 0) {
    x = 30;
}
```

Если позже вернуться сюда через `longjmp()`, значение `x` будет неопределённым. Исправим это с помощью `volatile`:

```
volatile int x = 20;

if (setjmp(env) == 0) {
    x = 30;
}
```

Теперь после возвращения `longjmp()` значение будет равно `30`. Причина в том, что компилятор может временно хранить переменную в регистре, пока её значение в памяти не обновлено; при восстановлении состояния содержимое регистра может измениться.

34.2.2. Сколько состояния сохраняется?

После `longjmp()` выполнение возобновляется в точке соответствующего `setjmp()` — и только. Стандарт описывает это примерно так: как будто мы вернулись в функцию, а локальные переменные имеют значения на момент вызова `longjmp()`, с учётом ограничений предыдущего раздела.

Не восстанавливаются, в частности:

- флаги состояния вычислений с плавающей точкой;
- открытые файлы;
- другие компоненты абстрактной машины.

34.2.3. Нельзя использовать имя setjmp

Нельзя объявлять внешние идентификаторы с именем `setjmp`. Если `setjmp` реализован как макрос, нельзя отменять его определение. Оба случая приводят к неопределённому поведению.

34.2.4. setjmp() нельзя использовать в сложном выражении

Следующий код недопустим:

```
if (x == 12 && setjmp(env) == 0) { ... }
```

Выражение слишком сложное для ограничений стандарта: нельзя вернуться через `longjmp()` в частично вычисленное сложное выражение. Разрешены следующие формы:

- Всё управляющее выражение условия:

  ```
  if (setjmp(env)) {...}
  switch (setjmp(env)) {...}
  ```

- Часть выражения сравнения, если второй операнд — целочисленная константа, а всё сравнение является управляющим выражением:

  ```
  if (setjmp(env) == 0) {...}
  ```

- Операнд логического отрицания, если всё выражение является управляющим:

```
if (!setjmp(env)) {...}
```

- Отдельное выражение, возможно с приведением к `void`:

```
setjmp(env);
(void)setjmp(env);
```

34.2.5. Когда нельзя вызывать longjmp()?

Поведение неопределённо, если:

- соответствующий `setjmp()` ранее не вызывался;
- `setjmp()` был вызван в другом потоке;
- `setjmp()` был вызван в области видимости массива переменной длины (VLA), а выполнение покинуло эту область до вызова `longjmp()`;
- функция, содержащая вызов `setjmp()`, завершилась до вызова `longjmp()`.

В последнем случае завершение включает как обычный возврат из функции, так и переход другим `longjmp()` в ещё более раннюю функцию в цепочке вызовов.

34.2.6. Передача нуля в longjmp()

Если передать `longjmp()` значение `0`, оно будет автоматически заменено на `1`. Поскольку нулевой результат `setjmp()` имеет особое значение первоначальной установки закладки, вернуть ноль при переходе нельзя.

34.2.7. longjmp() и массивы переменной длины

Если выйти через `longjmp()` из области видимости VLA, выделенная для него память может утечь: она останется занятой до завершения программы без возможности её освободить. То же относится к переходу через вызовы предыдущих функций, в которых VLA ещё находились в области видимости.

Автор отмечает, что возможность написать формально допустимый код C, который таким образом расходует память, — одна из причин его неприязни к VLA.

Глава 35. Неполные типы

Вас может удивить, что этот пример компилируется без ошибки в среде автора:

```
extern int a[];
int main(void)
{
    struct foo *x;
    union bar *y;
    enum baz *z;
}
```

Мы не указали размер `a`, а указатели ссылаются на `foo`, `bar` и `baz`, которые, похоже, нигде не определены. В примере автора единственные предупреждения касаются неиспользуемых `x`, `y` и `z`.

Это примеры **неполных типов**. Для такого типа ещё не известен размер, который вернул бы `sizeof`: можно сказать, что объявление типа ещё не завершено. Можно иметь указатель на неполный тип, но нельзя разыменовать его, выполнять с ним арифметику указателей или применять к самому неполному типу `sizeof`.

35.1. Самоссылочные структуры

Один из важных случаев применения — предварительные ссылки на `struct` или `union` в самоссылочных и взаимно ссылающихся структурах. Дальнейшие примеры используют `struct`, но идея также применима к `union`.

При объявлении структура остаётся неполной до закрывающей фигурной скобки:

```
struct antelope {          // struct antelope is incomplete here
    int leg_count;         // Still incomplete
    float stomach_fullness; // Still incomplete
    float top_speed;       // Still incomplete
    char *nickname;        // Still incomplete
};                         // NOW it's complete.
```

А что, если мы строим связный список и каждому узлу нужна ссылка на следующий узел? Объявить сам узел внутри ещё не завершённого объявления нельзя, зато можно объявить указатель на него:

```
struct node {
    int val;
    struct node *next; // struct node is incomplete, but that's OK!
};
```

Подобным образом могут ссылаться друг на друга и разные структуры:

```
struct a {
    struct b *x; // Refers to a `struct b`
};
```

```
struct b {
    struct a *x; // Refers to a `struct a`
};
```

Без возможности ссылаться на неполный тип такую пару структур определить было бы невозможно.

35.2. Сообщения об ошибках

Если компилятор сообщает, что `sizeof` применён к неполному типу, используется неопределённый тип или разыменовывается указатель на неполный тип, вероятная причина — забытый `#include` заголовочного файла, определяющего тип.

35.3. Другие неполные типы

Объявление `struct` или `union` без тела создаёт неполный тип, например `struct foo;`. Тип перечисления неполон до закрывающей фигурной скобки его определения. `void` — тоже неполный тип.

Массив, объявленный с `extern` без размера, неполон:

```
extern int a[];
```

Массив без `extern`, объявленный без размера, но с инициализатором, остаётся неполным до закрывающей скобки инициализатора.

35.4. Массивы в заголовочных файлах

Неполный тип массива удобно объявлять в заголовочном файле, а сам массив определять в одном `.c`-файле. Определение массива в заголовке, включённом из нескольких исходных файлов, может привести к нескольким определениям.

```
// File: bar.h

#ifndef BAR_H
#define BAR_H

extern int my_array[]; // Incomplete type

#endif
```

В `.c`-файле определим массив:

```
// File: bar.c

int my_array[1024]; // Complete type!
```

Теперь заголовок можно включать в разные файлы: они будут обращаться к одному и тому же массиву.

```
// File: foo.c

#include <stdio.h>
#include "bar.h" // includes the incomplete type for my_array

int main(void)
{
    my_array[0] = 10;

    printf("%d\n", my_array[0]);
}
```

При сборке указывайте компилятору все `.c`-файлы, но не `.h`-файлы:

```
gcc -o foo foo.c bar.c
```

35.5. Завершение неполного типа

Неполный тип можно завершить, определив соответствующую структуру, объединение, перечисление или массив в той же области видимости:

```
struct foo;     // incomplete type
struct foo *p;  // pointer, no problem
// struct foo f; // Error: incomplete type!

struct foo {
    int x, y, z;
}; // Now the struct foo is complete!

struct foo f;   // Success!
```

Тип `void` неполный, но завершить его нельзя. Это объясняет, почему можно объявить указатель:

```
void *p; // OK: pointer to incomplete type
```

Но нельзя объявить объект типа `void` или разыменовать такой указатель:

```
void v;             // Error: declare variable of incomplete type
printf("%d\n", *p); // Error: dereference incomplete type
```

Глава 36. Комплексные числа

Комплексное число записывается в виде *a* + *bi*, где *a* и *b* — действительные числа, а *i* — мнимая единица, для которой $i^2 = -1$. Значение *a* называют действительной частью, а *b* — мнимой. Предположим, что вы уже знакомы с комплексными числами; здесь мы рассмотрим средства C для работы с ними.

Поддержка комплексных чисел необязательна для компилятора; степень поддержки тоже может различаться. Проверить её можно так:

```
#ifdef __STDC_NO_COMPLEX__
#error Complex numbers not supported!
#endif
```

Существует также макрос, указывающий на соответствие ISO 60559 (IEEE 754) для операций с комплексными числами и наличие типа `_Imaginary`:

```
#if __STDC_IEC_559_COMPLEX__ != 1
#error Need IEC 60559 complex support!
#endif
```

Подробности приведены в приложении G стандарта C11.

36.1. Комплексные типы

Подключите `<complex.h>`. Имена `_Complex` и `complex` обозначают одно и то же, так что можно использовать более удобное `complex`.

При соответствующей поддержке реализации также доступны обозначения мнимого типа `_Imaginary` и `imaginary`. Для мнимой единицы имеются `I`, `_Complex_I` и, при поддержке, `_Imaginary_I`. Макрос `I` равен `_Imaginary_I`, если он доступен, или `_Complex_I` в противном случае, поэтому используйте просто `I`.

Автор отмечает: хотя по его прочтению стандарта значение `__STDC_IEC_559_COMPLEX__`, равное 1, подразумевает поддержку `_Imaginary`, ему не удалось найти компилятор, который действительно поддерживает этот тип. Поэтому примеры с ним он не смог проверить.

36.2. Присваивание комплексных чисел

Действительная и мнимая части хранятся как числа с плавающей точкой, поэтому нужно указать их точность с помощью `float`, `double` или `long double`. Порядок слов не важен:

```
float complex c; // Spec prefers this way
complex float c; // Same thing--order doesn't matter
```

Инициализация может выглядеть естественно:

```
double complex x = 5 + 2*I;
double complex y = 10 + 3*I;
```

Это числа 5 + 2i и 10 + 3i соответственно.

36.3. Создание, разбор и вывод

Комплексное число можно построить и из переменных:

```
double a = 5;
double b = 2;
double complex x = a + b*I;
```

Для построения есть и макрос `CMPLX()`:

```
double complex x = CMPLX(5, 2);
```

Автор отмечает возможное различие при работе с отрицательным нулём мнимой части: макрос `CMPLX()` должен сохранять знак нуля, тогда как выражение с `I` может дать другой результат в зависимости от представления мнимой единицы. В случаях, когда знак нуля важен, стоит учитывать это различие. Для `float` и `long double` существуют варианты `CMPLXF()` и `CMPLXL()`.

Чтобы извлечь действительную и мнимую части, используйте `creal()` и `cimag()`:

```
double complex x = 5 + 2*I;
double complex y = 10 + 3*I;
printf("x = %f + %fi\n", creal(x), cimag(x));
printf("y = %f + %fi\n", creal(y), cimag(y));
```

Результат:

```
x = 5.000000 + 2.000000i
y = 10.000000 + 3.000000i
```

Буква `i` в строке формата — обычный выводимый символ, а не часть спецификатора. Обе функции возвращают `double`. Варианты для `float` и `long double`: `crealf()`, `cimagf()`, `creall()` и `cimagl()`.

36.4. Арифметика и сравнение

С комплексными числами можно выполнять обычные арифметические операции; их математический разбор выходит за рамки книги:

```
#include <stdio.h>
#include <complex.h>

int main(void)
```

```
{
    double complex x = 1 + 2*I;
    double complex y = 3 + 4*I;
    double complex z;

    z = x + y;
    printf("x + y = %f + %fi\n", creal(z), cimag(z));
    z = x - y;
    printf("x - y = %f + %fi\n", creal(z), cimag(z));
    z = x * y;
    printf("x * y = %f + %fi\n", creal(z), cimag(z));
    z = x / y;
    printf("x / y = %f + %fi\n", creal(z), cimag(z));
}
```

Результат:

```
x + y = 4.000000 + 6.000000i
x - y = -2.000000 + -2.000000i
x * y = -5.000000 + 10.000000i
x / y = 0.440000 + 0.080000i
```

Комплексные числа можно сравнивать на равенство и неравенство:

```
#include <stdio.h>
#include <complex.h>

int main(void)
{
    double complex x = 1 + 2*I;
    double complex y = 3 + 4*I;

    printf("x == y = %d\n", x == y); // 0
    printf("x != y = %d\n", x != y); // 1
}
```

Они равны, если равны обе соответствующие части. Как и при любых вычислениях с плавающей точкой, результаты сравнения могут зависеть от округления.

36.5. Математические функции

Ниже перечислены варианты для `double`. Добавьте суффикс `f` для `float` или `l` для `long double`: например, `cabsf()` и `cabsl()`.

Тригонометрические функции

Функция	Назначение
`ccos()`, `csin()`, `ctan()`	Косинус, синус, тангенс
`cacos()`, `casin()`, `catan()`	Арккосинус, арксинус, арктангенс
`ccosh()`, `csinh()`, `ctanh()`	Гиперболические косинус, синус, тангенс

Функция	Назначение
`cacosh()`, `casinh()`, `catanh()`	Обратные гиперболические функции

Экспонента, логарифм, степень

Функция	Назначение
`cexp()`	Экспонента с основанием e
`clog()`	Натуральный логарифм
`cabs()`	Абсолютная величина
`cpow()`	Возведение в степень
`csqrt()`	Квадратный корень

Другие операции

Функция	Назначение
`creal()`, `cimag()`	Получить действительную и мнимую части
`CMPLX()`	Построить комплексное число
`carg()`	Аргумент, или фазовый угол
`conj()`	Комплексно-сопряжённое число
`cproj()`	Проекция на сферу Римана

Глава 37. Целочисленные типы фиксированной ширины

В C есть целочисленные типы различных размеров, например `int` и `long`. Их пределы можно узнать по константам вроде `INT_MAX`, а размер в байтах — с помощью `sizeof`.

Но что, если нужен тип размером ровно 32 бита или не менее 16 бит? Для этого служит заголовочный файл `<stdint.h>`.

37.1. Типы с заданным числом бит

Для знаковых и беззнаковых целых предусмотрены три основных семейства типов. Вместо `N` подставляется число бит:

- `intN_t` — целое число ровно указанного размера;
- `int_leastN_t` — целое число размером не менее указанного;
- `int_fastN_t` — максимально быстрый тип размером не менее указанного.

Насколько быстрее означает «быстрый»? Стандарт не устанавливает конкретного выигрыша. На некоторых архитектурах процессор и память быстрее работают с определёнными размерами данных, поэтому быстрый тип размером не менее восьми бит может занимать 16 или 32 бита. Компиляторы и так обычно хорошо оптимизируют код; эти типы особенно полезны, когда нужна максимально быстрая доступная обработка.

Беззнаковые варианты начинаются с `u`. Примеры:

```
int32_t w;       // w is exactly 32 bits, signed
uint16_t x;      // x is exactly 16 bits, unsigned
int_least8_t y;  // y is at least 8 bits, signed
uint_fast64_t z; // z is the fastest representation at least 64 bits, unsigned
```

Гарантированно определены следующие типы:

```
int_least8_t    uint_least8_t
int_least16_t   uint_least16_t
int_least32_t   uint_least32_t
int_least64_t   uint_least64_t
int_fast8_t     uint_fast8_t
int_fast16_t    uint_fast16_t
int_fast32_t    uint_fast32_t
int_fast64_t    uint_fast64_t
```

Другие ширины тоже могут поддерживаться, но необязательны.

А где типы точной ширины вроде `int16_t`? Их наличие зависит от реализации. Если система предоставляет соответствующий знаковый или беззнаковый целочисленный тип нужной ширины без добавочных битов, такие типы определяются. На большинстве современных компьютеров доступны:

```
int8_t   uint8_t
int16_t  uint16_t
int32_t  uint32_t
int64_t  uint64_t
```

Другие варианты точной ширины также могут существовать.

37.2. Типы максимальной ширины

Для представления наиболее широких доступных целых значений используются знаковый и беззнаковый типы:

```
intmax_t
uintmax_t
```

Используйте их, когда нужен максимально широкий тип соответствующей знаковости.

37.3. Константы заданной ширины

Для целочисленных констант предусмотрены макросы, позволяющие получить константу подходящего типа, при необходимости добавив суффикс, как в `22L` или `3490ULL`:

```
INT8_C(x)    UINT8_C(x)
INT16_C(x)   UINT16_C(x)
INT32_C(x)   UINT32_C(x)
INT64_C(x)   UINT64_C(x)
INTMAX_C(x)  UINTMAX_C(x)
```

Они применяются к целочисленным константам, например:

```
uint16_t x = UINT16_C(12);
intmax_t y = INTMAX_C(3490);
```

37.4. Пределы целочисленных типов

Для максимальных и минимальных значений определены макросы:

```
INT8_MAX        INT8_MIN        UINT8_MAX
INT16_MAX       INT16_MIN       UINT16_MAX
INT32_MAX       INT32_MIN       UINT32_MAX
INT64_MAX       INT64_MIN       UINT64_MAX
INT_LEAST8_MAX  INT_LEAST8_MIN  UINT_LEAST8_MAX
INT_LEAST16_MAX INT_LEAST16_MIN UINT_LEAST16_MAX
INT_LEAST32_MAX INT_LEAST32_MIN UINT_LEAST32_MAX
INT_LEAST64_MAX INT_LEAST64_MIN UINT_LEAST64_MAX
INT_FAST8_MAX   INT_FAST8_MIN   UINT_FAST8_MAX
INT_FAST16_MAX  INT_FAST16_MIN  UINT_FAST16_MAX
INT_FAST32_MAX  INT_FAST32_MIN  UINT_FAST32_MAX
INT_FAST64_MAX  INT_FAST64_MIN  UINT_FAST64_MAX
INTMAX_MAX      INTMAX_MIN      UINTMAX_MAX
```

Минимум любого беззнакового типа равен нулю, поэтому отдельные макросы минимума для них не нужны.

37.5. Спецификаторы формата

Чтобы вывести значение через `printf()`, нужен подходящий спецификатор формата; аналогичная задача возникает при вводе через `scanf()`. Поскольку внутреннее представление типа зависит от системы, используйте макросы из `<inttypes.h>`.

Ниже `n` обозначает `8`, `16`, `32` или `64`. Макросы для вывода знаковых целых:

```
PRIdn PRIdLEASTn PRIdFASTn PRIdMAX
PRIin PRIiLEASTn PRIiFASTn PRIiMAX
```

Варианты соответствуют типам точной, минимальной, быстрой и максимальной ширины. Буквы `d` и `i` соответствуют спецификаторам `printf()` `%d` и `%i`.

Например, для `int_least16_t` можно использовать `PRIdLEAST16`. Макрос раскрывается в строковый литерал, содержащий нужную часть спецификатора, например `"d"` или `"ld"`. Соседние строковые литералы в C автоматически объединяются:

```
printf("Hello, " "world!\n"); // Prints "Hello, world!"
```

Поэтому макрос можно включить непосредственно в строку формата:

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

int main(void)
{
    int_least16_t x = 3490;

    printf("The value is %" PRIdLEAST16 "!\n", x);
}
```

Для вывода беззнаковых значений есть соответствующие макросы:

```
PRIon PRIoLEASTn PRIoFASTn PRIoMAX
PRIun PRIuLEASTn PRIuFASTn PRIuMAX
PRIxn PRIxLEASTn PRIxFASTn PRIxMAX
PRIXn PRIXLEASTn PRIXFASTn PRIXMAX
```

Буквы `o`, `u`, `x` и `X` соответствуют спецификаторам `printf()`. Вместо `n` снова подставляется нужная ширина.

Для `scanf()` предусмотрен аналогичный набор:

```
SCNdn SCNdLEASTn SCNdFASTn SCNdMAX
SCNin SCNiLEASTn SCNiFASTn SCNiMAX
SCNon SCNoLEASTn SCNoFASTn SCNoMAX
```

```
SCNun SCNuLEASTn SCNuFASTn SCNuMAX
SCNxn SCNxLEASTn SCNxFASTn SCNxMAX
```

Для ввода и вывода целых типов заданной ширины используйте соответствующий макрос из `<inttypes.h>`.

Глава 38. Дата и время

Работа с датой и временем не слишком сложна, но поначалу может пугать: существуют разные типы и способы преобразования между ними. Добавьте UTC и местное время — и получите все обычные сложности работы с датами.

Не забывайте золотое правило: не пытайтесь самостоятельно писать средства работы с датами и временем, если можно воспользоваться библиотекой. Время слишком сложно, чтобы правильно учесть все детали без готовых инструментов.

38.1. Терминология

- **UTC** — всемирное координированное время. В любой точке Земли в один и тот же момент значение UTC одинаково, хотя местное время отличается.
- **GMT** — среднее время по Гринвичу. На практике его часто используют почти как синоним UTC, хотя GMT технически относится к часовому поясу, а UTC — к системе отсчёта времени. Многие функции C появились до UTC и по-прежнему содержат в имени `gmt`; в данном контексте они работают с универсальным временем.
- **Местное время** — время в месте, для которого настроен компьютер. Оно описывается относительно UTC. Несмотря на множество часовых поясов, программы часто работают либо с местным временем, либо с UTC.

Как правило, для события, происходящего в один конкретный момент, например записи журнала или запуска ракеты, используйте UTC. Для событий, происходящих в одинаковое время в каждом часовом поясе, например наступления Нового года или времени ужина, используйте местное время.

Если хранить даты в неподходящей форме, последующие преобразования между UTC и местным временем могут доставить немало хлопот.

38.2. Типы для даты и времени

Два основных типа C — `time_t` и `struct tm`:

- `time_t` — вещественный тип, способный хранить календарное время. Стандарт допускает целочисленное или плавающее представление. В POSIX используется целочисленное.
- `struct tm` хранит календарное время по компонентам: часы, минуты, секунды, день, месяц, год и так далее.

Во многих системах `time_t` обозначает число секунд с начала эпохи — обычно с 1 января 1970 года UTC. Отрицательные значения могут представлять предшествующие моменты. Однако стандарт C не требует именно такого представления.

Поля `struct tm`:

```
struct tm {
    int tm_sec;   // seconds after the minute -- [0, 60]
    int tm_min;   // minutes after the hour -- [0, 59]
    int tm_hour;  // hours since midnight -- [0, 23]
    int tm_mday;  // day of the month -- [1, 31]
    int tm_mon;   // months since January -- [0, 11]
    int tm_year;  // years since 1900
    int tm_wday;  // days since Sunday -- [0, 6]
    int tm_yday;  // days since January 1 -- [0, 365]
    int tm_isdst; // Daylight Saving Time flag
};
```

Заметьте: отсчёт большинства полей начинается с нуля, но день месяца — с единицы.

Эти типы могут хранить различные моменты, а не только текущее время. Существуют функции для получения текущего времени, инициализации значений и преобразования между типами.

38.3. Инициализация и преобразование типов

Функция `time()` получает текущее время в `time_t`:

```
time_t now;       // Variable to hold the time now
now = time(NULL); // You can get it like this...
time(&now);       // ...or this. Same as the previous line.
```

Для вывода `time_t` в местном времени можно использовать `ctime()`:

```
now = time(NULL);
printf("%s", ctime(&now));
```

Она возвращает строку определённого формата с завершающим переводом строки:

```
Sun Feb 28 18:47:25 2021
```

Такой формат не слишком гибок. Для большего контроля преобразуем `time_t` в `struct tm`.

38.3.1. Преобразование time_t в struct tm

Для этого есть две функции:

- `localtime()` преобразует `time_t` в `struct tm` с местным временем.
- `gmtime()` преобразует `time_t` в `struct tm` с UTC; старое название GMT сохранилось в имени функции.

Выведем результат с помощью `asctime()`:

```
printf("Local: %s", asctime(localtime(&now)));
printf("  UTC: %s", asctime(gmtime(&now)));
```

Пример вывода для североамериканского тихоокеанского времени:

```
Local: Sun Feb 28 20:15:27 2021
  UTC: Mon Mar 1 04:15:27 2021
```

Получив `struct tm`, можно выводить дату разными способами, определять день недели и выполнять другие операции, а также преобразовать значение обратно в `time_t`.

38.3.2. Преобразование struct tm в time_t

Для обратного преобразования используйте `mktime()`.

Функция сама устанавливает `tm_wday` и `tm_yday`, поэтому заполнять их заранее не нужно: значения будут перезаписаны. Поле `tm_isdst` можно установить в `-1`, чтобы функция определила действие летнего времени сама, либо явно задать истинное или ложное значение.

```
// Don't be tempted to put leading zeros on these numbers (unless you
// mean for them to be in octal)!
struct tm some_time = {
    .tm_year=82,  // years since 1900
    .tm_mon=3,    // months since January -- [0, 11]
    .tm_mday=12,  // day of the month -- [1, 31]
    .tm_hour=12,  // hours since midnight -- [0, 23]
    .tm_min=0,    // minutes after the hour -- [0, 59]
    .tm_sec=4,    // seconds after the minute -- [0, 60]
    .tm_isdst=-1, // Daylight Saving Time flag
};

time_t some_time_epoch;
some_time_epoch = mktime(&some_time);

printf("%s", ctime(&some_time_epoch));
printf("Is DST: %d\n", some_time.tm_isdst);
```

Вывод:

```
Mon Apr 12 12:00:04 1982
Is DST: 0
```

Заполненная вручную структура здесь обозначает *местное* время. `mktime()` преобразует его в календарное значение `time_t`.

Автор отмечает, что стандарт, рассматриваемый в книге, не предоставляет прямого способа преобразовать `struct tm`, заданную в UTC, в `time_t`. На Unix-подобных системах может быть доступна нестандартная `timegm()`, а на Windows — `_mkgmtime()`.

38.4. Форматированный вывод даты

Мы уже использовали `ctime()` для `time_t` и `asctime()` для `struct tm`:

```
time_t now = time(NULL);

struct tm *local = localtime(&now);
struct tm *utc = gmtime(&now);

printf("Local time: %s", ctime(&now));     // Local time with time_t
printf("Local time: %s", asctime(local));  // Local time with struct tm
printf("UTC       : %s", asctime(utc));    // UTC with a struct tm
```

Для более гибкого формата можно извлечь поля `struct tm` вручную, но функция `strftime()` сделает большую часть работы. Она напоминает `printf()`, только предназначена для дат.

Ей передают буфер назначения, его максимальный размер и строку формата, определяющую, какие компоненты `struct tm` выводить и как. В формат можно включать и обычные символы. В примере структура получена через `localtime()`, но может поступать и из другого источника.

```
#include <stdio.h>
#include <time.h>

int main(void)
{
    char s[128];
    time_t now = time(NULL);

    // %c: print date as per current locale
    strftime(s, sizeof s, "%c", localtime(&now));
    puts(s);  // Sun Feb 28 22:29:00 2021

    // %A: full weekday name
    // %B: full month name
    // %d: day of the month
    strftime(s, sizeof s, "%A, %B %d", localtime(&now));
    puts(s);  // Sunday, February 28

    // %I: hour (12 hour clock)
    // %M: minute
    // %S: second
    // %p: AM or PM
    strftime(s, sizeof s, "It's %I:%M:%S %p", localtime(&now));
    puts(s);  // It's 10:29:00 PM

    // %F: ISO 8601 yyyy-mm-dd
    // %T: ISO 8601 hh:mm:ss
    // %z: ISO 8601 time zone offset
    strftime(s, sizeof s, "ISO 8601: %FT%T%z", localtime(&now));
    puts(s);  // ISO 8601: 2021-02-28T22:29:00-0800
}
```

У `strftime()` много других спецификаторов — загляните в справочник функции.

38.5. Более высокая точность с timespec_get()

С помощью `timespec_get()` можно получить секунды и наносекунды с начала эпохи. Впрочем, реализация может не иметь наносекундного разрешения, поэтому количество значащих разрядов зависит от системы.

Функция принимает указатель на `struct timespec` для результата и основание времени. Стандарт предоставляет `TIME_UTC`; реализации могут поддерживать и другие значения. Структура содержит два поля:

```
struct timespec {
    time_t tv_sec;  // Seconds
    long tv_nsec;   // Nanoseconds (billionths of a second)
};
```

Получим время и выведем его как целые значения и как число с плавающей точкой:

```
struct timespec ts;

timespec_get(&ts, TIME_UTC);

printf("%ld s, %ld ns\n", ts.tv_sec, ts.tv_nsec);

double float_time = ts.tv_sec + ts.tv_nsec/1000000000.0;
printf("%f seconds since epoch\n", float_time);
```

Пример вывода:

```
1614581530 s, 806325800 ns
1614581530.806326 seconds since epoch
```

`struct timespec` также используется некоторыми функциями работы с потоками, которым требуется такая детализация времени.

38.6. Разница между моментами времени

Стандарт не предписывает конкретное представление `time_t`, поэтому простое вычитание двух значений не обязательно даст осмысленный результат. В POSIX такое вычитание возможно, но для переносимости лучше использовать `difftime()`, которая возвращает разницу в секундах.

```
#include <stdio.h>
#include <time.h>

int main(void)
{
    struct tm time_a = {
        .tm_year=82,  // years since 1900
        .tm_mon=3,    // months since January -- [0, 11]
        .tm_mday=12,  // day of the month -- [1, 31]
        .tm_hour=4,   // hours since midnight -- [0, 23]
        .tm_min=00,   // minutes after the hour -- [0, 59]
        .tm_sec=04,   // seconds after the minute -- [0, 60]
        .tm_isdst=-1, // Daylight Saving Time flag
    };

    struct tm time_b = {
        .tm_year=120, // years since 1900
        .tm_mon=10,   // months since January -- [0, 11]
        .tm_mday=15,  // day of the month -- [1, 31]
```

```
        .tm_hour=16,  // hours since midnight -- [0, 23]
        .tm_min=27,   // minutes after the hour -- [0, 59]
        .tm_sec=00,   // seconds after the minute -- [0, 60]
        .tm_isdst=-1, // Daylight Saving Time flag
    };

    time_t cal_a = mktime(&time_a);
    time_t cal_b = mktime(&time_b);

    double diff = difftime(cal_b, cal_a);

    double years = diff / 60 / 60 / 24 / 365.2425;  // close enough

    printf("%f seconds (%f years) between events\n", diff, years);
}
```

Пример результата:

```
1217996816.000000 seconds (38.596783 years) between events
```

Помните о `difftime()` при вычислении промежутков между моментами времени: даже если на POSIX доступно обычное вычитание, переносимый вариант предпочтительнее.

Глава 39. Многопоточность

Стандарт C11 официально добавил многопоточность, во многом похожую на POSIX threads. Эта глава знакомит с основными средствами, но не заменяет подробного руководства по параллельному программированию.

Поддержка потоков необязательна. Если реализация определяет `__STDC_NO_THREADS__`, библиотечные средства потоков недоступны:

```
#ifdef __STDC_NO_THREADS__
#error I need threads to build this program!
#endif
```

При сборке могут потребоваться дополнительные параметры компоновщика. На Unix-подобных системах для GCC часто используют `-pthread`; точный вариант зависит от компилятора и платформы.

39.1. Общие сведения

Потоки позволяют нескольким частям одной программы выполняться одновременно и использовать доступные ядра процессора. Система обычно распределяет потоки между ядрами. Потоков может быть больше, чем ядер, но тогда они конкурируют за процессорное время.

39.2. Возможности

Можно создать поток, передав ему функцию для выполнения. Создавший поток продолжает работу. Можно дождаться завершения созданного потока (*join*) или отсоединить его (*detach*), если ждать не требуется. Поток завершается возвратом из своей функции или явным вызовом завершения; он также может спать заданное время. `main()` выполняется в собственном потоке. Кроме того, C предоставляет локальные данные потока, мьютексы и условные переменные.

39.3. Гонки данных и стандартная библиотека

Некоторые функции стандартной библиотеки, например `asctime()` и `strtok()`, используют общие статические данные и могут быть небезопасны при одновременных вызовах. Обычно библиотека стремится обеспечить безопасность, если не оговорено иное, но проверяйте документацию: функции, сохраняющие внутреннее состояние или возвращающие указатель на внутренние данные, требуют особого внимания.

39.4. Создание потока и ожидание завершения

Поток обозначается непрозрачным значением `thrd_t`. При создании нужно передать функцию и указатель на её аргумент, либо `NULL`. Функция потока имеет тип `thrd_start_t`: она

принимает `void *` и возвращает `int`.

Обычная последовательность: написать функцию, создать поток через `thrd_create()`, продолжить работу и затем дождаться его через `thrd_join()`. Неотсоединённый поток обычно нужно присоединить, чтобы освободить связанные ресурсы.

```
#include <stdio.h>
#include <threads.h>

int run(void *arg)
{
    int *a = arg;
    printf("THREAD: Running thread with arg %d\n", *a);
    return 12; // Результат для thrd_join()
}

int main(void)
{
    thrd_t t;
    int arg = 3490;
    printf("Launching a thread\n");
    thrd_create(&t, run, &arg);
    printf("Doing other things while the thread runs\n");
    printf("Waiting for thread to complete...\n");
    int res;
    thrd_join(t, &res);
    printf("Thread exited with return value %d\n", res);
}
```

Порядок сообщений может меняться. Переданный аргумент должен существовать достаточно долго, чтобы поток успел им воспользоваться; другой поток не должен одновременно изменять его без синхронизации. В производственном коде проверяйте результаты `thrd_create()` и `thrd_join()`.

Посмотрим на ошибочный пример с пятью потоками:

```
#include <stdio.h>
#include <threads.h>

int run(void *arg)
{
    int i = *(int*)arg;
    printf("THREAD %d: running!\n", i);
    return i;
}

#define THREAD_COUNT 5

int main(void)
{
    thrd_t t[THREAD_COUNT];
    int i;
    printf("Launching threads...\n");
    for (i = 0; i < THREAD_COUNT; i++)
        thrd_create(t + i, run, &i); // Все получают адрес одной i!

    printf("Doing other things while the thread runs...\n");
    printf("Waiting for thread to complete...\n");
```

```
    for (int i = 0; i < THREAD_COUNT; i++) {
        int res;
        thrd_join(t[i], &res);
        printf("Thread %d complete!\n", res);
    }
    printf("All threads complete!\n");
}
```

Главный поток меняет `i`, пока другие потоки её читают. Может показаться, что некоторые номера повторяются или появляется 5. Строго говоря, несинхронизированные чтение и запись создают гонку данных и неопределённое поведение: полагаться нельзя ни на какой конкретный результат.

Нужно выделить отдельное значение каждому потоку, например через `malloc()`. Поток копирует его и освобождает память:

```
#include <stdio.h>
#include <stdlib.h>
#include <threads.h>

int run(void *arg)
{
    int i = *(int*)arg;
    free(arg);
    printf("THREAD %d: running!\n", i);
    return i;
}

#define THREAD_COUNT 5

int main(void)
{
    thrd_t t[THREAD_COUNT];
    printf("Launching threads...\n");
    for (int i = 0; i < THREAD_COUNT; i++) {
        int *arg = malloc(sizeof *arg);
        *arg = i;
        thrd_create(t + i, run, arg);
    }
    for (int i = 0; i < THREAD_COUNT; i++) {
        int res;
        thrd_join(t[i], &res);
        printf("Thread %d complete!\n", res);
    }
}
```

Порядок работы потоков не гарантирован. В реальной программе также проверяйте `malloc()` и создание потока; если создание не удалось, выделенную память должен освободить вызывающий код.

39.5. Отсоединение потоков

Если результат потока не нужен, его можно отсоединить вместо ожидания:

```
thrd_create(&t, run, NULL);
```

```
thrd_detach(t);
```

Ресурсы отсоединённого потока освобождаются после его завершения, но получить его результат через `thrd_join()` уже нельзя.

```
#include <stdio.h>
#include <threads.h>

int run(void *arg)
{
    (void)arg;
    // printf("Thread running! %lu\n", thrd_current()); // Непереносимо!
    printf("Thread running!\n");
    return 0;
}

#define THREAD_COUNT 10

int main(void)
{
    thrd_t t;
    for (int i = 0; i < THREAD_COUNT; i++) {
        thrd_create(&t, run, NULL);
        thrd_detach(t);
    }
    thrd_sleep(&(struct timespec){.tv_sec=1}, NULL);
}
```

Пауза на секунду лишь даёт потокам время: она не гарантирует, что все завершатся до конца программы. `thrd_t` — непрозрачный тип и не обязан подходить для `%lu`; закомментированный вывод непереносим. Идентификаторы завершившихся потоков могут переиспользоваться.

39.6. Локальные данные потока

Автоматические локальные переменные каждого вызова отдельны, но статическая переменная функции или переменная области файла обычно общая для всех потоков. Без синхронизации это создаёт гонки:

```
#include <stdio.h>
#include <stdlib.h>
#include <threads.h>

int run(void *arg)
{
    int n = *(int*)arg;
    free(arg);
    static int foo = 10; // Общая переменная
    int x = foo;         // Отдельная локальная переменная
    printf("Thread %d: x = %d, foo = %d\n", n, x, foo);
    if (x != foo)
        printf("Thread %d: Craziness! x != foo! %d != %d\n", n, x, foo);
    foo++;
    return 0;
}
```

```
#define THREAD_COUNT 5

int main(void)
{
    thrd_t t[THREAD_COUNT];
    for (int i = 0; i < THREAD_COUNT; i++) {
        int *n = malloc(sizeof *n);
        *n = i;
        thrd_create(t + i, run, n);
    }
    for (int i = 0; i < THREAD_COUNT; i++)
        thrd_join(t[i], NULL);
}
```

Другой поток может изменить `foo` между обращениями. Но важно: это гонка данных и неопределённое поведение, поэтому даже пример наблюдаемого результата не гарантирован. Если переменная должна быть общей, защитите её мьютексом; если каждому нужна отдельная копия, используйте локальное хранилище потока.

39.6.1. Класс хранения thread_local

Объявление `thread_local` создаёт отдельный экземпляр переменной для каждого потока. В C11/C17 заголовок `<threads.h>` определяет это имя как макрос для `_Thread_local`; в C23 `thread_local` — ключевое слово.

```
int run(void *arg)
{
    int n = *(int*)arg;
    free(arg);
    thread_local static int foo = 10; // У каждого потока своя копия
    int x = foo;
    printf("Thread %d: x = %d, foo = %d\n", n, x, foo);
    if (x != foo)
        printf("Thread %d: Craziness! x != foo! %d != %d\n", n, x, foo);
    foo++;
    return 0;
}
```

Теперь каждый поток начинает со своей `foo == 10`. Для переменной с областью блока `thread_local` должен сочетаться с `static` или `extern`; автоматические локальные переменные и так принадлежат отдельным вызовам.

39.6.2. Хранилище, специфичное для потока (TSS)

Другой механизм — TSS. Его особенность: можно задать деструктор для данных потока, например `free()`. Тип `tss_dtor_t` — указатель на функцию, принимающую `void *` и возвращающую `void`.

Создайте ключ `tss_t` через `tss_create()`, устанавливайте и получайте значения функциями `tss_set()` и `tss_get()`, затем удалите ключ через `tss_delete()`. Деструкторы вызываются при завершении соответствующих потоков, а не при самом `tss_delete()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <threads.h>

tss_t str;

void some_function(void)
{
    char *tss_string = tss_get(str);
    printf("TSS string: %s\n", tss_string);
}

int run(void *arg)
{
    int serial = *(int*)arg;
    free(arg);
    char *s = malloc(64);
    sprintf(s, "thread %d! :)", serial);
    tss_set(str, s);
    some_function();
    return 0;
}

#define THREAD_COUNT 15

int main(void)
{
    thrd_t t[THREAD_COUNT];
    tss_create(&str, free);
    for (int i = 0; i < THREAD_COUNT; i++) {
        int *n = malloc(sizeof *n);
        *n = i;
        thrd_create(t + i, run, n);
    }
    for (int i = 0; i < THREAD_COUNT; i++)
        thrd_join(t[i], NULL);
    tss_delete(str);
}
```

Пример опускает проверки ошибок выделения, создания потока и TSS. Если деструктор не нужен, `thread_local` обычно проще.

39.7. Мьютексы

Мьютекс обеспечивает взаимное исключение: только один поток одновременно владеет блокировкой и выполняет защищённый критический участок. Другие потоки ждут освобождения мьютекса.

Мьютекс типа `mtx_t` создаётся через `mtx_init()`, захватывается через `mtx_lock()`, освобождается через `mtx_unlock()` и уничтожается через `mtx_destroy()`.

Без защиты следующий доступ к общей переменной создаёт гонку данных:

```
int run(void *arg)
{
    (void)arg;
    static int serial = 0;
```

```
    printf("Thread running! %d\n", serial);
    serial++;
    return 0;
}
```

Несколько потоков могут прочитать одно значение, а поведение программы в целом неопределённо. Защитим чтение и увеличение вместе:

```
#include <stdio.h>
#include <threads.h>

mtx_t serial_mtx;

int run(void *arg)
{
    (void)arg;
    static int serial = 0;
    mtx_lock(&serial_mtx);
    printf("Thread running! %d\n", serial);
    serial++;
    mtx_unlock(&serial_mtx);
    return 0;
}

#define THREAD_COUNT 10

int main(void)
{
    thrd_t t[THREAD_COUNT];
    mtx_init(&serial_mtx, mtx_plain);
    for (int i = 0; i < THREAD_COUNT; i++)
        thrd_create(t + i, run, NULL);
    for (int i = 0; i < THREAD_COUNT; i++)
        thrd_join(t[i], NULL);
    mtx_destroy(&serial_mtx);
}
```

Между `mtx_lock()` и `mtx_unlock()` находится критический участок. Числа будут выданы от 0 до 9, хотя порядок самих потоков не определён. Проверяйте возвращаемые значения функций. При работе с несколькими мьютексами используйте единый порядок их захвата во всех потоках, чтобы избежать взаимной блокировки; освобождают их обычно в обратном порядке.

39.7.1. Типы мьютексов

Тип	Назначение
`mtx_plain`	Обычный мьютекс
`mtx_timed`	Поддерживает ограниченное по времени ожидание
`mtx_plain | mtx_recursive`	Рекурсивный мьютекс
`mtx_timed | mtx_recursive`	Рекурсивный с ограниченным ожиданием

Рекурсивный мьютекс позволяет владельцу захватывать его повторно; освободить его нужно столько же раз. Для ограниченного ожидания используют абсолютное время UTC и `mtx_timedlock()`:

```
mtx_init(&serial_mtx, mtx_timed);
struct timespec timeout;
timespec_get(&timeout, TIME_UTC);
timeout.tv_sec += 1;
int result = mtx_timedlock(&serial_mtx, &timeout);
if (result == thrd_timedout) {
    printf("Mutex lock timed out!\n");
}
```

`timespec` содержит секунды `tv_sec` и наносекунды `tv_nsec`. При успешном захвате мьютекс нужно позднее освободить.

39.8. Условные переменные

Условная переменная позволяет потоку спать, пока другой поток не сообщит о возможном изменении условия. Само условие обычно относится к общим данным, которые защищает мьютекс; `cnd_t` не хранит эти данные.

Создание и удаление: `cnd_init()` и `cnd_destroy()`. Ожидающий поток захватывает мьютекс и вызывает `cnd_wait()`, пока условие не выполнено. Вызов атомарно освобождает мьютекс на время ожидания, а перед возвратом снова захватывает его. Сигнализирующий поток меняет данные под мьютексом, вызывает `cnd_signal()` и освобождает мьютекс. Сигнал сам мьютекс не освобождает.

Условие проверяют в `while`, а не в `if`: возможны ложные пробуждения, а также ситуация, когда другой поток успел изменить данные до повторного захвата мьютекса.

Рассмотрим задачу производителя и потребителя: главный поток вводит числа группами по пять, а дочерний суммирует каждую группу. Ниже приведён оригинальный пример автора, сохранённый для изучения. В нём есть ошибки: поток запускается до инициализации мьютекса и условной переменной; кроме того, производитель может записать шестой элемент до обработки пятого. Поэтому этот пример не следует использовать как готовое решение.

```
#include <stdio.h>
#include <threads.h>

#define VALUE_COUNT_MAX 5

int value[VALUE_COUNT_MAX]; // Shared global
int value_count = 0; // Shared global, too

mtx_t value_mtx; // Mutex around value
cnd_t value_cnd; // Condition variable on value

int run(void *arg)
{
    (void)arg;
```

```
    for (;;) {
        mtx_lock(&value_mtx); // <-- GRAB THE MUTEX

        while (value_count < VALUE_COUNT_MAX) {
            printf("Thread: is waiting \n");
            cnd_wait(&value_cnd, &value_mtx); // <-- CONDITION WAIT
        }

        printf("Thread: is awake! \n");

        int t = 0;

        // Add everything up
        for (int i = 0; i < VALUE_COUNT_MAX; i++)
            t += value[i];

        printf("Thread: total is %d\n", t);

        // Reset input index for main thread
        value_count = 0;

        mtx_unlock(&value_mtx); // <-- MUTEX UNLOCK
    }

    return 0;
}

int main(void)
{
    thrd_t t;

    // Spawn a new thread
    thrd_create(&t, run, NULL);
    thrd_detach(t);

    // Set up the mutex and condition variable
    mtx_init(&value_mtx, mtx_plain);
    cnd_init(&value_cnd);

    for (;;) {
        int n;

        scanf("%d", &n);

        mtx_lock(&value_mtx); // <-- LOCK MUTEX

        value[value_count++] = n;

        if (value_count == VALUE_COUNT_MAX) {
            printf("Main: signaling thread \n");
            cnd_signal(&value_cnd); // <-- SIGNAL CONDITION
        }

        mtx_unlock(&value_mtx); // <-- UNLOCK MUTEX
    }

    // Clean up (I know that's an infinite loop above here, but I
    // want to at least pretend to be proper):
    mtx_destroy(&value_mtx);
    cnd_destroy(&value_cnd);
}
```

Исправленный вариант ниже инициализирует синхронизацию заранее и ждёт освобождения буфера:

```
#include <stdio.h>
#include <threads.h>

#define VALUE_COUNT_MAX 5
int value[VALUE_COUNT_MAX];
int value_count = 0;
mtx_t value_mtx;
cnd_t value_cnd;

int run(void *arg)
{
    (void)arg;
    for (;;) {
        mtx_lock(&value_mtx);
        while (value_count < VALUE_COUNT_MAX)
            cnd_wait(&value_cnd, &value_mtx);
        int total = 0;
        for (int i = 0; i < VALUE_COUNT_MAX; i++)
            total += value[i];
        printf("Thread: total is %d\n", total);
        value_count = 0;
        cnd_broadcast(&value_cnd);
        mtx_unlock(&value_mtx);
    }
    return 0;
}

int main(void)
{
    thrd_t t;
    mtx_init(&value_mtx, mtx_plain);
    cnd_init(&value_cnd);
    thrd_create(&t, run, NULL);
    thrd_detach(t);
    for (;;) {
        int n;
        if (scanf("%d", &n) != 1) break;
        mtx_lock(&value_mtx);
        while (value_count == VALUE_COUNT_MAX)
            cnd_wait(&value_cnd, &value_mtx);
        value[value_count++] = n;
        if (value_count == VALUE_COUNT_MAX)
            cnd_broadcast(&value_cnd);
        mtx_unlock(&value_mtx);
    }
    // Для управляемого завершения нужен отдельный флаг остановки
    // и присоединение потока; пример сосредоточен на ожидании.
}
```

При вводе пяти единиц сумма будет 5; следующая группа `2 8 5 9 0` даст 24. На практике предусмотрите корректную остановку и проверку ошибок. Преимущество ожидания перед постоянным опросом — поток не тратит процессорное время впустую.

39.8.1. Ожидание с тайм-аутом

`cnd_timedwait()` принимает абсолютное время UTC и возвращает `thrd_timedout` по истечении срока. Перед возвратом поток всё равно должен снова захватить мьютекс, поэтому возврат может произойти позднее самой границы времени.

```
struct timespec timeout;
timespec_get(&timeout, TIME_UTC);
timeout.tv_sec += 1;
int result = cnd_timedwait(&condition, &mutex, &timeout);
if (result == thrd_timedout) {
    printf("Condition variable timed out!\n");
}
```

39.8.2. Пробуждение всех ожидающих потоков

`cnd_signal()` пробуждает один ожидающий поток; `cnd_broadcast()` сообщает всем ожидающим на данной условной переменной. Хотя одновременно мьютекс получит только один, широковещательное уведомление бывает нужно, если несколько потоков должны повторно проверить условие и нельзя заранее знать, какой из них сможет продолжить.

39.9. Однократный запуск функции

`call_once()` гарантирует однократный вызов функции для данного флага, даже если к ней одновременно обращаются несколько потоков. Функция не принимает аргументов и ничего не возвращает:

```
once_flag of = ONCE_FLAG_INIT;

void run_once_function(void)
{
    printf("I'll only run once!\n");
}

int run(void *arg)
{
    (void)arg;
    call_once(&of, run_once_function);
    // ...
    return 0;
}
```

Сколько бы потоков ни вызвали `run()`, `run_once_function()` для этого флага выполнится один раз.

Глава 40. Атомарные операции

> — Они пытались и потерпели неудачу, все до одного? — Нет, — она покачала головой. — Они пытались и погибли.
>
> Пол Атрейдес и преподобная мать Гайус Хелен Мохайам, «Дюна»

Это одна из наиболее сложных тем многопоточного программирования на C. Но мы постараемся разобраться спокойно.

Я расскажу о более простых способах применения атомарных переменных, о том, что это такое и как они работают. Также упомяну некоторые невероятно сложные возможности, но подробно рассматривать их не буду. Если вы знаете, что они вам нужны, то, вероятно, знаете об этом больше меня.

Даже в основах здесь есть странности. Так что пристегните ремни: Канзас остаётся позади.

40.1. Проверка поддержки атомарных операций

Атомарные операции — необязательная возможность. Макрос `__STDC_NO_ATOMICS__` равен 1, если поддержки нет.

До C11 этот макрос может отсутствовать, поэтому проверим версию языка с помощью `__STDC_VERSION__`:

```
#if __STDC_VERSION__ < 201112L || __STDC_NO_ATOMICS__ == 1
#define HAS_ATOMICS 0
#else
#define HAS_ATOMICS 1
#endif
```

В раннем C89 макроса `__STDC_VERSION__` не было; если это важно, проверяйте его наличие через `#ifdef`. После успешной проверки можно подключить `<stdatomic.h>`, на котором основана остальная часть главы. Если атомарные операции не поддерживаются, заголовок может отсутствовать. В некоторых системах для функций из него может понадобиться добавить `-latomic` в конец команды компиляции.

40.2. Атомарные переменные

Часть смысла атомарности такова: если один поток записывает общую атомарную переменную, другой видит запись целиком, а не частично. Например, он не увидит половину записанного 32-битного значения.

Это похоже на небольшую блокировку вокруг чтения и записи одной переменной. Иногда такая блокировка действительно используется; см. раздел об атомарных переменных без блокировок.

Можно вообще обойтись без атомарных переменных, защищая критические секции мьютексами. Существуют и структуры данных без блокировок, позволяющие потокам продвигаться без ожидания мьютекса, но правильно создать их с нуля непросто и выходит за рамки книги.

Это лишь часть истории, но с неё мы начнём. Как объявить атомарную переменную? Сначала подключить `<stdatomic.h>`, который предоставляет типы вроде `atomic_int`, а затем объявить переменную этого типа.

Рассмотрим два потока. Первый некоторое время работает, устанавливает переменную в определённое значение и завершается. Второй ждёт, пока увидит это значение, и тоже завершается:

```
#include <stdio.h>
#include <threads.h>
#include <stdatomic.h>

atomic_int x; // THE POWER OF ATOMICS! BWHAHAHA!

int thread1(void *arg)
{
    (void)arg;

    printf("Thread 1: Sleeping for 1.5 seconds\n");
    thrd_sleep(&(struct timespec){.tv_sec=1, .tv_nsec=500000000}, NULL);

    printf("Thread 1: Setting x to 3490\n");
    x = 3490;

    printf("Thread 1: Exiting\n");
    return 0;
}

int thread2(void *arg)
{
    (void)arg;

    printf("Thread 2: Waiting for 3490\n");
    while (x != 3490) {} // spin here

    printf("Thread 2: Got 3490--exiting!\n");
    return 0;
}

int main(void)
{
    x = 0;

    thrd_t t1, t2;

    thrd_create(&t1, thread1, NULL);
    thrd_create(&t2, thread2, NULL);

    thrd_join(t1, NULL);
    thrd_join(t2, NULL);

    printf("Main : Threads are done, so x better be 3490\n");
```

```
    printf("Main : And indeed, x == %d\n", x);
}
```

Второй поток постоянно проверяет флаг, ожидая `3490`, а первый устанавливает это значение. У меня выводится:

```
Thread 1: Sleeping for 1.5 seconds
Thread 2: Waiting for 3490
Thread 1: Setting x to 3490
Thread 1: Exiting
Thread 2: Got 3490--exiting!
Main : Threads are done, so x better be 3490
Main : And indeed, x == 3490
```

Смотри, мама! Мы обращаемся к переменной из разных потоков без мьютекса! Это работает благодаря атомарности переменной.

Что если заменить её обычным неатомарным `int`? В моей системе пример иногда работает, но при оптимизированной сборке второй поток может навсегда застрять в ожидании. Компилятор может держать значение в регистре и не замечать запись другого потока. Кроме того, несинхронизированные чтение и запись обычной переменной создают гонку данных и неопределённое поведение.

Это только начало. Следующая часть посвящена *синхронизации* и потребует больше умственных усилий.

40.3. Синхронизация

Теперь поговорим о том, когда записи в память одного потока становятся видимы другому.

Может показаться, что немедленно. Но всё сложнее. Компилятор может переупорядочить обращения к памяти, процессор тоже может выполнять операции иначе, а архитектура может задерживать видимость записей между ядрами.

Общий смысл: несинхронизированные обращения к памяти могут наблюдаться разными потоками в разном порядке, как будто строки программы переставлены.

Например, что произойдёт первым: запись в `x` или в `y`?

```
int x, y; // global

// ...

x = 2;
y = 3;

printf("%d %d\n", x, y);
```

Компилятор или процессор могут организовать действия иначе, если однопоточное наблюдаемое поведение останется таким, будто код выполнен по порядку.

Представим многопоточный псевдокод:

```
int x = 0, y = 0;

thread1() {
    x = 2;
    y = 3;
}

thread2() {
    while (y != 3) {} // spin
    printf("x is now %d\n", x); // 2? ...or 0?
}
```

Можно ожидать `x is now 2`, но без синхронизации порядок видимости не гарантирован. Более того, это пример гонки данных: поведение программы не определено и не ограничивается выводом 0 или 2.

Нужен способ сказать: «К этому моменту предыдущие записи другого потока должны быть видимы здесь». Потоки синхронизируются, когда устанавливают согласованный порядок операций с общей памятью.

Атомарные операции с подходящим порядком памяти позволяют это сделать. Пока мы говорим об операциях с последовательной согласованностью, используемой по умолчанию. Если один поток записывает атомарную переменную, а другой считывает соответствующее значение с нужной семантикой, записи первого потока до этой атомарной операции становятся видимы второму.

Исправим пример:

```
int x = 0;
atomic int y = 0; // Make y atomic

thread1() {
    x = 2;
    y = 3; // Synchronize on write
}

thread2() {
    while (y != 3) {} // Synchronize on read
    printf("x is now %d\n", x); // 2, period.
}
```

Когда второй поток считывает записанное первым значение `y`, синхронизация делает предшествующую запись `x` видимой. Здесь важно:

В зависимости от версии C и реализации для объявления `y` может потребоваться стандартная запись `atomic_int y = 0;` или `_Atomic int y = 0;`. Строка выше сохранена по оригиналу.

1. Само ожидание не усыпляет поток: цикл активно проверяет значение.
2. Синхронизация возникает между подходящей записью и чтением, которое наблюдает её значение или соответствующую последовательность.

3. `x` не обязана быть атомарной, поскольку её запись упорядочена до чтения через синхронизацию по `y`.

Синхронизация может стоить дороже обычного доступа к переменной. Поэтому атомарные операции используют там, где они нужны.

40.4. Acquire и release

Новые термины! *Acquire* — семантика приобретения, *release* — освобождения. При операциях по умолчанию атомарное чтение имеет свойства acquire, а атомарная запись — release. Не всякая атомарная операция в любом порядке памяти обладает этими свойствами: например, `relaxed` их не даёт.

При согласованной паре запись с release и чтение с acquire одного атомарного объекта могут синхронизировать потоки: если acquire наблюдает соответствующую release-запись, операции до release становятся видимыми после acquire.

В общих чертах acquire действует как односторонний барьер: последующие обращения не должны наблюдаться до него так, чтобы нарушить гарантии синхронизации. Release действует в противоположную сторону: предшествующие обращения не должны наблюдаться после него в нарушение этих гарантий.

```
int x, y, z = 0;
atomic_int a = 0;

thread1() {
    x = 10;
    y = 20;
    a = 999; // Release
    z = 30;
}

thread2()
{
    while (a != 999) { } // Acquire
    assert(x == 10); // never asserts, x is always 10
    assert(y == 20); // never asserts, y is always 20
    assert(z == 0);  // might assert!!
}
```

Второй поток может рассчитывать на `x` и `y`, поскольку они записаны до release. Запись `z` происходит после неё и не покрывается этой синхронизацией. Более того, как написан пример, одновременная запись и чтение обычной `z` могут создать гонку данных и неопределённое поведение; чтобы исследовать возможные значения `z`, её тоже нужно сделать атомарной или дополнительно синхронизировать.

Синхронизация не возникает автоматически только потому, что один поток выполнил release одной атомарной переменной, а другой — acquire другой. Необходима предусмотренная стандартом связь между операциями.

40.5. Последовательная согласованность

Держитесь? Основную часть простого применения атомарных операций мы прошли.

Последовательная согласованность — один из порядков памяти. С точки зрения программиста это наиболее понятный вариант, и он применяется по умолчанию. Для других порядков их нужно указать явно.

Упрощённо, последовательно согласованные атомарные операции участвуют в едином общем порядке, совместимом с необходимыми ограничениями порядка операций. Это дополняет гарантии acquire/release и облегчает рассуждения о взаимодействии потоков. Нельзя, однако, считать, что все произвольные обращения к памяти всех потоков автоматически упорядочены без синхронизации.

Другие порядки ослабляют некоторые ограничения. Их используют, когда хорошо понимают модель памяти и нужна производительность. Здесь водятся целые армии драконов.

40.6. Атомарные присваивания и операторы

Некоторые операторы над атомарными переменными представляют собой единую атомарную операцию, другие — нет. Начнём с примера:

```
atomic_int x = 0;

thread1() {
    x = x + 3; // NOT atomic!
}
```

Здесь чтение `x` справа и запись слева — две отдельные атомарные операции, но всё выражение целиком не является одной атомарной операцией чтения-изменения-записи. Между ними может вмешаться другой поток.

Сокращённая форма выполняет атомарное обновление:

```
atomic_int x = 0;

thread1() {
    x += 3; // ATOMIC!
}
```

Теперь `x` увеличивается на 3 одной атомарной операцией. Следующие составные операторы и операторы увеличения/уменьшения для подходящих атомарных типов выполняют атомарное чтение-изменение-запись с последовательной согласованностью:

```
a++  a--  --a  ++a
a += b  a -= b  a *= b  a /= b  a %= b
a &= b  a |= b  a ^= b  a >>= b  a <<= b
```

40.7. Библиотечные функции с автоматической синхронизацией

Некоторые библиотечные функции сами выполняют ограниченную синхронизацию:

```
call_once()   thrd_create()   thrd_join()
mtx_lock()    mtx_timedlock() mtx_trylock()
malloc()      calloc()        realloc()
aligned_alloc()
```

- `call_once()` синхронизирует завершение однократной инициализации с другими вызовами для того же флага.
- `thrd_create()` синхронизируется с началом нового потока: новый поток видит предшествующие записи родительского.
- `thrd_join()` синхронизируется с завершением ожидаемого потока, так что вызывающий поток видит его завершённые записи.
- `mtx_lock()` синхронизируется с предыдущим освобождением того же мьютекса. Это напоминает acquire/release: изменения в защищённой критической секции становятся видимыми следующему владельцу мьютекса.
- Успешные `mtx_timedlock()` и `mtx_trylock()` работают аналогично.
- Функции динамического выделения связаны с предыдущим освобождением того же участка; операции выделения и освобождения конкретного участка имеют определённый общий порядок.

40.8. Спецификатор и квалификатор атомарного типа

Посмотрим на встроенные атомарные типы и их эквиваленты. Спойлер: `_Atomic` может быть квалификатором типа!

Атомарный тип	Эквивалент
`atomic_bool`	`_Atomic _Bool`
`atomic_char`	`_Atomic char`
`atomic_schar`	`_Atomic signed char`
`atomic_uchar`	`_Atomic unsigned char`
`atomic_short`	`_Atomic short`
`atomic_ushort`	`_Atomic unsigned short`
`atomic_int`	`_Atomic int`
`atomic_uint`	`_Atomic unsigned int`
`atomic_long`	`_Atomic long`

Атомарный тип	Эквивалент
`atomic_ulong`	`_Atomic unsigned long`
`atomic_llong`	`_Atomic long long`
`atomic_ullong`	`_Atomic unsigned long long`
`atomic_char16_t`	`_Atomic char16_t`
`atomic_char32_t`	`_Atomic char32_t`
`atomic_wchar_t`	`_Atomic wchar_t`
`atomic_int_least8_t`	`_Atomic int_least8_t`
`atomic_uint_least8_t`	`_Atomic uint_least8_t`
`atomic_int_least16_t`	`_Atomic int_least16_t`
`atomic_uint_least16_t`	`_Atomic uint_least16_t`
`atomic_int_least32_t`	`_Atomic int_least32_t`
`atomic_uint_least32_t`	`_Atomic uint_least32_t`
`atomic_int_least64_t`	`_Atomic int_least64_t`
`atomic_uint_least64_t`	`_Atomic uint_least64_t`
`atomic_int_fast8_t`	`_Atomic int_fast8_t`
`atomic_uint_fast8_t`	`_Atomic uint_fast8_t`
`atomic_int_fast16_t`	`_Atomic int_fast16_t`
`atomic_uint_fast16_t`	`_Atomic uint_fast16_t`
`atomic_int_fast32_t`	`_Atomic int_fast32_t`
`atomic_uint_fast32_t`	`_Atomic uint_fast32_t`
`atomic_int_fast64_t`	`_Atomic int_fast64_t`
`atomic_uint_fast64_t`	`_Atomic uint_fast64_t`
`atomic_intptr_t`	`_Atomic intptr_t`
`atomic_uintptr_t`	`_Atomic uintptr_t`
`atomic_size_t`	`_Atomic size_t`
`atomic_ptrdiff_t`	`_Atomic ptrdiff_t`
`atomic_intmax_t`	`_Atomic intmax_t`
`atomic_uintmax_t`	`_Atomic uintmax_t`

Пользуйтесь ими свободно! Они согласуются с аналогичными атомарными именами C++.

Если нужно больше, можно использовать спецификатор или квалификатор. Спецификатор — `_Atomic` с типом в скобках; он подходит, например, для `typedef`:

```
typedef _Atomic(double) atomic_double;
atomic_double f;
```

Тип в скобках не может быть массивом, функцией, атомарным или иным квалифицированным типом.

Квалификатор — `_Atomic` без типа в скобках. Эти объявления похожи:

```
_Atomic(int) i; // type specifier
_Atomic int j;  // type qualifier
```

Во втором варианте можно добавить другие квалификаторы:

```
_Atomic volatile int k; // qualified atomic variable
```

Массив или функцию атомарным квалификатором объявить нельзя. Стандарт допускает различия в размере, представлении и выравнивании атомарных и соответствующих обычных типов.

40.9. Атомарные переменные без блокировок

Аппаратные архитектуры ограничены в том, какой объём данных могут читать и записывать атомарно. Если оборудование не обеспечивает нужную операцию напрямую, реализация может использовать блокировку: захватить её, выполнить доступ и освободить. Это может быть медленнее и важно для обработчиков сигналов.

Из атомарных типов стандарт гарантирует отсутствие блокировок только для `atomic_flag`. В обычных настольных системах и другие типы часто работают без блокировок.

Макросы ниже можно проверять при компиляции через `#if`; они применяются к соответствующим знаковым и беззнаковым типам:

Тип	Макрос
`atomic_bool`	`ATOMIC_BOOL_LOCK_FREE`
`atomic_char`	`ATOMIC_CHAR_LOCK_FREE`
`atomic_char16_t`	`ATOMIC_CHAR16_T_LOCK_FREE`
`atomic_char32_t`	`ATOMIC_CHAR32_T_LOCK_FREE`
`atomic_wchar_t`	`ATOMIC_WCHAR_T_LOCK_FREE`
`atomic_short`	`ATOMIC_SHORT_LOCK_FREE`
`atomic_int`	`ATOMIC_INT_LOCK_FREE`

Тип	Макрос
`atomic_long`	`ATOMIC_LONG_LOCK_FREE`
`atomic_llong`	`ATOMIC_LLONG_LOCK_FREE`
Атомарные указатели	`ATOMIC_POINTER_LOCK_FREE`

Значения макросов:

Значение	Смысл
0	Никогда не работает без блокировок.
1	Иногда работает без блокировок.
2	Всегда работает без блокировок.

«Иногда» означает, что ответ может зависеть от условий реализации и быть известен во время выполнения. Проверить конкретный объект можно функцией `atomic_is_lock_free()`.

Зачем это знать? Операция без блокировок может быть быстрее; также это важно при использовании атомарной переменной в обработчике сигнала.

40.9.1. Обработчики сигналов и атомарные операции без блокировок

При обращении обработчика сигнала к общим объектам действуют строгие ограничения. Среди предусмотренных вариантов — присваивание объекту `volatile sig_atomic_t` и использование подходящих атомарных объектов без блокировок в соответствии с правилами стандарта.

Автор отмечает, что детали порядка операций обработчика сигнала тонки: сигнал может возникнуть в разный момент, а аналогичные положения C++ отдельно оговаривают связь с `raise()`. При написании такого кода необходимо учитывать действующий стандарт и ограничения конкретной реализации.

40.10. Атомарные флаги

`atomic_flag` — единственный атомарный тип, для которого стандарт гарантирует работу без блокировок. Это непрозрачный тип для операции «проверить и установить».

Флаг может быть установлен или сброшен. Инициализировать сброшенный флаг можно так:

```
atomic_flag f = ATOMIC_FLAG_INIT;
```

`atomic_flag_test_and_set()` атомарно устанавливает флаг и возвращает его прежнее состояние как `_Bool`: `true`, если он уже был установлен. `atomic_flag_clear()` атомарно сбрасывает его.

В примере флаг сначала сброшен, затем дважды устанавливается, сбрасывается и устанавливается ещё раз:

```
#include <stdio.h>
#include <stdbool.h> // Not needed in C23
#include <stdatomic.h>

atomic_flag f = ATOMIC_FLAG_INIT;

int main(void)
{
    bool r = atomic_flag_test_and_set(&f);
    printf("Value was: %d\n", r); // 0

    r = atomic_flag_test_and_set(&f);
    printf("Value was: %d\n", r); // 1

    atomic_flag_clear(&f);

    r = atomic_flag_test_and_set(&f);
    printf("Value was: %d\n", r); // 0
}
```

40.11. Атомарные структуры и объединения

С помощью `_Atomic` можно сделать атомарной структуру или объединение! Если данных немного, полученный тип может работать без блокировок; проверим через `atomic_is_lock_free()`:

```
#include <stdio.h>
#include <stdatomic.h>

int main(void)
{
    struct point {
        float x, y;
    };

    _Atomic(struct point) p;

    printf("Is lock free: %d\n", atomic_is_lock_free(&p));
}
```

Подвох: обращаться к отдельным полям атомарной структуры или объединения напрямую нельзя. Зато можно атомарно скопировать всю структуру в обычную переменную и работать с ней. Копирование в обратную сторону тоже возможно:

```
#include <stdio.h>
#include <stdatomic.h>

int main(void)
{
    struct point {
        float x, y;
    };
```

```
    _Atomic(struct point) p;
    struct point t;

    p = (struct point){1, 2}; // Atomic copy

    //printf("%f\n", p.x); // Error

    t = p; // Atomic copy

    printf("%f\n", t.x); // OK!
}
```

Можно также объявить структуру, отдельные поля которой атомарны. Поддержка атомарных типов для битовых полей зависит от реализации.

40.12. Атомарные указатели

Важно расположение `_Atomic`. Сначала указатель на атомарный объект: сам указатель не атомарен, но объект — атомарен.

```
_Atomic int x;
_Atomic int *p; // p is a pointer to an atomic int
p = &x;        // OK!
```

Теперь атомарный указатель на неатомарное значение:

```
int x;
int * _Atomic p; // p is an atomic pointer to an int
p = &x;          // OK!
```

Наконец, атомарный указатель на атомарное значение:

```
_Atomic int x;
_Atomic int * _Atomic p; // p is an atomic pointer to an atomic int
p = &x;                  // OK!
```

40.13. Порядок памяти

Мы уже обсудили последовательную согласованность, но есть и другие порядки:

Константа	Описание
`memory_order_seq_cst`	Последовательная согласованность
`memory_order_acq_rel`	Acquire/release
`memory_order_release`	Release
`memory_order_acquire`	Acquire
`memory_order_consume`	Consume

Константа	Описание
`memory_order_relaxed`	Ослабленный порядок

Некоторые библиотечные функции позволяют указать порядок явно. Например, добавить число можно так:

```
atomic_int x = 0;
x += 5; // Sequential consistency, the default
```

Или так:

```
atomic_int x = 0;
atomic_fetch_add(&x, 5); // Sequential consistency, the default
```

Или с явно указанным порядком:

```
atomic_int x = 0;
atomic_fetch_add_explicit(&x, 5, memory_order_seq_cst);
```

Если по какой-то причине нужен acquire/release, укажите его:

```
atomic_int x = 0;
atomic_fetch_add_explicit(&x, 5, memory_order_acq_rel);
```

Далее разберём порядки по отдельности. Не меняйте стандартный последовательный порядок, пока не понимаете последствий: ошибки здесь могут проявляться редко и плохо воспроизводиться.

40.13.1. Последовательная согласованность

- Загрузка обладает свойствами acquire.
- Запись обладает свойствами release.
- Чтение-изменение-запись объединяет acquire и release.

Дополнительно последовательно согласованные операции участвуют в едином общем порядке. Это более сильное ограничение, чем отдельные гарантии acquire/release.

40.13.2. Acquire

Это порядок для чтения атомарной переменной. Если чтение наблюдает соответствующую release-запись другого потока, предшествующие ей записи становятся видимы; последующие обращения текущего потока не могут быть переупорядочены вопреки этой гарантии.

40.13.3. Release

Это порядок для записи атомарной переменной. Если другой поток затем выполнит связанное acquire-чтение, записи текущего потока до release станут ему видимы. Предшествующие обращения не могут быть переупорядочены вопреки этой гарантии.

40.13.4. Consume

Это более слабый вариант, похожий на acquire, но распространяющийся на операции, зависящие от прочитанного значения. Грубо говоря, значение атомарной переменной используется в последующем вычислении. Зависимые операции получают соответствующие гарантии видимости, тогда как acquire распространяет их шире.

Аналогично ограничивается перенос зависимых операций до consume. На практике этот порядок требует особой осторожности и учёта поддержки компилятора.

40.13.5. Acquire/release

Этот порядок применяется к операциям чтения-изменения-записи. Он объединяет acquire для чтения и release для записи.

40.13.6. Relaxed

Никаких правил, анархия! Все всё переставляют! Собаки и кошки живут вместе — массовая истерия!

На самом деле правило есть: чтение и запись остаются атомарными. Но `relaxed` сам по себе не создаёт синхронизацию между потоками. Для него есть применения, например некоторые простые счётчики. В отдельных схемах для синхронизации можно использовать барьеры вместе с атомарными операциями.

40.14. Барьеры памяти

Операции acquire и release могут быть выражены и отдельными барьерами — *fences*. Они позволяют задавать ограничения порядка отдельно от конкретной атомарной операции.

Для синхронизации потоков одних двух барьеров недостаточно: нужна предусмотренная стандартом связь через атомарные операции. `memory_order_consume` для барьера рассматривается как acquire, поскольку сам барьер не считывает значение, от которого могли бы зависеть операции.

Пример:

```
atomic_thread_fence(memory_order_release);
```

Есть и облегчённый вариант для обработчиков сигналов — `atomic_signal_fence()`. Он работает внутри одного потока между потоком и обработчиком сигнала и не синхронизирует

разные потоки; аппаратный барьер для него обычно не требуется. Его используют для ограничения переупорядочивания относительно обработчика, соблюдая при этом отдельные правила доступа к данным из обработчиков сигналов.

40.15. Дополнительные материалы

Чтобы глубже изучить тему, автор рекомендует:

- Доклад Herb Sutter *atomic<> Weapons*: часть 1 и часть 2.
- Материалы Jeff Preshing, в том числе статьи о программировании без блокировок, acquire/release, отношениях happens-before и synchronizes-with, consume и атомарных операциях чтения-изменения-записи.
- CPPReference: порядок памяти и атомарные типы.
- Bruce Dawson, Lockless Programming Considerations.
- Сообщество r/C_Programming.

Глава 41. Спецификаторы функций и выравнивания

По опыту автора, эти средства используются нечасто, но для полноты картины рассмотрим и их.

41.1. Спецификаторы функций

При объявлении функции можно сообщить компилятору некоторые сведения о её использовании. Это может помочь ему применить оптимизации или проверить код.

41.1.1. inline для скорости — возможно

Функцию можно объявить как `inline`:

```
static inline int add(int x, int y) {
    return x + y;
}
```

Это побуждает компилятор сделать вызов быстрым. Исторически один из способов — *встраивание*: тело функции копируется непосредственно в место вызова. Так можно избежать накладных расходов на вызов и возврат, но размер программы может увеличиться, поскольку код функции повторяется.

Краткие рекомендации автора:

1. Для ускорения, вероятно, не нужно использовать `inline`: современные компиляторы сами принимают решения об оптимизации.
2. Если используете его для скорости, применяйте функцию с внутренним связыванием — `static inline`. Так проще избежать сложных правил внешнего связывания и `inline`.

На этом можно закончить чтение раздела. Всё ещё хотите разобраться? Тогда уберём `static`:

```
#include <stdio.h>

inline int add(int x, int y)
{
    return x + y;
}

int main(void)
{
    printf("%d\n", add(1, 2));
}
```

GCC может сообщить об ошибке компоновки для `add()`: при использовании функции с внешним связыванием может понадобиться её внешнее определение. Если в текущем

файле есть `inline`-определение, а в другом месте — внешняя версия, реализация может выбрать, какую вызвать; поэтому автор рекомендует, чтобы они вели себя одинаково.

Другой вариант — объявление `extern inline`: оно позволяет использовать определение в текущей единице трансляции и создаёт внешнее определение. Правила зависят от режима языка и расширений компилятора, поэтому при необходимости проверьте документацию своего компилятора.

41.1.2. noreturn и _Noreturn

Этот спецификатор сообщает, что функция не вернёт управление вызывающему коду, например потому, что завершит программу. Это может помочь компилятору оптимизировать код и показывает другим разработчикам, что логика зависит от отсутствия возврата. Примеры стандартных функций — `exit()` и `abort()`.

В C11 встроенный спецификатор называется `_Noreturn`; заголовок `<stdnoreturn.h>` предоставляет более читаемое имя `noreturn`. В C23 предпочтителен стандартный атрибут `[[noreturn]]`. Если функция, объявленная как не возвращающая управление, всё же вернётся, поведение неопределённо.

Правильный пример:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdnoreturn.h>

noreturn void foo(void) // Эта функция не должна возвращаться!
{
    printf("Happy days\n");
    exit(1); // И не возвращается: здесь программа завершается
}

int main(void)
{
    foo();
}
```

Если заменить функцию на следующую, компилятор может предупредить, что она способна вернуться:

```
noreturn void foo(void)
{
    printf("Breakin' the law\n");
}
```

Пример предупреждения: `function declared 'noreturn' should not return` — функция, объявленная `noreturn`, не должна возвращать управление.

41.2. Спецификаторы и операторы выравнивания

Выравнивание определяет допустимые адреса размещения объекта: например, требуется ли адрес, кратный 2, 8 или 16. Это важно в низкоуровневом коде, например при создании распределителя памяти, работающего с ОС. Большинству разработчиков редко приходится задавать выравнивание вручную.

41.2.1. alignas и _Alignas

Это не функция, а спецификатор выравнивания для объявления объекта. В C11/C17 встроенная форма — `_Alignas`, а `<stdalign.h>` предоставляет `alignas`. В C23 `alignas` является ключевым словом.

Можно потребовать для `char` такое же выравнивание, как у `int`:

```
char alignas(int) c;
```

Можно также указать целочисленную константу или константное выражение, поддерживаемое системой:

```
char alignas(8) c; // Выравнивание по границе в 8 байтов
```

Малые степени двойки часто поддерживаются, но конкретные допустимые значения зависят от реализации. Чтобы запросить выравнивание типа `max_align_t`, подключите `<stddef.h>`:

```
char alignas(max_align_t) c;
```

Расширенное выравнивание, превышающее выравнивание `max_align_t`, может поддерживаться, но это зависит от системы.

41.2.2. alignof и _Alignof

Оператор сообщает требуемое выравнивание типа на данной системе. Например, для `char` оно может равняться 1, а для `int` — 4. В C11/C17 встроенная форма `_Alignof`, а `<stdalign.h>` предоставляет `alignof`; в C23 это ключевое слово.

```
#include <stdalign.h>
#include <stdio.h>  // for printf()
#include <stddef.h> // for max_align_t

struct t {
    int a;
    char b;
    float c;
};

int main(void)
{
    printf("char       : %zu\n", alignof(char));
```

```
    printf("short      : %zu\n", alignof(short));
    printf("int        : %zu\n", alignof(int));
    printf("long       : %zu\n", alignof(long));
    printf("long long  : %zu\n", alignof(long long));
    printf("double     : %zu\n", alignof(double));
    printf("long double: %zu\n", alignof(long double));
    printf("struct t   : %zu\n", alignof(struct t));
    printf("max_align_t: %zu\n", alignof(max_align_t));
}
```

Пример вывода автора — на другой системе значения могут отличаться:

```
char       : 1
short      : 2
int        : 4
long       : 8
long long  : 8
double     : 8
long double: 16
struct t   : 16
max_align_t: 16
```

`max_align_t` имеет выравнивание, достаточное для скалярных типов с фундаментальными требованиями к выравниванию. Расширенные требования могут быть больше.

41.3. Функция memalignment()

Автор описывает `memalignment()` как новое средство C23 и отмечает, что во время написания доступные ему компиляторы его ещё не поддерживали, поэтому пример почти не проверен. Наличие и поддержку этой функции следует отдельно проверять по документации конкретной реализации.

`alignof` полезен, когда тип известен. Но что делать, если есть лишь `void *` на участок памяти, а нужно узнать, подходит ли его выравнивание для объекта с более строгими требованиями? Это может понадобиться в низкоуровневом коде, например при работе с атомарными типами.

Автор приводит следующий незаконченный фрагмент:

```
void foo(void *p)
{
    if (memalignment(p) >= alignof(atomic int)) {
        atomic int *i = p;
        do_things(i);
    } else
        puts("This pointer is no good as an atomic int\n");
    ...
```

Это иллюстрация идеи, а не законченная программа. Само по себе достаточное выравнивание также не устанавливает, что в памяти существует подходящий объект нужного типа. Большинству программ такие проверки не нужны; они относятся преимущественно к низкоуровневой работе с памятью.